

Numerical Methods with Matlab

Ryuichi Ashino and Rémi Vaillancourt

DEPARTMENT OF MATHEMATICS AND STATISTICS, UNIVERSITY OF OTTAWA, OTTAWA, ON, CANADA, K1N 6N5

E-mail address: `remi@uottawa.ca`

Contents

Chapter 1. Solutions of Nonlinear Equations	1
1.1. Computer Arithmetics	1
1.2. Review of Calculus	4
1.3. The Bisection Method	4
1.4. Fixed Point Iteration	8
1.5. Newton's, Secant, and False Position Methods	13
1.6. Accelerating Convergence	20
1.7. Horner's Method and the Synthetic Division	22
1.8. Müller's Method	25
Chapter 2. Interpolation and Extrapolation	27
2.1. Lagrange Interpolating Polynomial	27
2.2. Newton's Divided Difference Interpolating Polynomial	29
2.3. Gregory–Newton Forward-Difference Polynomial	32
2.4. Gregory–Newton Backward-Difference Polynomial	35
2.5. Hermite Interpolating Polynomial	36
2.6. Cubic Spline Interpolation	37
Chapter 3. Numerical Differentiation and Integration	41
3.1. Numerical Differentiation	41
3.2. The Effect of Roundoff and Truncation Errors	43
3.3. Richardson's Extrapolation	45
3.4. Basic Numerical Integration Rules	47
3.5. The Composite Midpoint Rule	49
3.6. The Composite Trapezoidal Rule	51
3.7. The Composite Simpson's Rule	53
3.8. Romberg Integration for the Trapezoidal Rule	55
3.9. Adaptive Quadrature Methods	57
Chapter 4. Matrix Computations	59
4.1. LU Solution of $A\mathbf{x} = \mathbf{b}$	59
4.2. Cholesky Decomposition	67
4.3. Matrix Norms	71
4.4. Iterative Methods	73
4.5. Overdetermined Systems	75
4.6. Matrix Eigenvalues and Eigenvectors	78
4.7. The QR Decomposition	82
4.8. The QR algorithm	83
4.9. The Singular Value Decomposition	84

Chapter 5. Numerical Solution of Differential Equations	87
5.1. Initial Value Problems	87
5.2. Euler's and Improved Euler's Method	88
5.3. Low-Order Explicit Runge–Kutta Methods	91
5.4. Convergence of Numerical Methods	99
5.5. Absolutely Stable Numerical Methods	100
5.6. Stability of Runge–Kutta methods	101
5.7. Embedded Pairs of Runge–Kutta methods	104
5.8. Multistep Predictor–Corrector Methods	109
5.9. Stiff Systems of Differential Equations	122
Chapter 6. The Matlab ODE Suite	131
6.1. Introduction	131
6.2. The Methods in the Matlab ODE Suite	131
6.3. The <code>odeset</code> Options	134
6.4. Nonstiff Problems of the Matlab <code>odedemo</code>	136
6.5. Stiff Problems of the Matlab <code>odedemo</code>	136
6.6. Concluding Remarks	140
Bibliography	141
Chapter 7. Orthogonal polynomials	143
7.1. Fourier–Legendre Series	143
7.2. Derivation of Gaussian Quadratures	145
7.3. Numerical Solution of Integral Equations of the Second Kind	149
Chapter 8. Formulae and Tables	155
8.1. Legendre Polynomials $P_n(x)$ on $[-1, 1]$	155
8.2. Laguerre Polynomials on $0 \leq x < \infty$	156
8.3. Fourier–Legendre Series Expansion	157
Exercises for Numerical Methods	159
Exercises for Chapter 1	159
Exercises for Chapter 2	161
Exercises for Chapter 3	162
Exercises for Chapter 4	164
Exercises for Chapter 5	166
Solutions to Exercises for Numerical Methods	169
Solutions to Exercises for Chapter 1	169
Solutions to Exercises for Chapter 2	171
Solutions to Exercises for Chapter 4	172
Solutions to Exercises for Chapter 5	176
Index	183

CHAPTER 1

Solutions of Nonlinear Equations

1.1. Computer Arithmetics

1.1.1. Definitions. The following notation and terminology will be used.

- (1) If a is the exact value of a computation and $\tilde{a}$ is an approximate value for the same computation, then

$$\epsilon = \tilde{a} - a$$

is the **error** in $\tilde{a}$ and $|\epsilon|$ is the **absolute error**. If $a \neq 0$,

$$\epsilon_r = \frac{\tilde{a} - a}{a} = \frac{\epsilon}{a}$$

is the **relative error** in $\tilde{a}$.

- (2) **Upper bounds** for the absolute and relative errors in $\tilde{a}$ are numbers B_a and B_r such that

$$|\epsilon| = |\tilde{a} - a| < B_a, \quad |\epsilon_r| = \left| \frac{\tilde{a} - a}{a} \right| < B_r,$$

respectively.

- (3) A **roundoff error** occurs when a computer approximates a real number by a number with only a finite number of digits to the right of the decimal point (see Subsection 1.1.2).
- (4) In scientific computation, the **floating point representation** of a number c of length d in the base β is

$$c = \pm 0.b_1b_2 \cdots b_d \times \beta^N,$$

where $b_1 \neq 0$, $0 \leq b_i < \beta$. We call $b_1b_2 \cdots b_d$ the **mantissa** or **decimal part** and N the **exponent** of c . For instance, with $d = 5$ and $\beta = 10$,

$$0.27120 \times 10^2, \quad -0.31224 \times 10^3.$$

- (5) The number of **significant digits** of a floating point number is the number of digits counted from the first to the last nonzero digits. For example, with $d = 4$ and $\beta = 10$, the number of significant digits of the three numbers:

$$0.1203 \times 10^2, \quad 0.1230 \times 10^{-2}, \quad 0.1000 \times 10^3,$$

is 4, 3, and 1, respectively.

- (6) The term **truncation error** is used for the error committed when an infinite series is truncated after a finite number of terms.

REMARK 1.1. For simplicity, we shall often write floating point numbers without exponent and with zeros immediately to the right of the decimal point or with nonzero numbers to the left of the decimal point:

$$0.001203, \quad 12300.04$$

1.1.2. Rounding and chopping numbers. Real numbers are rounded away from the origin. The floating-point number, say in base 10,

$$c = \pm 0.b_1b_2 \dots b_d \times 10^N$$

is rounded to k digits as follows:

- (i) If $0.b_{k+1}b_{k+2} \dots b_m \geq 0.5$, round c to

$$(0.b_1b_2 \dots b_{k-1}b_k + 0.1 \times 10^{-k+1}) \times 10^N.$$

- (ii) If $0.b_{k+1}b_{k+2} \dots b_m < 0.5$, round c to

$$0.b_1b_2 \dots b_{k-1}b_k \times 10^N.$$

EXAMPLE 1.1. Numbers rounded to three digits:

$$1.9234542 \approx 1.92$$

$$2.5952100 \approx 2.60$$

$$1.9950000 \approx 2.00$$

$$-4.9850000 \approx -4.99$$

Floating-point numbers are chopped to k digits by replacing the digits to the right of the k th digit by zeros.

1.1.3. Cancellation in computations. Cancellation due to the subtraction of two almost equal numbers leads to a loss of significant digits. It is better to avoid cancellation than to try to estimate the error due to cancellation. Example 1.2 illustrates these points.

EXAMPLE 1.2. Use 10-digit rounded arithmetic to solve the quadratic equation

$$x^2 - 1634x + 2 = 0.$$

SOLUTION. The usual formula yields

$$x = 817 \pm \sqrt{2\,669\,948}.$$

Thus,

$$x_1 = 817 + 816.998\,776\,0 = 1.633\,998\,776 \times 10^3,$$

$$x_2 = 817 - 816.998\,776\,0 = 1.224\,000\,000 \times 10^{-3}.$$

Four of the six zeros at the end of the fractional part of x_2 are the result of cancellation and thus are meaningless. A more accurate result for x_2 can be obtained if we use the relation

$$x_1x_2 = 2.$$

In this case

$$x_2 = 1.223\,991\,125 \times 10^{-3},$$

where all digits are significant. □

From Example 1.2, it is seen that a numerically stable formula for solving the quadratic equation

$$ax^2 + bx + c = 0, \quad a \neq 0,$$

is

$$x_1 = \frac{1}{2a} \left[-b - \text{sign}(b) \sqrt{b^2 - 4ac} \right], \quad x_2 = \frac{c}{ax_1},$$

where the signum function is

$$\text{sign}(x) = \begin{cases} +1, & \text{if } x \geq 0, \\ -1, & \text{if } x < 0. \end{cases}$$

EXAMPLE 1.3. If the value of x rounded to three digits is 4.81 and the value of y rounded to five digits is 12.752, find the smallest interval which contains the exact value of $x - y$.

SOLUTION. Since

$$4.805 \leq x < 4.815 \quad \text{and} \quad 12.7515 \leq y < 12.7525,$$

then

$$4.805 - 12.7525 < x - y < 4.815 - 12.7515 \Leftrightarrow -7.9475 < x - y < -7.9365. \quad \square$$

EXAMPLE 1.4. Find the error and the relative error in the commonly used rational approximations $22/7$ and $355/113$ to the transcendental number π and express your answer in three-digit floating point numbers.

SOLUTION. The error and the relative error in $22/7$ are

$$\epsilon = 22/7 - \pi, \quad \epsilon_r = \epsilon/\pi,$$

which Matlab evaluates as

```
pp = pi
pp = 3.14159265358979
r1 = 22/7.
r1 = 3.14285714285714
abserr1 = r1 - pi
abserr1 = 0.00126448926735
relerr1 = abserr1/pi
relerr1 = 4.024994347707008e-04
```

Hence, the error and the relative error in $22/7$ rounded to three digits are

$$\epsilon = 0.126 \times 10^{-2} \quad \text{and} \quad \epsilon_r = 0.402 \times 10^{-3},$$

respectively. Similarly, Matlab computes the error and relative error in $355/113$ as

```
r2 = 355/113.
r2 = 3.14159292035398
abserr2 = r2 - pi
abserr2 = 2.667641894049666e-07
relerr2 = abserr2/pi
relerr2 = 8.491367876740610e-08
```

Hence, the error and the relative error in 355/113 rounded to three digits are

$$\epsilon = 0.267 \times 10^{-6} \quad \text{and} \quad \epsilon_r = 0.849 \times 10^{-7}. \quad \square$$

1.2. Review of Calculus

The following results from elementary calculus are needed to justify the methods of solution presented here.

THEOREM 1.1 (Intermediate Value Theorem). *Let $a < b$ and $f(x)$ be a continuous function on $[a, b]$. If w is a number strictly between $f(a)$ and $f(b)$, then there exists a number c such that $a < c < b$ and $f(c) = w$.*

COROLLARY 1.1. *Let $a < b$ and $f(x)$ be a continuous function on $[a, b]$. If $f(a)f(b) < 0$, then there exists a zero of $f(x)$ in the open interval $]a, b[$.*

PROOF. Since $f(a)$ and $f(b)$ have opposite signs, 0 lies between $f(a)$ and $f(b)$. The result follows from the intermediate value theorem with $w = 0$. $\square$

THEOREM 1.2 (Extreme Value Theorem). *Let $a < b$ and $f(x)$ be a continuous function on $[a, b]$. Then there exist numbers $\alpha \in [a, b]$ and $\beta \in [a, b]$ such that, for all $x \in [a, b]$, we have*

$$f(\alpha) \leq f(x) \leq f(\beta).$$

THEOREM 1.3 (Mean Value Theorem). *Let $a < b$ and $f(x)$ be a continuous function on $[a, b]$ which is differentiable on $]a, b[$. Then there exists a number c such that $a < c < b$ and*

$$f'(c) = \frac{f(b) - f(a)}{b - a}.$$

THEOREM 1.4 (Mean Value Theorem for Integrals). *Let $a < b$ and $f(x)$ be a continuous function on $[a, b]$. If $g(x)$ is an integrable function on $[a, b]$ which does not change sign on $[a, b]$, then there exists a number c such that $a < c < b$ and*

$$\int_a^b f(x)g(x)dx = f(c) \int_a^b g(x)dx.$$

A similar theorem holds for sums.

THEOREM 1.5 (Mean Value Theorem for Sums). *Let $\{w_i\}$, $i = 1, 2, \dots, n$, be a set of n distinct real numbers and let $f(x)$ be a continuous function on an interval $[a, b]$. If the numbers w_i all have the same sign and all the points $x_i \in [a, b]$, then there exists a number $c \in [a, b]$ such that*

$$\sum_{i=1}^n w_i f(x_i) = f(c) \sum_{i=1}^n w_i.$$

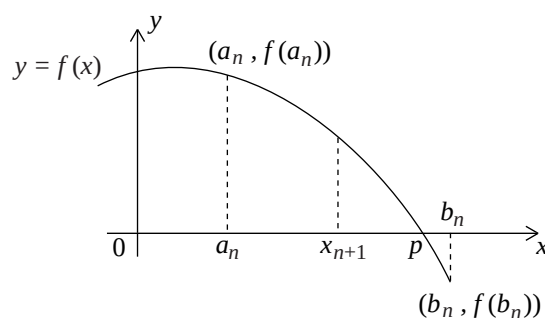
1.3. The Bisection Method

The bisection method constructs a sequence of intervals of decreasing length which contain a root p of $f(x) = 0$. If

$$f(a)f(b) < 0 \quad \text{and} \quad f \text{ is continuous on } [a, b],$$

then, by Corollary 1.1, $f(x) = 0$ has a root between a and b . The root is either between

$$a \quad \text{and} \quad \frac{a+b}{2}, \quad \text{if} \quad f(a)f\left(\frac{a+b}{2}\right) < 0,$$

FIGURE 1.1. The n th step of the bisection method.

or between

$$\frac{a+b}{2} \quad \text{and} \quad b, \quad \text{if} \quad f\left(\frac{a+b}{2}\right) f(b) < 0,$$

or exactly at

$$\frac{a+b}{2}, \quad \text{if} \quad f\left(\frac{a+b}{2}\right) = 0.$$

The n th step of the bisection method is shown in Fig. 1.1.

The algorithm of the **bisection method** is as follows.

ALGORITHM 1.1 (Bisection Method). Given that $f(x)$ is continuous on $[a, b]$ and $f(a)f(b) < 0$:

- (1) Choose $a_0 = a$, $b_0 = b$; tolerance N ; maximum number of iteration N_0 .
- (2) For $n = 0, 1, 2, \dots, N_0$, compute

$$x_{n+1} = \frac{a_n + b_n}{2}.$$

- (3) If $f(x_{n+1}) = 0$ or $(b_n - a_n)/2 < TOL$, then output $p (= x_{n+1})$ and stop.
- (4) Else if $f(x_{n+1})$ and $f(a_n)$ have opposite signs, set $a_{n+1} = a_n$ and $b_{n+1} = x_{n+1}$.
- (5) Else set $a_{n+1} = x_{n+1}$ and $b_{n+1} = b_n$.
- (6) Repeat (2), (3), (4) and (5).
- (7) Ouput 'Method failed after N_0 iterations' and stop.

Other stopping criteria are described in Subsection 1.4.1. The rate of convergence of the bisection method is low but the method always converges.

The bisection method is programmed in the following Matlab function M-file which is found in <ftp://ftp.cs.cornell.edu/pub/cv>.

```
function root = Bisection(fname,a,b,delta)
%
% Pre:
%   fname    string that names a continuous function f(x) of
%             a single variable.
%
%   a,b      define an interval [a,b]
%             f is continuous, f(a)f(b) < 0
%
```

```

%   delta   non-negative real number.
%
% Post:
%   root    the midpoint of an interval [alpha,beta]
%           with the property that f(alpha)f(beta)<=0 and
%           |beta-alpha| <= delta+eps*max(|alpha|,|beta|)
%
fa = feval(fname,a);
fb = feval(fname,b);
if fa*fb > 0
    disp('Initial interval is not bracketing.')
    return
end
if nargin==3
    delta = 0;
end
while abs(a-b) > delta+eps*max(abs(a),abs(b))
    mid = (a+b)/2;
    fmid = feval(fname,mid);
    if fa*fmid<=0
        % There is a root in [a,mid].
        b = mid;
        fb = fmid;
    else
        % There is a root in [mid,b].
        a = mid;
        fa = fmid;
    end
end
root = (a+b)/2;

```

EXAMPLE 1.5. Find an approximation to $\sqrt{2}$ using the bisection method. Stop iterating when $|x_{n+1} - x_n| < 10^{-2}$.

SOLUTION. We need to find a root of $f(x) = x^2 - 2 = 0$. Choose $a_0 = 1$ and $b_0 = 2$, and obtain recursively

$$x_{n+1} = \frac{a_n + b_n}{2}$$

by the bisection method. The results are listed in Table 1.1. The answer is $\sqrt{2} \approx 1.414063$ with an accuracy of 10^{-2} . Note that a root lies in the interval $[1.414063, 1.421875]$. $\square$

EXAMPLE 1.6. Show that the function $f(x) = x^3 + 4x^2 - 10$ has a unique root in the interval $[1, 2]$ and give an approximation to this root using eight iterations of the bisection method. Give a bound for the absolute error.

SOLUTION. Since

$$f(1) = -5 < 0 \quad \text{and} \quad f(2) = 14 > 0,$$

TABLE 1.1. Results of Example 1.5.

n	x_n	a_n	b_n	$ x_{n-1} - x_n $	$f(x_n)$	$f(a_n)$
0		1	2			—
1	1.500000	1	1.500000	.500000	+	—
2	1.250000	1.250000	1.500000	.250000	—	—
3	1.375000	1.375000	1.500000	.125000	—	—
4	1.437500	1.375000	1.437500	.062500	+	—
5	1.406250	1.406250	1.437500	.031250	—	—
6	1.421875	1.406250	1.421875	.015625	+	—
7	1.414063	1.414063	1.421875	.007812	—	—

TABLE 1.2. Results of Example 1.6.

n	x_n	a_n	b_n	$f(x_n)$	$f(a_n)$
0		1	2		—
1	1.500000000	1	1.500000000	+	—
2	1.250000000	1.250000000	1.500000000	—	—
3	1.375000000	1.250000000	1.375000000	+	—
4	1.312500000	1.312500000	1.375000000	—	—
5	1.343750000	1.343750000	1.375000000	—	—
6	1.359375000	1.359375000	1.375000000	—	—
7	1.367187500	1.359375000	1.367187500	+	—
8	1.363281250	1.363281250	1.367187500	—	—

then $f(x)$ has a root, p , in $[1, 2]$. This root is unique since $f(x)$ is strictly increasing on $[1, 2]$; in fact

$$f'(x) = 3x^2 + 4x > 0 \quad \text{for all } x \text{ between 1 and 2.}$$

The results are listed in Table 1.2.

After eight iterations, we find that p lies between 1.363281250 and 1.367187500. Therefore, the absolute error in p is bounded by

$$1.367187500 - 1.363281250 = 0.00390625. \quad \square$$

EXAMPLE 1.7. Find the number of iterations needed in Example 1.6 to have an absolute error less than 10^{-4} .

SOLUTION. Since the root, p , lies in each interval $[a_n, b_n]$, after n iterations the error is at most $b_n - a_n$. Thus, we want to find n such that $b_n - a_n < 10^{-4}$. Since, at each iteration, the length of the interval is halved, it is easy to see that

$$b_n - a_n = (2 - 1)/2^n.$$

Therefore, n satisfies the inequality

$$2^{-n} < 10^{-4},$$

that is,

$$\ln 2^{-n} < \ln 10^{-4}, \quad \text{or} \quad -n \ln 2 < -4 \ln 10.$$

Thus,

$$n > 4 \ln 10 / \ln 2 = 13.28771238 \implies n = 14.$$

Hence, we need 14 iterations. $\square$

1.4. Fixed Point Iteration

Let $f(x)$ be a real-valued function of a real variable x . In this section, we present iterative methods for solving equations of the form

$$f(x) = 0. \quad (1.1)$$

A **root** of the equation $f(x) = 0$, or a **zero** of $f(x)$, is a number p such that $f(p) = 0$.

To find a root of equation (1.1), we rewrite this equation in an equivalent form

$$x = g(x), \quad (1.2)$$

for instance, $g(x) = x - f(x)$.

We say that (1.1) and (1.2) are **equivalent** (on a given interval) if any root of (1.1) is a fixed point for (1.2) and vice-versa.

Conversely, if, for a given initial value x_0 , the sequence $x_0, x_1, \dots$, defined by the recurrence

$$x_{n+1} = g(x_n), \quad n = 0, 1, \dots, \quad (1.3)$$

converges to a number p , we say that the fixed point method converges. If $g(x)$ is continuous, then $p = g(p)$. This is seen by taking the limit in equation (1.3) as $n \rightarrow \infty$. The number p is called a **fixed point** for the function $g(x)$ of the fixed point iteration (1.2).

It is easily seen that the two equations

$$x^3 + 9x - 9 = 0, \quad x = (9 - x^3)/9$$

are equivalent. The problem is to choose a suitable function $g(x)$ and a suitable initial value x_0 to have convergence. To treat this question we need to define the different types of fixed points.

DEFINITION 1.1. A fixed point, $p = g(p)$, of an iterative scheme

$$x_{n+1} = g(x_n),$$

is said to be *attractive*, *repulsive* or *indifferent* if the *multiplier*, $g'(p)$, of $g(x)$ satisfies

$$|g'(p)| < 1, \quad |g'(p)| > 1, \quad \text{or} \quad |g'(p)| = 1,$$

respectively.

THEOREM 1.6 (Fixed Point Theorem). Let $g(x)$ be a real-valued function satisfying the following conditions:

- (1) $g(x) \in [a, b]$ for all $x \in [a, b]$.
- (2) $g(x)$ is differentiable on $[a, b]$.
- (3) There exists a number K , $0 < K < 1$, such that $|g'(x)| \leq K$ for all $x \in (a, b)$.

Then $g(x)$ has a unique attractive fixed point $p \in [a, b]$. Moreover, for arbitrary $x_0 \in [a, b]$, the sequence $x_0, x_1, x_2, \dots$ defined by

$$x_{n+1} = g(x_n), \quad n = 0, 1, 2, \dots,$$

converges to p .

PROOF. If $g(a) = a$ or $g(b) = b$, the existence of an attractive fixed point is obvious. Suppose not, then it follows that $g(a) > a$ and $g(b) < b$. Define the auxiliary function

$$h(x) = g(x) - x.$$

Then h is continuous on $[a, b]$ and

$$h(a) = g(a) - a > 0, \quad h(b) = g(b) - b < 0.$$

By Corollary 1.1, there exists a number $p \in]a, b[$ such that $h(p) = 0$, that is, $g(p) = p$ and p is a fixed point for $g(x)$.

To prove uniqueness, suppose that p and q are distinct fixed points for $g(x)$ in $[a, b]$. By the Mean Value Theorem 1.3, there exists a number c between p and q (and hence in $[a, b]$) such that

$$|p - q| = |g(p) - g(q)| = |g'(c)| |p - q| \leq K |p - q| < |p - q|,$$

which is a contradiction. Thus $p = q$ and the attractive fixed point in $[a, b]$ is unique.

We now prove convergence. By the Mean Value Theorem 1.3, for each pair of numbers x and y in $[a, b]$, there exists a number c between x and y such that

$$g(x) - g(y) = g'(c)(x - y).$$

Hence,

$$|g(x) - g(y)| \leq K |x - y|.$$

In particular,

$$|x_{n+1} - p| = |g(x_n) - g(p)| \leq K |x_n - p|.$$

Repeating this procedure $n + 1$ times, we have

$$|x_{n+1} - p| \leq K^{n+1} |x_0 - p| \rightarrow 0, \quad \text{as } n \rightarrow \infty,$$

since $0 < K < 1$. Thus the sequence $\{x_n\}$ converges to p . $\square$

EXAMPLE 1.8. Find a root of the equation

$$f(x) = x^3 + 9x - 9 = 0$$

in the interval $[0, 1]$ by a fixed point iterative scheme.

SOLUTION. Solving this equation is equivalent to finding a fixed point for

$$g(x) = (9 - x^3)/9.$$

Since

$$f(0)f(1) = -9 < 0,$$

Corollary 1.1 implies that $f(x)$ has a root, p , between 0 and 1. Condition (3) of Theorem 1.6 is satisfied with $K = 1/3$ since

$$|g'(x)| = |-x^2/3| \leq 1/3$$

for all x between 0 and 1. The other conditions are also satisfied.

Five iterations are performed with Matlab starting with $x_0 = 0.5$. The function M-file `exp8_8.m` is

```
function x1 = exp8_8(x0); % Example 8.8.
x1 = (9-x0^3)/9;
```

TABLE 1.3. Results of Example 1.8.

n	x_n	error ϵ_n	$\epsilon_n/\epsilon_{n-1}$
0	0.500000000000000	-0.41490784153366	1.000000000000000
1	0.986111111111111	0.07120326957745	0.07120326957745
2	0.89345451579409	-0.02145332573957	-0.30129691890395
3	0.92075445888550	0.00584661735184	-0.01940483617658
4	0.91326607850598	-0.00164176302768	0.08460586900804
5	0.91536510274262	0.00045726120896	0.00540460389243

The exact solution

$$0.91490784153366$$

is obtained by means of some 30 iterations. The following iterative procedure solves the problem.

```

xexact = 0.91490784153366;
N = 5; x=zeros(N+1,4);
x0 = 0.5; x(1,:) = [0 x0 (x0-xexact), 1];
for i = 1:N
    xt=exp8_8(x(i,2));
    x(i+1,:) = [i xt (xt-xexact), (xt-xexact)/x(i,4)];
end

```

The iterates, their errors and the ratios of successive errors are listed in Table 1.3. One sees that the ratios of successive errors are decreasing; therefore the order of convergence, defined in Subsection 1.4.2, is greater than one, but smaller than two since the number of correct digits does not double from one iterate to the next. $\square$

In Example 1.9 below, we shall show that the convergence of an iterative scheme $x_{n+1} = g(x_n)$ to an attractive fixed point depends upon a judicious rearrangement of the equation $f(x) = 0$ to be solved. In fact, besides fixed points, an iterative scheme may have cycles which are defined in Definition 1.2, where $g^2(x) = g(g(x))$, $g^3(x) = g(g^2(x))$ etc.

DEFINITION 1.2. Given an iterative scheme

$$x_{n+1} = g(x_n),$$

a k -cycle of $g(x)$ is a set of k distinct points,

$$x_0, \quad x_1, \quad x_2, \quad \dots, \quad x_{k-1},$$

satisfying the relations

$$x_1 = g(x_0), \quad x_2 = g^2(x_0), \quad \dots, \quad x_{k-1} = g^{k-1}(x_0), \quad x_0 = g^k(x_0).$$

The *multiplier* of a k cycle is

$$(g^k)'(x_j) = g'(x_{k-1}) \cdots g'(x_0), \quad j = 0, 1, \dots, k-1.$$

A k -cycle is *attractive*, *repulsive*, or *indifferent* as

$$|(g^k)'(x_j)| < 1, \quad > 1, \quad = 1.$$

A fixed point is a 1-cycle.

The multiplier of a cycle is seen to be the same at every point of the cycle.

EXAMPLE 1.9. Find a root of the equation

$$f(x) = x^3 + 4x^2 - 10 = 0$$

in the interval $[1, 2]$ by fixed point iterative schemes and study their convergence properties.

SOLUTION. Since $f(1)f(2) = -70 < 0$, the equation $f(x) = 0$ has a root in the interval $[1, 2]$. The exact roots are given by the Matlab command `roots`

```
p=[1 4 0 -10]; % the polynomial f(x)
r =roots(p)
r =
-2.68261500670705 + 0.35825935992404i
-2.68261500670705 - 0.35825935992404i
1.36523001341410
```

There is one real root, which we denote by x_∞ , in the interval $[1, 2]$, and a pair of complex conjugate roots.

Six iterations are performed with the following five rearrangements $x = g_j(x)$, $j = 1, 2, 3, 4, 5$, of the given equation $f(x) = 0$. The derivative of $g'_j(x)$ is evaluated at the real root $x_\infty \approx 1.365$.

$$\begin{aligned} x = g_1(x) &=: 10 + x - 4x^2 - x^3, & g'_1(x_\infty) &\approx -15.51, \\ x = g_2(x) &=: \sqrt{(10/x) - 4x}, & g'_2(x_\infty) &\approx -3.42, \\ x = g_3(x) &=: \frac{1}{2}\sqrt{10 - x^3}, & g'_3(x_\infty) &\approx -0.51, \\ x = g_4(x) &=: \sqrt{10/(4+x)}, & g'_4(x_\infty) &\approx -0.13 \\ x = g_5(x) &=: x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}, & g'_5(x_\infty) &= 0. \end{aligned}$$

The Matlab function M-file `exp1_9.m` is

```
function y = exp1_9(x); % Example 1.9.
y = [10+x(1)-4*x(1)^2-x(1)^3; sqrt((10/x(2))-4*x(2));
sqrt(10-x(3)^3)/2; sqrt(10/(4+x(4)));
x(5)-(x(5)^3+4*x(5)^2-10)/(3*x(5)^2+8*x(5))];
```

The following iterative procedure is used.

```
N = 6; x=zeros(N+1,5);
x0 = 1.5; x(1,:) = [0 x0 x0 x0 x0];
for i = 1:N
xt=exp1_9(x(i,2:5));
x(i+1,:) = [i xt];
end
```

The results are summarized in Table 1.4. We see from the table that x_∞ is an attractive fixed point of $g_3(x)$, $g_4(x)$ and $g_5(x)$. Moreover, $g_4(x_n)$ converges more quickly to the root 1.365 230 013 than $g_3(x_n)$, and $g_5(x)$ converges even faster. In fact, these three fixed point methods need 30, 15 and 4 iterations, respectively,

TABLE 1.4. Results of Example 1.9.

	$g_1(x)$	$g_2(x)$	$g_3(x)$	$g_4(x)$	$g_5(x)$
n	$10 + x - 4x^2 - x^3$	$\sqrt{(10/x) - 4x}$	$0.5\sqrt{10 - x^3}$	$\sqrt{10/(4 + x)}$	$x - \frac{x^3 + 4x^2 - 10}{2x^2 + 8x}$
0	1.5	1.5	1.5	1.5	1.5
1	-0.8750	0.816	1.286953	1.348399	1.373333333
2	6.732421875	2.996	1.402540	1.367376	1.365262015
3	-4.6972001×10^2	$0.00 - 2.94i$	1.345458	1.364957	1.365230014
4	1.0275×10^8	$2.75 - 2.75i$	1.375170	1.365264	1.365230013
5	-1.08×10^{24}	$1.81 - 3.53i$	1.360094	1.365225	
6	1.3×10^{72}	$2.38 - 3.43i$	1.367846	1.365230	

to produce a 10-digit correct answer. On the other hand, the sequence $g_2(x_n)$ is trapped in an attractive two-cycle,

$$z_{\pm} = 2.27475487839820 \pm 3.60881272309733i,$$

with multiplier

$$g'_2(z_+)g'_2(z_-) = 0.19790433047378$$

which is smaller than one in absolute value. Once in an attractive cycle, an iteration cannot converge to a fixed point. Finally x_{∞} is a repulsive fixed point of $g_1(x)$ and $x_{n+1} = g(x_n)$ diverges to $-\infty$. $\square$

REMARK 1.2. An iteration started in the basin of attraction of an attractive fixed point (or cycle) will converge to that fixed point (or cycle). An iteration started near a repulsive fixed point (or cycle) will not converge to that fixed point (or cycle). Convergence to an indifferent fixed point is very slow, but can be accelerated by different acceleration processes.

1.4.1. Stopping criteria. Three usual criteria that are used to decide when to stop an iteration procedure to find a zero of $f(x)$ are:

- (1) Stop after N iterations (for a given N).
- (2) Stop when $|x_{n+1} - x_n| < \epsilon$ (for a given ϵ).
- (3) Stop when $|f(x_n)| < \eta$ (for a given η).

The usefulness of any of these criteria is problem dependent.

1.4.2. Order and rate of convergence of an iterative method. We are often interested in the rate of convergence of an iterative scheme. Suppose that the function $g(x)$ for the iterative method

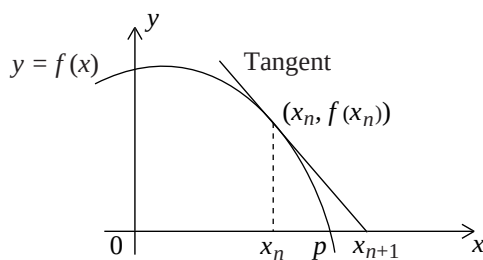
$$x_{n+1} = g(x_n)$$

has a Taylor expansion about the fixed point p ($p = g(p)$) and let

$$\epsilon_n = x_n - p.$$

Then, we have

$$\begin{aligned} x_{n+1} &= g(x_n) = g(p + \epsilon_n) = g(p) + g'(p)\epsilon_n + \frac{g''(p)}{2!}\epsilon_n^2 + \dots \\ &= p + g'(p)\epsilon_n + \frac{g''(p)}{2!}\epsilon_n^2 + \dots \end{aligned}$$

FIGURE 1.2. The n th step of Newton's method.

Hence,

$$\epsilon_{n+1} = x_{n+1} - p = g'(p)\epsilon_n + \frac{g''(p)}{2!}\epsilon_n^2 + \dots \quad (1.4)$$

DEFINITION 1.3. The **order of an iterative method** $x_{n+1} = g(x_n)$ is the order of the first non-zero derivative of $g(x)$ at p . A method of order p is said to have a **rate of convergence** p .

In Example 1.9, the iterative schemes $g_3(x)$ and $g_4(x)$ converge to first order, while $g_5(x)$ converges to second order.

Note that, for a second-order iterative scheme, we have

$$\frac{\epsilon_{n+1}}{\epsilon_n^2} \approx \frac{g''(p)}{2} = \text{constant}.$$

1.5. Newton's, Secant, and False Position Methods

1.5.1. Newton's method. Let x_n be an approximation to a root, p , of $f(x) = 0$. Draw the tangent line

$$y = f(x_n) + f'(x_n)(x - x_n)$$

to the curve $y = f(x)$ at the point $(x_n, f(x_n))$ as shown in Fig. 1.2. Then x_{n+1} is determined by the point of intersection, $(x_{n+1}, 0)$, of this line with the x -axis,

$$0 = f(x_n) + f'(x_n)(x_{n+1} - x_n).$$

If $f'(x_n) \neq 0$, solving this equation for x_{n+1} we obtain **Newton's method**, also called the **Newton-Raphson method**,

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \quad (1.5)$$

Note that Newton's method is a fixed point method since it can be rewritten in the form

$$x_{n+1} = g(x_n), \quad \text{where} \quad g(x) = x - \frac{f(x)}{f'(x)}.$$

EXAMPLE 1.10. Approximate $\sqrt{2}$ by Newton's method. Stop when $|x_{n+1} - x_n| < 10^{-4}$.

SOLUTION. We wish to find a root to the equation

$$f(x) = x^2 - 2 = 0.$$

TABLE 1.5. Results of Example 1.10.

n	x_n	$ x_n - x_{n-1} $
0	2	
1	1.5	0.5
2	1.416667	0.083333
3	1.414216	0.002451
4	1.414214	0.000002

TABLE 1.6. Results of Example 1.11.

n	x_n	$ x_n - x_{n-1} $
0	1.5	
1	1.37333333333333	0.126667
2	1.36526201487463	0.00807132
3	1.36523001391615	0.000032001
4	1.3652300134141	5.0205×10^{-10}
5	1.3652300134141	2.22045×10^{-16}
6	1.3652300134141	2.22045×10^{-16}

In this case, Newton's method becomes

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} = x_n - \frac{x_n^2 - 2}{2x_n} = \frac{x_n^2 + 2}{2x_n}.$$

With $x_0 = 2$, we obtain the results listed in Table 1.5. Therefore,

$$\sqrt{2} \approx 1.414214.$$

Note that the number of zeros in the errors roughly doubles as it is the case with methods of second order. $\square$

EXAMPLE 1.11. Use six iterations of Newton's method to approximate a root $p \in [1, 2]$ of the polynomial

$$f(x) = x^3 + 4x^2 - 10 = 0$$

given in Example 1.9.

SOLUTION. In this case, Newton's method becomes

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} = x_n - \frac{x_n^3 + 4x_n^2 - 10}{3x_n^2 + 8x_n} = \frac{2(x_n^3 + 2x_n^2 + 5)}{3x_n^2 + 8x_n}.$$

We take $x_0 = 1.5$. The results are listed in Table 1.6. $\square$

THEOREM 1.7. Let p be a simple root of $f(x) = 0$, that is, $f(p) = 0$ and $f'(p) \neq 0$. If $f''(p)$ exists, then Newton's method is at least of second order near p .

PROOF. Differentiating the function

$$g(x) = x - \frac{f(x)}{f'(x)}$$

TABLE 1.7. Results of Example 1.12.

n	x_n	$\epsilon_{n+1}/\epsilon_n$	x_n	$\epsilon_{n+1}/\epsilon_n^2$
0	0.000		0.000000000000000	
1	0.400	0.600	0.800000000000000	-0.2000
2	0.652	2.245	0.98461538461538	-0.3846
3	0.806	0.143	0.99988432620012	-0.4887
4	0.895	0.537	0.99999999331095	-0.4999
5	0.945	0.522	1	
6	0.972	0.512	1	

we have

$$\begin{aligned} g'(x) &= 1 - \frac{(f'(x))^2 - f(x)f''(x)}{(f'(x))^2} \\ &= \frac{f(x)f''(x)}{(f'(x))^2}. \end{aligned}$$

Since $f(p) = 0$, we have

$$g'(p) = 0.$$

Therefore, Newton's method is of order two near a simple zero of f . $\square$

REMARK 1.3. Taking the second derivative of $g(x)$ in Newton's method, we have

$$g''(x) = \frac{(f'(x))^2 f''(x) + f(x)f'(x)f'''(x) - 2f(x)(f''(x))^2}{(f'(x))^3}.$$

If $f'''(p)$ exists, we obtain

$$g''(p) = -\frac{f''(p)}{f'(p)}.$$

Thus, by (1.4), the successive errors satisfy the approximate relation

$$\epsilon_{n+1} \approx -\frac{1}{2} \frac{f''(p)}{f'(p)} \epsilon_n^2,$$

which explains the doubling of the number of leading zeros in the error of Newton's method near a simple root of $f(x) = 0$.

EXAMPLE 1.12. Use six iterations of the ordinary and modified Newton's methods

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad x_{n+1} = x_n - 2 \frac{f(x_n)}{f'(x_n)}$$

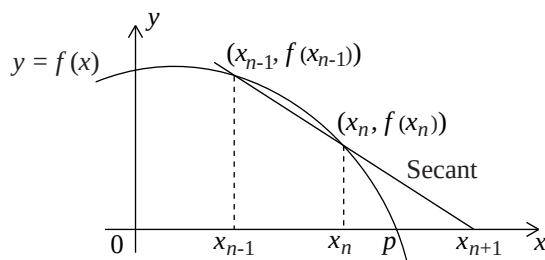
to approximate the double root, $x = 1$, of the polynomial

$$f(x) = (x-1)^2(x-2).$$

SOLUTION. The two methods have iteration functions

$$g_1(x) = x - \frac{(x-1)(x-2)}{2(x-2) + (x-1)}, \quad g_2(x) = x - \frac{(x-1)(x-2)}{(x-2) + (x-1)},$$

respectively. We take $x_0 = 0$. The results are listed in Table 1.7. One sees that Newton's method has first-order convergence near a double zero of $f(x)$, but one

FIGURE 1.3. The n th step of the secant method.

can verify that the modified Newton method has second-order convergence. In fact, near a root of multiplicity m the modified Newton method

$$x_{n+1} = x_n - m \frac{f(x_n)}{f'(x_n)}$$

has second-order convergence. $\square$

In general, Newton's method may converge to the desired root, to another root, or to an attractive cycle, especially in the complex plane.

1.5.2. The secant method. Let x_{n-1} and x_n be two approximations to a root, p , of $f(x) = 0$. Draw the secant to the curve $y = f(x)$ through the points $(x_{n-1}, f(x_{n-1}))$ and $(x_n, f(x_n))$. The equation of this secant is

$$y = f(x_n) + \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}} (x - x_n).$$

The $(n+1)$ st iterate x_{n+1} is determined by the point of intersection $(x_{n+1}, 0)$ of the secant with the x -axis as shown in Fig. 1.3,

$$0 = f(x_n) + \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}} (x_{n+1} - x_n).$$

Solving for x_{n+1} , we obtain the **secant method**:

$$x_{n+1} = x_n - \frac{(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})} f(x_n). \quad (1.6)$$

The algorithm for the secant method is as follows.

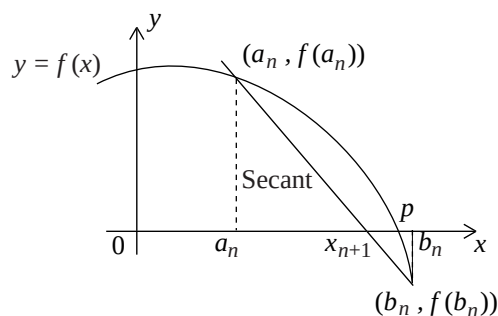
ALGORITHM 1.2 (Secant Method). Given that $f(x)$ is continuous on $[a, b]$ and has a root in $[a, b]$.

- (1) Choose x_0 and x_1 near the root p that is sought.
- (2) Given x_{n-1} and x_n , x_{n+1} is obtained by the formula

$$x_{n+1} = x_n - \frac{(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})} f(x_n),$$

provided that $f(x_n) - f(x_{n-1}) \neq 0$. If $f(x_n) - f(x_{n-1}) = 0$, try other starting values x_0 and x_1 .

- (3) Repeat (2) until the selected stopping criterion is satisfied (see Subsection 1.4.1).

FIGURE 1.4. The n th step of the method of false position.

This method converges to a simple root to order 1.618 and may not converge to a multiple root. Thus it is generally slower than Newton's method. However, it does not require the derivative of $f(x)$. In general applications of Newton's method, the derivative of the function $f(x)$ is approximated numerically by the slope of a secant to the curve.

1.5.3. The method of false position. The method of false position, also called *regula falsi*, is similar to the secant method, but with the additional condition that, for each $n = 0, 1, 2, \dots$, the pair of approximate values, a_n and b_n , to the root, p , of $f(x) = 0$ be such that $f(a_n)f(b_n) < 0$. The next iterate, x_{n+1} , is determined by the intersection of the secant passing through the points $(a_n, f(a_n))$ and $(b_n, f(b_n))$ with the x -axis.

The equation for the secant through $(a_n, f(a_n))$ and $(b_n, f(b_n))$, shown in Fig. 1.4, is

$$y = f(a_n) + \frac{f(b_n) - f(a_n)}{b_n - a_n} (x - a_n).$$

Hence, x_{n+1} satisfies the equation

$$0 = f(a_n) + \frac{f(b_n) - f(a_n)}{b_n - a_n} (x_{n+1} - a_n),$$

which leads to the **method of false position**:

$$x_{n+1} = \frac{a_n f(b_n) - b_n f(a_n)}{f(b_n) - f(a_n)}. \quad (1.7)$$

The algorithm for the method of false position is as follows.

ALGORITHM 1.3 (False Position Method). Given that $f(x)$ is continuous on $[a, b]$ and that $f(a)f(b) < 0$.

- (1) Pick $a_0 = a$ and $b_0 = b$.
- (2) Given a_n and b_n such that $f(a_n)f(b_n) < 0$, compute

$$x_{n+1} = \frac{a_n f(b_n) - b_n f(a_n)}{f(b_n) - f(a_n)}.$$

- (3) If $f(x_{n+1}) = 0$, stop.
- (4) Else if $f(x_{n+1})$ and $f(a_n)$ have opposite signs, set $a_{n+1} = a_n$ and $b_{n+1} = x_{n+1}$;
- (5) Else set $a_{n+1} = x_{n+1}$ and $b_{n+1} = b_n$.

TABLE 1.8. Results of Example 1.13.

n	x_n	a_n	b_n	$ x_{n-1} - x_n $	$f(x_n)$	$f(a_n)$
0		1	2			—
1	1.333333	1.333333	2		—	—
2	1.400000	1.400000	2	0.066667	—	—
3	1.411765	1.411765	2	0.011765	—	—
4	1.413793	1.413793	2	0.002028	—	—
5	1.414141	1.414141	2	0.000348	—	—

- (6) Repeat (2)–(5) until the selected stopping criterion is satisfied (see Subsection 1.4.1).

This method is generally slower than Newton’s method, but it does not require the derivative of $f(x)$ and it always converges to a nested root. If the approach to the root is one-sided, convergence can be accelerated by replacing the value of $f(x)$ at the stagnant end position with $f(x)/2$.

EXAMPLE 1.13. Find an approximation to $\sqrt{2}$ using the method of false position. Stop iterating when $|x_{n+1} - x_n| < 10^{-3}$.

SOLUTION. This problem is equivalent to the problem of finding a root of the equation

$$f(x) = x^2 - 2 = 0.$$

We have

$$x_{n+1} = \frac{a_n(b_n^2 - 2) - b_n(a_n^2 - 2)}{(b_n^2 - 2) - (a_n^2 - 2)} = \frac{a_n b_n + 2}{a_n + b_n}.$$

Choose $a_0 = 1$ and $b_0 = 2$. Notice that $f(1) < 0$ and $f(2) > 0$. The results are listed in Table 1.8. Therefore, $\sqrt{2} \approx 1.414141$. $\square$

1.5.4. A global Newton-bisection method. The many difficulties that can occur with Newton’s method can be handled with success by combining the Newton and bisection ideas in a way that captures the best features of each framework. At the beginning, it is assumed that we have a bracketing interval $[a, b]$ for $f(x)$, that is, $f(a)f(b) < 0$, and that the initial value x_c is one of the endpoints. If

$$x_+ = x_c - \frac{f(x_c)}{f'(x_c)} \in [a, b],$$

we proceed with either $[a, x_+]$ or $[x_+, b]$, whichever is bracketing. The new x_c equals x_+ . If the Newton step falls out of $[a, b]$, we take a bisection step setting the new x_c to $(a + b)/2$. In a typical situation, a number of bisection steps are taken before the Newton iteration takes over. This globalization of the Newton iteration is programmed in the following Matlab function M-file which is found in <ftp://ftp.cs.cornell.edu/pub/cv>.

```
function [x,fx,nEvals,aF,bF] = ...
    GlobalNewton(fName,fpName,a,b,tolx,tolf,nEvalsMax)

% Pre:
%   fName      string that names a function f(x).
```

```

% fpName      string that names the derivative function f'(x).
% a,b         A root of f(x) is sought in the interval [a,b]
%             and f(a)*f(b)<=0.
% tolx,tolf   Nonnegative termination criteria.
% nEvalsMax   Maximum number of derivative evaluations.
%
% Post:
% x           An approximate zero of f.
% fx          The value of f at x.
% nEvals      The number of derivative evaluations required.
% aF,bF       The final bracketing interval is [aF,bF].
%
% Comments:
% Iteration terminates as soon as x is within tolx of a true zero
% or if |f(x)|<= tolf or after nEvalMax f-evaluations

fa = feval(fName,a);
fb = feval(fName,b);
if fa*fb>0
    disp('Initial interval not bracketing.')
    return
end
x = a;
fx = feval(fName,x);
fpx = feval(fpName,x);
disp(sprintf('%20.15f %20.15f %20.15f',a,x,b))

nEvals = 1;
while (abs(a-b) > tolx ) & (abs(fx) > tolf) &
    ((nEvals<nEvalsMax) | (nEvals==1))
    %[a,b] brackets a root and x = a or x = b.
    if StepIsIn(x,fx,fpx,a,b)
        %Take Newton Step
        disp('Newton')
        x = x-fx/fpx;
    else
        %Take a Bisection Step:
        disp('Bisection')
        x = (a+b)/2;
    end
    fx = feval(fName,x);
    fpx = feval(fpName,x);
    nEvals = nEvals+1;
    if fa*fx<=0
        % There is a root in [a,x]. Bring in right endpoint.
        b = x;
        fb = fx;
    else

```

```

        % There is a root in [x,b]. Bring in left endpoint.
        a = x;
        fa = fx;
    end
    disp(sprintf('%20.15f  %20.15f  %20.15f',a,x,b))
end
aF = a;
bF = b;

```

1.5.5. The Matlab fzero function. The MATLAB `fzero` function is a general-purpose root finder that does not require derivatives. A simple call involves only the name of the function and a starting value x_0 . For example

```
aroot = fzero('function_name', x0)
```

The value returned is near a point where the function changes sign, or NaN if the search fails. Other options are described in `help fzero`.

1.6. Accelerating Convergence

The linear convergence of an iterative method can be accelerated by Aitken's process. Suppose that the sequence $\{x_n\}$ converges to a fixed point p to first order. Then the following ratios are approximately equal:

$$\frac{x_{n+1} - p}{x_n - p} \approx \frac{x_{n+2} - p}{x_{n+1} - p}.$$

We make this an equality by substituting a_n for p ,

$$\frac{x_{n+1} - a_n}{x_n - a_n} = \frac{x_{n+2} - a_n}{x_{n+1} - a_n}$$

and solve for a_n which, after some algebraic manipulation, becomes

$$a_n = x_n - \frac{(x_{n+1} - x_n)^2}{x_{n+2} - 2x_{n+1} + x_n}.$$

This is Aitken's process which accelerates convergence in the sense that

$$\lim_{n \rightarrow \infty} \frac{a_n - p}{x_n - p} = 0.$$

If we introduce the first- and second-order forward differences:

$$\Delta x_n = x_{n+1} - x_n, \quad \Delta^2 x_n = \Delta(\Delta x_n) = x_{n+2} - 2x_{n+1} + x_n,$$

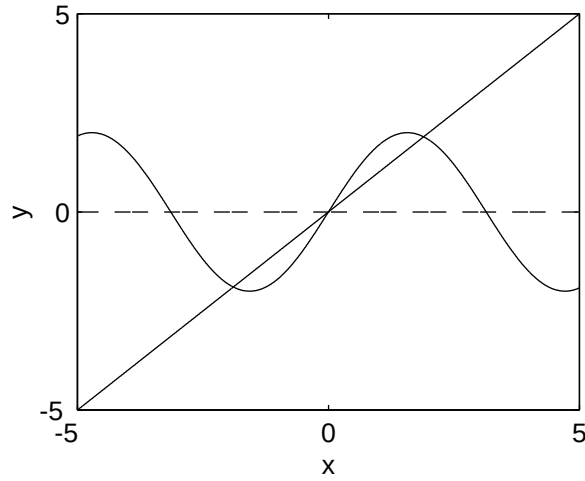
then **Aitken's process** becomes

$$a_n = x_n - \frac{(\Delta x_n)^2}{\Delta^2 x_n}. \quad (1.8)$$

Steffensen's process assumes that $s_1 = a_0$ is a better value than x_2 . Thus $s_0 = x_0$, $z_1 = g(s_0)$ and $z_2 = g(z_1)$ are used to produce s_1 . Next, s_1 , $z_1 = g(s_1)$ and $z_2 = g(z_2)$ are used to produce s_2 . And so on. The algorithm is as follows.

ALGORITHM 1.4 (Steffensen's Algorithm). Set

$$s_0 = x_0,$$

FIGURE 1.5. The three real roots of $x = 2 \sin x$ in Example 1.14.

and, for $n = 0, 1, 2, \dots$,

$$\begin{aligned} z_1 &= g(s_n), \\ z_2 &= g(z_1), \\ s_{n+1} &= s_n - \frac{(z_1 - s_n)^2}{z_2 - 2z_1 + s_n}. \end{aligned}$$

Steffensen's process applied to a first-order fixed point method produces a second-order method.

EXAMPLE 1.14. Consider the fixed point iteration $x_{n+1} = g(x_n)$:

$$x_{n+1} = 2 \sin x_n, \quad x_0 = 1.$$

Do seven iterations and perform Aitken's and Steffensen's accelerations.

SOLUTION. The three real fixed points of $x = 2 \sin x$ can be seen in Fig. 1.5. The Matlab function `fzero` produces the fixed point near $x = 1$:

```
p = fzero('x-2*sin(x)',1.)
p = 1.89549426703398
```

The convergence is linear since

$$g'(p) = -0.63804504828524 \neq 0.$$

The following Matlab M function and script produce the results listed in Table 1.9. The second, third, and fourth columns are the iterates x_n , Aitken's and Steffensen's accelerated sequences a_n and s_n , respectively. The fifth column, which lists $\epsilon_{n+1}/\epsilon_n^2 = (s_{n+2} - s_{n+1})/(s_{n+1} - s_n)^2$ tending to a constant, indicates that the Steffensen sequence s_n converges to second order.

The M function function is:

```
function f = twosine(x);
f = 2*sin(x);
```

TABLE 1.9. Results of Example 1.14.

n	x_n	a_n	s_n	$\epsilon_{n+1}/\epsilon_n^2$
0	1.000000000000000	2.23242945471637	1.000000000000000	-0.2620
1	1.68294196961579	1.88318435428750	2.23242945471637	0.3770
2	1.98743653027215	1.89201364327283	1.83453173271065	0.3560
3	1.82890755262358	1.89399129067379	1.89422502453561	0.3689
4	1.93374764234016	1.89492839486397	1.89549367325365	0.3691
5	1.86970615363078	1.89525656226218	1.89549426703385	
6	1.91131617912526		1.89549426703398	
7	1.88516234821223		NaN	

The M script function is:

```

n = 7;
x = ones(1,n+1);
x(1) = 1.0;
for k = 1:n
    x(k+1)=twosine(x(k)); % iterating x(k+1) = 2*sin(x(k))
end

a = ones(1,n-1);
for k = 1:n-1
    a(k) = x(k) - (x(k+1)-x(k))^2/(x(k+2)-2*x(k+1)+x(k)); % Aitken
end

s = ones(1,n+1);
s(1) = 1.0;
for k = 1:n
    z1=twosine(s(k));
    z2=twosine(z1);
    s(k+1) = s(k) - (z1-s(k))^2/(z2-2*z1+s(k)); % Steffensen
end

d = ones(1,n-2);
for k = 1:n-2
    d(k) = (s(k+2)-s(k+1))/(s(k+1)-s(k))^2; % 2nd order convergence
end

```

Note that the Matlab program produced NaN (not a number) for s_7 because of a division by zero. $\square$

1.7. Horner's Method and the Synthetic Division

1.7.1. Horner's method. To reduce the number of products in the evaluation of polynomials, these should be expressed in nested form. For instance,

$$\begin{aligned}
 p(x) &= a_3x^3 + a_2x^2 + a_1x + a_0 \\
 &= ((a_3x + a_2)x + a_1)x + a_0.
 \end{aligned}$$

In this simple case, the reduction is from 8 to 3 products.

The Matlab command `horner` transforms a symbolic polynomial into its Horner, or nested, representation.

```
syms x
p = x^3-6*x^2+11*x-6
p = x^3-6*x^2+11*x-6
hp = horner(p)
hp = -6+(11+(-6+x)*x)*x
```

Horner's method incorporates this nesting technique.

THEOREM 1.8 (Horner's Method). *Let*

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0.$$

If $b_n = a_n$ and

$$b_k = a_k + b_{k+1}x_0, \quad \text{for } k = n-1, n-2, \dots, 1, 0,$$

then

$$b_0 = p(x_0).$$

Moreover, if

$$q(x) = b_n x^{n-1} + b_{n-1} x^{n-2} + \cdots + b_2 x + b_1,$$

then

$$p(x) = (x - x_0)q(x) + b_0.$$

PROOF. By the definition of $q(x)$,

$$\begin{aligned} (x - x_0)q(x) + b_0 &= (x - x_0)(b_n x^{n-1} + b_{n-1} x^{n-2} + \cdots + b_2 x + b_1) + b_0 \\ &= (b_n x^n + b_{n-1} x^{n-1} + \cdots + b_2 x^2 + b_1 x) \\ &\quad - (b_n x_0 x^{n-1} + b_{n-1} x_0 x^{n-2} + \cdots + b_2 x_0 x + b_1 x_0) + b_0 \\ &= b_n x^n + (b_{n-1} - b_n x_0) x^{n-1} + \cdots + (b_1 - b_2 x_0) x + (b_0 - b_1 x_0) \\ &= a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \\ &= p(x) \end{aligned}$$

and

$$b_0 = p(x_0). \quad \square$$

1.7.2. Synthetic division. Evaluating a polynomial at $x = x_0$ by Horner's method is equivalent to applying the synthetic division as shown in Example 1.15.

EXAMPLE 1.15. Find the value of the polynomial

$$p(x) = 2x^4 - 3x^2 + 3x - 4$$

at $x_0 = -2$ by Horner's method.

SOLUTION. By successively multiplying the elements of the third line of the following tableau by $x_0 = -2$ and adding to the first line, one gets the value of $p(-2)$.

$a_4 = 2$	$a_3 = 0$	$a_2 = -3$	$a_1 = 3$	$a_0 = -4$
	-4	8	-10	14
$b_4 = 2$	$b_3 = -4$	$b_2 = 5$	$b_1 = -7$	$b_0 = 10$

Thus

$$p(x) = (x + 2)(2x^3 - 4x^2 + 5x - 7) + 10$$

and

$$p(-2) = 10. \quad \square$$

Horner's method can be used efficiently with Newton's method to find zeros of a polynomial $p(x)$. Differentiating

$$p(x) = (x - x_0)q(x) + b_0$$

we obtain

$$p'(x) = (x - x_0)q'(x) + q(x).$$

Hence

$$p'(x_0) = q(x_0).$$

Putting this in Newton's method we have

$$\begin{aligned} x_n &= x_{n-1} - \frac{p(x_{n-1})}{p'(x_{n-1})} \\ &= x_{n-1} - \frac{p(x_{n-1})}{q(x_{n-1})}. \end{aligned}$$

This procedure is shown in Example 1.16.

EXAMPLE 1.16. Compute the value of the polynomial

$$p(x) = 2x^4 = 3x^3 + 3x - 4$$

and of its derivative $p'(x)$ at $x_0 = -2$ by Horner's method and apply the results to Newton's method to find the first iterate x_1 .

SOLUTION. By successively multiplying the elements of the third line of the following tableau by $x_0 = -2$ and adding to the first line, one gets the value of $p(-2)$. Then by successively multiplying the elements of the fifth line of the tableau by $x_0 = -2$ and adding to the third line, one gets the value of $p'(-2)$.

2	0	-3	= 3	-4
	-4	8	-10	14
2	-4	5	-7	10 = $p(-2)$
	-4	16	-42	
2	-8	21	-49 = $p'(-2)$	

Thus

$$p(-2) = 10, \quad p'(-2) = -49,$$

and

$$x_1 = -2 - \frac{10}{-49} \approx -1.7959. \quad \square$$

1.8. Müller's Method

Müller's, or the parabola, method finds the real or complex roots of an equation

$$f(x) = 0.$$

This method uses three initial approximations, x_0 , x_1 , and x_2 , to construct a parabola,

$$p(x) = a(x - x_2)^2 + b(x - x_2) + c,$$

through the three points $(x_0, f(x_0))$, $(x_1, f(x_1))$, and $(x_2, f(x_2))$ on the curve $f(x)$ and determines the next approximation x_3 as the point of intersection of the parabola with the real axis closer to x_2 .

The coefficients a , b and c defining the parabola are obtained by solving the linear system

$$f(x_0) = a(x_0 - x_2)^2 + b(x_0 - x_2) + c,$$

$$f(x_1) = a(x_1 - x_2)^2 + b(x_1 - x_2) + c,$$

$$f(x_2) = c.$$

We immediately have

$$c = f(x_2)$$

and obtain a and b from the linear system

$$\begin{bmatrix} (x_0 - x_2)^2 & (x_0 - x_2) \\ (x_1 - x_2)^2 & (x_1 - x_2) \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} f(x_0) - f(x_2) \\ f(x_1) - f(x_2) \end{bmatrix}.$$

Then, we set

$$p(x_3) = a(x_3 - x_2)^2 + b(x_3 - x_2) + c = 0$$

and solve for $x_3 - x_2$:

$$\begin{aligned} x_3 - x_2 &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \\ &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \times \frac{-b \mp \sqrt{b^2 - 4ac}}{-b \mp \sqrt{b^2 - 4ac}} \\ &= \frac{-2c}{b \pm \sqrt{b^2 - 4ac}}. \end{aligned}$$

To find x_3 closer to x_2 , we maximize the denominator:

$$x_3 = x_2 - \frac{2c}{b + \text{sign}(b)\sqrt{b^2 - 4ac}}.$$

Müller's method converges approximately to order 1.839 to a simple or double root. It may not converge to a triple root.

EXAMPLE 1.17. Find the four zeros of the polynomial

$$16x^4 - 40x^3 + 5x^2 + 20x + 6,$$

whose graph is shown in Fig. 1.6, by means of Müller's method.

SOLUTION. The following Matlab commands do one iteration of Müller's method on the given polynomial which is transformed into its nested form:

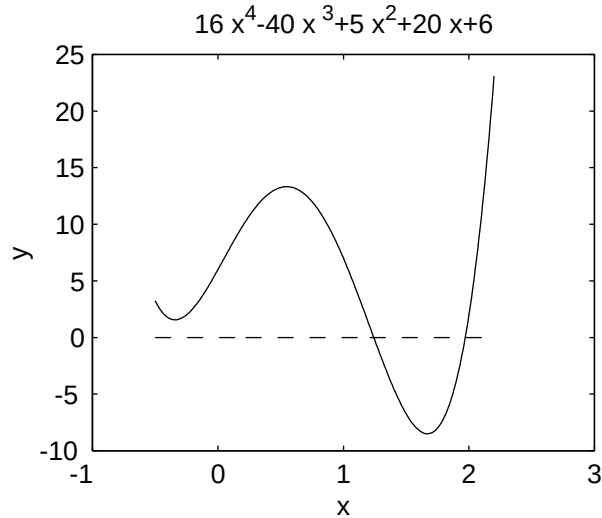


FIGURE 1.6. The graph of the polynomial $16x^4 - 40x^3 + 5x^2 + 20x + 6$ for Example 1.17.

```
syms x
pp = 16*x^4-40*x^3+5*x^2+20*x+6
pp = 16*x^4-40*x^3+5*x^2+20*x+6
pp = horner(pp)
pp = 6+(20+(5+(-40+16*x)*x)*x)*x
```

The polynomial is evaluated by the Matlab M function:

```
function pp = mullerpol(x);
pp = 6+(20+(5+(-40+16*x)*x)*x)*x;
```

Müller's method obtains x_3 with the given three starting values:

```
x0 = 0.5; x1 = -0.5; x2 = 0; % starting values
m = [(x0-x2)^2 x0-x2; (x1-x2)^2 x1-x2];
rhs = [mullerpol(x0)-mullerpol(x2); mullerpol(x1)-mullerpol(x2)];
ab = m\rhs; a = ab(1); b = ab(2); % coefficients a and b
c = mullerpol(x2); % coefficient c
x3 = x2 -(2*c)/(b+sign(b)*sqrt(b^2-4*a*c))
x3 = -0.5556 + 0.5984i
```

The method is iterated until convergence. The four roots of this polynomial are

```
rr = roots([16 -40 5 20 6])'
rr = 1.9704 1.2417 -0.3561 - 0.1628i -0.3561 + 0.1628i
```

The two real roots can be obtained by Müller's method with starting values $[0.5, 1.0, 1.5]$ and $[2.5, 2.0, 2.25]$, respectively. $\square$

CHAPTER 2

Interpolation and Extrapolation

Quite often, experimental results provide only a few values of an unknown function $f(x)$, say,

$$(x_0, f_0), \quad (x_1, f_1), \quad (x_2, f_2), \quad \dots, \quad (x_n, f_n), \quad (2.1)$$

where f_i is the observed value for $f(x_i)$. We would like to use these data to approximate $f(x)$ at an arbitrary point $x \neq x_i$.

When we want to estimate $f(x)$ for x between two of the x_i 's, we talk about **interpolation** of $f(x)$ at x . When x is not between two of the x_i 's, we talk about **extrapolation** of $f(x)$ at x .

The idea is to construct an interpolating polynomial, $p_n(x)$, of degree n whose graph passes through the $n + 1$ points listed in (2.1). This polynomial will be used to estimate $f(x)$.

2.1. Lagrange Interpolating Polynomial

The Lagrange interpolating polynomial, $p_n(x)$, of degree n through the $n + 1$ points $(x_k, f(x_k))$, $k = 0, 1, \dots, n$, is expressed in terms of the following Lagrange basis:

$$L_k(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0)(x_k - x_1) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}.$$

Clearly, $L_k(x)$ is a polynomial of degree n and

$$L_k(x) = \begin{cases} 1, & x = x_k, \\ 0, & x = x_j, \quad j \neq k. \end{cases}$$

Then the **Lagrange interpolating polynomial** of $f(x)$ is

$$p_n(x) = f(x_0)L_0(x) + f(x_1)L_1(x) + \cdots + f(x_n)L_n(x). \quad (2.2)$$

It is of degree n and interpolates $f(x)$ at the points listed in (2.1).

EXAMPLE 2.1. Interpolate $f(x) = 1/x$ at the nodes $x_0 = 2$, $x_1 = 2.5$ and $x_2 = 4$ with the Lagrange interpolating polynomial of degree 2.

SOLUTION. The Lagrange basis, in nested form, is

$$\begin{aligned} L_0(x) &= \frac{(x - 2.5)(x - 4)}{(2 - 2.5)(2 - 4)} = (x - 6.5)x + 10, \\ L_1(x) &= \frac{(x - 2)(x - 4)}{(2.5 - 2)(2.5 - 4)} = \frac{(-4x + 24)x - 32}{3}, \\ L_2(x) &= \frac{(x - 2)(x - 2.5)}{(4 - 2)(4 - 2.5)} = \frac{(x - 4.5)x + 5}{3}. \end{aligned}$$

Thus,

$$\begin{aligned} p(x) &= \frac{1}{2} [(x - 6.5)x + 10] + \frac{1}{2.5} \frac{(-4x + 24)x - 32}{3} + \frac{1}{4} \frac{(x - 4.5)x + 5}{3} \\ &= (0.05x - 0.425)x + 1.15. \quad \square \end{aligned}$$

THEOREM 2.1. *Suppose $x_0, x_1, \dots, x_n$ are $n+1$ distinct points in the interval $[a, b]$ and $f \in C^{n+1}[a, b]$. Then there exists a number $\xi(x) \in [a, b]$ such that*

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi(x))}{(n+1)!} (x - x_0)(x - x_1) \cdots (x - x_n), \quad (2.3)$$

where $p_n(x)$ is the Lagrange interpolating polynomial. In particular, if

$$m_{n+1} = \min_{a \leq x \leq b} |f^{(n+1)}(x)| \quad \text{and} \quad M_{n+1} = \max_{a \leq x \leq b} |f^{(n+1)}(x)|,$$

then the absolute error in $p_n(x)$ is bounded by the inequalities:

$$\begin{aligned} \frac{m_{n+1}}{(n+1)!} |(x - x_0)(x - x_1) \cdots (x - x_n)| &\leq |f(x) - p_n(x)| \\ &\leq \frac{M_{n+1}}{(n+1)!} |(x - x_0)(x - x_1) \cdots (x - x_n)| \end{aligned}$$

for $a \leq x \leq b$.

PROOF. First, note that the error is 0 at $x = x_0, x_1, \dots, x_n$ since

$$p_n(x_k) = f(x_k), \quad k = 0, 1, \dots, n,$$

from the interpolating property of $p_n(x)$. For $x \neq x_k$, define the auxiliary function

$$\begin{aligned} g(t) &= f(t) - p_n(t) - [f(x) - p_n(x)] \frac{(t - x_0)(t - x_1) \cdots (t - x_n)}{(x - x_0)(x - x_1) \cdots (x - x_n)} \\ &= f(t) - p_n(t) - [f(x) - p_n(x)] \prod_{i=0}^n \frac{t - x_i}{x - x_i}. \end{aligned}$$

For $t = x_k$,

$$g(x_k) = f(x_k) - p_n(x_k) - [f(x) - p_n(x)] \times 0 = 0$$

and for $t = x$,

$$g(x) = f(x) - p_n(x) - [f(x) - p_n(x)] \times 1 = 0.$$

Thus $g \in C^{n+1}[a, b]$ and it has $n+2$ zeros in $[a, b]$. By the generalized Rolle theorem, $g'(t)$ has $n+1$ zeros in $[a, b]$, $g''(t)$ has n zeros in $[a, b]$, $\dots$, $g^{(n+1)}(t)$ has 1 zero, $\xi \in [a, b]$,

$$\begin{aligned} g^{(n+1)}(\xi) &= f^{(n+1)}(\xi) - p_n^{(n+1)}(\xi) - [f(x) - p_n(x)] \frac{d^{n+1}}{dt^{n+1}} \left[\prod_{i=0}^n \frac{t - x_i}{x - x_i} \right]_{t=\xi} \\ &= f^{(n+1)}(\xi) - 0 - [f(x) - p_n(x)] \frac{(n+1)!}{\prod_{i=0}^n (x - x_i)} \\ &= 0 \end{aligned}$$

since $p_n(x)$ is a polynomial of degree n so that its $(n+1)$ st derivative is zero and only the top term, t^{n+1} , in the product $\prod_{i=0}^n (t - x_i)$ contributes to $(n+1)!$ in its $(n+1)$ st derivative. Hence

$$f(x) = p_n(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!} (x - x_0)(x - x_1) \cdots (x - x_n). \quad \square$$

From a computational point of view, (2.2) is not the best representation of $p_n(x)$ because it is computationally costly and has to be redone from scratch if we want to increase the degree of $p_n(x)$ to improve the interpolation.

If the points x_i are distinct, this polynomial is unique. For, suppose $p_n(x)$ and $q_n(x)$ of degree n both interpolate $f(x)$ at $n+1$ distinct points, then

$$p_n(x) - q_n(x)$$

is a polynomial of degree n which admits $n+1$ distinct zeros, hence it is identically zero.

2.2. Newton's Divided Difference Interpolating Polynomial

Newton's divided difference interpolating polynomials, $p_n(x)$, of degree n use a factorial basis in the form

$$p_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}).$$

The values of the coefficients a_k are determined by recurrence. We denote

$$f_k = f(x_k).$$

Let $x_0 \neq x_1$ and consider the two data points: (x_0, f_0) and (x_1, f_1) . Then the interpolating property of the polynomial

$$p_1(x) = a_0 + a_1(x - x_0)$$

implies that

$$p_1(x_0) = a_0 = f_0, \quad p_1(x_1) = f_0 + a_1(x_1 - x_0) = f_1.$$

Solving for a_1 we have

$$a_1 = \frac{f_1 - f_0}{x_1 - x_0}.$$

If we let

$$f[x_0, x_1] = \frac{f_1 - f_0}{x_1 - x_0}$$

be the **first divided difference**, then the divided difference interpolating polynomial of degree one is

$$p_1(x) = f_0 + (x - x_0) f[x_0, x_1].$$

EXAMPLE 2.2. Consider a function $f(x)$ which passes through the points (2.2, 6.2) and (2.5, 6.7). Find the divided difference interpolating polynomial of degree one for $f(x)$ and use it to interpolate f at $x = 2.35$.

SOLUTION. Since

$$f[2.2, 2.5] = \frac{6.7 - 6.2}{2.5 - 2.2} = 1.6667,$$

then

$$p_1(x) = 6.2 + (x - 2.2) \times 1.6667 = 2.5333 + 1.6667x.$$

In particular, $p_1(2.35) = 6.45$. $\square$

EXAMPLE 2.3. Approximate $\cos 0.2$ linearly using the values of $\cos 0$ and $\cos \pi/8$.

SOLUTION. We have the points

$$(0, \cos 0) = (0, 1) \quad \text{and} \quad \left(\frac{\pi}{8}, \cos \frac{\pi}{8}\right) = \left(\frac{\pi}{8}, \frac{1}{2}\sqrt{\sqrt{2}+2}\right)$$

(Substitute $\theta = \pi/8$ into the formula

$$\cos^2 \theta = \frac{1 + \cos(2\theta)}{2}$$

to get

$$\cos \frac{\pi}{8} = \frac{1}{2}\sqrt{\sqrt{2}+2}$$

since $\cos(\pi/4) = \sqrt{2}/2$.) Thus

$$f[0, \pi/8] = \frac{\left(\frac{\sqrt{\sqrt{2}+2}}{2} - 1\right)}{\pi/8 - 0} = \frac{4}{\pi} \left(\sqrt{\sqrt{2}+2} - 2\right).$$

This leads to

$$p_1(x) = 1 + \frac{4}{\pi} \left(\sqrt{\sqrt{2}+2} - 2\right) x.$$

In particular,

$$p_1(0.2) = 0.96125.$$

Note that $\cos 0.2 = 0.98007$ (rounded to five digits). The absolute error is 0.01882. $\square$

Consider the three data points

$$(x_0, f_0), \quad (x_1, f_1), \quad (x_2, f_2), \quad \text{where } x_i \neq x_j \text{ for } i \neq j.$$

Then the divided difference interpolating polynomial of degree two through these points is

$$p_2(x) = f_0 + (x - x_0) f[x_0, x_1] + (x - x_0)(x - x_1) f[x_0, x_1, x_2]$$

where

$$f[x_0, x_1] := \frac{f_1 - f_0}{x_1 - x_0} \quad \text{and} \quad f[x_0, x_1, x_2] := \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

are the **first** and **second** divided differences, respectively.

EXAMPLE 2.4. Interpolate a given function $f(x)$ through the three points

$$(2.2, 6.2), \quad (2.5, 6.7), \quad (2.7, 6.5),$$

by means the divided difference interpolating polynomial of degree two, $p_2(x)$, and interpolate $f(x)$ at $x = 2.35$ by means of $p_2(2.35)$.

SOLUTION. We have

$$f[2.2, 2.5] = 1.6667, \quad f[2.5, 2.7] = -1$$

and

$$f[2.2, 2.5, 2.7] = \frac{f[2.5, 2.7] - f[2.2, 2.5]}{2.7 - 2.2} = \frac{-1 - 1.6667}{2.7 - 2.2} = -5.3334.$$

Therefore,

$$p_2(x) = 6.2 + (x - 2.2) \times 1.6667 + (x - 2.2)(x - 2.5) \times (-5.3334).$$

In particular, $p_2(2.35) = 6.57$. $\square$

EXAMPLE 2.5. Construct the divided difference interpolating polynomial of degree two for $\cos x$ using the values $\cos 0$, $\cos \pi/8$ and $\cos \pi/4$, and approximate $\cos 0.2$.

SOLUTION. It was seen in Example 2.3 that

$$\cos \frac{\pi}{8} = \frac{1}{2} \sqrt{\sqrt{2} + 2}.$$

Hence, from the three data points

$$(0, 1), \quad (\pi/8, \cos \pi/8), \quad (\pi/4, \sqrt{2}/2),$$

we obtain the divided differences

$$f[0, \pi/8] = \frac{4}{\pi} \left(\sqrt{\sqrt{2} + 2} - 2 \right), \quad f[\pi/8, \pi/4] = \frac{4}{\pi} \left(\sqrt{2} - \sqrt{\sqrt{2} + 2} \right),$$

and

$$\begin{aligned} f[0, \pi/8, \pi/4] &= \frac{f[\pi/8, \pi/4] - f[0, \pi/8]}{\pi/4 - 0} \\ &= \frac{4}{\pi} \left[\frac{\sqrt{2}/2 - (\sqrt{\sqrt{2} + 2})/2}{\pi/4 - \pi/8} - \frac{4\sqrt{\sqrt{2} + 2} - 8}{\pi} \right] \\ &= \frac{16}{\pi^2} \left(\sqrt{2} - 2\sqrt{\sqrt{2} + 2} \right). \end{aligned}$$

Hence,

$$p_2(x) = 1 + x \frac{4}{\pi} \left(\sqrt{\sqrt{2} + 2} - 2 \right) + x \left(x - \frac{\pi}{8} \right) \frac{16}{\pi^2} \left(\sqrt{2} - 2\sqrt{\sqrt{2} + 2} \right).$$

Evaluating this polynomial at $x = 0.2$, we obtain

$$p_2(0.2) = 0.97881.$$

The absolute error is 0.00189. $\square$

In general, given $n + 1$ data points

$$(x_0, f_0), \quad (x_1, f_1), \quad \dots, \quad (x_n, f_n),$$

where $x_i \neq x_j$ for $i \neq j$, Newton's divided difference interpolating polynomial of degree n is

$$\begin{aligned} p_n(x) &= f_0 + (x - x_0) f[x_0, x_1] + (x - x_0)(x - x_1) f[x_0, x_1, x_2] + \dots \\ &\quad + (x - x_0)(x - x_1) \dots (x - x_{n-1}) f[x_0, x_1, \dots, x_n], \end{aligned} \quad (2.4)$$

where, by definition,

$$f[x_j, x_{j+1}, \dots, x_k] = \frac{f[x_{j+1}, \dots, x_k] - f[x_j, x_{j+1}, \dots, x_{k-1}]}{x_k - x_j}$$

is a $(k - j)$ **th divided difference**. This formula can be obtained by recurrence.

A divided difference table is shown in Table 2.1.

TABLE 2.1. Ddivided difference table

x	$f(x)$	First divided differences	Second divided differences	Third divided differences
x_0	$f[x_0]$			
		$f[x_0, x_1]$		
x_1	$f[x_1]$		$f[x_0, x_1, x_2]$	
		$f[x_1, x_2]$		$f[x_0, x_1, x_2, x_3]$
x_2	$f[x_2]$		$f[x_1, x_2, x_3]$	
		$f[x_2, x_3]$		$f[x_1, x_2, x_3, x_4]$
x_3	$f[x_3]$		$f[x_2, x_3, x_4]$	
		$f[x_3, x_4]$		$f[x_2, x_3, x_4, x_5]$
x_4	$f[x_4]$		$f[x_3, x_4, x_5]$	
		$f[x_4, x_5]$		
x_5	$f[x_5]$			

EXAMPLE 2.6. Construct the cubic interpolating polynomial through the four unequally spaced points

$$(1.0, 2.4), \quad (1.3, 2.2), \quad (1.5, 2.3), \quad (1.7, 2.4),$$

on the graph of a certain function $f(x)$ and approximate $f(1.4)$.

SOLUTION. Newton's divided difference table is

x_i	$f(x_i)$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$f[x_i, x_{i+1}, x_{i+2}, x_{i+3}]$
1.0	2.4			
		-0.66667		
1.3	2.2		2.33333	
		0.500000		-3.33333
1.5	2.3		0.00000	
		0.500000		
1.7	2.4			

Therefore,

$$p_3(x) = 2.4 + (x - 1.0)(-0.66667) + (x - 1.0)(x - 1.3) \times 2.33333 \\ + (x - 1.0)(x - 1.3)(x - 1.5)(-3.33333).$$

The approximation to $f(1.4)$ is

$$p_3(1.4) = 2.2400. \quad \square$$

2.3. Gregory–Newton Forward-Difference Polynomial

We rewrite (2.4) in the special case where the nodes x_i are equidistant,

$$x_i = x_0 + i h.$$

The **first** and **second forward differences** of $f(x)$ at x_j are

$$\Delta f_j := f_{j+1} - f_j, \quad \Delta^2 f_j := \Delta f_{j+1} - \Delta f_j,$$

respectively, and in general, the k th forward difference of $f(x)$ at x_j is

$$\Delta^k f_j := \Delta^{k-1} f_{j+1} - \Delta^{k-1} f_j.$$

It is seen by mathematical induction that

$$f[x_0, \dots, x_k] := \frac{1}{k! h^k} \Delta^k f_0.$$

If we set

$$r = \frac{x - x_0}{h},$$

then, for equidistant nodes,

$$x - x_k = x - x_0 - (x_k - x_0) = hr - hk = h(r - k)$$

and

$$(x - x_0)(x - x_1) \cdots (x - x_{k-1}) = h^k r(r-1)(r-2) \cdots (r-k+1).$$

Thus (2.4) becomes

$$\begin{aligned} p_n(r) &= f_0 + \sum_{k=1}^n \frac{r(r-1) \cdots (r-k+1)}{k!} \Delta^k f_0 \\ &= \sum_{k=0}^n \binom{r}{k} \Delta^k f_0, \end{aligned} \quad (2.5)$$

where

$$\binom{r}{k} = \begin{cases} \frac{r(r-1) \cdots (r-k+1)}{k!} & \text{if } k > 0, \\ 1 & \text{if } k = 0. \end{cases}$$

Polynomial (2.5) is the **Gregory-Newton forward-difference interpolating polynomial**.

EXAMPLE 2.7. Suppose that we are given the following equally spaced data:

x	1988	1989	1990	1991	1992	1993
y	35000	36000	36500	37000	37800	39000

Extrapolate the value of y in year 1994.

SOLUTION. The forward difference table is

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$
0	1988	35000					
			1000				
1	1989	36000		-500			
			500		500		
2	1990	36500		0		-200	
			500		300		0
3	1991	37000		300		-200	
			800		100		
4	1992	37800		400			
			1200				
5	1993	39000					

Setting $r = (x - 1988)/1$, we have

$$p_5(r) = 35000 + r(1000) + \frac{r(r-1)}{2}(-500) + \frac{r(r-1)(r-2)}{6}(500) \\ + \frac{r(r-1)(r-2)(r-3)}{24}(-200) + \frac{r(r-1)(r-2)(r-3)(r-4)}{120}(0).$$

Extrapolating the data at 1994 we have $r = 6$ and

$$p_5(6) = 40500.$$

An iterative use of the Matlab `diff(y,n)` command produces a difference table.

```
y = [35000 36000 36500 37000 37800 39000]
dy = diff(y);
    dy = 1000          500          500          800          1200
d2y = diff(y,2)
    d2y = -500         0         300         400
d3y = diff(y,3)
    d3y = 500         300         100
d4y = diff(y,4)
    d4y = -200        -200
d5y = diff(y,5)
    d5y = 0
```

□

EXAMPLE 2.8. Use the following equally spaced data to approximate $f(1.5)$.

x	1.0	1.3	1.6	1.9	2.2
$f(x)$	0.7651977	0.6200860	0.4554022	0.2818186	0.1103623

SOLUTION. The forward difference table is

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$
0	1.0	0.7651977				
			-0.145112			
1	1.3	0.6200860		-0.0195721		
			-0.164684		0.0106723	
2	1.6	0.4554022		-0.0088998		0.0003548
			-0.173584		0.0110271	
3	1.9	0.2818186		0.0021273		
			-0.170856			
4	2.2	0.1103623				

Setting $r = (x - 1.0)/0.3$, we have

$$p_4(r) = 0.7651977 + r(-0.145112) + \frac{r(r-1)}{2}(-0.0195721) \\ + \frac{r(r-1)(r-2)}{6}(0.0106723) + \frac{r(r-1)(r-2)(r-3)}{24}(0.0003548).$$

Interpolating $f(x)$ at $x = 1$, we have $r = 5/3$ and

$$p_4(5/3) = 0.511819. \quad \square$$

2.4. Gregory-Newton Backward-Difference Polynomial

To interpolate near the bottom of a difference table with equidistant nodes, one uses the Gregory-Newton backward-difference interpolating polynomial for the data

$$(x_{-n}, f_{-n}), \quad (x_{-n+1}, f_{-n+1}), \quad \dots, \quad (x_0, f_0).$$

If we set

$$r = \frac{x - x_0}{h},$$

then, for equidistant nodes,

$$x - x_{-k} = x - x_0 - (x_{-k} - x_0) = hr + hk = h(r + k)$$

and

$$(x - x_0)(x - x_{-1}) \cdots (x - x_{-(k-1)}) = h^k r(r+1)(r+2) \cdots (r+k-1).$$

Thus (2.5) becomes

$$\begin{aligned} p_n(r) &= f_0 + \sum_{k=1}^n \frac{r(r+1) \cdots (r+k-1)}{k!} \Delta^k f_{-k} \\ &= \sum_{k=0}^n \binom{r+k-1}{k} \Delta^k f_{-k}, \end{aligned} \quad (2.6)$$

The polynomial (2.6) is the **Gregory-Newton backward-difference interpolating polynomial**.

EXAMPLE 2.9. Interpolate the equally spaced data of Example 2.8 at $x = 2.1$

SOLUTION. The difference table is

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$
-4	1.0	0.7651977				
			-0.145112			
-3	1.3	0.6200860		-0.0195721		
			-0.164684		0.0106723	
-2	1.6	0.4554022		-0.0088998		0.0003548
			-0.173584		0.0110271	
-1	1.9	0.2818186		0.0021273		
			-0.170856			
0	2.2	0.1103623				

Setting $r = (x - 2.2)/0.3$, we have

$$\begin{aligned} p_4(r) &= 0.1103623 + r(-0.170856) + \frac{r(r+1)}{2}(0.0021273) \\ &\quad + \frac{r(r+1)(r+2)}{6}(0.0110271) + \frac{r(r+1)(r+2)(r+3)}{24}(0.0003548). \end{aligned}$$

Since

$$r = \frac{2.1 - 2.2}{0.3} = -\frac{1}{3},$$

then

$$p_4(-1/3) = 0.115904. \quad \square$$

2.5. Hermite Interpolating Polynomial

Given $n + 1$ distinct nodes $x_0, x_1, \dots, x_n$ and $2n + 2$ values $f_k = f(x_k)$ and $f'_k = f'(x_k)$, the Hermite interpolating polynomial $p_{2n+1}(x)$ of degree $2n + 1$,

$$p_{2n+1}(x) = \sum_{m=0}^n h_m(x) f_m + \sum_{m=0}^n \hat{h}_m(x) f'_m,$$

takes the values

$$p_{2n+1}(x_k) = f_k, \quad p'_{2n+1}(x_k) = f'_k, \quad k = 0, 1, \dots, n.$$

We look for polynomials $h_m(x)$ and $\hat{h}_m(x)$ of degree at most $2n + 1$ satisfying the following conditions:

$$\begin{aligned} h_m(x_k) &= h'_m(x_k) = 0, & k &\neq m, \\ h_m(x_m) &= 1, \\ h'_m(x_m) &= 0, \end{aligned}$$

and

$$\begin{aligned} \hat{h}_m(x_k) &= \hat{h}'_m(x_k) = 0, & k &\neq m, \\ \hat{h}_m(x_m) &= 0, \\ \hat{h}'_m(x_m) &= 1. \end{aligned}$$

These conditions are satisfied by the polynomials

$$h_m(x) = [1 - 2(x - x_m)L'_m(x_m)]L_m^2(x)$$

and

$$\hat{h}_m(x) = (x - x_m)L_m^2(x),$$

where

$$L_m(x) = \prod_{k=0, k \neq m}^n \frac{x - x_k}{x_m - x_k}$$

are the elements of the Lagrange basis of degree n .

A practical method of constructing a Hermite interpolating polynomial over the $n + 1$ distinct nodes $x_0, x_1, \dots, x_n$ is to set

$$z_{2i} = z_{2i+1} = x_i, \quad i = 0, 1, \dots, n,$$

and take

$$f'(x_0) \text{ for } f[z_0, z_1], \quad f'(x_1) \text{ for } f[z_2, z_3], \quad \dots, \quad f'(x_j) \text{ for } f[z_{2n}, z_{2n+1}]$$

in the divided difference table for the Hermite interpolating polynomial of degree $2n + 1$. Thus,

$$p_{2n+1}(x) = f[z_0] + \sum_{k=1}^{2n+1} f[z_0, z_1, \dots, z_k](x - z_0)(x - z_1) \cdots (x - z_{k-1}).$$

A divided difference table for a Hermite interpolating polynomial is as follows.

z	$f(z)$	First divided differences	Second divided differences	Third divided differences
$z_0 = x_0$	$f[z_0] = f(x_0)$			
		$f[z_0, z_1] = f'(x_0)$		
$z_1 = x_0$	$f[z_1] = f(x_0)$		$f[z_0, z_1, z_2]$	
		$f[z_1, z_2]$		$f[z_0, z_1, z_2, z_3]$
$z_2 = x_1$	$f[z_2] = f(x_1)$		$f[z_1, z_2, z_3]$	
		$f[z_2, z_3] = f'(x_1)$		$f[z_1, z_2, z_3, z_4]$
$z_3 = x_1$	$f[z_3] = f(x_1)$		$f[z_2, z_3, z_4]$	
		$f[z_3, z_4]$		$f[z_2, z_3, z_4, z_5]$
$z_4 = x_2$	$f[z_4] = f(x_2)$		$f[z_3, z_4, z_5]$	
		$f[z_4, z_5] = f'(x_2)$		
$z_5 = x_2$	$f[z_5] = f(x_2)$			

EXAMPLE 2.10. Interpolate the underlined data, given in the table below, at $x = 1.5$ by a Hermite interpolating polynomial of degree five.

SOLUTION. In the difference table the underlined entries are the given data. The remaining entries are generated by standard divided differences.

<u>1.3</u>	<u>0.6200860</u>				
		<u>-0.5220232</u>			
<u>1.3</u>	<u>0.6200860</u>	-0.0897427			
		-0.5489460	0.0663657		
<u>1.6</u>	<u>0.4554022</u>	-0.0698330	0.0026663		
		<u>-0.5698959</u>	0.0679655	-0.0027738	
<u>1.6</u>	<u>0.4554022</u>	-0.0290537	0.0010020		
		-0.5786120	0.0685667		
<u>1.9</u>	<u>0.2818186</u>	-0.0084837			
		<u>-0.5811571</u>			
<u>1.9</u>	<u>0.2818186</u>				

Taking the elements along the top downward diagonal, we have

$$\begin{aligned}
P(1.5) &= 0.6200860 + (1.5 - 1.3)(-0.5220232) + (1.5 - 1.3)^2(-0.0897427) \\
&\quad + (1.5 - 1.3)^2(1.5 - 1.6)(0.0663657) + (1.5 - 1.3)^2(1.5 - 1.6)^2(0.0026663) \\
&\quad + (1.5 - 1.3)^2(1.5 - 1.6)^2(1.5 - 1.9)(-0.0027738) \\
&= 0.5118277. \quad \square
\end{aligned}$$

2.6. Cubic Spline Interpolation

In this section, we interpolate functions by piecewise cubic polynomials which satisfy some global smoothness conditions. Piecewise polynomials avoid the oscillatory nature of high-degree polynomials over a large interval.

DEFINITION 2.1. Given a function $f(x)$ defined on the interval $[a, b]$ and a set of nodes

$$a = x_0 < x_1 < \cdots < x_n = b,$$

a cubic splines interpolant S for f is a piecewise cubic polynomial that satisfies the following conditions:

- (a) $S(x)$ is a cubic polynomial, denoted $S_j(x)$, on the subinterval $[x_j, x_{j+1}]$ for each $j = 0, 1, \dots, n-1$;
- (b) $S(x_j) = f(x_j)$ for each $j = 0, 1, \dots, n$;
- (c) $S_{j+1}(x_{j+1}) = S_j(x_{j+1})$ for each $j = 0, 1, \dots, n-2$;
- (d) $S'_{j+1}(x_{j+1}) = S'_j(x_{j+1})$ for each $j = 0, 1, \dots, n-2$;
- (e) $S''_{j+1}(x_{j+1}) = S''_j(x_{j+1})$ for each $j = 0, 1, \dots, n-2$;
- (f) One of the sets of boundary conditions is satisfied:
 - (i) $S'''(x_0) = S'''(x_n) = 0$ (free or natural boundary);
 - (ii) $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$ (clamped boundary).

Other boundary conditions can be used in the definition of splines. When free or clamped boundary conditions occur, the spline is called a **natural spline** or a **clamped spline**, respectively.

To construct the cubic spline interpolant for a given function f , the conditions in the definition are applied to the cubic polynomials

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3,$$

for each $j = 0, 1, \dots, n-1$.

The following existence and uniqueness theorems hold for natural and clamped spline interpolants, respectively.

THEOREM 2.2 (Natural Spline). *If f is defined at $a = x_0 < x_1 < \dots < x_n = b$, then f has a unique natural spline interpolant S on the nodes $x_0, x_1, \dots, x_n$ with boundary conditions $S''(a) = 0$ and $S''(b) = 0$.*

THEOREM 2.3 (Clamped Spline). *If f is defined at $a = x_0 < x_1 < \dots < x_n = b$ and is differentiable at a and b , then f has a unique clamped spline interpolant S on the nodes $x_0, x_1, \dots, x_n$ with boundary conditions $S'(a) = f'(a)$ and $S'(b) = f'(b)$.*

The following Matlab commands generate a sine curve and sample the spline over a finer mesh:

```
x = 0:10; y = sin(x);
xx = 0:0.25:10;
yy = spline(x,y,xx);
subplot(2,2,1); plot(x,y,'o',xx,yy);
```

The result is shown in Fig 2.1.

The following Matlab commands illustrate the use of clamped spline interpolation where the end slopes are prescribed. Zero slopes at the ends of an interpolant to the values of a certain distribution are enforced:

```
x = -4:4; y = [0 .15 1.12 2.36 2.36 1.46 .49 .06 0];
cs = spline(x,[0 y 0]);
xx = linspace(-4,4,101);
plot(x,y,'o',xx,ppval(cs,xx),'-');
```

The result is shown in Fig 2.2.

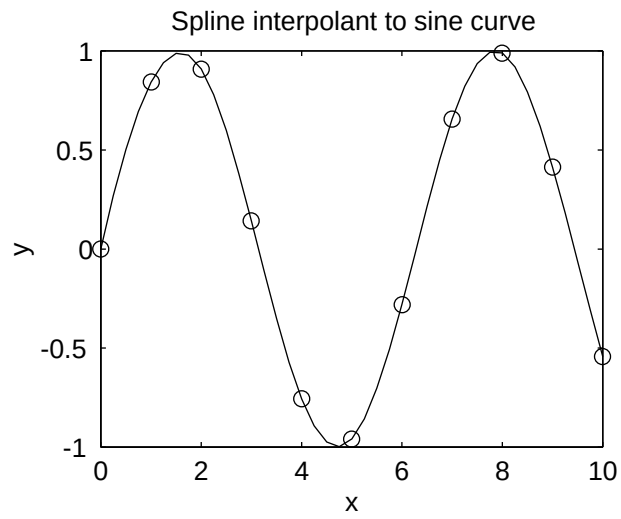


FIGURE 2.1. Spline interpolant of sine curve.

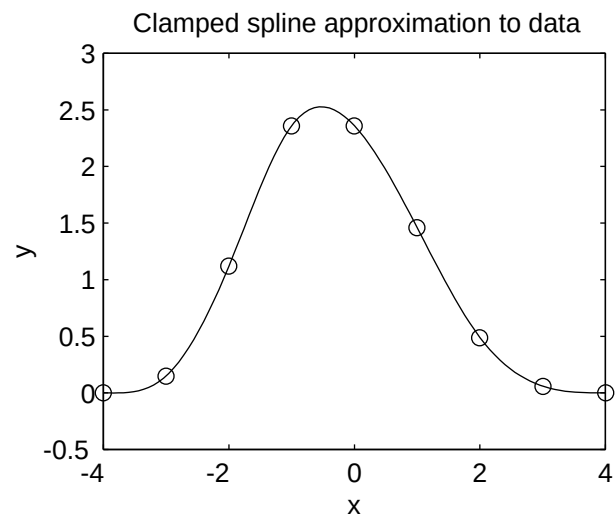


FIGURE 2.2. Clamped spline approximation to data.

Numerical Differentiation and Integration

3.1. Numerical Differentiation

3.1.1. Two-point formula for $f'(x)$. The Lagrange interpolating polynomial of degree 1 for $f(x)$ at x_0 and $x_1 = x_0 + h$ is

$$f(x) = f(x_0) \frac{x - x_1}{-h} + f(x_1) \frac{x - x_0}{h} + \frac{(x - x_0)(x - x_1)}{2!} f''(\xi(x)), \quad x_0 < \xi(x) < x_0 + h.$$

Differentiating this polynomial, we have

$$f'(x) = f(x_0) \frac{1}{-h} + f(x_1) \frac{1}{h} + \frac{(x - x_1) + (x - x_0)}{2!} f''(\xi(x)) + \frac{(x - x_0)(x - x_1)}{2!} \frac{d}{dx} [f''(\xi(x))].$$

Putting $x = x_0$ in $f'(x)$, we obtain the first-order two-point formula

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2} f''(\xi). \quad (3.1)$$

If $h > 0$, this is a forward difference formula and, if $h < 0$, this is a backward difference formula.

3.1.2. Three-point formula for $f'(x)$. The Lagrange interpolating polynomial of degree 2 for $f(x)$ at x_0 , $x_1 = x_0 + h$ and $x_2 = x_0 + 2h$ is

$$f(x) = f(x_0) \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} + f(x_1) \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} + f(x_2) \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} + \frac{(x - x_0)(x - x_1)(x - x_2)}{3!} f'''(\xi(x)),$$

where $x_0 < \xi(x) < x_2$. Differentiating this polynomial and substituting $x = x_j$, we have

$$f'(x_j) = f(x_0) \frac{2x_j - x_1 - x_2}{(x_0 - x_1)(x_0 - x_2)} + f(x_1) \frac{2x_j - x_0 - x_2}{(x_1 - x_0)(x_1 - x_2)} + f(x_2) \frac{2x_j - x_0 - x_1}{(x_2 - x_0)(x_2 - x_1)} + \frac{1}{6} f'''(\xi(x_j)) \prod_{k=0, k \neq j}^2 (x_j - x_k).$$

With $j = 0, 1, 2$, $f'(x_j)$ gives three second-order three-point formulae:

$$\begin{aligned} f'(x_0) &= f(x_0) \frac{-3h}{2h^2} + f(x_1) \frac{-2h}{-h^2} + f(x_2) \frac{-h}{2h^2} + \frac{2h^2}{6} f'''(\xi_0) \\ &= \frac{1}{h} \left[-\frac{3}{2} f(x_0) + 2f(x_1) - \frac{1}{2} f(x_2) \right] + \frac{h^2}{3} f'''(\xi_0), \end{aligned}$$

$$\begin{aligned} f'(x_1) &= f(x_0) \frac{-h}{2h^2} + f(x_1) \frac{0}{-h^2} + f(x_2) \frac{h}{2h^2} - \frac{h^2}{6} f'''(\xi_1) \\ &= \frac{1}{h} \left[-\frac{1}{2} f(x_0) + \frac{1}{2} f(x_2) \right] - \frac{h^2}{6} f'''(\xi_1), \end{aligned}$$

and, similarly,

$$f'(x_2) = \frac{1}{h} \left[\frac{1}{2} f(x_0) - 2f(x_1) + \frac{3}{2} f(x_2) \right] + \frac{h^2}{3} f'''(\xi_2).$$

These three-point formulae are usually written at x_0 :

$$f'(x_0) = \frac{1}{2h} [-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)] + \frac{h^2}{3} f'''(\xi_0), \quad (3.2)$$

$$f'(x_0) = \frac{1}{2h} [f(x_0 + h) - f(x_0 - h)] - \frac{h^2}{6} f'''(\xi_1). \quad (3.3)$$

The third formula is obtained from (3.2) by replacing h with $-h$. It is to be noted that the centred formula (3.3) is more precise than (3.2) since its error coefficient is half the error coefficient of the other formula.

3.1.3. Three-point centered difference formula for $f''(x)$. We use truncated Taylor's expansions for $f(x + h)$ and $f(x - h)$:

$$\begin{aligned} f(x_0 + h) &= f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + \frac{1}{6}f'''(x_0)h^3 + \frac{1}{24}f^{(4)}(\xi_0)h^4, \\ f(x_0 - h) &= f(x_0) - f'(x_0)h + \frac{1}{2}f''(x_0)h^2 - \frac{1}{6}f'''(x_0)h^3 + \frac{1}{24}f^{(4)}(\xi_1)h^4. \end{aligned}$$

Adding these expansions, we have

$$f(x_0 + h) + f(x_0 - h) = 2f(x_0) + f''(x_0)h^2 + \frac{1}{24} [f^{(4)}(\xi_0) + f^{(4)}(\xi_1)] h^4.$$

Solving for $f''(x_0)$, we have

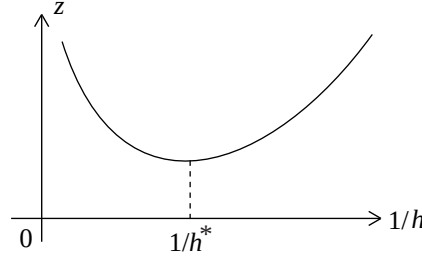
$$f''(x_0) = \frac{1}{h^2} [f(x_0 - h) - 2f(x_0) + f(x_0 + h)] - \frac{1}{24} [f^{(4)}(\xi_0) + f^{(4)}(\xi_1)] h^2.$$

By the Mean Value Theorem 1.5 for sums, there is a value ξ , $x_0 - h < \xi < x_0 + h$, such that

$$\frac{1}{2} [f^{(4)}(\xi_0) + f^{(4)}(\xi_1)] = f^{(4)}(\xi).$$

We thus obtain the three-point second-order centered difference formula

$$f''(x_0) = \frac{1}{h^2} [f(x_0 - h) - 2f(x_0) + f(x_0 + h)] - \frac{h^2}{12} f^{(4)}(\xi). \quad (3.4)$$

FIGURE 3.1. Truncation and roundoff error curve as a function of $1/h$.

3.2. The Effect of Roundoff and Truncation Errors

The presence of the stepsize h in the denominator of numerical differentiation formulae may produce large errors due to roundoff errors. We consider the case of the two-point centred formula (3.3) for $f'(x)$. Other cases are treated similarly.

Suppose that the roundoff error in the evaluated value $\tilde{f}(x_j)$ for $f(x_j)$ is $e(x_j)$. Thus,

$$f(x_0 + h) = \tilde{f}(x_0 + h) + e(x_0 + h), \quad f(x_0 - h) = \tilde{f}(x_0 - h) + e(x_0 - h).$$

Substituting these values in (3.3), we have the total error, which is the sum of the roundoff and the truncation errors,

$$f'(x_0) - \frac{\tilde{f}(x_0 + h) - \tilde{f}(x_0 - h)}{2h} = \frac{e(x_0 + h) - e(x_0 - h)}{2h} - \frac{h^2}{6} f^{(3)}(\xi).$$

Taking the absolute value of the right-hand side and applying the triangle inequality, we have

$$\left| \frac{e(x_0 + h) - e(x_0 - h)}{2h} - \frac{h^2}{6} f^{(3)}(\xi) \right| \leq \frac{1}{2h} (|e(x_0 + h)| + |e(x_0 - h)|) + \frac{h^2}{6} |f^{(3)}(\xi)|.$$

If

$$|e(x_0 \pm h)| \leq \varepsilon, \quad |f^{(3)}(x)| \leq M,$$

then

$$\left| f'(x_0) - \frac{\tilde{f}(x_0 + h) - \tilde{f}(x_0 - h)}{2h} \right| \leq \frac{\varepsilon}{h} + \frac{h^2}{6} M.$$

We remark that the expression

$$z(h) = \frac{\varepsilon}{h} + \frac{h^2}{6} M$$

first decreases and afterwards increases as $1/h$ increases, as shown in Fig. 3.1. The term $Mh^2/6$ is due to the truncation error and the term ε/h is due to roundoff errors.

EXAMPLE 3.1. (a) Given the function $f(x)$ and its first derivative $f'(x)$:

$$f(x) = \cos x, \quad f'(x) = -\sin x,$$

approximate $f'(0.7)$ with $h = 0.1$ by the five-point formula, without the truncation error term,

$$f'(x) = \frac{1}{12h} [-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)] + \frac{h^4}{30} f^{(5)}(\xi),$$

where ξ , in the truncation error, satisfies the inequalities $x - 2h \leq \xi \leq x + 2h$.

(b) Given that the roundoff error in each evaluation of $f(x)$ is bounded by $\epsilon = 5 \times 10^{-7}$, find a bound for the total error in $f'(0.7)$ by adding bounds for the roundoff and the truncation errors).

(c) Finally, find the value of h that minimizes the total error.

SOLUTION. (a) A simple computation with the given formula, without the truncation error, gives the approximation

$$f'(0.7) \approx -0.644\,215\,542.$$

(b) Since

$$f^{(5)}(x) = -\sin x$$

is negative and decreasing on the interval $0.5 \leq x \leq 0.9$, then

$$M = \max_{0.5 \leq x \leq 0.9} |-\sin x| = \sin 0.9 = 0.7833.$$

Hence, a bound for the total error is

$$\begin{aligned} \text{Total error} &\leq \frac{1}{12 \times 0.1} (1 + 8 + 8 + 1) \times 5 \times 10^{-7} + \frac{(0.1)^4}{30} \times 0.7833 \\ &= 7.5000 \times 10^{-6} + 2.6111 \times 10^{-6} \\ &= 1.0111 \times 10^{-5}. \end{aligned}$$

(c) The minimum of the total error, as a function of h ,

$$\frac{90 \times 10^{-7}}{12h} + \frac{0.7833}{30} h^4,$$

will be attained at a zero of its derivative with respect to h , that is,

$$\frac{d}{dh} \left(\frac{90 \times 10^{-7}}{12h} + \frac{0.7833}{30} h^4 \right) = 0.$$

Performing the derivative and multiplying both sides by h^2 , we obtain a quintic equation for h :

$$-7.5 \times 10^{-7} + \frac{4 \times 0.7833}{30} h^5 = 0.$$

Hence,

$$\begin{aligned} h &= \left(\frac{7.5 \times 10^{-7} \times 30}{4 \times 0.7833} \right)^{1/5} \\ &= 0.0936 \end{aligned}$$

minimizes the total error. □

3.3. Richardson's Extrapolation

Suppose it is known that a numerical formula, $N(h)$, approximates an exact value M with an error in the form of a series in h^j ,

$$M = N(h) + K_1 h + K_2 h^2 + K_3 h^3 + \dots,$$

where the constants K_j are independent of h . Then computing $N(h/2)$, we have

$$M = N\left(\frac{h}{2}\right) + \frac{1}{2} K_1 h + \frac{1}{4} K_2 h^2 + \frac{1}{8} K_3 h^3 + \dots$$

Subtracting the first expression from twice the second, we eliminate the error in h :

$$M = N\left(\frac{h}{2}\right) + \left[N\left(\frac{h}{2}\right) - N(h)\right] + \left(\frac{h^2}{2} - h^2\right) K_2 + \left(\frac{h^3}{4} - h^3\right) K_3 + \dots$$

If we put

$$\begin{aligned} N_1(h) &= N(h), \\ N_2(h) &= N_1\left(\frac{h}{2}\right) + \left[N_1\left(\frac{h}{2}\right) - N_1(h)\right], \end{aligned}$$

the last expression for M becomes

$$M = N_2(h) - \frac{1}{2} K_2 h^2 - \frac{3}{4} K_3 h^3 - \dots$$

Now with $h/4$, we have

$$M = N_2\left(\frac{h}{2}\right) - \frac{1}{8} K_2 h^2 - \frac{3}{32} K_3 h^3 + \dots$$

Subtracting the second last expression for M from 4 times the last one and dividing the result by 3, we eliminate the term in h^2 :

$$M = \left[N_2\left(\frac{h}{2}\right) + \frac{N_2(h/2) - N_2(h)}{3}\right] + \frac{1}{8} K_3 h^3 + \dots$$

Now, putting

$$N_3(h) = N_2\left(\frac{h}{2}\right) + \frac{N_2(h/2) - N_2(h)}{3},$$

we have

$$M = N_3(h) + \frac{1}{8} K_3 h^3 + \dots$$

The presence of the number $2^{j-1} - 1$ in the denominator of the second term of $N_j(h)$ ensures convergence. It is clear how to continue this process which is called **Richardson's extrapolation**.

An important case of Richardson's extrapolation is when $N(h)$ is the centred difference formula (3.3) for $f'(x)$, that is,

$$f'(x_0) = \frac{1}{2h} [f(x_0 + h) - f(x_0 - h)] - \frac{h^2}{6} f'''(x_0) - \frac{h^4}{120} f^{(5)}(x_0) - \dots$$

Since, in this case, the error term contains only even powers of h , the convergence of Richardson's extrapolation is very fast. Putting

$$N_1(h) = N(h) = \frac{1}{2h} [f(x_0 + h) - f(x_0 - h)],$$

TABLE 3.1. Richardson's extrapolation to the derivative of $x e^x$.

$N_1(0.2) = 22.414\,160$		
$N_1(0.1) = 22.228\,786$	$N_2(0.2) = 22.166\,995$	
$N_1(0.05) = 22.182\,564$	$N_2(0.1) = 22.167\,157$	$N_3(0.2) = 22.167\,168$

the above formula for $f'(x_0)$ becomes

$$f'(x_0) = N_1(h) - \frac{h^2}{6} f'''(x_0) - \frac{h^4}{120} f^{(5)}(x_0) - \dots$$

Replacing h with $h/2$ in this formula gives the approximation

$$f'(x_0) = N_1\left(\frac{h}{2}\right) - \frac{h^2}{24} f'''(x_0) - \frac{h^4}{1920} f^{(5)}(x_0) - \dots$$

Subtracting the second last formula for $f'(x_0)$ from 4 times the last one and dividing by 3, we have

$$f'(x_0) = N_2(h) - \frac{h^4}{480} f^{(5)}(x_0) + \dots,$$

where

$$N_2(h) = N_1\left(\frac{h}{2}\right) + \frac{N_1(h/2) - N_1(h)}{3}.$$

The presence of the number $4^{j-1} - 1$ in the denominator of the second term of $N_j(h)$ provides fast convergence.

EXAMPLE 3.2. Let

$$f(x) = x e^x.$$

Apply Richardson's extrapolation to the centred difference formula to compute $f'(x)$ at $x_0 = 2$ with $h = 0.2$.

SOLUTION. We have

$$N_1(0.2) = N(0.2) = \frac{1}{0.4} [f(2.2) - f(1.8)] = 22.414\,160,$$

$$N_1(0.1) = N(0.1) = \frac{1}{0.2} [f(2.1) - f(1.9)] = 22.228\,786,$$

$$N_1(0.05) = N(0.05) = \frac{1}{0.1} [f(2.05) - f(1.95)] = 22.182\,564.$$

Next,

$$N_2(0.2) = N_1(0.1) + \frac{N_1(0.1) - N_1(0.2)}{3} = 22.166\,995,$$

$$N_2(0.1) = N_1(0.05) + \frac{N_1(0.05) - N_1(0.1)}{3} = 22.167\,157.$$

Finally,

$$N_3(0.2) = N_2(0.1) + \frac{N_2(0.1) - N_2(0.2)}{15} = 22.167\,168,$$

which is correct to all 6 decimals. The results are listed in Table 3.1. One sees the fast convergence of Richardson's extrapolation for the centred difference formula. $\square$

3.4. Basic Numerical Integration Rules

To approximate the value of the definite integral

$$\int_a^b f(x) dx,$$

where the function $f(x)$ is smooth on $[a, b]$ and $a < b$, we subdivide the interval $[a, b]$ into n subintervals of equal length $h = (b - a)/n$. The function $f(x)$ is approximated on each of these subintervals by an interpolating polynomial and the polynomials are integrated.

For the **midpoint rule**, $f(x)$ is interpolated on each subinterval $[x_{i-1}, x_i]$ by $f([x_{i-1} + x_i]/2)$, and the integral of $f(x)$ over a subinterval is estimated by the area of a rectangle (see Fig. 3.2).

For the **trapezoidal rule**, $f(x)$ is interpolated on each subinterval $[x_{i-1}, x_i]$ by a polynomial of degree one, and the integral of $f(x)$ over a subinterval is estimated by the area of a trapezoid (see Fig. 3.3).

For **Simpson's rule**, $f(x)$ is interpolated on each pair of subintervals, $[x_{2i}, x_{2i+1}]$ and $[x_{2i+1}, x_{2i+2}]$, by a polynomial of degree two (parabola), and the integral of $f(x)$ over such pair of subintervals is estimated by the area under the parabola (see Fig. 3.4).

3.4.1. Midpoint rule. The midpoint rule,

$$\int_{x_0}^{x_1} f(x) dx = hf(x_1^*) + \frac{1}{24} f''(\xi)h^3, \quad x_0 < \xi < x_1, \quad (3.5)$$

approximates the integral of $f(x)$ on the interval $x_0 \leq x \leq x_1$ by the area of a rectangle with height $f(x_1^*)$ and base $h = x_1 - x_0$, where x_1^* is the midpoint of the interval $[x_0, x_1]$,

$$x_1^* = \frac{x_0 + x_1}{2},$$

(see Fig. 3.2).

To derive formula (3.5), we expand $f(x)$ in a truncated Taylor series with center at $x = x_1^*$,

$$f(x) = f(x_1^*) + f'(x_1^*)(x - x_1^*) + \frac{1}{2} f''(\xi)(x - x_1^*)^2, \quad x_0 < \xi < x_1.$$

Integrating this expression from x_0 to x_1 , we have

$$\begin{aligned} \int_{x_0}^{x_1} f(x) dx &= hf(x_1^*) + \int_{x_0}^{x_1} f'(x_1^*)(x - x_1^*) dx + \frac{1}{2} \int_{x_0}^{x_1} f''(\xi)(x - x_1^*)^2 dx \\ &= hf(x_1^*) + \frac{1}{2} f''(\xi) \int_{x_0}^{x_1} (x - x_1^*)^2 dx. \end{aligned}$$

where the integral over the linear term $(x - x_1^*)$ is zero because this term is an odd function with respect to the midpoint $x = x_1^*$ and the Mean Value Theorem 1.4 for integrals has been used in the integral of the quadratic term $(x - x_1^*)^2$ which does not change sign over the interval $[x_0, x_1]$. The result follows from the value of the integral

$$\frac{1}{2} \int_{x_0}^{x_1} (x - x_1^*)^2 dx = \frac{1}{6} [(x - x_1^*)^3]_{x_0}^{x_1} = \frac{1}{24} h^3.$$

3.4.2. Trapezoidal rule. The trapezoidal rule,

$$\int_{x_0}^{x_1} f(x) dx = \frac{h}{2} [f(x_0) + f(x_1)] - \frac{1}{12} f''(\xi) h^3, \quad x_0 < \xi < x_1, \quad (3.6)$$

approximates the integral of $f(x)$ on the interval $x_0 \leq x \leq x_1$ by the area of a trapezoid with heights $f(x_0)$ and $f(x_1)$ and base $h = x_1 - x_0$ (see Fig. 3.3).

To derive formula (3.6), we interpolate $f(x)$ at $x = x_0$ and $x = x_1$ by the linear Lagrange polynomial

$$p_1(x) = f(x_0) \frac{x - x_1}{x_0 - x_1} + f(x_1) \frac{x - x_0}{x_1 - x_0}.$$

Thus,

$$f(x) = p_1(x) + \frac{f''(\xi)}{2} (x - x_0)(x - x_1), \quad x_0 < \xi < x_1.$$

Since

$$\int_{x_0}^{x_1} p_1(x) dx = \frac{h}{2} [f(x_0) + f(x_1)],$$

we have

$$\begin{aligned} \int_{x_0}^{x_1} f(x) dx - \frac{h}{2} [f(x_0) + f(x_1)] &= \int_{x_0}^{x_1} [f(x) - p_1(x)] dx \\ &= \int_{x_0}^{x_1} \frac{f''(\xi(x))}{2} (x - x_0)(x - x_1) dx \\ &= \frac{f''(\xi)}{2} \int_{x_0}^{x_1} (x - x_0)(x - x_1) dx \\ &= \frac{f''(\xi)}{2} \left[\frac{x^3}{3} - \frac{x_0 + x_1}{2} x^2 + x_0 x_1 x \right]_{x_0}^{x_1} \\ &= -\frac{f''(\xi)}{12} h^3, \end{aligned}$$

where the Mean Value Theorem 1.4 for integrals has been used to obtain the third equality since the term $(x - x_0)(x - x_1)$ does not change sign over the interval $[x_0, x_1]$. The last equality follows by some algebraic manipulation.

3.4.3. Simpson's rule. Simpson's rule

$$\int_{x_0}^{x_2} f(x) dx = \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)] - \frac{h^5}{90} f^{(4)}(\xi), \quad x_0 < \xi < x_2, \quad (3.7)$$

approximates the integral of $f(x)$ on the interval $x_0 \leq x \leq x_2$ by the area under a parabola which interpolates $f(x)$ at $x = x_0$, x_1 and x_2 (see Fig. 3.4).

To derive formula (3.7), we expand $f(x)$ in a truncated Taylor series with center at $x = x_1$,

$$f(x) = f(x_1) + f'(x_1)(x - x_1) + \frac{f''(x_1)}{2} (x - x_1)^2 + \frac{f'''(x_1)}{6} (x - x_1)^3 + \frac{f^{(4)}(\xi(x))}{24} (x - x_1)^4.$$

Integrating this expression from x_0 to x_2 and noticing that the odd terms $(x - x_1)$ and $(x - x_1)^3$ are odd functions with respect to the point $x = x_1$ so that their

integrals vanish, we have

$$\begin{aligned}\int_{x_0}^{x_2} f(x) dx &= \left[f(x)x + \frac{f''(x_1)}{6}(x-x_1)^3 + \frac{f^{(4)}(\xi_1)}{120}(x-x_1)^5 \right]_{x_0}^{x_2} \\ &= 2hf(x_1) + \frac{h^3}{3}f''(x_1) + \frac{f^{(4)}(\xi_1)}{60}h^5,\end{aligned}$$

where the Mean Value Theorem 1.4 for integrals was used in the integral of the error term because the factor $(x-x_1)^4$ does not change sign over the interval $[x_0, x_2]$.

Substituting the three-point centered difference formula (3.4) for $f''(x_1)$ in terms of $f(x_0)$, $f(x_1)$ and $f(x_2)$:

$$f''(x_1) = \frac{1}{h^2} [f(x_0) - 2f(x_1) + f(x_2)] - \frac{1}{12} f^{(4)}(\xi_2)h^2,$$

we obtain

$$\int_{x_0}^{x_2} f(x) dx = \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)] - \frac{h^5}{12} \left[\frac{1}{3} f^{(4)}(\xi_2) - \frac{1}{5} f^{(4)}(\xi_2) \right].$$

In this case, we cannot apply the Mean Value Theorem 1.5 for sums to express the error term in the form of $f^{(4)}(\xi)$ evaluated at one point since the weights $1/3$ and $-1/5$ have different signs. However, since the formula is exact for polynomials of degree less than or equal to 4, to obtain the factor $1/90$ it suffices to apply the formula to the monomial $f(x) = x^4$ and, for simplicity, integrate from $-h$ to h :

$$\begin{aligned}\int_{-h}^h x^4 dx &= \frac{h}{3} [(-h)^4 + 4(0)^4 + h^4] + kf^{(4)}(\xi) \\ &= \frac{2}{3}h^5 + 4!k = \frac{2}{5}h^5,\end{aligned}$$

where the last term is the exact value of the integral. It follows that

$$k = \frac{1}{4!} \left[\frac{2}{5} - \frac{2}{3} \right] h^5 = -\frac{1}{90}h^5,$$

which yields (3.7).

3.5. The Composite Midpoint Rule

We subdivide the interval $[a, b]$ into n subintervals of equal length $h = (b-a)/n$ with end-points

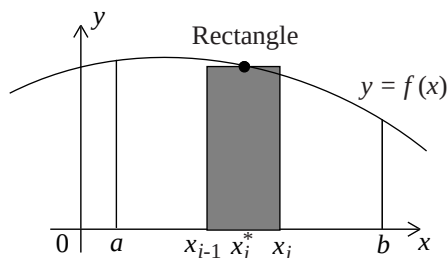
$$x_0 = a, \quad x_1 = a + h, \quad \dots, \quad x_i = a + ih, \quad \dots, \quad x_n = b.$$

On the subinterval $[x_{i-1}, x_i]$, the integral of $f(x)$ is approximated by the signed area of the rectangle with base $[x_{i-1}, x_i]$ and height $f(x_i^*)$, where

$$x_i^* = \frac{1}{2}(x_{i-1} + x_i)$$

is the mid-point of the segment $[x_{i-1}, x_i]$, as shown in Fig. 3.2. Thus, by the basic midpoint rule (3.5),

$$\int_{x_{i-1}}^{x_i} f(x) dx = hf(x_i^*) + \frac{1}{24} f''(\xi_i)h^3, \quad x_{i-1} < \xi_i < x_i,$$

FIGURE 3.2. The i th panel of the midpoint rule.

Summing over all the subintervals, we have

$$\int_a^b f(x) dx = h \sum_{i=1}^n f(x_i^*) + \frac{h^3}{24} \sum_{i=1}^n f''(\xi_i).$$

Multiplying and dividing the error term by n , applying the Mean Value Theorem 1.5 for sums to this term and using the fact that $nh = b - a$, we have

$$\frac{nh^3}{12} \sum_{i=1}^n \frac{1}{n} f''(\xi_i) = \frac{(b-a)h^2}{12} f''(\xi) h^2, \quad a < \xi < b.$$

Thus, we obtain the **composite midpoint rule**:

$$\begin{aligned} \int_a^b f(x) dx &= h[f(x_1^*) + f(x_2^*) + \cdots + f(x_n^*)] \\ &\quad + \frac{(b-a)h^2}{24} f''(\xi), \quad a < \xi < b. \end{aligned} \quad (3.8)$$

We see that the composite midpoint rule is a method of order $O(h^2)$, which is exact for polynomials of degree smaller than or equal to 1.

EXAMPLE 3.3. Use the composite midpoint rule to approximate the integral

$$I = \int_0^1 e^{x^2} dx$$

with step size h such that the absolute truncation error is bounded by 10^{-4} .

SOLUTION. Since

$$f(x) = e^{x^2} \quad \text{and} \quad f''(x) = (2 + 4x^2)e^{x^2},$$

then

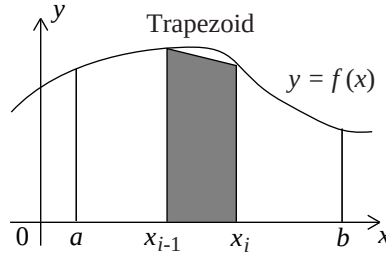
$$0 \leq f''(x) \leq 6e \quad \text{for } x \in [0, 1].$$

Therefore, a bound for the absolute truncation error is

$$|\epsilon_M| \leq \frac{1}{24} 6e(1-0)h^2 = \frac{1}{4} eh^2 < 10^{-4}.$$

Thus

$$h < 0.0121 \quad \frac{1}{h} = 82.4361.$$

FIGURE 3.3. The i th panel of the trapezoidal rule.

We take $n = 83 \geq 1/h = 82.4361$ and $h = 1/83$. The approximate value of I is

$$I \approx \frac{1}{83} \left[e^{(0.5/83)^2} + e^{(1.5/83)^2} + \dots + e^{(13590.5/83)^2} + e^{(82.5/83)^2} \right] \\ \approx 1.46262 \quad \square$$

The following Matlab commands produce the midpoint integration.

```
x = 0.5:82.5; y = exp((x/83).^2);
z = 1/83*sum(y)
z = 1.4626
```

□

3.6. The Composite Trapezoidal Rule

We divide the interval $[a, b]$ into n subintervals of equal length $h = (b - a)/n$, with end-points

$$x_0 = a, \quad x_1 = a + h, \quad \dots, \quad x_i = a + ih, \quad \dots, \quad x_n = b.$$

On each subinterval $[x_{i-1}, x_i]$, the integral of $f(x)$ is approximated by the signed area of the trapezoid with vertices

$$(x_{i-1}, 0), \quad (x_i, 0), \quad (x_i, f(x_i)), \quad (x_{i-1}, f(x_{i-1})),$$

as shown in Fig. 3.3. Thus, by the basic trapezoidal rule (3.6),

$$\int_{x_{i-1}}^{x_i} f(x) dx = \frac{h}{2} [f(x_{i-1}) + f(x_i)] - \frac{h^3}{12} f''(\xi_i).$$

Summing over all the subintervals, we have

$$\int_a^b f(x) dx = \frac{h}{2} \sum_{i=1}^n [f(x_{i-1}) + f(x_i)] - \frac{h^3}{12} \sum_{i=1}^n f''(\xi_i).$$

Multiplying and dividing the error term by n , applying the Mean Value Theorem 1.5 for sums to this term and using the fact that $nh = b - a$, we have

$$-\frac{nh^3}{12} \sum_{i=1}^n \frac{1}{n} f''(\xi_i) = -\frac{(b-a)h^2}{12} f''(\xi), \quad a < \xi < b.$$

Thus, we obtain the **composite trapezoidal rule**:

$$\int_a^b f(x) dx = \frac{h}{2} [f(x_0) + 2f(x_1) + 2f(x_2) + \cdots + 2f(x_{n-2}) + 2f(x_{n-1}) + f(x_n)] - \frac{(b-a)h^2}{12} f''(\xi), \quad a < \xi < b. \quad (3.9)$$

We see that the composite trapezoidal rule is a method of order $O(h^2)$, which is exact for polynomials of degree smaller than or equal to 1. Its absolute truncation error is twice the absolute truncation error of the midpoint rule.

EXAMPLE 3.4. Use the composite trapezoidal rule to approximate the integral

$$I = \int_0^1 e^{x^2} dx$$

with step size h such that the absolute truncation error is bounded by 10^{-4} . Compare with Examples 3.3 and 3.6.

SOLUTION. Since

$$f(x) = e^{x^2} \quad \text{and} \quad f''(x) = (2 + 4x^2) e^{x^2},$$

then

$$0 \leq f''(x) \leq 6e \quad \text{for} \quad x \in [0, 1].$$

Therefore,

$$|\epsilon_T| \leq \frac{1}{12} 6e(1-0)h^2 = \frac{1}{2} eh^2 < 10^{-4}, \quad \text{that is,} \quad h < 0.008577638.$$

We take $n = 117 \geq 1/h = 116.6$ (compared to 83 for the composite midpoint rule). The approximate value of I is

$$\begin{aligned} I &\approx \frac{1}{117 \times 2} \left[e^{(0/117)^2} + 2e^{(1/117)^2} + 2e^{(2/117)^2} + \cdots \right. \\ &\quad \left. + 2e^{(115/117)^2} + 2e^{(116/117)^2} + e^{(117/117)^2} \right] \\ &= 1.46268. \end{aligned}$$

The following Matlab commands produce the trapezoidal integration of numerical values y_k at nodes $k/117$, $k = 0, 1, \dots, 117$, with stepsize $h = 1/117$.

```
x = 0:117; y = exp((x/117).^2);
z = trapz(x,y)/117
z = 1.4627
```

□

EXAMPLE 3.5. How many subintervals are necessary for the composite trapezoidal rule to approximate the integral

$$I = \int_1^2 \left[x^2 - \frac{1}{12}(x-1.5)^4 \right] dx$$

with step size h such that the absolute truncation error is bounded by 10^{-3} .

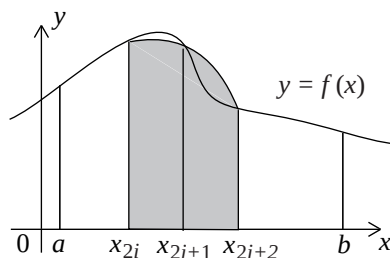


FIGURE 3.4. A double panel of Simpson's rule.

SOLUTION. Denote the integrand by

$$f(x) = x^2 - \frac{1}{12}(x - 1.5)^4.$$

Then

$$f''(x) = 2 - (x - 1.5)^2.$$

It is clear that

$$M = \max_{1 \leq x \leq 2} f''(x) = f(1.5) = 2.$$

To bound the absolute truncation error by 10^{-3} , we need

$$\begin{aligned} \left| \frac{(b-a)h^2}{12} f''(\xi) \right| &\leq \frac{h^2}{12} M \\ &= \frac{h^2}{6} \\ &\leq 10^{-3}. \end{aligned}$$

This gives

$$h \leq \sqrt{6 \times 10^{-3}} = 0.0775$$

and

$$\frac{1}{h} = 12.9099 \leq n = 13.$$

Thus it suffices to take

$$h = \frac{1}{13}, \quad n = 13. \quad \square$$

3.7. The Composite Simpson's Rule

We subdivide the interval $[a, b]$ into an even number, $n = 2m$, of subintervals of equal length, $h = (b - a)/(2m)$, with end-points

$$x_0 = a, \quad x_1 = a + h, \quad \dots, \quad x_i = a + i h, \quad \dots, \quad x_{2m} = b.$$

On the subinterval $[x_{2i}, x_{2i+2}]$, the function $f(x)$ is interpolated by the quadratic polynomial $p_2(x)$ which passes through the points

$$(x_{2i}, f(x_{2i})), \quad (x_{2i+1}, f(x_{2i+1})), \quad (x_{2i+2}, f(x_{2i+2})),$$

as shown in Fig. 3.4.

Thus, by the basic Simpson's rule (3.7),

$$\int_{x_{2i}}^{x_{2i+2}} f(x) dx = \frac{h}{3} [f(x_{2i}) + 4f(x_{2i+1}) + f(x_{2i+2})] - \frac{h^5}{90} f^{(4)}(\xi_i), \quad x_{2i} < \xi < x_{2i+2}.$$

Summing over all the subintervals, we have

$$\int_a^b f(x) dx = \frac{h}{3} \sum_{i=1}^m [f(x_{2i}) + 4f(x_{2i+1}) + f(x_{2i+2})] - \frac{h^5}{90} \sum_{i=1}^m f^{(4)}(\xi_i).$$

Multiplying and dividing the error term by m , applying the Mean Value Theorem 1.5 for sums to this term and using the fact that $2mh = nh = b - a$, we have

$$-\frac{2mh^5}{2 \times 90} \sum_{i=1}^m \frac{1}{m} f^{(4)}(\xi_i) = -\frac{(b-a)h^4}{180} f^{(4)}(\xi), \quad a < \xi < b.$$

Thus, we obtain the **composite Simpson's rule**:

$$\begin{aligned} \int_a^b f(x) dx &= \frac{h}{3} [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots \\ &+ 2f(x_{2m-2}) + 4f(x_{2m-1}) + f(x_{2m})] - \frac{(b-a)h^4}{180} f^{(4)}(\xi), \quad a < \xi < b. \end{aligned} \quad (3.10)$$

We see that the composite Simpson's rule is a method of order $O(h^4)$, which is exact for polynomials of degree smaller than or equal to 3.

EXAMPLE 3.6. Use the composite Simpson's rule to approximate the integral

$$I = \int_0^1 e^{x^2} dx$$

with stepsize h such that the absolute truncation error is bounded by 10^{-4} . Compare with Examples 3.3 and 3.4.

SOLUTION. We have

$$f(x) = e^{x^2} \quad \text{and} \quad f^{(4)}(x) = 4e^{x^2} (3 + 12x^2 + 4x^4).$$

Thus

$$0 \leq f^{(4)}(x) \leq 76e \quad \text{on} \quad [0, 1].$$

The absolute truncation error is thus less than or equal to $\frac{76}{180} e(1-0)h^4$. Hence, h must satisfy the inequality

$$\frac{76}{180} eh^4 < 10^{-4}, \quad \text{that is,} \quad h < 0.096614232.$$

To satisfy the inequality

$$2m \geq \frac{1}{h} = 10.4$$

we take

$$n = 2m = 12 \quad \text{and} \quad h = \frac{1}{12}.$$

The approximation is

$$\begin{aligned} I &\approx \frac{1}{12 \times 3} \left[e^{(0/12)^2} + 4e^{(1/12)^2} + 2e^{(2/12)^2} + \cdots + 2e^{(10/12)^2} + 4e^{(11/12)^2} + e^{(12/12)^2} \right] \\ &= 1.46267. \end{aligned}$$

We obtain a value which is similar to those found in Examples 3.3 and 3.4. However, the number of arithmetic operations is much less when using Simpson's rule (hence cost and truncation errors are reduced). In general, Simpson's rule is preferred to the midpoint and trapezoidal rules. $\square$

EXAMPLE 3.7. Use the composite Simpson's rule to approximate the integral

$$I = \int_0^2 \sqrt{1 + \cos^2 x} dx$$

within an accuracy of 0.0001.

SOLUTION. We must determine the step size h such that the absolute truncation error, $|\epsilon_S|$, will be bounded by 0.0001. For

$$f(x) = \sqrt{1 + \cos^2 x},$$

we have

$$\begin{aligned} f^{(4)}(x) = & \frac{-3 \cos^4(x)}{(1 + \cos^2(x))^{3/2}} + \frac{4 \cos^2(x)}{\sqrt{1 + \cos^2(x)}} - \frac{18 \cos^4(x) \sin^2(x)}{(1 + \cos^2(x))^{5/2}} \\ & + \frac{22 \cos^2(x) \sin^2(x)}{(1 + \cos^2(x))^{3/2}} - \frac{4 \sin^2(x)}{\sqrt{1 + \cos^2(x)}} - \frac{15 \cos^4(x) \sin^4(x)}{(1 + \cos^2(x))^{7/2}} \\ & + \frac{18 \cos^2(x) \sin^4(x)}{(1 + \cos^2(x))^{5/2}} - \frac{3 \sin^4(x)}{(1 + \cos^2(x))^{3/2}}. \end{aligned}$$

Since every denominator is greater than one, we have

$$|f^{(4)}(x)| \leq 3 + 4 + 18 + 22 + 4 + 15 + 18 + 3 = 87.$$

Therefore, we need

$$|\epsilon_S| < \frac{87}{180} (2 - 0)h^4.$$

Hence,

$$h < 0.100851140, \quad \frac{1}{h} > 9.915604269.$$

To have $2m \geq 1/h = 9.9$ we take $n = 2m = 10$ and $h = 1/10$. The approximation is

$$\begin{aligned} I &\approx \frac{1}{20 \times 3} \left[\sqrt{1 + \cos^2(0/20)} + 4 \sqrt{1 + \cos^2(1/20)} + 2 \sqrt{1 + \cos^2(2/20)} + \dots \right. \\ &\quad \left. + 2 \sqrt{1 + \cos^2(18/20)} + 4 \sqrt{1 + \cos^2(19/20)} + \sqrt{1 + \cos^2(20/20)} \right] \\ &= 2.48332. \quad \square \end{aligned}$$

3.8. Romberg Integration for the Trapezoidal Rule

Romberg integration uses Richardson's extrapolation to improve the trapezoidal rule approximation, $R_{k,1}$, with step size h_k , to an integral

$$I = \int_a^b f(x) dx.$$

It can be shown that

$$I = R_{k,1} + K_1 h_k^2 + K_2 h_k^4 + K_3 h_k^6 + \dots,$$

where the constants K_j are independent of h_k . With step sizes

$$h_1 = h, \quad h_2 = \frac{h}{2}, \quad h_3 = \frac{h}{2^2}, \quad \dots, \quad h_k = \frac{h}{2^{k-1}}, \quad \dots,$$

one can cancel errors of order h^2 , h^4 , etc. as follows. Suppose $R_{k,1}$ and $R_{k+1,1}$ have been computed, then we have

$$I = R_{k,1} + K_1 h_k^2 + K_2 h_k^4 + K_3 h_k^6 + \dots$$

and

$$I = R_{k+1,1} + K_1 \frac{h_k^2}{4} + K_2 \frac{h_k^2}{16} + K_3 \frac{h_k^2}{64} + \dots$$

Subtracting the first expression for I from 4 times the second expression and dividing by 3, we obtain

$$I = \left[R_{k+1,1} + \frac{R_{k+1,1} - R_{k,1}}{3} \right] + \frac{K_2}{3} \left[\frac{1}{4} - 1 \right] h_k^4 + \frac{K_3}{3} \left[\frac{1}{16} - 1 \right] h_k^4 + \dots$$

Put

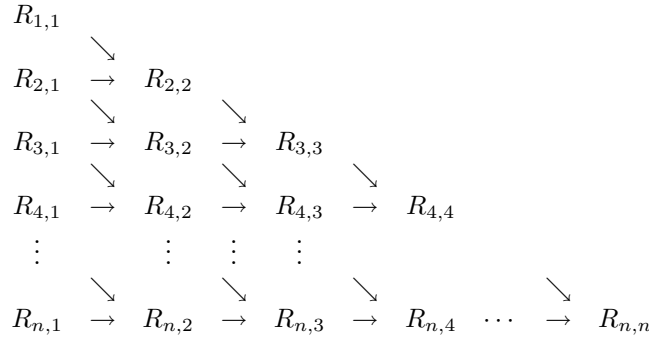
$$R_{k,2} = R_{k,1} + \frac{R_{k,1} - R_{k-1,1}}{3}$$

and, in general,

$$R_{k,j} = R_{k,j-1} + \frac{R_{k,j-1} - R_{k-1,j-1}}{4^{j-1} - 1}.$$

Then $R_{k,j}$ is a better approximation to I than $R_{k,j-1}$ and $R_{k-1,j-1}$. The relations between the $R_{k,j}$ are shown in Table 3.2.

TABLE 3.2. Romberg integration table with n levels



EXAMPLE 3.8. Use 6 levels of Romberg integration, with $h_1 = h = \pi/4$, to approximate the integral

$$I = \int_0^{\pi/4} \tan x \, dx.$$

SOLUTION. The following results are obtained by a simple Matlab program.

Romberg integration table:

```

0.39269908
0.35901083  0.34778141
0.34975833  0.34667417  0.34660035
0.34737499  0.34658054  0.34657430  0.34657388
0.34677428  0.34657404  0.34657360  0.34657359  0.34657359
0.34662378  0.34657362  0.34657359  0.34657359  0.34657359  0.34657359

```

□

3.9. Adaptive Quadrature Methods

Uniformly spaced composite rules that are exact for degree d polynomials are efficient if the $(d+1)$ st derivative $f^{(d+1)}$ is uniformly behaved across the interval of integration $[a, b]$. However, if the magnitude of this derivative varies widely across this interval, the error control process may result in an unnecessary number of function evaluations. This is because the number n of nodes is determined by an interval-wide derivative bound M_{d+1} . In regions where $f^{(d+1)}$ is small compared to this value, the subintervals are (possibly) much shorter than necessary. *Adaptive quadrature* methods addresses this problem by discovering where the integrand is ill behaved and shortening the subintervals accordingly.

We take Simpson's rule as a typical example:

$$I =: \int_a^b f(x) dx = S(a, b) - \frac{h^5}{90} f^{(4)}(\xi), \quad 0 < \xi < b,$$

where

$$S(a, b) = \frac{h}{3} [f(a) + 4f(a+h) + f(b)], \quad h = \frac{b-a}{2}.$$

The aim of adaptive quadrature is to take h large over regions where $|f^{(4)}(x)|$ is small and take h small over regions where $|f^{(4)}(x)|$ is large to have a uniformly small error. A simple way to estimate the error is to use h and $h/2$ as follows:

$$I = S(a, b) - \frac{h^5}{90} f^{(4)}(\xi_1), \quad (3.11)$$

$$I = S\left(a, \frac{a+b}{2}\right) + S\left(\frac{a+b}{2}, b\right) - \frac{2}{32} \frac{h^5}{90} f^{(4)}(\xi_2). \quad (3.12)$$

Assuming that

$$f^{(4)}(\xi_2) \approx f^{(4)}(\xi_1)$$

and subtracting the second expression for I from the first we have an expression for the error term:

$$\frac{h^5}{90} f^{(4)}(\xi_1) \approx \frac{16}{15} \left[S(a, b) - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right) \right].$$

Putting this expression in (3.12), we obtain an estimate for the absolute error:

$$\left| I - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right) \right| \approx \frac{1}{15} \left| S(a, b) - S\left(a, \frac{a+b}{2}\right) - S\left(\frac{a+b}{2}, b\right) \right|.$$

If the right-hand side of this estimate is smaller than a given tolerance, then

$$S\left(a, \frac{a+b}{2}\right) + S\left(\frac{a+b}{2}, b\right)$$

is taken as a good approximation to the value of I .

The adaptive quadrature for Simpson's rule is often better than the composite Simpson's rule. For example, in integrating the function

$$f(x) = \frac{100}{x^2} \sin\left(\frac{10}{x}\right), \quad 1 \leq x \leq 3,$$

shown in Fig. 3.5, with tolerancelance 10^{-4} , the adaptive quadrature uses 23 subintervals and requires 93 evaluations of f . On the other hand, the composite Simpson's rule uses a constant value of $h = 1/88$ and requires 177 evaluations of f . It is seen from the figure that f varies quickly over the interval $[1, 1.5]$. The adaptive

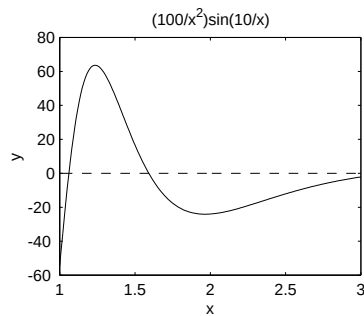


FIGURE 3.5. A fast varying function for adaptive quadrature.

quadrature needs 11 subintervals on the short interval $[1, 1.5]$ and only 12 on the longer interval $[1.5, 3]$.

The MATLAB quadrature routines `quad`, `quadl` and `dblquad` are adaptive routines.

Matlab's adaptive Simpson's rule `quad` and adaptive Newton–Cotes 8-panel rule `quad8` evaluate the integral

$$I = \int_0^{\pi/2} \sin x \, dx$$

as follows.

```
>> v1 = quad('sin',0,pi/2)
    v1 = 1.00000829552397
>> v2 = quad8('sin',0,pi/2)
    v2 = 1.00000000000000
```

respectively, within a relative error of 10^{-3} .

CHAPTER 4

Matrix Computations

With the advent of digitized systems in many areas of science and engineering, matrix computation is occupying a central place in modern computer software. In this chapter, we study the solutions of linear systems,

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{m \times n},$$

and eigenvalue problems,

$$A\mathbf{x} = \lambda\mathbf{x}, \quad A \in \mathbb{R}^{n \times n}, \quad \mathbf{x} \neq 0,$$

as implemented in softwares, where accuracy, stability and algorithmic complexity are of the utmost importance.

4.1. *LU* Solution of $A\mathbf{x} = \mathbf{b}$

The solution of a linear system

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{n \times n},$$

with partial pivoting, to be explained below, is obtained by the *LU* decomposition of A ,

$$A = LU,$$

where L is a row-permutation of a lower triangular matrix M with $m_{ii} = 1$ and $|m_{ij}| \leq 1$, for $i > j$, and U is an upper triangular matrix. Thus the system becomes

$$LU\mathbf{x} = \mathbf{b}.$$

The solution is obtained in two steps. First,

$$L\mathbf{y} = \mathbf{b}$$

is solved for $\mathbf{y}$ by forward substitution and, second,

$$U\mathbf{x} = \mathbf{y}$$

is solved for $\mathbf{x}$ by backward substitution. The following example illustrates the above steps.

EXAMPLE 4.1. Solve the system $A\mathbf{x} = \mathbf{b}$,

$$\begin{bmatrix} 3 & 9 & 6 \\ 18 & 48 & 39 \\ 9 & -27 & 42 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 23 \\ 136 \\ 45 \end{bmatrix}$$

by the *LU* decomposition **with partial pivoting**.

SOLUTION. Since $a_{21} = 18$ is the largest pivot in absolute value in the first column of A ,

$$|18| > |3|, \quad |18| > |9|,$$

we interchange the second and first rows of A ,

$$P_1 A = \begin{bmatrix} 18 & 48 & 39 \\ 3 & 9 & 6 \\ 9 & -27 & 42 \end{bmatrix}, \quad \text{where} \quad P_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

We now apply a Gaussian transformation, M_1 , on $P_1 A$ to put zeros under 18 in the first column,

$$\begin{bmatrix} 1 & 0 & 0 \\ -1/6 & 1 & 0 \\ -1/2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 18 & 48 & 39 \\ 3 & 9 & 6 \\ 9 & -27 & 42 \end{bmatrix} = \begin{bmatrix} 18 & 48 & 39 \\ 0 & 1 & -1/2 \\ 0 & -51 & 45/2 \end{bmatrix},$$

with multipliers $-1/6$ and $-1/2$. Thus

$$M_1 P_1 A = A_1.$$

Considering the 2×2 submatrix

$$\begin{bmatrix} 1 & -1/2 \\ -51 & 45/2 \end{bmatrix},$$

we see that -51 is the pivot in the first column since

$$|-51| > |1|.$$

Hence we interchange the third and second row,

$$P_2 A_1 = \begin{bmatrix} 18 & 48 & 39 \\ 0 & -51 & 45/2 \\ 0 & 1 & -1/2 \end{bmatrix}, \quad \text{where} \quad P_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

To zero the $(3, 2)$ element we apply a Gaussian transformation, M_2 , on $P_2 A_1$,

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1/51 & 1 \end{bmatrix} \begin{bmatrix} 18 & 48 & 39 \\ 0 & -51 & 45/2 \\ 0 & 1 & -1/2 \end{bmatrix} = \begin{bmatrix} 18 & 48 & 39 \\ 0 & -51 & 22.5 \\ 0 & 0 & -0.0588 \end{bmatrix},$$

where $1/51$ is the multiplier. Thus

$$M_2 P_2 A_1 = U.$$

Therefore

$$M_2 P_2 M_1 P_1 A = U,$$

and

$$A = P_1^{-1} M_1^{-1} P_2^{-1} M_2^{-1} U = LU.$$

The inverse of a Gaussian transformation is easily written:

$$\begin{aligned} M_1 = \begin{bmatrix} 1 & 0 & 0 \\ -a & 1 & 0 \\ -b & 0 & 1 \end{bmatrix} &\implies M_1^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}, \\ M_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -c & 1 \end{bmatrix} &\implies M_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & c & 1 \end{bmatrix}, \end{aligned}$$

once the multipliers $-a$, $-b$, $-c$ are known. Moreover the product $M_1^{-1}M_2^{-1}$ can be easily written:

$$M_1^{-1}M_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ a & 1 & 0 \\ b & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & c & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ a & 1 & 0 \\ b & c & 1 \end{bmatrix}.$$

It is easily seen that a permutation P , which consists of the identity matrix I with permuted rows, is an orthogonal matrix. Hence,

$$P^{-1} = P^T.$$

Therefore, if

$$L = P_1^T M_1^{-1} P_2^T M_2^{-1},$$

then, solely by a rearrangement of the elements of M_1^{-1} and M_2^{-1} without any arithmetic operations, we obtain

$$\begin{aligned} L &= \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 1/6 & 1 & 0 \\ 1/2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1/51 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 1/6 & 1 & 0 \\ 1 & 0 & 0 \\ 1/2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1/51 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1/6 & -1/51 & 1 \\ 1 & 0 & 0 \\ 1/2 & 1 & 0 \end{bmatrix} \end{aligned}$$

which is the row-permutation of a lower triangular matrix, that is, it becomes lower triangular if the second and first rows are interchanged, and then the new second row is interchanged with the third row, namely, $P_2 P_1 L$ is lower triangular.

The system

$$L\mathbf{y} = \mathbf{b}$$

is solved by forward substitution:

$$\begin{bmatrix} 1/6 & -1/51 & 1 \\ 1 & 0 & 0 \\ 1/2 & 1 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 23 \\ 136 \\ 45 \end{bmatrix},$$

$$y_1 = 136,$$

$$y_2 = 45 - 136/2 = -23,$$

$$y_3 = 23 - 136/6 - 23/51 = -0.1176.$$

Finally, the system

$$U\mathbf{x} = \mathbf{y}$$

is solved by backward substitution:

$$\begin{bmatrix} 18 & 48 & 39 \\ 0 & -51 & 22.5 \\ 0 & 0 & -0.0588 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 136 \\ -23 \\ -0.1176 \end{bmatrix},$$

$$x_3 = 0.1176/0.0588 = 2,$$

$$x_2 = (-23 - 22.5 \times 2)/(-51) = 1.3333,$$

$$x_1 = (136 - 48 \times 1.3333 - 39 \times 2)/18 = -0.3333.$$

□

The following Matlab session does exactly that.

```
>> A = [3 9 6; 18 48 39; 9 -27 42]
```

```
A =
```

```
    3    9    6
   18   48   39
    9  -27   42
```

```
>> [L,U] = lu(A)
```

```
L =
```

```
    0.1667   -0.0196    1.0000
    1.0000         0         0
    0.5000    1.0000         0
```

```
U =
```

```
   18.0000   48.0000   39.0000
         0  -51.0000   22.5000
         0         0   -0.0588
```

```
>> b = [23; 136; 45]
```

```
b =
```

```
    23
   136
    45
```

```
>> y = L\b    % forward substitution
```

```
y =
```

```
   136.0000
  -23.0000
   -0.1176
```

```
>> x = U\y    % backward substitution
```

```
x =
```

```
  -0.3333
   1.3333
   2.0000
```

```
>> z = A\b    % Matlab left-inverse to solve Az = b by the LU decomposition
```

```
z =
```

```
  -0.3333
   1.3333
   2.0000
```

The didactic Matlab command

```
[L,U,P] = lu(A)
```

finds the permutation matrix P which does all the pivoting at once on the system

$$A\mathbf{x} = \mathbf{b}$$

and produces the equivalent permuted system

$$PA\mathbf{x} = P\mathbf{b}$$

which requires no further pivoting. Then it computes the LU decomposition of PA ,

$$PA = LU,$$

where the matrix L is unit lower triangular with $l_{ij} \leq 1$, for $i > j$, and the matrix U is upper triangular.

We repeat the previous Matlab session making use of the matrix P .

```
A = [3 9 6; 18 48 39; 9 -27 42]
A =      3      9      6
      18     48     39
      9    -27     42

b = [23; 136; 45]
b =      23
      136
      45

[L,U,P] = lu(A)
L =      1.0000      0      0
      0.5000      1.0000      0
      0.1667    -0.0196      1.0000

U =  18.0000   48.0000   39.0000
      0   -51.0000   22.5000
      0      0   -0.0588

P =      0      1      0
      0      0      1
      1      0      0

y = L\P*b
y = 136.0000
   -23.0000
    -0.1176

x = U\y
x = -0.3333
     1.3333
     2.0000
```

THEOREM 4.1. *The LU decomposition of a matrix A exists if and only if all its principal minors are nonzero.*

The principal minors of A are the determinants of the top left submatrices of A . Partial pivoting attempts to make the principal minors of PA nonzero.

EXAMPLE 4.2. Given

$$A = \begin{bmatrix} 3 & 2 & 0 \\ 12 & 13 & 6 \\ -3 & 8 & 9 \end{bmatrix}, \quad b = \begin{bmatrix} 14 \\ 40 \\ -28 \end{bmatrix},$$

find the LU decomposition of A **without pivoting** and solve

$$A\mathbf{x} = \mathbf{b}.$$

SOLUTION. For $M_1A = A_1$, we have

$$\begin{bmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 & 2 & 0 \\ 12 & 13 & 6 \\ -3 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 0 \\ 0 & 5 & 6 \\ 0 & 10 & 9 \end{bmatrix}.$$

For $M_2A_1 = U$, we have

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix} \begin{bmatrix} 3 & 2 & 0 \\ 0 & 5 & 6 \\ 0 & 10 & 9 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 0 \\ 0 & 5 & 6 \\ 0 & 0 & -3 \end{bmatrix} = U,$$

that is

$$M_2M_1A = U, \quad A = M_1^{-1}M_2^{-1}U = LU.$$

Thus

$$L = M_1^{-1}M_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ -1 & 2 & 1 \end{bmatrix}.$$

Forward substitution is used to obtain $\mathbf{y}$ from $L\mathbf{y} = \mathbf{b}$,

$$\begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ -1 & 2 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 14 \\ 40 \\ -28 \end{bmatrix};$$

thus

$$\begin{aligned} y_1 &= 14, \\ y_2 &= 40 - 56 = -16, \\ y_3 &= -28 + 14 + 32 = 18. \end{aligned}$$

Finally, backward substitution is used to obtain $\mathbf{x}$ from $U\mathbf{x} = \mathbf{y}$,

$$\begin{bmatrix} 3 & 2 & 0 \\ 0 & 5 & 6 \\ 0 & 0 & -3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 14 \\ -16 \\ 18 \end{bmatrix};$$

thus

$$\begin{aligned} x_3 &= -6, \\ x_2 &= (-16 + 36)/5 = 4, \\ x_1 &= (14 - 8)/3 = 2. \end{aligned}$$

□

We note that, without pivoting, $|l_{ij}|$, $i > j$, may be larger than 1.

The LU decomposition without partial pivoting is an unstable procedure which may lead to large errors in the solution. In practice, partial pivoting is usually stable. However, in some cases, one needs to resort to complete pivoting on rows and columns to ensure stability, or to use the stable QR decomposition.

Sometimes it is useful to scale the rows or columns of the matrix of a linear system before solving it. This may alter the choice of the pivots. In practice, one has to consider the meaning and physical dimensions of the unknown variables to decide upon the type of scaling or balancing of the matrix. Softwares provide some of these options. Scaling in the l_∞ -norm is used in the following example.

EXAMPLE 4.3. Scale each equation in the l_∞ -norm, so that the largest coefficient of each row on the left-hand side is equal to 1 in absolute value, and solve the following system:

$$\begin{array}{rcl} 30.00x_1 & + & 591400x_2 = 591700 \\ 5.29x_1 & - & 6.130x_2 = 46.70 \end{array}$$

by the LU decomposition with pivoting with four-digit arithmetic.

SOLUTION. Dividing the first equation by

$$s_1 = \max\{|30.00|, |591400|\} = 591400$$

and the second equation by

$$s_2 = \max\{|5.291|, |6.130|\} = 6.130,$$

we find that

$$\frac{|a_{11}|}{s_1} = \frac{30.00}{591400} = 0.5073 \times 10^{-4}, \quad \frac{|a_{21}|}{s_2} = \frac{5.291}{6.130} = 0.8631.$$

Hence the scaled pivot is in the second equation. Note that the scaling is done only for comparison purposes and the division to determine the scaled pivots produces no roundoff error in solving the system. Thus the LU decomposition applied to the interchanged system

$$\begin{array}{rcl} 5.29x_1 & - & 6.130x_2 = 46.70 \\ 30.00x_1 & + & 591400x_2 = 591700 \end{array}$$

produces the correct results:

$$x_1 = 10.00, \quad x_2 = 1.000.$$

On the other hand, the LU decomposition with four-digit arithmetic applied to the non-interchanged system produces the erroneous results $x_1 \approx -10.00$ and $x_2 \approx 1.001$. $\square$

The following Matlab function M-files are found in <ftp://ftp.cs.cornell.edu/pub/cv>. The forward substitution algorithm solves a lower triangular system:

```
function x = LTriSol(L,b)
%
% Pre:
%   L   n-by-n nonsingular lower triangular matrix
%   b   n-by-1
%
```

```

% Post:
%   x   Lx = b

    n = length(b);
    x = zeros(n,1);
    for j=1:n-1
        x(j) = b(j)/L(j,j);
        b(j+1:n) = b(j+1:n) - L(j+1:n,j)*x(j);
    end
    x(n) = b(n)/L(n,n);

```

The backward substitution algorithm solves a upper triangular system:

```

function x = UTriSol(U,b)
%
% Pre:
%   U   n-by-n nonsingular upper triangular matrix
%   b   n-by-1
%
% Post:
%   x   Lx = b

    n = length(b);
    x = zeros(n,1);
    for j=n:-1:2
        x(j) = b(j)/U(j,j);
        b(1:j-1) = b(1:j-1) - x(j)*U(1:j-1,j);
    end
    x(1) = b(1)/U(1,1);

```

The LU decomposition without pivoting is performed by the following function.

```

function [L,U] = GE(A);
%
% Pre:
%   A       n-by-n
%
% Post:
%   L       n-by-n unit lower triangular with |L(i,j)| <= 1.
%   U       n-by-n upper triangular.
%           A = LU

    [n,n] = size(A);
    for k=1:n-1
        A(k+1:n,k) = A(k+1:n,k)/A(k,k);
        A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - A(k+1:n,k)*A(k,k+1:n);
    end
    L = eye(n,n) + tril(A,-1);
    U = triu(A);

```

The LU decomposition with pivoting is performed by the following function.

```

function [L,U,piv] = GEpiv(A);

```

```

%
% Pre:
%   A      n-by-n
%
% Post:
%   L      n-by-n unit lower triangular with |L(i,j)| <= 1.
%   U      n-by-n upper triangular
%   piv     integer n-vector that is a permutation of 1:n.
%
%           A(piv,:) = LU

[n,n] = size(A);
piv = 1:n;
for k=1:n-1
    [maxv,r] = max(abs(A(k:n,k)));
    q = r+k-1;
    piv([k q]) = piv([q k]);
    A([k q],:) = A([q k],:);
    if A(k,k) ~= 0
        A(k+1:n,k) = A(k+1:n,k)/A(k,k);
        A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - A(k+1:n,k)*A(k,k+1:n);
    end
end
L = eye(n,n) + tril(A,-1);
U = triu(A);

```

4.2. Cholesky Decomposition

The important class of positive definite symmetric matrices admits the Cholesky decomposition

$$A = GG^T$$

where G is lower triangular.

DEFINITION 4.1. A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is said to be positive definite if

$$\mathbf{x}^T A \mathbf{x} > 0, \quad \text{for all } \mathbf{x} \neq 0, \quad \mathbf{x} \in \mathbb{R}^n.$$

In that case we write $A > 0$.

A symmetric matrix, A , is positive definite if and only if all its eigenvalues λ ,

$$A\mathbf{x} = \lambda\mathbf{x}, \quad \mathbf{x} \neq 0,$$

are positive, $\lambda > 0$.

A symmetric matrix A is positive definite if and only if all its principal minors are positive. For example,

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} > 0$$

if and only if

$$\det a_{11} = a_{11} > 0, \quad \det \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} > 0, \quad \det A > 0.$$

If $A > 0$, then $a_{ii} > 0$, $i = 1, 2, \dots, n$.

An $n \times n$ matrix A is *diagonally dominant* if

$$|a_{ii}| > |a_{i1}| + |a_{i2}| + \cdots + |a_{i,i-1}| + |a_{i,i+1}| + \cdots + |a_{in}|, \quad i = 1, 2, \dots, n.$$

A diagonally dominant symmetric matrix with positive diagonal entries is positive definite.

THEOREM 4.2. *If A is positive definite, the Cholesky decomposition*

$$A = GG^T$$

does not require any pivoting, and hence $A\mathbf{x} = \mathbf{b}$ can be solved by the Cholesky decomposition without pivoting, by forward and backward substitutions:

$$G\mathbf{y} = \mathbf{b}, \quad G^T\mathbf{x} = \mathbf{y}.$$

EXAMPLE 4.4. Let

$$A = \begin{bmatrix} 4 & 6 & 8 \\ 6 & 34 & 52 \\ 8 & 52 & 129 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ -160 \\ -452 \end{bmatrix}.$$

Find the Cholesky decomposition of A and use it to compute the determinant of A and to solve the system

$$A\mathbf{x} = \mathbf{b}.$$

SOLUTION. The Cholesky decomposition is obtained (without pivoting) by solving the following system for g_{ij} :

$$\begin{bmatrix} g_{11} & 0 & 0 \\ g_{21} & g_{22} & 0 \\ g_{31} & g_{32} & g_{33} \end{bmatrix} \begin{bmatrix} g_{11} & g_{21} & g_{31} \\ 0 & g_{22} & g_{32} \\ 0 & 0 & g_{33} \end{bmatrix} = \begin{bmatrix} 4 & 6 & 8 \\ 6 & 34 & 52 \\ 8 & 52 & 129 \end{bmatrix}.$$

$$\begin{aligned} g_{11}^2 &= 4 && \implies g_{11} = 2 > 0, \\ g_{11}g_{21} &= 6 && \implies g_{21} = 3, \\ g_{11}g_{31} &= 8 && \implies g_{31} = 4, \\ g_{21}^2 + g_{22}^2 &= 34 && \implies g_{22} = 5 > 0, \\ g_{21}g_{31} + g_{22}g_{32} &= 52 && \implies g_{32} = 8, \\ g_{31}^2 + g_{32}^2 + g_{33}^2 &= 129 && \implies g_{33} = 7 > 0. \end{aligned}$$

Hence

$$G = \begin{bmatrix} 2 & 0 & 0 \\ 3 & 5 & 0 \\ 4 & 8 & 7 \end{bmatrix},$$

and

$$\det A = \det G \det G^T = (\det G)^2 = (2 \times 5 \times 7)^2 > 0.$$

Solving $G\mathbf{y} = \mathbf{b}$ by forward substitution,

$$\begin{bmatrix} 2 & 0 & 0 \\ 3 & 5 & 0 \\ 4 & 8 & 7 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 0 \\ -160 \\ -452 \end{bmatrix},$$

we have

$$\begin{aligned}y_1 &= 0, \\y_2 &= -32, \\y_3 &= (-452 + 256)/7 = -28.\end{aligned}$$

Solving $G^T \mathbf{x} = \mathbf{y}$ by backward substitution,

$$\begin{bmatrix} 2 & 3 & 4 \\ 0 & 5 & 8 \\ 0 & 0 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ -32 \\ -28 \end{bmatrix},$$

we have

$$\begin{aligned}x_3 &= -4, \\x_2 &= (-32 + 32)/5 = 0, \\x_1 &= (0 - 3 \times 0 + 16)/2 = 8. \quad \square\end{aligned}$$

The numeric Matlab command `chol` find the Cholesky decomposition $R^T R$ of the symmetric matrix A as follows.

```
>> A = [4 6 8;6 34 52;8 52 129];
>> R = chol(A)
R =
```

```
 2     3     4
 0     5     8
 0     0     7
```

The following Matlab function M-files are found in <ftp://ftp.cs.cornell.edu/pub/cv>. They are introduced here to illustrate the different levels of matrix-vector multiplications.

The simplest “scalar” Cholesky decomposition is obtained by the following function.

```
function G = CholScalar(A);
%
% Pre: A is a symmetric and positive definite matrix.
% Post: G is lower triangular and A = G*G'.

[n,n] = size(A);
G = zeros(n,n);
for i=1:n
    % Compute G(i,1:i)
    for j=1:i
        s = A(j,i);
        for k=1:j-1
            s = s - G(j,k)*G(i,k);
        end
        if j<i
            G(i,j) = s/G(j,j);
        else
            G(i,i) = sqrt(s);
        end
    end
end
```

```
end
```

```
end
```

The dot product of two vectors returns a scalar, $c = x^T y$. Noticing that the k -loop in `CholScalar` oversees an inner product between subrows of G , we obtain the following level-1 dot product implementation.

```
function G = CholDot(A);
%
% Pre:  A is a symmetric and positive definite matrix.
% Post: G is lower triangular and A = G*G'.

[n,n] = size(A);
G = zeros(n,n);
for i=1:n
    % Compute G(i,1:i)
    for j=1:i
        if j==1
            s = A(j,i);
        else
            s = A(j,i) - G(j,1:j-1)*G(i,1:j-1)';
        end
        if j<i
            G(i,j) = s/G(j,j);
        else
            G(i,i) = sqrt(s);
        end
    end
end
```

An update of the form

$$\text{vector} \leftarrow \text{vector} + \text{vector} \cdot \text{scalar}$$

is called a *saxpy* operation, which stands for “scalar a times x plus y ”, that is $y = ax + y$. A column-orientation version that features the saxpy operation is the following implementation.

```
function G = CholSax(A);
%
% Pre:  A is a symmetric and positive definite matrix.
% Post: G is lower triangular and A = G*G'.

[n,n] = size(A);
G = zeros(n,n);
s = zeros(n,1);
for j=1:n
    s(j:n) = A(j:n,j);
    for k=1:j-1
        s(j:n) = s(j:n) - G(j:n,k)*G(j,k);
    end
    G(j:n,j) = s(j:n)/sqrt(s(j));
end
```

An update of the form

$$\text{vector} \leftarrow \text{vector} + \text{matrix} \times \text{vector}$$

is called a *gaxpy* operation, which stands for “general A times $\mathbf{x}$ plus $\mathbf{y}$ ” (general saxpy), that is $\mathbf{y} = A\mathbf{x} + \mathbf{y}$. A version that features level-2 gaxpy operation is the following implementation.

```
function G = CholGax(A);
%
% Pre:  A is a symmetric and positive definite matrix.
% Post: G is lower triangular and A = G*G'.

[n,n] = size(A);
G = zeros(n,n);
s = zeros(n,1);
for j=1:n
    if j==1
        s(j:n) = A(j:n,j);
    else
        s(j:n) = A(j:n,j) - G(j:n,1:j-1)*G(j,1:j-1)';
    end
    G(j:n,j) = s(j:n)/sqrt(s(j));
end
```

There is also a recursive implementation which computes the Cholesky factor row by row, just like `CholScalar`

```
function G = CholRecur(A);
%
% Pre:  A is a symmetric and positive definite matrix.
% Post: G is lower triangular and A = G*G'.

[n,n] = size(A);
if n==1
    G = sqrt(A);
else
    G(1:n-1,1:n-1) = CholRecur(A(1:n-1,1:n-1));
    G(n,1:n-1)      = LTriSol(G(1:n-1,1:n-1),A(1:n-1,n))';
    G(n,n)           = sqrt(A(n,n) - G(n,1:n-1)*G(n,1:n-1)');
end
```

There is even a high performance level-3 implementation of the Cholesky decomposition `CholBlock`

4.3. Matrix Norms

In matrix computations, norms are used to quantify results, like error estimates and to study the convergence of iterative schemes.

Given a matrix $A \in \mathbb{R}^{n \times n}$ or $\mathbb{C}^{n \times n}$, and a vector norm $\|\mathbf{x}\|$ for $\mathbf{x} \in \mathbb{R}^n$, a *subordinate matrix norm*, $\|A\|$, is defined by the supremum

$$\|A\| = \sup_{\mathbf{x} \neq 0} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|} = \sup_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|.$$

There are three important vector norms in scientific computation, the l_1 -norm of $\mathbf{x}$,

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| = |x_1| + |x_2| + \cdots + |x_n|,$$

the *Euclidean norm*, or l_2 -norm, of $\mathbf{x}$,

$$\|\mathbf{x}\|_2 = \left[\sum_{i=1}^n |x_i|^2 \right]^{1/2} = [|x_1|^2 + |x_2|^2 + \cdots + |x_n|^2]^{1/2},$$

and the *supremum norm*, or l_∞ -norm, of $\mathbf{x}$,

$$\|\mathbf{x}\|_\infty = \sup_{i=1,2,\dots,n} |x_i| = \sup\{|x_1|, |x_2|, \dots, |x_n|\}.$$

It can be shown that the corresponding matrix norms are given by the following formulae.

The l_1 -norm, or column “sum” norm, of A is

$$\|A\|_1 = \max_{j=1,2,\dots,n} \sum_{i=1}^n |a_{ij}| \quad (\text{largest column in the } l_1 \text{ vector norm}),$$

the l_∞ -norm, or a row “sum” norm, of A is

$$\|A\|_\infty = \max_{i=1,2,\dots,n} \sum_{j=1}^n |a_{ij}| \quad (\text{largest row in the } l_1 \text{ vector norm}),$$

and the l_2 -norm of A is

$$\|A\|_2 = \max_{i=1,2,\dots,n} \{\sigma_i^{1/2}\} \quad (\text{largest singular value of } A),$$

where the $\sigma_i^2 \geq 0$ are the eigenvalues of $A^T A$. The singular values of a matrix are considered in Subsection 4.9.

An important non-subordinate matrix norm is the *Frobenius norm*, or *Euclidean matrix norm*,

$$\|A\|_F = \left[\sum_{j=1}^n \sum_{i=1}^n |a_{ij}|^2 \right]^{1/2}.$$

DEFINITION 4.2 (Condition number). The *condition number* of a matrix $A \in \mathbb{R}^{n \times n}$ is the number

$$\kappa(A) = \|A\| \|A^{-1}\|. \quad (4.1)$$

Note that $\kappa(A) \geq 1$ if $\|I\| = 1$.

The condition number of A appears in an upper bound for the relative error in the solution to the system

$$A\mathbf{x} = \mathbf{b}.$$

In fact, let $\hat{\mathbf{x}}$ be the exact solution to the perturbed system

$$(A + \Delta A)\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b},$$

where all experimental and numerical roundoff errors are lumped into ΔA and $\delta \mathbf{b}$. Then we have the bound

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(A) \left[\frac{\|\Delta A\|}{\|A\|} + \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|} \right]. \quad (4.2)$$

We say that a system $A\mathbf{x} = \mathbf{b}$ is well conditioned if $\kappa(A)$ is small; otherwise it is ill conditioned.

EXAMPLE 4.5. Study the ill condition of the following system

$$\begin{bmatrix} 1.0001 & 1 \\ 1 & 1.0001 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2.0001 \\ 2.0001 \end{bmatrix}$$

with exact and some approximate solutions

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \hat{\mathbf{x}} = \begin{bmatrix} 2.0000 \\ 0.0001 \end{bmatrix},$$

respectively.

SOLUTION. The approximate solution has a very small residual (to 4 decimals), $\mathbf{r} = \mathbf{b} - A\hat{\mathbf{x}}$,

$$\begin{aligned} \mathbf{r} &= \begin{bmatrix} 2.0001 \\ 2.0001 \end{bmatrix} - \begin{bmatrix} 1.0001 & 1 \\ 1 & 1.0001 \end{bmatrix} \begin{bmatrix} 2.0000 \\ 0.0001 \end{bmatrix} \\ &= \begin{bmatrix} 2.0001 \\ 2.0001 \end{bmatrix} - \begin{bmatrix} 2.0003 \\ 2.0001 \end{bmatrix} = \begin{bmatrix} -0.0002 \\ 0.0000 \end{bmatrix}. \end{aligned}$$

However, the relative error in $\hat{\mathbf{x}}$ is

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|_1}{\|\mathbf{x}\|_1} = \frac{(1.0000 + 0.9999)}{1 + 1} \approx 1,$$

that is 100%. This is explained by the fact that the system is very ill conditioned. In fact,

$$A^{-1} = \frac{1}{0.0002} \begin{bmatrix} 1.0001 & -1.0000 \\ -1.0000 & 1.0001 \end{bmatrix} = \begin{bmatrix} 5000.5 & -5000.0 \\ -5000.0 & 5000.5 \end{bmatrix},$$

and

$$\kappa_1(A) = (1.0001 + 1.0000)(5000.5 + 5000.0) = 20\,002. \quad \square$$

The l_1 -norm of the matrix A of the previous example and the l_1 -condition number of A are obtained by the following numeric Matlab commands:

```
>> A = [1.0001 1; 1 1.0001];
>> N1 = norm(A,1)
N1 = 2.0001
>> K1 = cond(A,1)
K1 = 2.0001e+04
```

4.4. Iterative Methods

One can solve linear systems by iterative methods, especially when dealing with very large systems. One such method is Gauss–Seidel’s method which uses the latest values for the variables as soon as they are obtained. This method is best explained by means of an example.

EXAMPLE 4.6. Apply two iterations of Gauss–Seidel’s iterative scheme to the system

$$\begin{array}{rclcl} 4x_1 & + & 2x_2 & + & x_3 & = & 14, & x_1^{(0)} & = & 1, \\ x_1 & + & 5x_2 & - & x_3 & = & 10, & \text{with } x_2^{(0)} & = & 1, \\ x_1 & + & x_2 & + & 8x_3 & = & 20, & x_3^{(0)} & = & 1. \end{array}$$

SOLUTION. Since the system is diagonally dominant, Gauss–Seidel’s iterative scheme will converge. This scheme is

$$\begin{array}{rclcl} x_1^{(n+1)} & = & \frac{1}{4}(14 & - & 2x_2^{(n)} & - & x_3^{(n)}), & x_1^{(0)} & = & 1, \\ x_2^{(n+1)} & = & \frac{1}{5}(10 & - & x_1^{(n+1)} & + & x_3^{(n)}), & x_2^{(0)} & = & 1, \\ x_3^{(n+1)} & = & \frac{1}{8}(20 & - & x_1^{(n+1)} & - & x_2^{(n+1)}), & x_3^{(0)} & = & 1. \end{array}$$

For $n = 0$, we have

$$\begin{aligned} x_1^{(1)} &= \frac{1}{4}(14 - 2 - 1) = \frac{11}{4} = 2.75 \\ x_2^{(1)} &= \frac{1}{5}(10 - 2.75 + 1) = 1.65 \\ x_3^{(1)} &= \frac{1}{8}(20 - 2.75 - 1.65) = 1.95. \end{aligned}$$

For $n = 1$:

$$\begin{aligned} x_1^{(2)} &= \frac{1}{4}(14 - 2 \times 1.65 - 1.95) = 2.1875 \\ x_2^{(2)} &= \frac{1}{5}(10 - 2.1875 + 1.95) = 1.9525 \\ x_3^{(2)} &= \frac{1}{8}(20 - 2.1875 - 1.9525) = 1.9825 \quad \square \end{aligned}$$

Gauss–Seidel’s iteration to solve the system $A\mathbf{x} = \mathbf{b}$ is given by the following iterative scheme:

$$\mathbf{x}^{(m+1)} = D^{-1}(\mathbf{b} - L\mathbf{x}^{(m+1)} - U\mathbf{x}^{(m)}), \quad \text{with properly chosen } \mathbf{x}^{(0)},$$

where the matrix A has been split as the sum of three matrices,

$$A = D + L + U,$$

with D diagonal, L strictly lower triangular, and U strictly upper triangular.

This algorithm is programmed in Matlab to do $k = 5$ iterations for the following system:

```
A = [7 1 -1; 1 11 1; -1 1 9]; b = [3 0 -17]';
D = diag(A); L = tril(A,-1); U = triu(A,1);
m = size(b,1); % number of rows of b
x = ones(m,1); % starting value
y = zeros(m,1); % temporary storage
k = 5; % number of iterations
for j = 1:k
    uy = U*x(:,j);
    for i = 1:m
        y(i) = (1/D(i))*(b(i)-L(i,:)*y-uy(i));
    end
    x = [x,y];
```

```

end
x
x =
    1.0000    0.4286    0.1861    0.1380    0.1357    0.1356
    1.0000   -0.1299    0.1492    0.1588    0.1596    0.1596
    1.0000   -1.8268   -1.8848   -1.8912   -1.8915   -1.8916

```

It is important to rearrange the coefficient matrix of a given linear system in as much a diagonally dominant matrix as possible since this may assure or improve the convergence of the Gauss–Seidel iteration.

EXAMPLE 4.7. Rearrange the system

$$\begin{array}{rrrrrr} 2x_1 & + & 10x_2 & - & x_3 & = & -32 \\ -x_1 & + & 2x_2 & - & 15x_3 & = & 17 \\ 10x_1 & - & x_2 & + & 2x_3 & = & 35 \end{array}$$

such that Gauss–Seidel’s scheme converges.

SOLUTION. By placing the last equation first, the system will be diagonally dominant,

$$\begin{array}{rrrrrr} 10x_1 & - & x_2 & + & 2x_3 & = & 35 \\ 2x_1 & + & 10x_2 & - & x_3 & = & -32 \\ -x_1 & + & 2x_2 & - & 15x_3 & = & 17 \end{array} \quad \square$$

The Jacobi iteration solves the system $A\mathbf{x} = \mathbf{b}$ by the following simultaneous iterative scheme:

$$\mathbf{x}^{(m+1)} = D^{-1} \left(\mathbf{b} - L\mathbf{x}^{(m)} - U\mathbf{x}^{(m)} \right), \quad \text{with properly chosen } \mathbf{x}^{(0)},$$

where the matrices D , L and U are as defined above.

Applied to Example 4.6, Jacobi’s method is

$$\begin{array}{ll} x_1^{(n+1)} &= \frac{1}{4}(14 - 2x_2^{(n)} - x_3^{(n)}), & x_1^{(0)} &= 1, \\ x_2^{(n+1)} &= \frac{1}{5}(10 - x_1^{(n)} + x_3^{(n)}), & x_2^{(0)} &= 1, \\ x_3^{(n+1)} &= \frac{1}{8}(20 - x_1^{(n)} - x_2^{(n)}), & x_3^{(0)} &= 1. \end{array}$$

We state the following three theorems, without proof, on the convergence of iterative schemes.

THEOREM 4.3. *If the matrix A is diagonally dominant, then the Jacobi and Gauss–Seidel iterations converge.*

THEOREM 4.4. *Suppose the matrix $A \in \mathbb{R}^{n \times n}$ is such that $a_{ii} > 0$ and $a_{ij} \leq 0$ for $i \neq j$, $i, j = 1, 2, \dots, n$. If the Jacobi iterative scheme converges, then the Gauss–Seidel iteration converges faster. If the Jacobi iterative scheme diverges, then the Gauss–Seidel iteration diverges faster.*

THEOREM 4.5. *If $A \in \mathbb{R}^{n \times n}$ is symmetric and positive definite, then the Gauss–Seidel iteration converges for any $\mathbf{x}^{(0)}$.*

4.5. Overdetermined Systems

A linear system is said to be overdetermined if it has more equations than unknowns. In curve fitting we are given N points,

$$(x_1, y_1), \quad (x_2, y_2), \quad \dots, \quad (x_N, y_N),$$

and want to determine a function $f(x)$ such that

$$f(x_i) \approx y_i, \quad i = 1, 2, \dots, N.$$

For properly chosen functions, $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x)$, we put

$$f(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \dots + a_n\varphi_n(x),$$

and minimize the quadratic form

$$Q(a_0, a_1, \dots, a_n) = \sum_{i=1}^N (f(x_i) - y_i)^2.$$

Typically, $N \gg n + 1$. If the functions $\varphi_j(x)$ are “linearly independent”, the quadratic form is nondegenerate and the minimum is attained for values of $a_0, a_1, \dots, a_n$, such that

$$\frac{\partial Q}{\partial a_j} = 0, \quad j = 0, 1, 2, \dots, n.$$

Writing the quadratic form Q explicitly,

$$Q = \sum_{i=1}^N (a_0\varphi_0(x_i) + \dots + a_n\varphi_n(x_i) - y_i)^2,$$

and equating the partial derivatives of Q with respect to a_j to zero, we have

$$\frac{\partial Q}{\partial a_j} = 2 \sum_{i=1}^N (a_0\varphi_0(x_i) + \dots + a_n\varphi_n(x_i) - y_i)\varphi_j(x_i) = 0.$$

This is an $(n + 1) \times (n + 1)$ symmetric linear algebraic system

$$\begin{bmatrix} \sum \varphi_0(x_i)\varphi_0(x_i) & \sum \varphi_1(x_i)\varphi_0(x_i) & \cdots & \sum \varphi_n(x_i)\varphi_0(x_i) \\ \vdots & & & \vdots \\ \sum \varphi_0(x_i)\varphi_n(x_i) & \sum \varphi_1(x_i)\varphi_n(x_i) & \cdots & \sum \varphi_n(x_i)\varphi_n(x_i) \end{bmatrix} \begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} \sum \varphi_0(x_i)y_i \\ \vdots \\ \sum \varphi_n(x_i)y_i \end{bmatrix}, \quad (4.3)$$

where all sums are over i from 1 to N . Setting the $N \times (n + 1)$ matrix A , and the N vector $\mathbf{y}$ as

$$A = \begin{bmatrix} \varphi_0(x_1) & \varphi_1(x_1) & \cdots & \varphi_n(x_1) \\ \vdots & & & \vdots \\ \varphi_0(x_N) & \varphi_1(x_N) & \cdots & \varphi_n(x_N) \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix},$$

we see that the previous square system can be written in the form

$$A^T A \begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix} = A^T \mathbf{y}.$$

These equations are called the *normal equations*.

In the case of linear regression, we have

$$\varphi_0(x) = 1, \quad \varphi_1(x) = x,$$

and the normal equations are

$$\begin{bmatrix} N & \sum_{i=1}^N x_i \\ \sum_{i=1}^N x_i & \sum_{i=1}^N x_i^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^N y_i \\ \sum_{i=1}^N x_i y_i \end{bmatrix}.$$

This is the least-square fit by a straight line.

In the case of quadratic regression, we have

$$\varphi_0(x) = 1, \quad \varphi_1(x) = x, \quad \varphi_2(x) = x^2,$$

and the normal equations are

$$\begin{bmatrix} N & \sum_{i=1}^N x_i & \sum_{i=1}^N x_i^2 \\ \sum_{i=1}^N x_i & \sum_{i=1}^N x_i^2 & \sum_{i=1}^N x_i^3 \\ \sum_{i=1}^N x_i^2 & \sum_{i=1}^N x_i^3 & \sum_{i=1}^N x_i^4 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^N y_i \\ \sum_{i=1}^N x_i y_i \\ \sum_{i=1}^N x_i^2 y_i \end{bmatrix}.$$

This is the least-square fit by a parabola.

EXAMPLE 4.8. Using the method of least squares, fit a parabola

$$f(x) = a_0 + a_1 x + a_2 x^2$$

to the following data

i	1	2	3	4	5
x_i	0	1	2	4	6
y_i	3	1	0	1	4

SOLUTION. **(a) The analytic solution.**— The normal equations are

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 4 & 6 \\ 0 & 1 & 4 & 16 & 36 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 4 & 16 \\ 1 & 6 & 36 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 4 & 6 \\ 0 & 1 & 4 & 16 & 36 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \\ 0 \\ 1 \\ 4 \end{bmatrix},$$

that is

$$\begin{bmatrix} 5 & 13 & 57 \\ 13 & 57 & 289 \\ 57 & 289 & 1569 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 9 \\ 29 \\ 161 \end{bmatrix},$$

or

$$N\mathbf{a} = \mathbf{b}.$$

Using the Cholesky decomposition $N = GG^T$, we have

$$G = \begin{bmatrix} 2.2361 & 0 & 0 \\ 5.8138 & 4.8166 & 0 \\ 25.4921 & 29.2320 & 8.0430 \end{bmatrix}.$$

The solution $\mathbf{a}$ is obtained by forward and backward substitutions with $G\mathbf{w} = \mathbf{b}$ and $G^T\mathbf{a} = \mathbf{w}$,

$$\begin{aligned}a_0 &= 2.8252 \\a_1 &= -2.0490 \\a_2 &= 0.3774.\end{aligned}$$

(b) The Matlab numeric solution.—

```
x = [0 1 2 4 6]';
A = [x.^0 x x.^2];
y = [3 1 0 1 4]';
a = (A'*A \ (A'*y))'
a =    2.8252   -2.0490    0.3774
```

The result is plotted in Fig. 4.1

□

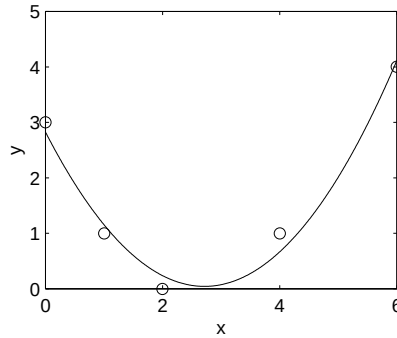


FIGURE 4.1. Quadratic least-square approximation in Example 4.8.

4.6. Matrix Eigenvalues and Eigenvectors

An *eigenvalue*, or *characteristic value*, of a matrix $A \in \mathbb{R}^{n \times n}$, or $\mathbb{C}^{n \times n}$, is a real or complex number such that the vector equation

$$A\mathbf{x} = \lambda\mathbf{x}, \quad \mathbf{x} \in \mathbb{R}^n \text{ or } \mathbb{C}^n, \quad (4.4)$$

has a nontrivial solution, $\mathbf{x} \neq 0$, called an *eigenvector*. We rewrite (4.4) in the form

$$(A - \lambda I)\mathbf{x} = 0, \quad (4.5)$$

where I is the $n \times n$ identity matrix. This equation has a nonzero solution $\mathbf{x}$ if and only if the *characteristic determinant* is zero,

$$\det(A - \lambda I) = 0, \quad (4.6)$$

that is, λ is a zero of the characteristic polynomial of A .

4.6.1. Gershgorin's disks. The inclusion theorem of Gershgorin states that each eigenvalue of A lies in a Gershgorin disk.

THEOREM 4.6 (Gershgorin Theorem). *Let λ be an eigenvalue of an arbitrary $n \times n$ matrix $A = (a_{ij})$. Then for some i , $1 \leq i \leq n$, we have*

$$|a_{ii} - \lambda| \leq |a_{i1}| + |a_{i2}| + \cdots + |a_{i,i-1}| + |a_{i,i+1}| + \cdots + |a_{in}|. \quad (4.7)$$

PROOF. Let $\mathbf{x}$ be an eigenvector corresponding to the eigenvalue λ , that is,

$$(A - \lambda I)\mathbf{x} = 0. \quad (4.8)$$

Let x_i be a component of $\mathbf{x}$ that is largest in absolute value. Then we have $|x_j/x_i| \leq 1$ for $j = 1, 2, \dots, n$. The vector equation (4.8) is a system of n equations and the i th equation is

$$a_{i1}x_1 + \cdots + a_{i,i-1}x_{i-1} + (a_{ii} - \lambda)x_i + a_{i,i+1}x_{i+1} + \cdots + a_{in}x_n = 0.$$

Division by x_i and reshuffling terms give

$$a_{ii} - \lambda = -a_{i1}\frac{x_1}{x_i} - \cdots - a_{i,i-1}\frac{x_{i-1}}{x_i} - a_{i,i+1}\frac{x_{i+1}}{x_i} - \cdots - a_{in}\frac{x_n}{x_i}.$$

Taking absolute values on both sides of this equation, applying the triangle inequality $|a+b| \leq |a|+|b|$ (where a and b are any complex numbers), and observing that because of the choice of i ,

$$\left| \frac{x_1}{x_i} \right| \leq 1, \quad \dots, \quad \left| \frac{x_n}{x_i} \right| \leq 1,$$

we obtain (4.7). □

EXAMPLE 4.9. Using Gershgorin Theorem, determine and sketch the Gershgorin disks D_k that contain the eigenvalues of the matrix

$$A = \begin{bmatrix} -3 & 0.5i & -i \\ 1-i & 1+i & 0 \\ 0.1i & 1 & -i \end{bmatrix}.$$

SOLUTION. The centres, c_i , and radii, r_i , of the disks are

$$\begin{aligned} c_1 &= -3, & r_1 &= |0.5i| + |-i| = 1.5 \\ c_2 &= 1+i, & r_2 &= |1-i| + |0| = \sqrt{2} \\ c_3 &= -i, & r_3 &= |0.1i| + 1 = 1.1 \end{aligned}$$

as shown in Fig. 4.2. □

The eigenvalues of the matrix A of Example 4.9 are

$$-3.2375 - 0.1548i, \quad 1.0347 + 1.1630i, \quad 0.2027 - 1.0082i.$$

In this case, there is one eigenvalue in each Gershgorin disk.

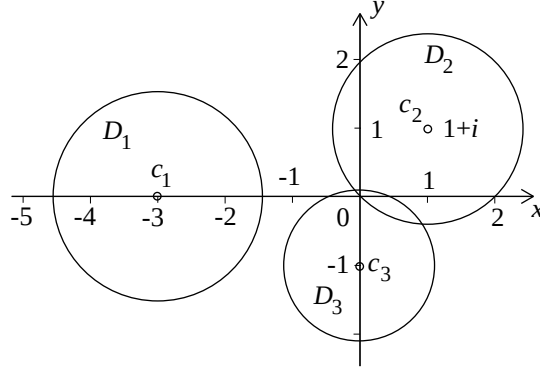


FIGURE 4.2. Gershgorin disks for Example 4.9.

4.6.2. The power method. The *power method* can be used to determine the eigenvalue of largest modulus of a matrix A and the corresponding eigenvector. The method is derived as follows.

For simplicity we assume that A admits n linearly independent eigenvectors $\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n$ corresponding to the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$, ordered such that

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|.$$

Then any vector $\mathbf{x}$ can be represented in the form

$$\mathbf{x} = a_1 \mathbf{z}_1 + a_2 \mathbf{z}_2 + \dots + a_n \mathbf{z}_n.$$

Applying A^k to $\mathbf{x}$, we have

$$\begin{aligned} A^k \mathbf{x} &= a_1 \lambda_1^k \mathbf{z}_1 + a_2 \lambda_2^k \mathbf{z}_2 + \dots + a_n \lambda_n^k \mathbf{z}_n \\ &= \lambda_1^k \left[a_1 \mathbf{z}_1 + a_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k \mathbf{z}_2 + \dots + a_n \left(\frac{\lambda_n}{\lambda_1} \right)^k \mathbf{z}_n \right] \\ &\rightarrow \lambda_1^k a_1 \mathbf{z}_1 = \mathbf{y} \quad \text{as } k \rightarrow \infty. \end{aligned}$$

Thus $A\mathbf{y} = \lambda_1 \mathbf{y}$. In practice, successive vectors are scaled to avoid overflows.

$$\begin{aligned} A\mathbf{x}^{(0)} &= \mathbf{x}^{(1)}, & \mathbf{u}^{(1)} &= \frac{\mathbf{x}^{(1)}}{\|\mathbf{x}^{(1)}\|_\infty}, \\ A\mathbf{u}^{(1)} &= \mathbf{x}^{(2)}, & \mathbf{u}^{(2)} &= \frac{\mathbf{x}^{(2)}}{\|\mathbf{x}^{(2)}\|_\infty}, \\ &\vdots \\ A\mathbf{u}^{(n)} &= \mathbf{x}^{(n+1)} \\ &\approx \lambda_1 \mathbf{u}^{(n)}. \end{aligned}$$

EXAMPLE 4.10. Using the power method, find the largest eigenvalue and the corresponding eigenvector of the matrix

$$\begin{bmatrix} 3 & 2 \\ 2 & 5 \end{bmatrix}.$$

SOLUTION. Letting $\mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, we have

$$\begin{aligned} \begin{bmatrix} 3 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} &= \begin{bmatrix} 5 \\ 7 \end{bmatrix} = \mathbf{x}^{(1)}, & \mathbf{u}^{(1)} &= \begin{bmatrix} 5/7 \\ 1 \end{bmatrix}, \\ \begin{bmatrix} 3 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 5/7 \\ 1 \end{bmatrix} &= \begin{bmatrix} 4.14 \\ 6.43 \end{bmatrix} = \mathbf{x}^{(2)}, & \mathbf{u}^{(2)} &= \begin{bmatrix} 0.644 \\ 1 \end{bmatrix}, \\ \begin{bmatrix} 3 & 2 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 0.644 \\ 1 \end{bmatrix} &= \begin{bmatrix} 3.933 \\ 6.288 \end{bmatrix} = \mathbf{x}^{(3)}, & \mathbf{u}^{(3)} &= \begin{bmatrix} 0.6254 \\ 1 \end{bmatrix}. \end{aligned}$$

Hence

$$\lambda_1 \approx 6.288, \quad \mathbf{x}_1 \approx \begin{bmatrix} 0.6254 \\ 1 \end{bmatrix}. \quad \square$$

Numeric Matlab has the command `eig` to find the eigenvalues and eigenvectors of a numeric matrix. For example

```
>> A = [3 2; 2 5];
>> [X,D] = eig(A)
X =
    0.8507    0.5257
   -0.5257    0.8507
D =
    1.7639         0
         0    6.2361
```

where the columns of the matrix X are the eigenvectors of A and the diagonal elements of the diagonal matrix D are the eigenvalues of A . The numeric command `eig` uses the QR algorithm with shifts to be described in Section 4.8.

4.6.3. The inverse power method. A more versatile method to determine any eigenvalue of a matrix $A \in \mathbb{R}^{n \times n}$, or $\mathbb{C}^{n \times n}$, is the *inverse power method*. It is derived as follows, under the simplifying assumption that A has n linearly independent eigenvectors $\mathbf{z}_1, \dots, \mathbf{z}_n$, and λ is near λ_1 .

We have

$$\begin{aligned} (A - \lambda I)\mathbf{x}^{(1)} &= \mathbf{x}^{(0)} = a_1\mathbf{z}_1 + \dots + a_n\mathbf{z}_n, \\ \mathbf{x}^{(1)} &= a_1 \frac{1}{\lambda_1 - \lambda} \mathbf{z}_1 + a_2 \frac{1}{\lambda_2 - \lambda} \mathbf{z}_2 + \dots + a_n \frac{1}{\lambda_n - \lambda} \mathbf{z}_n. \end{aligned}$$

Similarly, by recurrence,

$$\begin{aligned} \mathbf{x}^{(k)} &= a_1 \frac{1}{(\lambda_1 - \lambda)^k} \left[\mathbf{z}_1 + a_2 \left(\frac{\lambda_1 - \lambda}{\lambda_2 - \lambda} \right)^k \mathbf{z}_2 + \dots + a_n \left(\frac{\lambda_1 - \lambda}{\lambda_n - \lambda} \right)^k \mathbf{z}_n \right] \\ &\rightarrow a_1 \frac{1}{(\lambda_1 - \lambda)^k} \mathbf{z}_1, \quad \text{as } k \rightarrow \infty, \end{aligned}$$

since

$$\left| \frac{\lambda_1 - \lambda}{\lambda_j - \lambda} \right| < 1, \quad j \neq 1.$$

Thus, the sequence $\mathbf{x}^{(k)}$ converges in the direction of $\mathbf{z}_1$. In practice the vectors $\mathbf{x}^{(k)}$ are normalized and the system

$$(A - \lambda I)\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)}$$

is solved by the *LU* decomposition. The algorithm is as follows.

Choose $\mathbf{x}^{(0)}$
 For $k = 1, 2, 3, \dots$, do
 Solve
 $(A - \lambda I)\mathbf{y}^{(k)} = \mathbf{x}^{(k-1)}$ by the LU decomposition with partial pivoting.
 $\mathbf{x}^{(k)} = \mathbf{y}^{(k)} / \|\mathbf{y}^{(k)}\|_\infty$
 Stop if $\|(A - \lambda I)\mathbf{x}^{(k)}\|_\infty < c\epsilon\|A\|_\infty$, where c is a constant of order unity and ϵ is the machine epsilon.

4.7. The QR Decomposition

A very powerful method to solve ill-conditioned and overdetermined system

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{m \times n}, \quad m \geq n,$$

is the QR decomposition,

$$A = QR,$$

where Q is orthogonal, or unitary, and R is upper triangular. In this case,

$$\|A\mathbf{x} - \mathbf{b}\|_2 = \|QR\mathbf{x} - \mathbf{b}\|_2 = \|R\mathbf{x} - Q^T\mathbf{b}\|_2.$$

If A has full rank, that is, rank of A is equal to n , we can write

$$R = \begin{bmatrix} R_1 \\ 0 \end{bmatrix}, \quad Q^T\mathbf{b} = \begin{bmatrix} \mathbf{c} \\ \mathbf{d} \end{bmatrix},$$

where $R_1 \in \mathbb{R}^{n \times n}$, $0 \in \mathbb{R}^{(m-n) \times n}$, $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{d} \in \mathbb{R}^{m-n}$, and R_1 is upper triangular and non singular.

Then the least-square solution is

$$\mathbf{x} = R_1^{-1}\mathbf{c}$$

obtained by solving

$$R_1\mathbf{x} = \mathbf{c}$$

by backward substitution and the *residual* is

$$\rho = \min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2 = \|\mathbf{d}\|_2.$$

In the QR decomposition, the matrix A is transformed into an upper-triangular matrix by the successive application of $n - 1$ Householder reflections, the k th one zeroing the elements below the diagonal element in the k th column. For example, to zero the elements $x_2, x_3, \dots, x_n$ in the vector $\mathbf{x} \in \mathbb{R}^n$, one applies the Householder reflection

$$P = I - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}},$$

with

$$\mathbf{v} = \mathbf{x} + \text{sign}(x_1)\mathbf{e}_1, \quad \text{where } \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

In this case,

$$P \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} -\|\mathbf{x}\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

The matrix P is symmetric and orthogonal and it is equal to its own inverse, that is, it satisfies the relations

$$P^T = P = P^{-1}.$$

To minimize the number of floating point operations and memory allocation, the scalar

$$s = 2/\mathbf{v}^T \mathbf{v}$$

is first computed and then

$$P\mathbf{x} = \mathbf{x} - s(\mathbf{v}^T \mathbf{x})\mathbf{v}$$

is computed taking the special structure of the matrix P into account. To keep P in memory, only the number s and the vector $\mathbf{v}$ need be stored.

Softwares systematically use the QR decomposition to solve overdetermined systems. So does the Matlab left-division command `\` with an overdetermined or singular system.

The numeric Matlab command `qr` produces the QR decomposition of a matrix:

```
>> A = [1 2 3; 4 5 6; 7 8 9];
>> [Q,R] = qr(A)
Q =
   -0.1231    0.9045    0.4082
   -0.4924    0.3015   -0.8165
   -0.8616   -0.3015    0.4082
R =
   -8.1240   -9.6011  -11.0782
         0    0.9045    1.8091
         0         0   -0.0000
```

It is seen that the matrix A is singular since the diagonal element $r_{33} = 0$.

4.8. The QR algorithm

The QR algorithm uses a sequence of QR decompositions

$$\begin{aligned} A &= Q_1 R_1 \\ A_1 &= R_1 Q_1 = Q_2 R_2 \\ A_2 &= R_2 Q_2 = Q_3 R_3 \\ &\vdots \end{aligned}$$

to yield the eigenvalues of A , since A_n converges to an upper or quasi-upper triangular matrix with the real eigenvalues on the diagonal and complex eigenvalues in 2×2 diagonal blocks, respectively. Combined with simple shifts, double shifts, and other shifts, convergence is very fast.

For large matrices, of order $n \geq 100$, one seldom wants all the eigenvalues. To find selective eigenvalues, one may use Lanczos' method.

The Jacobi method to find the eigenvalues of a symmetric matrix is being revived since it is parallelizable for parallel computers.

4.9. The Singular Value Decomposition

The *singular value decomposition* is a very powerful tool in matrix computation. It is more expensive in time than the previous methods. Any matrix $A \in \mathbb{R}^{m \times n}$, say, with $m \geq n$, can be factored in the form

$$A = U\Sigma V^T,$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices and $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix, whose diagonal elements σ_i ordered in decreasing order

$$\sigma_1 \geq \sigma_2 \geq \cdots \sigma_n \geq 0,$$

are the singular values of A . If $A \in \mathbb{R}^{n \times n}$ is a square matrix, it is seen that

$$\|A\|_2 = \sigma_1, \quad \|A^{-1}\|_2 = \sigma_n.$$

The same decomposition holds for complex matrices $A \in \mathbb{C}^{m \times n}$. In this case U and V are unitary and the transpose V^T is replaced by the Hermitian transpose

$$V^* = \bar{V}^T.$$

The rank of a matrix A is the number of nonzero singular values of A .

The numeric Matlab command `svd` produces the singular values of a matrix:

```
A = [1 2 3; 4 5 6; 7 8 9];
[U,S,V] = svd(A)
U =
    0.2148    0.8872   -0.4082
    0.5206    0.2496    0.8165
    0.8263   -0.3879   -0.4082
S =
   16.8481         0         0
         0    1.0684         0
         0         0    0.0000

V =

    0.4797   -0.7767    0.4082
    0.5724   -0.0757   -0.8165
    0.6651    0.6253    0.4082
```

The diagonal elements of the matrix S are the singular values of A . The l_2 norm of A is $\|A\|_2 = \sigma_1 = 16.8481$. Since $\sigma_3 = 0$, the matrix A is singular.

If A is symmetric, $A^T = A$, Hermitian symmetric $A^H = A$ or, more generally, *normal*, $AA^H = A^HA$, then the moduli of the eigenvalues of A are the singular values of A .

THEOREM 4.7 (Schur Decomposition). *Any square matrix A admits the Schur decomposition*

$$A = UTU^H,$$

where the diagonal elements of the upper triangular matrix T are the eigenvalues of A and the matrix U is unitary.

For normal matrices, the matrix T of the Schur decomposition is diagonal.

THEOREM 4.8. *A matrix A is normal if and only if it admits the Schur decomposition*

$$A = UDU^H,$$

where the diagonal matrix D contains the eigenvalues of A and the columns of the unitary matrix U are the eigenvectors of A .

CHAPTER 5

Numerical Solution of Differential Equations

5.1. Initial Value Problems

Consider the first-order initial value problem:

$$y' = f(x, y), \quad y(x_0) = y_0. \quad (5.1)$$

To find an approximation to the solution $y(x)$ of (5.1) on the interval $a \leq x \leq b$, we choose N points, $a = x_0 < x_1 < x_2 < \dots < x_N = b$, and construct approximations y_n to $y(x_n)$, $n = 0, 1, \dots, N$.

It is important to know whether or not a *small perturbation* of (5.1) shall lead to a *large variation* in the solution. If this is the case, it is extremely unlikely that we will be able to find a good approximation to (5.1). Truncation errors, which occur when computing $f(x, y)$ and evaluating the initial condition, can be identified with perturbations of (5.1). The following theorem gives sufficient conditions for an initial value problem to be **well-posed**.

DEFINITION 5.1. Problem (5.1) is said to be well posed in the sense of Hadamard if it has one, and only one, solution and any small perturbation of the problem leads to a correspondingly small change in the solution.

THEOREM 5.1. *Let*

$$D = \{(x, y) : a \leq x \leq b \text{ and } -\infty < y < \infty\}.$$

*If $f(x, y)$ is continuous on D and satisfies the **Lipschitz condition***

$$|f(x, y_1) - f(x, y_2)| \leq L|y_1 - y_2| \quad (5.2)$$

for all (x, y_1) and (x, y_2) in D , where L is the Lipschitz constant, then the initial value problem (5.1) is well-posed.

In the sequel, we shall assume that the conditions of Theorem 5.1 hold and (5.1) is well posed. Moreover, we shall suppose that $f(x, y)$ has mixed partial derivatives of arbitrary order.

In considering numerical methods for the solution of (5.1) we shall use the following notation:

- $h > 0$ denotes the integration *step size*
- $x_n = x_0 + nh$ is the n -th *node*
- $y(x_n)$ is the *exact solution* at x_n
- y_n is the *numerical solution* at x_n
- $f_n = f(x_n, y_n)$ is the numerical value of $f(x)$ at (x_n, y_n)

A function, $g(x)$, is said to be of *order p* as $x \rightarrow x_0$, written $g \in O(|x - x_0|^p)$ if

$$|g(x)| < M|x - x_0|^p, \quad M \text{ a constant,}$$

for all x near x_0 .

5.2. Euler's and Improved Euler's Method

We begin with the simplest explicit methods.

5.2.1. Euler's method. We choose N points, $x_n = x_0 + nh$ where $h = (x_f - x_0)/N$. From Taylor's Theorem we get

$$y(x_{n+1}) = y(x_n) + y'(x_n)(x_{n+1} - x_n) + \frac{y''(\xi_n)}{2}(x_{n+1} - x_n)^2$$

for ξ_n between x_n and x_{n+1} , $n = 0, 1, \dots, N-1$. Since $y'(x_n) = f(x_n, y(x_n))$ and $x_{n+1} - x_n = h$, it follows that

$$y(x_{n+1}) = y(x_n) + f(x_n, y(x_n))h + \frac{y''(\xi_n)}{2}h^2.$$

We obtain Euler's method,

$$y_{n+1} = y_n + hf(x_n, y_n), \quad (5.3)$$

by deleting the term of order $O(h^2)$,

$$\frac{y''(\xi_n)}{2}h^2,$$

called the **local truncation error**.

The algorithm for **Euler's method** is as follows.

- (1) Choose h such that $N = (x_f - x_0)/h$ is an integer.
- (2) Given y_0 , for $n = 0, 1, \dots, N$, iterate the scheme

$$y_{n+1} = y_n + hf(x_0 + nh, y_n). \quad (5.4)$$

Then, y_n is as an approximation to $y(x_n)$.

EXAMPLE 5.1. Use Euler's method with $h = 0.1$ to approximate the solution to the initial value problem

$$y'(x) = 0.2xy, \quad y(1) = 1, \quad (5.5)$$

on the interval $1 \leq x \leq 1.5$.

SOLUTION. We have

$$x_0 = 1, \quad x_f = 1.5, \quad y_0 = 1, \quad f(x, y) = 0.2xy.$$

Hence

$$x_n = x_0 + hn = 1 + 0.1n, \quad N = \frac{1.5 - 1}{0.1} = 5,$$

and

$$y_{n+1} = y_n + 0.1 \times 0.2(1 + 0.1n)y_n, \quad \text{with } y_0 = 1,$$

for $n = 0, 1, \dots, 4$. The numerical results are listed in Table 5.1. Note that the differential equation in (5.5) is separable. The (unique) solution of (5.5) is

$$y(x) = e^{(0.1x^2 - 0.1)}.$$

This formula has been used to compute the exact values $y(x_n)$ in the previous table. $\square$

The next example illustrates the limitations of Euler's method. In the next subsections, we shall see more accurate methods than Euler's method.

TABLE 5.1. Numerical results of Example 5.1.

n	x_n	y_n	$y(x_n)$	Absolute error	Relative error
0	1.00	1.0000	1.0000	0.0000	0.00
1	1.10	1.0200	1.0212	0.0012	0.12
2	1.20	1.0424	1.0450	0.0025	0.24
3	1.30	1.0675	1.0714	0.0040	0.37
4	1.40	1.0952	1.1008	0.0055	0.50
5	1.50	1.1259	1.1331	0.0073	0.64

TABLE 5.2. Numerical results of Example 5.2.

n	x_n	y_n	$y(x_n)$	Absolute error	Relative error
0	1.00	1.0000	1.0000	0.0000	0.00
1	1.10	1.2000	1.2337	0.0337	2.73
2	1.20	1.4640	1.5527	0.0887	5.71
3	1.30	1.8154	1.9937	0.1784	8.95
4	1.40	2.2874	2.6117	0.3244	12.42
5	1.50	2.9278	3.4904	0.5625	16.12

EXAMPLE 5.2. Use Euler's method with $h = 0.1$ to approximate the solution to the initial value problem

$$y'(x) = 2xy, \quad y(1) = 1, \quad (5.6)$$

on the interval $1 \leq x \leq 1.5$.

SOLUTION. As in the previous example, we have

$$x_0 = 1, \quad x_f = 1.5, \quad y_0 = 1, \quad x_n = x_0 + hn = 1 + 0.1n, \quad N = \frac{1.5 - 1}{0.1} = 5,$$

However, $f(x, y) = 2xy$. Thus, Euler's method is

$$y_{n+1} = y_n + 0.1 \times 2(1 + 0.1n)y_n, \quad y_0 = 1,$$

for $n = 0, 1, 2, 3, 4$. The numerical results are listed in Table 5.2. The relative errors show that our approximations are not very good. $\square$

DEFINITION 5.2. The **local truncation error** of a method of the form

$$y_{n+1} = y_n + h \phi(x_n, y_n), \quad (5.7)$$

is dedfined by the expression

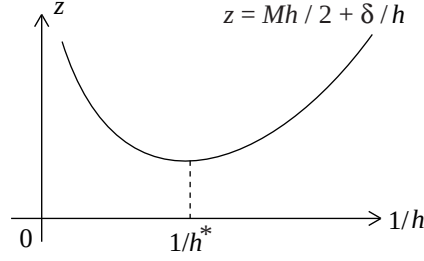
$$\tau_{n+1} = \frac{1}{h} [y(x_{n+1}) - y(x_n)] - \phi(x_n, y(x_n)) \quad \text{for } n = 0, 1, 2, \dots, N-1.$$

The method (5.7) is of **order** k if $|\tau_j| \leq M h^k$ for some constant M and for all j .

An equivalent definition is found in Section 5.4

EXAMPLE 5.3. The local truncation error of Euler's method is

$$\tau_{n+1} = \frac{1}{h} [y(x_{n+1}) - y(x_n)] - f(x_n, y(x_n)) = \frac{h}{2} y''(\xi_n)$$

FIGURE 5.1. Truncation and roundoff error curve as a function of $1/h$.

for some ξ_n between x_n and x_{n+1} . If

$$M = \max_{x_0 \leq x \leq x_f} |y''(x)|,$$

then $|\tau_n| \leq \frac{h}{2} M$ for all n . Hence, Euler's method is of order one.

REMARK 5.1. It is generally incorrect to say that by taking h sufficiently small one can obtain any desired level of precision, that is, get y_n as close to $y(x_n)$ as one wants. As the step size h decreases, at first the truncation error of the method decreases, but as the number of steps increases, the number of arithmetic operations increases, and, hence, the roundoff errors increase as shown in Fig. 5.1.

For instance, let y_n be the computed value for $y(x_n)$ in (5.4). Set

$$e_n = y(x_n) - y_n, \quad \text{for } n = 0, 1, \dots, N.$$

If

$$|e_0| < \delta_0$$

and the precision in the computations is bounded by δ , then it can be shown that

$$|e_n| \leq \frac{1}{L} \left(\frac{Mh}{2} + \frac{\delta}{h} \right) (e^{L(x_n - x_0)} - 1) + \delta_0 e^{L(x_n - x_0)},$$

where L is the Lipschitz constant defined in Theorem 5.1,

$$M = \max_{x_0 \leq x \leq x_f} |y''(x)|,$$

and $h = (x_f - x_0)/N$.

We remark that the expression

$$z(h) = \frac{Mh}{2} + \frac{\delta}{h}$$

first decreases and afterwards increases as $1/h$ increases, as shown in Fig. 5.1. The term $Mh/2$ is due to the truncation error and the term δ/h is due to the roundoff errors.

TABLE 5.3. Numerical results of Example 5.4.

n	x_n	y_n^P	y_n^C	$y(x_n)$	Absolute error	Relative error
0	1.00		1.0000	1.0000	0.0000	0.00
1	1.10	1.200	1.2320	1.2337	0.0017	0.14
2	1.20		1.5479	1.5527	0.0048	0.31
3	1.30		1.9832	1.9937	0.0106	0.53
4	1.40		2.5908	2.6117	0.0209	0.80
5	1.50		3.4509	3.4904	0.0344	1.13

5.2.2. Improved Euler's method. The improved Euler's method takes the average of the slopes at the left and right ends of each step. It is, here, formulated in terms of a predictor and a corrector:

$$y_{n+1}^P = y_n^C + hf(x_n, y_n^C),$$

$$y_{n+1}^C = y_n^C + \frac{1}{2}h [f(x_n, y_n^C) + f(x_{n+1}, y_{n+1}^P)].$$

This method is of order 2.

EXAMPLE 5.4. Use the improved Euler method with $h = 0.1$ to approximate the solution to the initial value problem of Example 5.2.

$$y'(x) = 2xy, \quad y(1) = 1,$$

$$1 \leq x \leq 1.5.$$

SOLUTION. We have

$$x_n = x_0 + hn = 1 + 0.1n, \quad n = 0, 1, \dots, 5.$$

The approximation y_n to $y(x_n)$ is given by the predictor-corrector scheme

$$y_0^C = 1,$$

$$y_{n+1}^P = y_n^C + 0.2x_n y_n,$$

$$y_{n+1}^C = y_n^C + 0.1(x_n y_n^C + x_{n+1} y_{n+1}^P)$$

for $n = 0, 1, \dots, 4$. The numerical results are listed in Table 5.3. These results are much better than those listed in Table 5.2 for Euler's method. $\square$

We need to develop methods of order greater than one, which, in general, are more precise than Euler's method.

5.3. Low-Order Explicit Runge-Kutta Methods

Runge-Kutta methods are one-step multistage methods.

5.3.1. Second-order Runge-Kutta method. Two-stage explicit Runge-Kutta methods are given by the formula (left) and, conveniently, in the form of a Butcher tableau (right):

$$k_1 = hf(x_n, y_n)$$

$$k_2 = hf(x_n + c_2h, y_n + a_{21}k_1)$$

$$y_{n+1} = y_n + b_1k_1 + b_2k_2$$

	$\mathbf{c}$	A
k_1	0	0
k_2	c_2	a_{21} 0
y_{n+1}	$\mathbf{b}^T$	b_1 b_2

In a Butcher tableau, the components of the vector $\mathbf{c}$ are the increments of x_n and the entries of the matrix A are the multipliers of the approximate slopes which, after multiplication by the step size h , increments y_n . The components of the vector $\mathbf{b}$ are the weights in the combination of the intermediary values k_j . The left-most column of the tableau is added here for the reader's convenience.

To attain second order, $\mathbf{c}$, A and $\mathbf{b}$ have to be chosen judiciously. We proceed to derive two-stage second-order Runge-Kutta methods.

By Taylor's Theorem, we have

$$y(x_{n+1}) = y(x_n) + y'(x_n)(x_{n+1} - x_n) + \frac{1}{2}y''(x_n)(x_{n+1} - x_n)^2 + \frac{1}{6}y'''(\xi_n)(x_{n+1} - x_n)^3 \quad (5.8)$$

for some ξ_n between x_n and x_{n+1} and $n = 0, 1, \dots, N-1$. From the differential equation

$$y'(x) = f(x, y(x)),$$

and its first total derivative with respect to x , we obtain expressions for $y'(x_n)$ and $y''(x_n)$,

$$\begin{aligned} y'(x_n) &= f(x_n, y(x_n)), \\ y''(x_n) &= \frac{d}{dx} f(x, y(x)) \big|_{x=x_n} \\ &= f_x(x_n, y(x_n)) + f_y(x_n, y(x_n)) f(x_n, y(x_n)). \end{aligned}$$

Therefore, putting $h = x_{n+1} - x_n$ and substituting these expressions in (5.8), we have

$$\begin{aligned} y(x_{n+1}) &= y(x_n) + f(x_n, y(x_n)) h \\ &\quad + \frac{1}{2} [f_x(x_n, y(x_n)) + f_y(x_n, y(x_n)) f(x_n, y(x_n))] h^2 \\ &\quad + \frac{1}{6} y'''(\xi_n) h^3 \end{aligned} \quad (5.9)$$

for $n = 0, 1, \dots, N-1$.

Our goal is to replace the expression

$$f(x_n, y(x_n))h + \frac{1}{2} [f_x(x_n, y(x_n)) + f_y(x_n, y(x_n)) f(x_n, y(x_n))] h + O(h^2)$$

by an expression of the form

$$af(x_n, y(x_n))h + bf(x_n + \alpha h, y(x_n) + \beta h f(x_n, y(x_n)))h + O(h^2). \quad (5.10)$$

The constants a , b , α and β are to be determined. This last expression is simpler to evaluate than the previous one since it does not involve partial derivatives.

Using Taylor's Theorem for functions of two variables, we get

$$\begin{aligned} f(x_n + \alpha h, y(x_n) + \beta h f(x_n, y(x_n))) &= f(x_n, y(x_n)) + \alpha h f_x(x_n, y(x_n)) \\ &\quad + \beta h f_y(x_n, y(x_n)) f(x_n, y(x_n)) + O(h^2). \end{aligned}$$

In order for the expressions (5.8) and (5.9) to be equal to order h , we must have

$$a + b = 1, \quad \alpha b = 1/2, \quad \beta b = 1/2.$$

Thus, we have three equations in four unknowns. This gives rise to a one-parameter family of solutions. Identifying the parameters:

$$c_1 = \alpha, \quad a_{21} = \beta, \quad b_1 = a, \quad b_2 = b,$$

we obtain second-order Runge-Kutta methods.

Here are some two-stage second-order Runge-Kutta methods.

The improved Euler's method can be written in the form of a two-stage explicit Runge-Kutta method (left) with Butcher tableau (right):

$$\begin{aligned} k_1 &= hf(x_n, y_n) \\ k_2 &= hf\left(x_n + h, y_n + k_1\right) \\ y_{n+1} &= y_n + \frac{1}{2}(k_1 + k_2) \end{aligned} \quad \begin{array}{c|cc} & \mathbf{c} & A \\ \hline k_1 & 0 & 0 \\ k_2 & 1 & 1 \quad 0 \\ \hline y_{n+1} & \mathbf{b}^T & 1/2 \quad 1/2 \end{array}$$

This is Heun's method of order 2.

Other two-stage second-order methods are the mid-point method:

$$\begin{aligned} k_1 &= hf(x_n, y_n) \\ k_2 &= hf\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1\right) \\ y_{n+1} &= y_n + k_2 \end{aligned} \quad \begin{array}{c|cc} & \mathbf{c} & A \\ \hline k_1 & 0 & 0 \\ k_2 & 1/2 & 1/2 \quad 0 \\ \hline y_{n+1} & \mathbf{b}^T & 0 \quad 1 \end{array}$$

and Heun's method:

$$\begin{aligned} k_1 &= hf(x_n, y_n) \\ k_2 &= hf\left(x_n + \frac{2}{3}h, y_n + \frac{2}{3}k_1\right) \\ y_{n+1} &= y_n + \frac{1}{4}k_1 + \frac{3}{4}k_2 \end{aligned} \quad \begin{array}{c|cc} & \mathbf{c} & A \\ \hline k_1 & 0 & 0 \\ k_2 & 2/3 & 2/3 \quad 0 \\ \hline y_{n+1} & \mathbf{b}^T & 1/4 \quad 3/4 \end{array}$$

5.3.2. Third-order Runge-Kutta method. We list two common three-stage third-order Runge-Kutta methods in their Butcher tableau, namely Heun's third-order formula and Kutta's third-order rule.

$$\begin{array}{c|cc} & \mathbf{c} & A \\ \hline k_1 & 0 & 0 \\ k_2 & 1/3 & 1/3 \quad 0 \\ k_3 & 2/3 & 0 \quad 2/3 \quad 0 \\ \hline y_{n+1} & \mathbf{b}^T & 1/4 \quad 0 \quad 3/4 \end{array}$$

Butcher tableau of Heun's third-order formula.

$$\begin{array}{c|cc} & \mathbf{c} & A \\ \hline k_1 & 0 & 0 \\ k_2 & 1/2 & 1/2 \quad 0 \\ k_3 & 1 & -1 \quad 2 \quad 0 \\ \hline y_{n+1} & \mathbf{b}^T & 1/6 \quad 2/3 \quad 1/6 \end{array}$$

Butcher tableau of Kutta's third-order rule.

5.3.3. Fourth-order Runge–Kutta method. The fourth-order (classic) Runge–Kutta method (also known as the classic Runge–Kutta method) is the very popular among the explicit one-step methods.

By Taylor’s Theorem, we have

$$y(x_{n+1}) = y(x_n) + y'(x_n)(x_{n+1} - x_n) + \frac{y''(x_n)}{2!}(x_{n+1} - x_n)^2 + \frac{y^{(3)}(x_n)}{3!}(x_{n+1} - x_n)^3 + \frac{y^{(4)}(x_n)}{4!}(x_{n+1} - x_n)^4 + \frac{y^{(5)}(\xi_n)}{5!}(x_{n+1} - x_n)^5$$

for some ξ_n between x_n and x_{n+1} and $n = 0, 1, \dots, N - 1$. To obtain the fourth-order Runge–Kutta method, we can proceed as we did for the second-order Runge–Kutta methods. That is, we seek values of a, b, c, d, α_j and β_j such that

$$y'(x_n)(x_{n+1} - x_n) + \frac{y''(x_n)}{2!}(x_{n+1} - x_n)^2 + \frac{y^{(3)}(x_n)}{3!}(x_{n+1} - x_n)^3 + \frac{y^{(4)}(x_n)}{4!}(x_{n+1} - x_n)^4 + O(h^5)$$

is equal to

$$ak_1 + bk_2 + ck_3 + dk_4 + O(h^5),$$

where

$$\begin{aligned} k_1 &= hf(x_n, y_n), \\ k_2 &= hf(x_n + \alpha_1 h, y_n + \beta_1 k_1), \\ k_3 &= hf(x_n + \alpha_2 h, y_n + \beta_2 k_2), \\ k_4 &= hf(x_n + \alpha_3 h, y_n + \beta_3 k_3). \end{aligned}$$

This follows from the relations

$$\begin{aligned} x_{n+1} - x_n &= h, \\ y'(x_n) &= f(x_n, y(x_n)), \\ y''(x_n) &= \frac{d}{dx} f(x, y(x))|_{t=x_n} \\ &= f_x(x_n, y(x_n)) + f_y(x_n, y(x_n)) f(x_n, y(x_n)), \dots, \end{aligned}$$

and Taylor’s Theorem for functions of two variables. The lengthy computation is omitted.

The (classic) four-stage Runge–Kutta method of order 4 given by its formula (left) and, conveniently, in the form of a Butcher tableau (right).

$$\begin{aligned} k_1 &= hf(x_n, y_n) \\ k_2 &= hf\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_1\right) \\ k_3 &= hf\left(x_n + \frac{1}{2}h, y_n + \frac{1}{2}k_2\right) \\ k_4 &= hf(x_n + h, y_n + k_3) \\ y_{n+1} &= y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

	c	A			
k_1	0	0			
k_2	1/2	1/2	0		
k_3	1/2	0	1/2	0	
k_4	1	0	0	1	0
y_{n+1}	b^T	1/6	2/6	2/6	1/6

TABLE 5.4. Numerical results for Example 5.5.

x_n	y_n	$y(x_n)$	Absolute error	Relative error
1.00	1.0000	1.0000	0.0000	0.0
1.10	1.2337	1.2337	0.0000	0.0
1.20	1.5527	1.5527	0.0000	0.0
1.30	1.9937	1.9937	0.0000	0.0
1.40	2.6116	2.6117	0.0001	0.0
1.50	3.4902	3.4904	0.0002	0.0

The next example shows that the fourth-order Runge–Kutta method yields better results for (5.6) than the previous methods.

EXAMPLE 5.5. Use the fourth-order Runge–Kutta method with $h = 0.1$ to approximate the solution to the initial value problem of Example 5.2,

$$y'(x) = 2xy, \quad y(1) = 1,$$

on the interval $1 \leq x \leq 1.5$.

SOLUTION. We have $f(x, y) = 2xy$ and

$$x_n = 1.0 + 0.1n, \quad \text{for } n = 0, 1, \dots, 5.$$

With the starting value $y_0 = 1.0$, the approximation y_n to $y(x_n)$ is given by the scheme

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

where

$$k_1 = 0.1 \times 2(1.0 + 0.1n)y_n,$$

$$k_2 = 0.1 \times 2(1.05 + 0.1n)(y_n + k_1/2),$$

$$k_3 = 0.1 \times 2(1.05 + 0.1n)(y_n + k_2/2),$$

$$k_4 = 0.1 \times 2(1.0 + 0.1(n+1))(y_n + k_3),$$

and $n = 0, 1, 2, 3, 4$. The numerical results are listed in Table 5.4. These results are much better than all those previously obtained. $\square$

EXAMPLE 5.6. Consider the initial value problem

$$y' = (y - x - 1)^2 + 2, \quad y(0) = 1.$$

Compute y_4 by means of Runge–Kutta’s method of order 4 with step size $h = 0.1$.

SOLUTION. The solution is given in tabular form.

n	x_n	y_n	Exact value $y(x_n)$	Global error $y(x_n) - y_n$
0	0.0	1.000 000 000	1.000 000 000	0.000 000 000
1	0.1	1.200 334 589	1.200 334 672	0.000 000 083
2	0.2	1.402 709 878	1.402 710 036	0.000 000 157
3	0.3	1.609 336 039	1.609 336 250	0.000 000 181
4	0.4	1.822 792 993	1.822 793 219	0.000 000 226

$\square$

EXAMPLE 5.7. Use the Runge–Kutta method of order 4 with $h = 0.01$ to obtain a six-decimal approximation for the initial value problem

$$y' = x + \arctan y, \quad y(0) = 0,$$

on $0 \leq x \leq 1$. Print every tenth value and plot the numerical solution.

SOLUTION. **The Matlab numeric solution.**— The M-file `exp5_7` for Example 5.7 is

```
function yprime = exp5_7(x,y); % Example 5.7.
yprime = x+atan(y);
```

The Runge–Kutta method of order 4 is applied to the given differential equation:

```
clear
h = 0.01; x0= 0; xf= 1; y0 = 0;
n = ceil((xf-x0)/h); % number of steps
%
count = 2; print_time = 10; % when to write to output
x = x0; y = y0; % initialize x and y
output = [0 x0 y0];
for i=1:n
    k1 = h*exp5_7(x,y);
    k2 = h*exp5_7(x+h/2,y+k1/2);
    k3 = h*exp5_7(x+h/2,y+k2/2);
    k4 = h*exp5_7(x+h,y+k3);
    z = y + (1/6)*(k1+2*k2+2*k3+k4);
    x = x + h;
    if count > print_time
        output = [output; i x z];
        count = count - print_time;
    end
    y = z;
count = count + 1;
end
output
save output %for printing the graph
```

The command `output` prints the values of n , x , and y .

n	x	y
0	0	0
10.0000	0.1000	0.0052
20.0000	0.2000	0.0214
30.0000	0.3000	0.0499
40.0000	0.4000	0.0918
50.0000	0.5000	0.1486
60.0000	0.6000	0.2218
70.0000	0.7000	0.3128
80.0000	0.8000	0.4228
90.0000	0.9000	0.5531

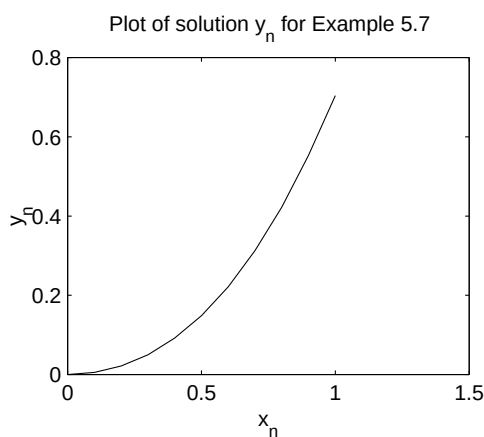


FIGURE 5.2. Graph of numerical solution of Example 5.7.

```
100.0000    1.0000    0.7040
```

The following commands print the output.

```
load output;
subplot(2,2,1); plot(output(:,2),output(:,3));
title('Plot of solution y_n for Example 5.7');
xlabel('x_n'); ylabel('y_n');
```

□

In the next example, the Runge–Kutta method of order 4 is used to solve the van der Pol system of two equations. This system is also solved by means of the Matlab `ode23` code and the graphs of the two solutions are compared.

EXAMPLE 5.8. Use the Runge–Kutta method of order 4 with fixed step size $h = 0.1$ to solve the second-order van der Pol equation

$$y'' + (y^2 - 1)y' + y = 0, \quad y(0) = 0, \quad y'(0) = 0.25, \quad (5.11)$$

on $0 \leq x \leq 20$, print every tenth value, and plot the numerical solution. Also, use the `ode23` code to solve (5.11) and plot the solution.

SOLUTION. We first rewrite problem (5.11) as a system of two first-order differential equations by putting $y_1 = y$ and $y_2 = y'_1$,

$$\begin{aligned} y'_1 &= y_2, \\ y'_2 &= y_2(1 - y_1^2) - y_1, \end{aligned}$$

with initial conditions $y_1(0) = 0$ and $y_2(0) = 0.25$.

Our MATLAB program will call the MATLAB function M-file `exp1vdp.m`:

```
function yprime = exp1vdp(t,y); % Example 5.8.
yprime = [y(2); y(2).*(1-y(1).^2)-y(1)]; % van der Pol system
```

The following program applies the Runge–Kutta method of order 4 to the differential equation defined in the M-file `exp1vdp.m`:

```

clear
h = 0.1; t0= 0; tf= 21; % step size, initial and final times
y0 = [0 0.25]'; % initial conditions
n = ceil((xf-t0)/h); % number of steps

count = 2; print_control = 10; % when to write to output
t = t0; y = y0; % initialize t and y
output = [t0 y0']; % first row of matrix of printed values
w = [t0, y0']; % first row of matrix of plotted values
for i=1:n
    k1 = h*exp1vdp(x,y);          k2 = h*exp1vdp(x+h/2,y+k1/2);
    k3 = h*exp1vdp(x+h/2,y+k2/2); k4 = h*exp1vdp(x+h,y+k3);
    z = y + (1/6)*(k1+2*k2+2*k3+k4);
    t = t + h;
    if count > print_control
        output = [output; t z']; % augmenting matrix of printed values
        count = count - print_control;
    end
    y = z;
    w = [w; t z']; % augmenting matrix of plotted values
    count = count + 1;
end
[output(1:11,:) output(12:22,:)] % print numerical values of solution
save w % save matrix to plot the solution

```

The command output prints the values of t , y_1 , and y_2 .

t	y(1)	y(2)	t	y(1)	y(2)
0	0	0.2500	11.0000	-1.9923	-0.2797
1.0000	0.3586	0.4297	12.0000	-1.6042	0.7195
2.0000	0.6876	0.1163	13.0000	-0.5411	1.6023
3.0000	0.4313	-0.6844	14.0000	1.6998	1.6113
4.0000	-0.7899	-1.6222	15.0000	1.8173	-0.5621
5.0000	-1.6075	0.1456	16.0000	0.9940	-1.1654
6.0000	-0.9759	1.0662	17.0000	-0.9519	-2.6628
7.0000	0.8487	2.5830	18.0000	-1.9688	0.3238
8.0000	1.9531	-0.2733	19.0000	-1.3332	0.9004
9.0000	1.3357	-0.8931	20.0000	0.1068	2.2766
10.0000	-0.0939	-2.2615	21.0000	1.9949	0.2625

The following commands graph the solution.

```

load w % load values to produce the graph
subplot(2,2,1); plot(w(:,1),w(:,2)); % plot RK4 solution
title('RK4 solution y_n for Example 5.8'); xlabel('t_n'); ylabel('y_n');

```

We now use the ode23 code. The command

```

load w % load values to produce the graph
v = [0 21 -3 3]; % set t and y axes

```

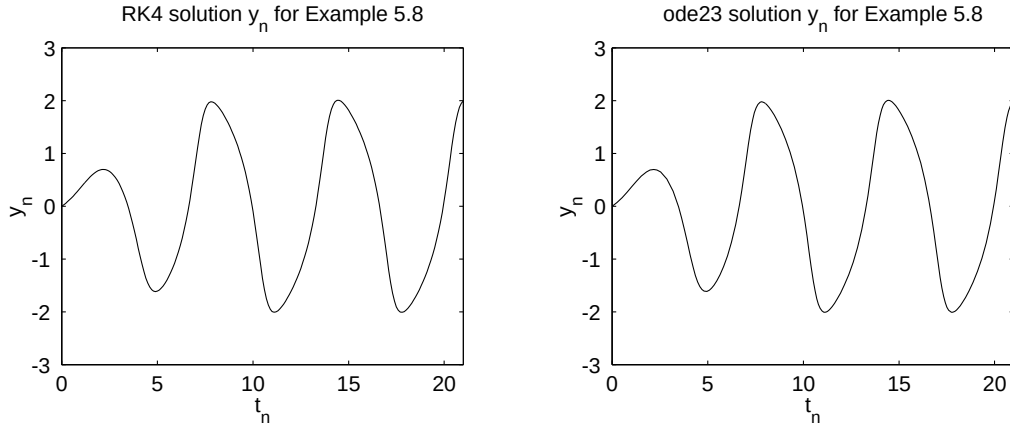


FIGURE 5.3. Graph of numerical solution of Example 5.8.

```

subplot(2,2,1);
plot(w(:,1),w(:,2)); % plot RK4 solution
axis(v);
title('RK4 solution y_n for Example 5.8'); xlabel('t_n'); ylabel('y_n');
subplot(2,2,2);
[t,y] = ode23('exp1vdp',[0 21], y0);
plot(x,y(:,1)); % plot ode23 solution
axis(v);
title('ode23 solution y_n for Example 5.8'); xlabel('t_n'); ylabel('y_n');

```

The code `ode23` produces three vectors, namely `t` of (144 unequally-spaced) nodes and corresponding solution values `y(1)` and `y(2)`, respectively. The left and right parts of Fig. 3.3 show the plots of the solutions obtained by `Rk4` and `ode23`, respectively. It is seen that the two graphs are identical. $\square$

5.4. Convergence of Numerical Methods

In this and the next sections, we introduce the concepts of convergence, consistency and stability of numerical ode solvers.

The numerical methods considered in this chapter can be written in the general form

$$\sum_{n=0}^k \alpha_j y_{n+j} = h \varphi_f(y_{n+k}, y_{n+k-1}, \dots, y_n, x_n; h). \quad (5.12)$$

where the subscript f to φ indicates the dependance of φ on the function $f(x, y)$ of (5.1). We impose the condition that

$$\varphi_{f \equiv 0}(y_{n+k}, y_{n+k-1}, \dots, y_n, x_n; h) \equiv 0,$$

and note that the Lipschitz continuity of φ with respect to y_{n+j} , $n = 0, 1, \dots, k$, follows from the Lipschitz continuity (5.2) of f .

DEFINITION 5.3. Method (5.12) with appropriate starting values is said to be **convergent** if, for all initial value problems (5.1), we have

$$y_n - y(x_n) \rightarrow 0 \quad \text{as } h \downarrow 0,$$

where $nh = x$ for all $x \in [a, b]$.

The **local truncation error** of (5.12) is the residual

$$R_{n+k} := \sum_{j=0}^k \alpha_j y(x_{n+j}) - h\varphi_f(y(x_{n+k}), y(x_{n+k-1}), \dots, y(x_n), x_n; h). \quad (5.13)$$

DEFINITION 5.4. Method (5.12) with appropriate starting values is said to be **consistent** if, for all initial value problems (5.1), we have

$$\frac{1}{h} R_{n+k} \rightarrow 0 \quad \text{as } h \downarrow 0,$$

where $nh = x$ for all $x \in [a, b]$.

DEFINITION 5.5. Method (5.12) is **zero-stable** if the roots of the characteristic polynomial

$$\sum_{j=0}^k \alpha_j r^{n+j}$$

lie inside or on the boundary of the unit disk, and those on the unit circle are simple.

We finally can state the following fundamental theorem.

THEOREM 5.2. *A method is **convergent** as $h \downarrow 0$ if and only if it is zero-stable and consistent.*

All numerical methods considered in this chapter are convergent.

5.5. Absolutely Stable Numerical Methods

We now turn attention to the application of a consistent and zero-stable numerical solver with small but nonvanishing step size.

For $n = 0, 1, 2, \dots$, let y_n be the numerical solution of (5.1) at $x = x_n$, and $y^{[n]}(x_{n+1})$ be the exact solution of the **local** problem:

$$y' = f(x, y), \quad y(x_n) = y_n. \quad (5.14)$$

A numerical method is said to have **local error**,

$$\varepsilon_{n+1} = y_{n+1} - y^{[n]}(x_{n+1}). \quad (5.15)$$

If we assume that $y(x) \in C^{p+1}[x_0, x_f]$ and

$$\varepsilon_{n+1} \approx C_{p+1} h_{n+1}^{p+1} y^{(p+1)}(x_n) + O(h_{n+1}^{p+2}), \quad (5.16)$$

then we say that the local error is of order $p+1$ and C_{p+1} is the error constant of the method. For consistent and zero-stable methods, the global error is of order p whenever the local error is of order $p+1$. In such case, we say that the method is of order p . We remark that a method of order $p \geq 1$ is consistent according to Definition 5.4.

Let us now apply the solver (5.12), with its small nonvanishing parameter h , to the linear test equation

$$y' = \lambda y, \quad \Re \lambda < 0. \quad (5.17)$$

The **region of absolute stability**, R , is that region in the complex $\hat{h}$ -plane, where $\hat{h} = h\lambda$, for which the numerical solution y_n of (5.17) goes to zero, as n goes to infinity.

The region of absolute stability of the explicit Euler method is the disk of radius 1 and center $(-1, 0)$, see curve $k = 1$ in Fig. 5.7. The region of stability of the implicit backward Euler method is the outside of the disk of radius 1 and center $(1, 0)$, hence it contains the left half-plane, see curve $k = 1$ in Fig. 5.10.

The region of absolute stability, R , of an explicit method is very roughly a disk or cardioid in the left half-plane (the cardioid overlaps with the right half-plane with a cusp at the origin). The boundary of R cuts the real axis at α , where $-\infty < \alpha < 0$, and at the origin. The interval $[\alpha, 0]$ is called the **interval of absolute stability**. For methods with real coefficients, R is symmetric with respect to the real axis. All methods considered in this work have real coefficients; hence Figs. 5.7, 5.8 and 5.10, below, show only the upper half of R .

The region of stability, R , of implicit methods extends to infinity in the left half-plane, that is $\alpha = -\infty$. The angle subtended at the origin by R in the left half-plane is usually smaller for higher order methods, see Fig. 5.10.

If the region R does not include the whole negative real axis, that is, $-\infty < \alpha < 0$, then the inclusion

$$h\lambda \in R$$

restricts the step size:

$$\alpha \leq h \operatorname{Re} \lambda \implies 0 < h \leq \frac{\alpha}{\operatorname{Re} \lambda}.$$

In practice, we want to use a step size h small enough to ensure accuracy of the numerical solution as implied by (5.15)–(5.16), but not too small.

5.6. Stability of Runge–Kutta methods

There are stable s -stage explicit Runge–Kutta methods of order $p = s$ for $s = 1, 2, 3, 4$. The minimal number of stages of a stable explicit Runge–Kutta method of order 5 is 6.

Applying a Runge–Kutta method to the test equation,

$$y' = \lambda y, \quad \Re \lambda < 0,$$

with solution $y(x) \rightarrow 0$ as $t \rightarrow \infty$, one obtains a one-step difference equation of the form

$$y_{n+1} = Q(\hat{h})y_n, \quad \hat{h} = h\lambda,$$

where $Q(\hat{h})$ is the **stability function** of the method. We see that $y_n \rightarrow 0$ as $n \rightarrow \infty$ if and only if

$$|Q(\hat{h})| < 1, \tag{5.18}$$

and the method is **absolutely stable** for those values of $\hat{h}$ in the complex plane for which (5.18) hold; those values form the **region of absolute stability** of the method. It can be shown that the stability function of explicit s -stage Runge–Kutta methods of order $p = s$, $s = 1, 2, 3, 4$, is

$$R(\hat{h}) = \frac{y_{n+1}}{y_n} = 1 + \hat{h} + \frac{1}{2!}\hat{h}^2 + \cdots + \frac{1}{s!}\hat{h}^s.$$

The regions of absolute stability, R , of s -stage explicit Runge–Kutta methods of order $k = s$, for $s = 1, 2, 3, 4$, are the interior of the closed regions whose upper halves are shown in Fig. 5.4. The left-most point α of R is -2 , -2 , 2.51 and -2.78 for the methods of order $s = 1, 2, 3$ and 4 , respectively

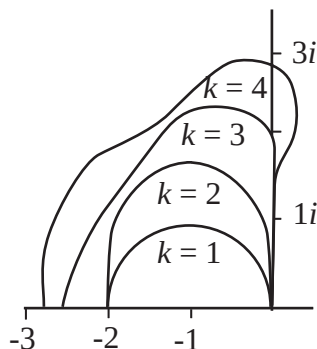


FIGURE 5.4. Region of absolute stability of s -stage explicit Runge-Kutta methods of order $k = s$.

Fixed stepsize Runge-Kutta methods of order 1 to 5 are implemented in the following Matlab function M-files which are found in <ftp://ftp.cs.cornell.edu/pub/cv>.

```
function [tvals,yvals] = FixedRK(fname,t0,y0,h,k,n)
%
% Produces approximate solution to the initial value problem
%
%       $y'(t) = f(t,y(t))$        $y(t_0) = y_0$ 
%
% using a strategy that is based upon a  $k$ -th order
% Runge-Kutta method. Stepsize is fixed.
%
% Pre:  fname = string that names the function  $f$ .
%       t0 = initial time.
%       y0 = initial condition vector.
%       h = stepsize.
%       k = order of method. ( $1 \leq k \leq 5$ ).
%       n = number of steps to be taken,
%
% Post: tvals(j) =  $t_0 + (j-1)h$ ,  $j=1:n+1$ 
%       yvals(:j) = approximate solution at  $t = tvals(j)$ ,  $j=1:n+1$ 
%
    tc = t0;
    yc = y0;
    tvals = tc;
    yvals = yc;
    fc = feval(fname,tc,yc);
    for j=1:n
        [tc,yc,fc] = RKstep(fname,tc,yc,fc,h,k);
        yvals = [yvals yc];
        tvals = [tvals tc];
    end
    function [tnew,ynew,fnew] = RKstep(fname,tc,yc,fc,h,k)
```

```

%
% Pre:  fname is a string that names a function of the form f(t,y)
%       where t is a scalar and y is a column d-vector.
%
%       yc is an approximate solution to  $y'(t) = f(t, y(t))$  at  $t=t_c$ .
%
%       fc = f(tc,yc).
%
%       h is the time step.
%
%       k is the order of the Runge-Kutta method used,  $1 \leq k \leq 5$ .
%
% Post: tnew=tc+h, ynew is an approximate solution at  $t=t_{\text{new}}$ , and
%       fnew = f(tnew,ynew).

if k==1
    k1 = h*fc;
    ynew = yc + k1;

elseif k==2
    k1 = h*fc;
    k2 = h*feval(fname,tc+h,yc+k1);
    ynew = yc + (k1 + k2)/2;

elseif k==3
    k1 = h*fc;
    k2 = h*feval(fname,tc+(h/2),yc+(k1/2));
    k3 = h*feval(fname,tc+h,yc-k1+2*k2);
    ynew = yc + (k1 + 4*k2 + k3)/6;

elseif k==4
    k1 = h*fc;
    k2 = h*feval(fname,tc+(h/2),yc+(k1/2));
    k3 = h*feval(fname,tc+(h/2),yc+(k2/2));
    k4 = h*feval(fname,tc+h,yc+k3);
    ynew = yc + (k1 + 2*k2 + 2*k3 + k4)/6;

elseif k==5
    k1 = h*fc;
    k2 = h*feval(fname,tc+(h/4),yc+(k1/4));
    k3 = h*feval(fname,tc+(3*h/8),yc+(3/32)*k1
                  +(9/32)*k2);
    k4 = h*feval(fname,tc+(12/13)*h,yc+(1932/2197)*k1
                  -(7200/2197)*k2+(7296/2197)*k3);
    k5 = h*feval(fname,tc+h,yc+(439/216)*k1
                  - 8*k2 + (3680/513)*k3 -(845/4104)*k4);
    k6 = h*feval(fname,tc+(1/2)*h,yc-(8/27)*k1
                  + 2*k2 -(3544/2565)*k3 + (1859/4104)*k4 - (11/40)*k5);

```

```

ynew = yc + (16/135)*k1 + (6656/12825)*k3 +
          (28561/56430)*k4 - (9/50)*k5 + (2/55)*k6;
end
tnew = tc+h;
fnew = feval(fname,tnew,ynew);

```

5.7. Embedded Pairs of Runge–Kutta methods

Thus far, we have only considered a constant step size h . In practice, it is advantageous to let h vary so that h is taken larger when $y(x)$ does not vary rapidly and smaller when $y(x)$ changes rapidly. We turn to this problem.

Embedded pairs of Runge–Kutta methods of orders p and $p+1$ have built-in local error and step-size controls by monitoring the difference between the higher and lower order solutions, $y_{n+1} - \hat{y}_{n+1}$. Some pairs include an interpolant which is used to interpolate the numerical solution between the nodes of the numerical solution and also, in some case, to control the step-size.

5.7.1. Matlab's four-stage RK pair ode23. The code `ode23` consists in a four-stage pair of embedded explicit Runge–Kutta methods of orders 2 and 3 with error control. It advances from y_n to y_{n+1} with the third-order method (so called local extrapolation) and controls the local error by taking the difference between the third-order and the second-order numerical solutions. The four stages are:

$$\begin{aligned}
k_1 &= h f(x_n, y_n), \\
k_2 &= h f(x_n + (1/2)h, y_n + (1/2)k_1), \\
k_3 &= h f(x_n + (3/4)h, y_n + (3/4)k_2), \\
k_4 &= h f(x_n + h, y_n + (2/9)k_1 + (1/3)k_2 + (4/9)k_3),
\end{aligned}$$

The first three stages produce the solution at the next time step:

$$y_{n+1} = y_n + \frac{2}{9}k_1 + \frac{1}{3}k_2 + \frac{4}{9}k_3,$$

and all four stages give the local error estimate:

$$E = -\frac{5}{72}k_1 + \frac{1}{12}k_2 + \frac{1}{9}k_3 - \frac{1}{8}k_4.$$

However, this is really a three-stage method since the first step at x_{n+1} is the same as the last step at x_n , that is $k_1^{[n+1]} = k_4^{[n]}$. Such methods are called FSAL methods.

The natural interpolant used in `ode23` is the two-point Hermite polynomial of degree 3 which interpolates y_n and $f(x_n, y_n)$ at $x = x_n$, and y_{n+1} and $f(x_{n+1}, y_{n+1})$ at $t = x_{n+1}$.

EXAMPLE 5.9. Use Matlab's four-stage FSAL `ode23` method with $h = 0.1$ to approximate $y(0.1)$ and $y(0.2)$ to 5 decimal places and estimate the local error for the initial value problem

$$y' = xy + 1, \quad y(0) = 1.$$

SOLUTION. The right-hand side of the differential equation is

$$f(x, y) = xy + 1.$$

With $n = 0$:

$$\begin{aligned} k_1 &= 0.1 \times 1 = 0.1 \\ k_2 &= 0.1 \times (0.05 \times 1.05 + 1) = 0.105\,25 \\ k_3 &= 0.1 \times (0.75 \times 1.078\,937\,5 + 1) = 0.108\,092\,031\,25 \\ k_4 &= 0.1 \times (0.1 \times 1.105\,346\,458\,333\,33 + 1) = 0.111\,053\,464\,583\,33 \\ y_1 &= 1.105\,346\,458\,333\,33 \end{aligned}$$

The estimate of the local error is

$$\text{Local error estimate} = -4.506\,848\,958\,333\,448e - 05$$

With $n = 1$:

$$\begin{aligned} k_1 &= 0.111\,053\,464\,583\,33 \\ k_2 &= 0.117\,413\,097\,859\,37 \\ k_3 &= 0.120\,884\,609\,930\,24 \\ k_4 &= 0.124\,457\,783\,972\,15 \\ y_2 &= 1.222\,889\,198\,607\,30 \end{aligned}$$

The estimate of the local error is

$$\text{Local error estimate} = -5.322\,100\,094\,209\,102e - 05$$

To use the numeric Matlab command `ode23` to solve and plot the given initial value problem on $[0, 1]$, one writes the function M-file `exp5_9.m`:

```
function yprime = exp5_9(x,y)
yprime = x.*y+1;
```

and use the commands

```
clear
xspan = [0 1]; y0 = 1; % xspan and initial value
[x,y] = ode23('exp5_9',xspan,y0);
subplot(2,2,1); plot(x,y); xlabel('x'); ylabel('y');
title('Solution to equation of Example 5.9');
print -deps2 Figexp5_9 % print figure to file Fig.exp5.9
```

□

The Matlab solver `ode23` is an implementation of the explicit Runge-Kutta (2,3) pair of Bogacki and Shampine called BS23. It uses a “free” interpolant of order 3. Local extrapolation is done, that is, the higher-order solution, namely of order 3, is used to advance the solution.

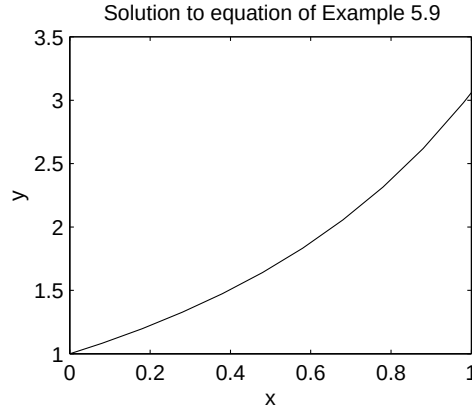


FIGURE 5.5. Graph of numerical solutions of Example 5.9.

5.7.2. Seven-stage Dormand–Prince pair DP(5,4)7M with interpolant. The seven-stage Dormand–Prince pair DP(5,4)7M [3] with local error estimate and interpolant is presented in a Butcher tableau. The number 5 in the designation DP(5,4)7M means that the solution is advanced with the solution y_{n+1} of order five (a procedure called *local extrapolation*). The number 4 means that the solution $\hat{y}_{n+1}$ of order four is used to obtain the local error estimate by means of the difference $y_{n+1} - \hat{y}_{n+1}$. In fact, $\hat{y}_{n+1}$ is not computed; rather the coefficients in the line $b^T - \hat{b}^T$ are used to obtain the local error estimate. The number 7 means that the method has seven stages. The letter M means that the constant C_6 in the top-order error term has been minimized, while maintaining stability. Six stages are necessary for the method of order 5. The seventh stage is necessary to have an interpolant. The last line of the tableau is used to produce an interpolant.

c		A						
k_1	0	0						
k_2	$\frac{1}{5}$	$\frac{1}{5}$	0					
k_3	$\frac{3}{10}$	$\frac{3}{40}$	$\frac{9}{40}$	0				
k_4	$\frac{4}{5}$	$\frac{44}{45}$	$-\frac{56}{15}$	$\frac{32}{9}$	0			
k_5	$\frac{8}{9}$	$\frac{19372}{6561}$	$-\frac{25360}{2187}$	$\frac{64448}{6561}$	$-\frac{212}{729}$	0		
k_6	1	$\frac{9017}{3168}$	$-\frac{355}{33}$	$\frac{46732}{5247}$	$\frac{49}{176}$	$-\frac{5103}{18656}$	0	
k_7	1	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$	
$\hat{y}_{n+1}$	$\hat{b}^T$	$\frac{5179}{57600}$	0	$\frac{7571}{16695}$	$\frac{393}{640}$	$-\frac{92097}{339200}$	$\frac{187}{2100}$	$\frac{1}{40}$
y_{n+1}	b^T	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$	0
$b^T - \hat{b}^T$		$\frac{71}{57600}$	0	$-\frac{71}{16695}$	$\frac{71}{1920}$	$-\frac{17253}{339200}$	$\frac{22}{525}$	$-\frac{1}{40}$
$y_{n+0.5}$		$\frac{5783653}{57600000}$	0	$\frac{466123}{1192500}$	$-\frac{41347}{1920000}$	$\frac{16122321}{339200000}$	$-\frac{7117}{20000}$	$\frac{183}{10000}$

(5.19)

Seven-stage Dormand–Prince pair DP(5,4)7M of order 5 and 4.

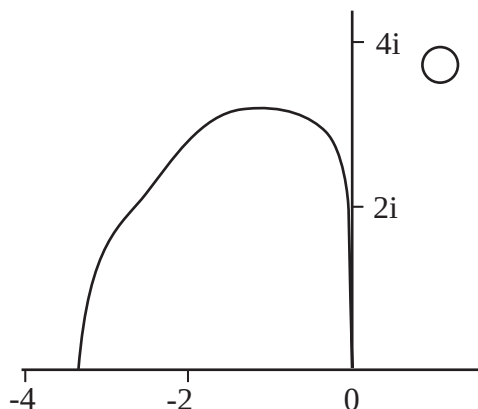


FIGURE 5.6. Region of absolute stability of the Dormand-Prince pair DP(5,4)7M.

This seven-stage method reduces, in practice, to a six-stage method since $k_1^{[n+1]} = k_7^{[n]}$; in fact the row vector b^T is the same as the 7-th line corresponding to k_7 . Such methods are called FSAL (First Same As Last) since the first line is the same as the last one.

The interval of absolute stability of the pair DP(5,4)7M is approximately $(-3.3, 0)$ (see Fig. 5.6).

One notices that the matrix A in the Butcher tableau of an explicit Runge-Kutta method is strictly lower triangular. Semi-explicit methods have a lower triangular matrix. Otherwise, the method is implicit. Solving semi-explicit methods for the vector solution y_{n+1} of a system is much cheaper than solving implicit methods.

Runge-Kutta methods constitute a clever and sensible idea [2]. The unique solution of a well-posed initial value problem is a single curve in $\mathbb{R}^{n+1}$, but due to truncation and roundoff error, any numerical solution is, in fact, going to wander off that integral curve, and the numerical solution is inevitably going to be affected by the behavior of neighboring curves. Thus, it is the behavior of the family of integral curves, and not just that of the unique solution curve, that is of importance. Runge-Kutta methods deliberately try to gather information about this family of curves, as it is most easily seen in the case of explicit Runge-Kutta methods.

The Matlab solver `ode45` is an implementation of the explicit Runge-Kutta (5,4) pair of Dormand and Prince called variously RK5(4)7FM, DOPRI5, DP(4,5) and DP54. It uses a “free” interpolant of order 4 communicated privately by Dormand and Prince. Local extrapolation is done.

Details on Matlab solvers `ode23`, `ode45` and other solvers can be found in *The MATLAB ODE Suite*, L. F. Shampine and M. W. Reichelt, SIAM Journal on Scientific Computing, **18**(1), 1997.

5.7.3. Six-stage Runge-Kutta-Fehlberg pair RKF(4,5). The six-stage Runge-Kutta-Fehlberg pair RKF(4,5) with local error estimate uses a method of order 4 to advance the numerical value from y_n to y_{n+1} , and a method of order 5 to obtain the auxiliary value $\hat{y}_{n+1}$ which serves in computing the local error

by means of the difference $y_{n+1} - \hat{y}_{n+1}$. We present this method in a Butcher tableau. The estimated local error is obtained from the last line. The method of order 4 minimizes the local error.

$$\begin{array}{c|cccccc}
 k_1 & 0 & 0 & & & & \\
 k_2 & \frac{1}{4} & \frac{1}{4} & 0 & & & \\
 k_3 & \frac{3}{8} & \frac{3}{32} & \frac{9}{32} & 0 & & \\
 k_4 & \frac{12}{13} & \frac{1932}{2197} & -\frac{7200}{2197} & \frac{7296}{2197} & 0 & \\
 k_5 & 1 & \frac{439}{216} & -8 & \frac{3680}{513} & -\frac{845}{4104} & 0 \\
 k_6 & \frac{1}{2} & -\frac{8}{27} & 2 & -\frac{3544}{2565} & \frac{1859}{4104} & -\frac{11}{40} & 0
 \end{array} \quad (5.20)$$

$$\begin{array}{c|cccccc}
 \frac{2197}{4104} & -\frac{1}{5} & 0 & & & & \\
 \hat{y}_{n+1} & \hat{\mathbf{b}}^T & \frac{16}{135} & 0 & \frac{6656}{12825} & \frac{28561}{56430} & -\frac{9}{50} & \frac{2}{55} \\
 & \hat{\mathbf{b}}^T - \mathbf{b}^T & \frac{1}{360} & 0 & -\frac{128}{4275} & -\frac{2197}{75240} & \frac{1}{50} & \frac{2}{55}
 \end{array}$$

Six-stage Runge–Kutta–Fehlberg pair RKF(4,5) of order 4 and 5.

The interval of absolute stability of the pair RKF(4,5) is approximately $(-3.78, 0)$.

The pair RKF45 of order four and five minimizes the error constant C_5 of the lower order method which is used to advance the solution from y_n to y_{n+1} , that is, without using local extrapolation. The algorithm follows.

ALGORITHM 5.1. Let y_0 be the initial condition. Suppose that the approximation y_n to $y(x_n)$ has been computed and satisfies $|y(x_n) - y_n| < \epsilon$ where ϵ is the desired precision. Let $h > 0$.

- (1) Compute two approximations for y_{n+1} : one using the fourth-order method

$$y_{n+1} = y_n + \left(\frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5 \right), \quad (5.21)$$

and the second using the fifth-order method,

$$\hat{y}_{j+1} = y_n + \left(\frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6 \right), \quad (5.22)$$

where

$$k_1 = hf(x_n, y_n),$$

$$k_2 = hf(x_n + h/4, y_n + k_1/4),$$

$$k_3 = hf(x_n + 3h/8, y_n + 3k_1/32 + 9k_2/32),$$

$$k_4 = hf(x_n + 12h/13, y_n + 1932k_1/2197 - 7200k_2/2197 + 7296k_3/2197),$$

$$k_5 = hf(x_n + h, y_n + 439k_1/216 - 8k_2 + 3680k_3/513 + 845k_4/4104),$$

$$k_6 = hf(x_n + h/2, y_n - 8k_1/27 + 2k_2 + 3544k_3/2565 + 1859k_4/4104 - 11k_5/40).$$

- (2) If $|\hat{y}_{j+1} - y_{n+1}| < \epsilon h$, accept y_{n+1} as the approximation to $y(x_{n+1})$. Replace h by qh where

$$q = [\epsilon h / (2|\hat{y}_{j+1} - y_{n+1}|)]^{1/4}$$

and go back to step (1) to compute an approximation for y_{j+2} .

(3) If $|\hat{y}_{j+1} - y_{n+1}| \geq \epsilon h$, replace h by qh where

$$q = [\epsilon h / (2|\hat{y}_{j+1} - y_{n+1}|)]^{1/4}$$

and go back to step (1) to compute the next approximation for y_{n+1} .

One can show that the local truncation error for (5.21) is approximately

$$|\hat{y}_{j+1} - y_{n+1}|/h.$$

At step (2), one requires that this error be smaller than ϵh in order to get $|y(x_n) - y_n| < \epsilon$ for all j (and in particular $|y(x_f) - y_f| < \epsilon$). The formula to compute q in (2) and (3) (and hence a new value for h) is derived from the relation between the local truncation errors of (5.21) and (5.22).

RKF(4,5) overestimate the error in the order-four solution because its local error constant is minimized. The next method, RKV, corrects this fault.

5.7.4. Eight-stage Runge–Kutta–Verner pair RKV(5,6). The eight-stage Runge–Kutta–Verner pair RKV(5,6) of order 5 and 6 is presented in a Butcher tableau. Note that 8 stages are necessary to get order 6. The method attempts to keep the global error proportional to a user-specified tolerance. It is efficient for nonstiff systems where the derivative evaluations are not expensive and where the solution is not required at a large number of finely spaced points (as might be required for graphical output).

	c	A						
k_1	0	0						
k_2	$\frac{1}{6}$	$\frac{1}{6}$	0					
k_3	$\frac{4}{15}$	$\frac{4}{75}$	$\frac{16}{75}$	0				
k_4	$\frac{2}{3}$	$\frac{5}{6}$	$-\frac{8}{3}$	$\frac{5}{2}$	0			
k_5	$\frac{5}{6}$	$-\frac{165}{64}$	$\frac{55}{6}$	$-\frac{425}{64}$	$\frac{85}{96}$	0		
k_6	1	$\frac{12}{5}$	-8	$\frac{4015}{612}$	$-\frac{11}{36}$	$\frac{88}{255}$	0	
k_7	$\frac{1}{15}$	$-\frac{8263}{15000}$	$\frac{124}{75}$	$-\frac{643}{680}$	$-\frac{81}{250}$	$\frac{2484}{10625}$	0	
k_8	1	$\frac{3501}{1720}$	$-\frac{300}{43}$	$\frac{297275}{52632}$	$-\frac{319}{2322}$	$\frac{24068}{84065}$	0	$\frac{3850}{26703}$
y_{n+1}	b ^T	$\frac{13}{160}$	0	$\frac{2375}{5984}$	$\frac{5}{16}$	$\frac{12}{85}$	$\frac{3}{44}$	
$\hat{y}_{n+1}$	$\hat{b}$ ^T	$\frac{3}{40}$	0	$\frac{875}{2244}$	$\frac{23}{72}$	$\frac{264}{1955}$	0	$\frac{125}{11592}$ $\frac{43}{616}$

Eight-stage Runge–Kutta–Verner pair RKV(5,6) of order 5 and 6.

5.8. Multistep Predictor-Corrector Methods

5.8.1. General multistep methods. Consider the initial value problem

$$y' = f(x, y), \quad y(a) = \eta, \quad (5.24)$$

where $f(x)$ is continuous with respect to x and Lipschitz continuous with respect to y on the strip $[a, b] \times (-\infty, \infty)$. Then, by Theorem 5.1, the exact solution, $y(x)$, exists and is unique on $[a, b]$.

We look for an approximate numerical solution $\{y_n\}$ at the nodes $x_n = a + nh$

where h is the step size and $n = (b - a)/h$.

For this purpose, we consider the k -step linear method:

$$\sum_{n=0}^k \alpha_j y_{n+j} = h \sum_{n=0}^k \beta_j f_{n+j}, \quad (5.25)$$

where $y_n \approx y(x_n)$ and $f_n := f(x_n, y_n)$. We normalize the method by the condition $\alpha_k = 1$ and insist that the number of steps be exactly k by imposing the condition

$$(\alpha_0, \beta_0) \neq (0, 0).$$

We choose k starting values $y_0, y_1, \dots, y_{k-1}$, say, by means of a Runge-Kutta method of the same order.

The method is *explicit* if $\beta_k = 0$; in this case, we obtain y_{n+1} directly. The method is *implicit* if $\beta_k \neq 0$; in this case, we have to solve for y_{n+k} by the recurrence formula:

$$y_{n+k}^{[s+1]} = h\beta_k f(x_{n+k}, y_{n+k}^{[s]}) + g, \quad y_{n+k}^{[0]} \text{ arbitrary}, \quad s = 0, 1, \dots, \quad (5.26)$$

where the function

$$g = g(x_n, \dots, x_{n+k-1}, y_0, \dots, y_{n+k-1})$$

contains only known values. The recurrence formula (5.26) converges as $s \rightarrow \infty$, if $0 \leq M < 1$ where M is the Lipschitz constant of the right-hand side of (5.26) with respect to y_{n+k} . If L is the Lipschitz constant of $f(x)$ with respect to y , then

$$M := Lh|\beta_k| < 1 \quad (5.27)$$

and the inequality

$$h < \frac{1}{L|\beta_k|}$$

implies convergence.

Applying (5.25) to the test equation,

$$y' = \lambda y, \quad \Re \lambda < 0,$$

with solution $y(x) \rightarrow 0$ as $t \rightarrow \infty$, one finds that the numerical solution $y_n \rightarrow 0$ as $n \rightarrow \infty$ if the zeros, $r_s(\hat{h})$, of the stability polynomial

$$\pi(r, \hat{h}) := \sum_{n=0}^k (\alpha_j - \hat{h}\beta_j) r^j$$

satisfy $|r_s(\hat{h})| < 1$, $s = 1, 2, \dots, k$. In that case, we say that the linear multistep method (5.25) is **absolutely stable** for given $\hat{h}$. The **region of absolute stability**, R , in the complex plane is the set of values of $\hat{h}$ for which the method is absolutely stable.

5.8.2. Adams-Bashforth-Moulton linear multistep methods. Popular linear k -step methods are (explicit) Adams-Bashforth (AB) and (implicit) Adams-Moulton (AM) methods,

$$y_{n+1} - y_n = h \sum_{j=0}^{k-1} \beta_j^* f_{n+j-k+1}, \quad y_{n+1} - y_n = h \sum_{j=0}^k \beta_j f_{n+j-k+1},$$

respectively. Tables 5.5 and 5.6 list the AB and AM methods of stepnumber 1 to 6, respectively. In the tables, the coefficients of the methods are to be divided by d , k is the stepnumber, p is the order, and C_{p+1}^* and C_{p+1} are the corresponding error constants of the methods.

TABLE 5.5. Coefficients of Adams–Bashforth methods of stepnumber 1–6.

β_5^*	β_4^*	β_3^*	β_2^*	β_1^*	β_0^*	d	k	p	C_{p+1}^*
					1	1	1	1	1/2
				3	−1	2	2	2	5/12
			23	−16	5	12	3	3	3/8
		55	−59	37	−9	24	4	4	251/720
	1901	−2774	1616	−1274	251	720	5	5	95/288
4277	−7923	9982	−7298	2877	−475	1440	6	6	19 087/60 480

TABLE 5.6. Coefficients of Adams–Moulton methods of stepnumber 1–6.

β_5	β_4	β_3	β_2	β_1	β_0	d	k	p	C_{p+1}
				1	1	2	1	2	−1/12
			5	8	−1	12	2	3	−1/24
		9	19	−5	1	24	3	4	−19/720
	251	646	−264	106	−19	720	4	5	−3/160
475	1427	−798	482	−173	27	1440	5	6	−863/60 480

The regions of absolute stability of k -step Adams–Bashforth and Adams–Moulton methods of order $k = 1, 2, 3, 4$, are the interior of the closed regions whose upper halves are shown in the left and right parts, respectively, of Fig. 5.7. The region of absolute stability of the Adams–Bashforth method of order 3 extends in a small triangular region in the right half-plane. The region of absolute stability of the Adams–Moulton method of order 1 is the whole left half-plane.

In practice, an AB method is used as a **predictor** to predict the next-step value y_{n+1}^* , which is then inserted in the right-hand side of an AM method used as a **corrector** to obtain the corrected value y_{n+1} . Such combination is called an ABM predictor-corrector which, when of the same order, comes with the Milne estimate for the principal local truncation error

$$\epsilon_{n+1} \approx \frac{C_{p+1}}{C_{p+1}^* - C_{p+1}} (y_{n+1} - y_{n+1}^*).$$

The procedure called *local approximation* improves the higher-order solution y_{n+1} by the addition of the error estimator, namely,

$$y_{n+1} + \frac{C_{p+1}}{C_{p+1}^* - C_{p+1}} (y_{n+1} - y_{n+1}^*).$$

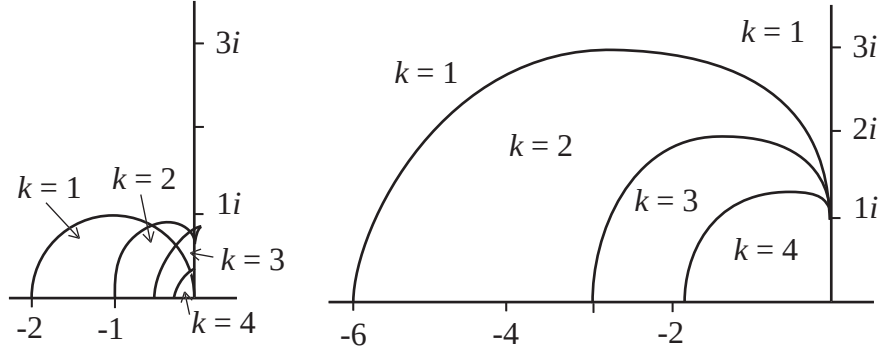


FIGURE 5.7. Left: Regions of absolute stability of k -step Adams–Bashforth methods. Right: Regions of absolute stability of k -step Adams–Moulton methods.

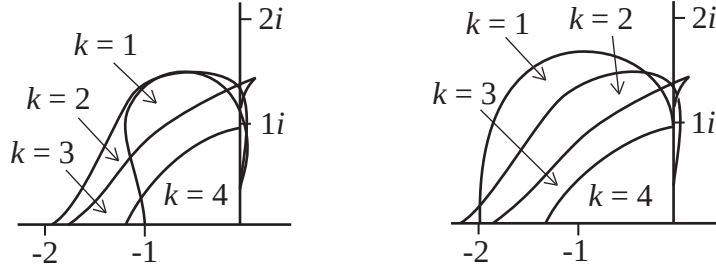


FIGURE 5.8. Regions of absolute stability of k -order Adams–Bashforth–Moulton methods, left in PECE mode, and right in PECLE mode.

The regions of absolute stability of k th-order Adams–Bashforth–Moulton pairs, for $k = 1, 2, 3, 4$, in *Predictor-Evaluation-Corrector-Evaluation* mode, denoted by PECE, are the interior of the closed regions whose upper halves are shown in the left part of Fig. 5.8. The regions of absolute stability of k th-order Adams–Bashforth–Moulton pairs, for $k = 1, 2, 3, 4$, in the PECLE mode where L stands for local extrapolation, are the interior of the closed regions whose upper halves are shown in the right part of Fig. 5.8.

5.8.3. Adams–Bashforth–Moulton methods of orders 3 and 4. As a first example of multistep methods, we consider the three-step Adams–Bashforth–Moulton method of order 3, given by the formula pair:

$$y_{n+1}^P = y_n^C + \frac{h}{12} (23f_n^C - 16f_{n-1}^C + 5f_{n-2}^C), \quad f_k^C = f(x_k, y_k^C), \quad (5.28)$$

$$y_{n+1}^C = y_n^C + \frac{h}{12} (5f_{n+1}^P + 8f_n^C - f_{n-1}^C), \quad f_k^P = f(x_k, y_k^P), \quad (5.29)$$

with local error estimate

$$\text{Err.} \approx -\frac{1}{10} [y_{n+1}^C - y_{n+1}^P]. \quad (5.30)$$

EXAMPLE 5.10. Solve to six decimal places the initial value problem

$$y' = x + \sin y, \quad y(0) = 0,$$

by means of the Adams–Bashforth–Moulton method of order 3 over the interval $[0, 2]$ with $h = 0.2$. The starting values have been obtained by a high precision method. Use formula (5.30) to estimate the local error at each step.

SOLUTION. The solution is given in a table.

n	x_n	Starting y_n^C	Predicted y_n^P	Corrected y_n^C	$10^5 \times \text{Local Error in } y_n^C$ $\approx -(y_n^C - y_n^P) \times 10^4$
0	0.0	0.000 000 0			
1	0.2	0.021 404 7			
2	0.4	0.091 819 5			
3	0.6		0.221 260	0.221 977	– 7
4	0.8		0.423 703	0.424 064	– 4
5	1.0		0.710 725	0.709 623	11
6	1.2		1.088 004	1.083 447	46
7	1.4		1.542 694	1.533 698	90
8	1.6		2.035 443	2.026 712	87
9	1.8		2.518 039	2.518 431	– 4
10	2.0		2.965 994	2.975 839	–98

□

As a second and better known example of multistep methods, we consider the four-step Adams–Bashforth–Moulton method of order 4.

The *Adams–Bashforth predictor* and the *Adams–Moulton corrector* of order 4 are

$$y_{n+1}^P = y_n^C + \frac{h}{24} (55f_n^C - 59f_{n-1}^C + 37f_{n-2}^C - 9f_{n-3}^C) \quad (5.31)$$

and

$$y_{n+1}^C = y_n^C + \frac{h}{24} (9f_{n+1}^P + 19f_n^C - 5f_{n-1}^C + f_{n-2}^C), \quad (5.32)$$

where

$$f_n^C = f(x_n, y_n^C) \quad \text{and} \quad f_n^P = f(x_n, y_n^P).$$

Starting values are obtained with a Runge–Kutta method or otherwise.

The local error is controlled by means of the estimate

$$C_5 h^5 y^{(5)}(x_{n+1}) \approx -\frac{19}{270} [y_{n+1}^C - y_{n+1}^P]. \quad (5.33)$$

A certain number of past values of y_n and f_n are kept in memory in order to extend the step size if the local error is small with respect to the given tolerance. If the local error is too large with respect to the given tolerance, the step size can be halved by means of the following formulae:

$$y_{n-1/2} = \frac{1}{128} (35y_n + 140y_{n-1} - 70y_{n-2} + 28y_{n-3} - y_{n-4}), \quad (5.34)$$

$$y_{n-3/2} = \frac{1}{162} (-y_n + 24y_{n-1} + 54y_{n-2} - 16y_{n-3} + 3y_{n-4}). \quad (5.35)$$

In PECE mode, the Adams–Bashforth–Moulton pair of order 4 has interval of absolute stability equal to $(-1.25, 0)$, that is, the method does not amplify past errors if the step size h is sufficiently small so that

$$-1.25 < h \frac{\partial f}{\partial y} < 0, \quad \text{where} \quad \frac{\partial f}{\partial y} < 0.$$

EXAMPLE 5.11. Consider the initial value problem

$$y' = x + y, \quad y(0) = 0.$$

Compute the solution at $x = 2$ by the Adams–Bashforth–Moulton method of order 4 with $h = 0.2$. Use Runge–Kutta method of order 4 to obtain the starting values. Use five decimal places and use the exact solution to compute the global error.

SOLUTION. The global error is computed by means of the exact solution

$$y(x) = e^x - x - 1.$$

We present the solution in the form of a table for starting values, predicted values, corrected values, exact values and global errors in the corrected solution.

n	x_n	Starting y_n^C	Predicted y_n^P	Corrected y_n^C	Exact $y(x_n)$	Error: $10^6 \times$ $(y(x_n) - y_n^C)$
0	0.0	0.000 000			0.000 000	0
1	0.2	0.021 400			0.021 403	3
2	0.4	0.091 818			0.091 825	7
3	0.6	0.222 107			0.222 119	12
4	0.8		0.425 361	0.425 529	0.425 541	12
5	1.0		0.718 066	0.718 270	0.718 282	12
6	1.2		1.119 855	1.120 106	1.120 117	11
7	1.4		1.654 885	1.655 191	1.655 200	9
8	1.6		2.352 653	2.353 026	2.353 032	6
9	1.8		3.249 190	3.249 646	3.249 647	1
10	2.0		4.388 505	4.389 062	4.389 056	-6

We see that the method is stable since the error does not grow. $\square$

EXAMPLE 5.12. Solve to six decimal places the initial value problem

$$y' = \arctan x + \arctan y, \quad y(0) = 0,$$

by means of the Adams–Bashforth–Moulton method of order 3 over the interval $[0, 2]$ with $h = 0.2$. Obtain the starting values by Runge–Kutta 4. Use formula (5.30) to estimate the local error at each step.

SOLUTION. **The Matlab numeric solution.**— The M-file `exp5_12` for Example 5.12 is

```
function yprime = exp5_12(x,y); % Example 5.12.
yprime = atan(x)+atan(y);
```

The initial conditions and the Runge–Kutta method of order 4 is used to obtain the four starting values

```

clear
h = 0.2; x0= 0; xf= 2; y0 = 0;
n = ceil((xf-x0)/h); % number of steps
%
count = 2; print_time = 1; % when to write to output
x = x0; y = y0; % initialize x and y
output = [0 x0 y0 0];
%RK4
for i=1:3
    k1 = h*exp5_12(x,y);
    k2 = h*exp5_12(x+h/2,y+k1/2);
    k3 = h*exp5_12(x+h/2,y+k2/2);
    k4 = h*exp5_12(x+h,y+k3);
    z = y + (1/6)*(k1+2*k2+2*k3+k4);
    x = x + h;
    if count > print_time
        output = [output; i x z 0];
        count = count - print_time;
    end
    y = z;
count = count + 1;
end
% ABM4
for i=4:n
    zp = y + (h/24)*(55*exp5_12(output(i,2),output(i,3))-...
        59*exp5_12(output(i-1,2),output(i-1,3))+...
        37*exp5_12(output(i-2,2),output(i-2,3))-...
        9*exp5_12(output(i-3,2),output(i-3,3)) );
    z = y + (h/24)*( 9*exp5_12(x+h,zp)+...
        19*exp5_12(output(i,2),output(i,3))-...
        5*exp5_12(output(i-1,2),output(i-1,3))+...
        exp5_12(output(i-2,2),output(i-2,3)) );
    x = x + h;
    if count > print_time
        errest = -(19/270)*(z-zp);
        output = [output;i x z errest];
        count = count - print_time;
    end
    y = z;
count = count + 1;
end
output
save output %for printing the graph

```

The command `output` prints the values of n , x , and y .

n	x	y	Error estimate
-----	-----	-----	----------------

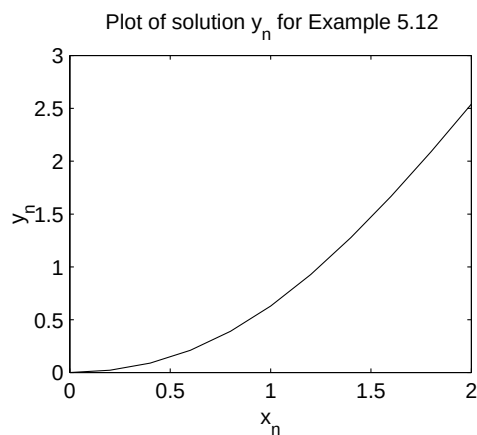


FIGURE 5.9. Graph of the numerical solution of Example 5.12.

0	0	0	0
1	0.2	0.02126422549044	0
2	0.4	0.08962325332457	0
3	0.6	0.21103407185113	0
4	0.8	0.39029787517821	0.00001007608281
5	1.0	0.62988482479868	0.00005216829834
6	1.2	0.92767891924367	0.00004381671342
7	1.4	1.27663327419538	-0.00003607372725
8	1.6	1.66738483675693	-0.00008228934754
9	1.8	2.09110753309673	-0.00005318684309
10	2.0	2.54068815072267	-0.00001234568256

The following commands print the output.

```
load output;
subplot(2,2,1); plot(output(:,2),output(:,3));
title('Plot of solution y_n for Example 5.12');
xlabel('x_n'); ylabel('y_n');
```

□

Fixed stepsize Adams–Bashforth–Moulton methods of order 1 to 5 are implemented in the following Matlab function M-files which are found in <ftp://ftp.cs.cornell.edu/pub/cv>.

```
function [tvals,yvals] = FixedPC(fname,t0,y0,h,k,n)
%
% Produces an approximate solution to the initial value problem
%
%      y'(t) = f(t,y(t))      y(t0) = y0
%
% using a strategy that is based upon a k-th order
% Adams PC method. Step size is fixed.
%
```



```

% Pre:  fname = string that names the function f.
%       t0 = initial time.
%       y0 = initial condition vector.
%       h = stepsize.
%       k = order of method. (1<=k<=5).
%       n = number of steps to be taken,
%
% Post: tvals(j) = t0 + (j-1)h, j=1:n+1
%       yvals(:j) = approximate solution at t = tvals(j), j=1:n+1
%
[tvals,yvals,fvals] = StartAB(fname,t0,y0,h,k);
tc = tvals(k);
yc = yvals(:,k);
fc = fvals(:,k);

for j=k:n
    % Take a step and then update.
    [tc,yPred,fPred,yc,fc] = PCstep(fname,tc,yc,fvals,h,k);
    tvals = [tvals tc];
    yvals = [yvals yc];
    fvals = [fc fvals(:,1:k-1)];
end

```

The starting values are obtained by the following M-file by means of a Runge-Kutta method.

```

function [tvals,yvals,fvals] = StartAB(fname,t0,y0,h,k)
%
% Uses k-th order Runge-Kutta to generate approximate
% solutions to
%            $y'(t) = f(t,y(t))$     $y(t_0) = y_0$ 
%
% at  $t = t_0, t_0+h, \dots, t_0 + (k-1)h$ .
%
% Pre:
%       fname is a string that names the function f.
%       t0 is the initial time.
%       y0 is the initial value.
%       h is the step size.
%       k is the order of the RK method used.
%
% Post:
%       tvals = [ t0, t0+h, ... , t0 + (k-1)h ].
%       For j =1:k, yvals(:,j) = y(tvals(j)) (approximately).
%       For j =1:k, fvals(:,j) = f(tvals(j),yvals(j)) .
%
tc = t0;
yc = y0;
fc = feval(fname,tc,yc);

```

```

tvals = tc;
yvals = yc;
fvals = fc;

for j=1:k-1
    [tc,yc,fc] = RKstep(fname,tc,yc,fc,h,k);
    tvals = [tvals tc];
    yvals = [yvals yc];
    fvals = [fc fvals];
end

```

The function M-file `Rkstep` is found in Subsection 5.6 The Adams-Bashforth predictor step is taken by the following M-file.

```

function [tnew,ynew,fnew] = ABstep(fname,tc,yc,fvals,h,k)
%
% Pre:  fname is a string that names a function of the form f(t,y)
%       where t is a scalar and y is a column d-vector.
%
%       yc is an approximate solution to  $y'(t) = f(t,y(t))$  at  $t=tc$ .
%
%       fvals is an d-by-k matrix where  $fvals(:,i)$  is an approximation
%       to  $f(t,y)$  at  $t = tc + (1-i)h$ ,  $i=1:k$ 
%
%       h is the time step.
%
%       k is the order of the AB method used,  $1 \leq k \leq 5$ .
%
% Post: tnew=tc+h, ynew is an approximate solution at  $t=tnew$ , and
%       fnew =  $f(tnew,ynew)$ .

if k==1
    ynew = yc + h*fvals;
elseif k==2
    ynew = yc + (h/2)*(fvals*[3;-1]);
elseif k==3
    ynew = yc + (h/12)*(fvals*[23;-16;5]);
elseif k==4
    ynew = yc + (h/24)*(fvals*[55;-59;37;-9]);
elseif k==5
    ynew = yc + (h/720)*(fvals*[1901;-2774;2616;-1274;251]);
end
tnew = tc+h;
fnew = feval(fname,tnew,ynew);

```

The Adams-Moulton corrector step is taken by the following M-file.

```

function [tnew,ynew,fnew] = AMstep(fname,tc,yc,fvals,h,k)
%
% Pre:  fname is a string that names a function of the form f(t,y)
%       where t is a scalar and y is a column d-vector.
%

```

```

%      yc is an approximate solution to  $y'(t) = f(t, y(t))$  at  $t=t_c$ .
%
%      fvals is an d-by-k matrix where fvals(:,i) is an approximation
%      to  $f(t, y)$  at  $t = t_c + (2-i)h$ ,  $i=1:k$ 
%
%      h is the time step.
%
%      k is the order of the AM method used,  $1 \leq k \leq 5$ .
%
% Post: tnew=tc+h, ynew is an approximate solution at  $t=t_{\text{new}}$ , and
%      fnew = f(tnew, ynew).

if k==1
    ynew = yc + h*fvals;
elseif k==2
    ynew = yc + (h/2)*(fvals*[1;1]);
elseif k==3
    ynew = yc + (h/12)*(fvals*[5;8;-1]);
elseif k==4
    ynew = yc + (h/24)*(fvals*[9;19;-5;1]);
elseif k==5
    ynew = yc + (h/720)*(fvals*[251;646;-264;106;-19]);
end
tnew = tc+h;
fnew = feval(fname, tnew, ynew);

```

The predictor-corrector step is taken by the following M-file.

```

function [tnew, yPred, fPred, yCorr, fCorr] = PCstep(fname, tc, yc, fvals, h, k)
%
% Pre:  fname is a string that names a function of the form  $f(t, y)$ 
%       where t is a scalar and y is a column d-vector.
%
%       yc is an approximate solution to  $y'(t) = f(t, y(t))$  at  $t=t_c$ .
%
%       fvals is an d-by-k matrix where fvals(:,i) is an approximation
%       to  $f(t, y)$  at  $t = t_c + (1-i)h$ ,  $i=1:k$ 
%
%       h is the time step.
%
%       k is the order of the Runge-Kutta method used,  $1 \leq k \leq 5$ .
%
% Post: tnew=tc+h,
%       yPred is the predicted solution at  $t=t_{\text{new}}$ 
%       fPred = f(tnew, yPred)
%       yCorr is the corrected solution at  $t=t_{\text{new}}$ 
%       fCorr = f(tnew, yCorr).

```

```
[tnew,yPred,fPred] = ABstep(fname,tc,yc,fvals,h,k);
[tnew,yCorr,fCorr] = AMstep(fname,tc,yc,[fPred fvals(:,1:k-1)],h,k);
```

5.8.4. Specification of multistep methods. The left-hand side of Adams methods is of the form

$$y_{n+1} - y_n.$$

Adams–Bashforth methods are explicit and Adams–Moulton methods are implicit. In the following formulae, Adams methods are obtained by taking $a = 0$ and $b = 0$. The integer k is the number of steps of the method. The integer p is the order of the method and the constant C_{p+1} is the constant of the top-order error term.

Explicit Methods

$k = 1 :$

$$\begin{aligned}\alpha_1 &= 1, \\ \alpha_0 &= -1, \quad \beta_0 = 1, \\ p &= 1; \quad C_{p+1} = \frac{1}{2}.\end{aligned}$$

$k = 2 :$

$$\begin{aligned}\alpha_2 &= 1, \\ \alpha_1 &= -1 - a, \quad \beta_1 = \frac{1}{2}(3 - a), \\ \alpha_0 &= a, \quad \beta_0 = \frac{1}{2}(-1 + a), \\ p &= 2; \quad C_{p+1} = \frac{1}{12}(5 + a).\end{aligned}$$

Absolute stability limits the order to 2.

$k = 3 :$

$$\begin{aligned}\alpha_3 &= 1, \\ \alpha_2 &= -1 - a, \quad \beta_2 = \frac{1}{12}(23 - 5a - b), \\ \alpha_1 &= a + b, \quad \beta_1 = \frac{1}{3}(-4 - 2a + 2b), \\ \alpha_0 &= -b, \quad \beta_0 = \frac{1}{12}(5 + a + 5b), \\ p &= 3; \quad C_{p+1} = \frac{1}{24}(9 + a + b).\end{aligned}$$

Absolute stability limits the order to 3.

$k = 4 :$

$$\begin{aligned}\alpha_4 &= 1, \\ \alpha_3 &= -1 - a, \quad \beta_3 = \frac{1}{24}(55 - 9a - b - c), \\ \alpha_2 &= a + b, \quad \beta_2 = \frac{1}{24}(-59 - 19a + 13b - 19c), \\ \alpha_1 &= -b - c, \quad \beta_1 = \frac{1}{24}(37 + 5a + 13b - 19c), \\ \alpha_0 &= c, \quad \beta_0 = \frac{1}{24}(-9 - a - b - 9c), \\ p &= 4; \quad C_{p+1} = \frac{1}{720}(251 + 19a + 11b + 19c).\end{aligned}$$

Absolute stability limits the order to 4.

Implicit Methods

$k = 1 :$

$$\begin{aligned}\alpha_1 &= 1, & \beta_1 &= \frac{1}{2}, \\ \alpha_0 &= -1, & \beta_0 &= \frac{1}{2}, \\ p &= 2; & C_{p+1} &= -\frac{1}{12}.\end{aligned}$$

$k = 2 :$

$$\begin{aligned}\alpha_2 &= 1, & \beta_2 &= \frac{1}{12}(5 + a), \\ \alpha_1 &= -1 - a, & \beta_1 &= \frac{2}{3}(1 - a), \\ \alpha_0 &= a, & \beta_0 &= \frac{1}{12}(-1 - 5a), \\ \text{If } a &\neq -1, & p &= 3; & C_{p+1} &= -\frac{1}{24}(1 + a), \\ \text{If } a &= -1, & p &= 4; & C_{p+1} &= -\frac{1}{90}.\end{aligned}$$

$k = 3 :$

$$\begin{aligned}\alpha_3 &= 1, & \beta_3 &= \frac{1}{24}(9 + a + b), \\ \alpha_2 &= -1 - a, & \beta_2 &= \frac{1}{24}(19 - 13a - 5b), \\ \alpha_1 &= a + b, & \beta_1 &= \frac{1}{24}(-5 - 13a + 19b), \\ \alpha_0 &= -b, & \beta_0 &= \frac{1}{24}(1 + a + 9b), \\ p &= 4; & C_{p+1} &= -\frac{1}{720}(19 + 11a + 19b).\end{aligned}$$

Absolute stability limits the order to 4.

$k = 4 :$

$$\begin{aligned}\alpha_4 &= 1, & \beta_4 &= \frac{1}{720}(251 + 19a + 11b + 19c), \\ \alpha_3 &= -1 - a, & \beta_3 &= \frac{1}{360}(323 - 173a - 37b - 53c), \\ \alpha_2 &= a + b, & \beta_2 &= \frac{1}{30}(-11 - 19a + 19b + 11c), \\ \alpha_1 &= -b - c, & \beta_1 &= \frac{1}{360}(53 + 37a + 173b - 323c), \\ \alpha_0 &= c, & \beta_0 &= \frac{1}{720}(-19 - 11a - 19b - 251c).\end{aligned}$$

If $27 + 11a + 11b + 27c \neq 0$, then

$$p = 5; \quad C_{p+1} = -\frac{1}{1440}(27 + 11a + 11b + 27c).$$

If $27 + 11a + 11b + 27c = 0$, then

$$p = 6; \quad C_{p+1} = -\frac{1}{15120}(74 + 10a - 10b - 74c).$$

Absolute stability limits the order to 6.

The Matlab solver `ode113` is a fully variable step size, PECE implementation in terms of modified divided differences of the Adams–Bashforth–Moulton family of formulae of orders 1 to 12. The natural “free” interpolants are used. Local extrapolation is done. Details are to be found in *The MATLAB ODE Suite*, L. F. Shampine and M. W. Reichelt, SIAM Journal on Scientific Computing, **18**(1), 1997.

5.9. Stiff Systems of Differential Equations

In this section, we illustrate the concept of stiff systems of differential equations by means of an example and mention some numerical methods that can handle such systems.

5.9.1. The phenomenon of stiffness. While the intuitive meaning of stiff is clear to all specialists, much controversy is going on about its correct mathematical definition. The most pragmatic opinion is also historically the first one: stiff equations are equations where certain implicit methods, in particular backward differentiation methods, perform much better than explicit ones (see [1], p. 1).

Consider a system of n differential equations,

$$\mathbf{y}' = \mathbf{f}(x, \mathbf{y}),$$

and let $\lambda_1, \lambda_2, \dots, \lambda_n$ be the eigenvalues of the $n \times n$ Jacobian matrix

$$J = \frac{\partial \mathbf{f}}{\partial \mathbf{y}} = \left(\frac{\partial f_i}{\partial y_j} \right), \quad i \downarrow 1, \dots, n, \quad j \rightarrow 1, \dots, n, \quad (5.36)$$

where Nagumo's matrix index notation has been used. We assume that the n eigenvalues, $\lambda_1, \dots, \lambda_n$, of the matrix J have negative real parts, $\operatorname{Re} \lambda_j < 0$, and are ordered as follows:

$$\operatorname{Re} \lambda_n \leq \dots \leq \operatorname{Re} \lambda_2 \leq \operatorname{Re} \lambda_1 < 0. \quad (5.37)$$

The following definition occurs in discussing stiffness.

DEFINITION 5.6. The **stiffness ratio** of the system $\mathbf{y}' = \mathbf{f}(x, \mathbf{y})$ is the positive number

$$r = \frac{\operatorname{Re} \lambda_n}{\operatorname{Re} \lambda_1}, \quad (5.38)$$

where the eigenvalues of the Jacobian matrix (5.36) of the system satisfy the relations (5.37).

The phenomenon of stiffness appears under various aspects (see [2], p. 217–221):

- A linear constant coefficient system is stiff if all of its eigenvalues have negative real parts and the stiffness ratio is large.
- Stiffness occurs when stability requirements, rather than those of accuracy, constrain the step length.
- Stiffness occurs when some components of the solution decay much more rapidly than others.
- A system is said to be stiff in a given interval I containing t if in I the neighboring solution curves approach the solution curve at a rate which is very large in comparison with the rate at which the solution varies in that interval.

A statement that we take as a definition of stiffness is one which merely relates what is observed happening in practice.

DEFINITION 5.7. If a numerical method with a region of absolute stability, applied to a system of differential equation with any initial conditions, is forced to use in a certain interval I of integration a step size which is *excessively small*

in relation to the smoothness of the exact solution in I , then the system is said to be **stiff** in I .

Explicit Runge–Kutta methods and predictor-corrector methods, which, in fact, are explicit pairs, cannot handle stiff systems in an economical way, if they can handle them at all. Implicit methods require the solution of nonlinear equations which are almost always solved by some form of Newton’s method. Two such implicit methods are in the following two sections.

5.9.2. Backward differentiation formulae. We define a k -step **backward differentiation formula** (BDF) in standard form by

$$\sum_{j=0}^k \alpha_j y_{n+j-k+1} = h\beta_k f_{n+1},$$

where $\alpha_k = 1$. BDF’s are implicit methods. Table 5.7 lists the BDF’s of step-number 1 to 6, respectively. In the table, k is the stepnumber, p is the order, C_{p+1} is the error constant, and α is half the angle subtended at the origin by the region of absolute stability R .

TABLE 5.7. Coefficients of the BDF methods.

k	α_6	α_5	α_4	α_3	α_2	α_1	α_0	β_k	p	C_{p+1}	α
1						1	−1	1	1	1	90°
2					1	− $\frac{4}{3}$	$\frac{1}{3}$	$\frac{2}{3}$	2	− $\frac{2}{9}$	90°
3				1	− $\frac{18}{11}$	$\frac{9}{11}$	− $\frac{2}{11}$	$\frac{6}{11}$	3	− $\frac{3}{22}$	86°
4			1	− $\frac{48}{25}$	$\frac{36}{25}$	− $\frac{16}{25}$	$\frac{3}{25}$	$\frac{12}{25}$	4	− $\frac{12}{125}$	73°
5		1	− $\frac{300}{137}$	$\frac{300}{137}$	− $\frac{200}{137}$	$\frac{75}{137}$	− $\frac{12}{137}$	$\frac{60}{137}$	5	− $\frac{110}{137}$	51°
6	1	− $\frac{360}{147}$	$\frac{450}{147}$	− $\frac{400}{147}$	$\frac{225}{147}$	− $\frac{72}{147}$	$\frac{10}{147}$	$\frac{60}{147}$	6	− $\frac{20}{343}$	18°

The left part of Fig. 5.10 shows the upper half of the region of absolute stability of the 1-step BDF, which is the exterior of the unit disk with center 1, and the regions of absolute stability of the 2- and 3-step BDF’s which are the exterior of closed regions in the right-hand plane. The angle subtended at the origin is $\alpha = 90^\circ$ in the first two cases and $\alpha = 88^\circ$ in the third case. The right part of Fig. 5.10 shows the regions of absolute stability of the 4-, 5-, and 6-steps BDF’s which include the negative real axis and make angles subtended at the origin of 73° , 51° , and 18° , respectively.

A short proof of the instability of the BDF formulae for $k \geq 7$ is found in [4]. BDF methods are used to solve stiff systems.

5.9.3. Numerical differentiation formulae. Numerical differentiation formulae (NDF) are a modification of BDF’s. Letting

$$\nabla y_n = y_n - y_{n-1}$$

denote the backward difference of y_n , we rewrite the k -step BDF of order $p = k$ in the form

$$\sum_{m=1}^k \frac{1}{m} \nabla^m y_{n+1} = h f_{n+1}.$$

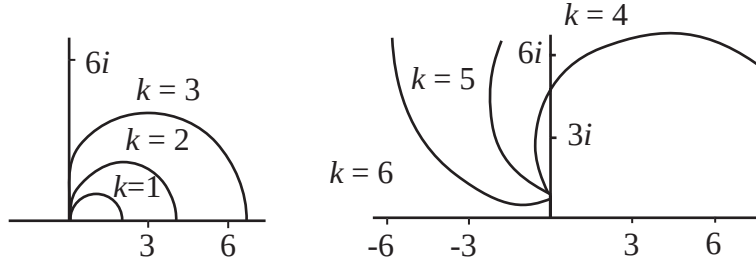


FIGURE 5.10. Left: Regions of absolute stability for k -step BDF for $k = 1, 2, \dots, 6$. These regions include the negative real axis.

The algebraic equation for y_{n+1} is solved with a simplified Newton (chord) iteration. The iteration is started with the predicted value

$$y_{n+1}^{[0]} = \sum_{m=0}^k \frac{1}{m} \nabla^m y_n.$$

Then the k -step NDF of order $p = k$ is

$$\sum_{m=1}^k \frac{1}{m} \nabla^m y_{n+1} = h f_{n+1} + \kappa \gamma_k (y_{n+1} - y_{n+1}^{[0]}),$$

where κ is a scalar parameter and $\gamma_k = \sum_{j=1}^k 1/j$. The NDF of order 1 to 5 are given in Table 5.8.

TABLE 5.8. Coefficients of the NDF methods.

k	κ	α_5	α_4	α_3	α_2	α_1	α_0	β_k	p	C_{p+1}	α
1	$-37/200$					1	-1	1	1	1	90°
2	$-1/9$				1	$-\frac{4}{3}$	$\frac{1}{3}$	$\frac{2}{3}$	2	$-\frac{2}{9}$	90°
3	-0.0823			1	$-\frac{18}{11}$	$\frac{9}{11}$	$-\frac{2}{11}$	$\frac{6}{11}$	3	$-\frac{3}{22}$	80°
4	-0.0415		1	$-\frac{48}{25}$	$\frac{36}{25}$	$-\frac{16}{25}$	$\frac{3}{25}$	$\frac{12}{25}$	4	$-\frac{12}{125}$	66°
5	0	1	$-\frac{300}{137}$	$\frac{300}{137}$	$-\frac{200}{137}$	$\frac{75}{137}$	$-\frac{12}{137}$	$\frac{60}{137}$	5	$-\frac{110}{137}$	51°

In [5], the choice of the number κ is a compromise made in balancing efficiency in step size and stability angle. Compared with the BDF's, there is a step ratio gain of 26% in NDF's of order 1, 2, and 3, 12% in NDF of order 4, and no change in NDF of order 5. The percent change in the stability angle is 0%, 0%, -7%, -10%, and 0%, respectively. No NDF of order 6 is considered because, in this case, the angle α is too small.

5.9.4. The effect of a large stiffness ratio. In the following example, we analyze the effect of the large stiffness ratio of a simple decoupled system of two differential equations with constant coefficients on the step size of the five methods of the ODE Suite. Such problems are called pseudo-stiff since they are quite tractable by implicit methods.

Consider the initial value problem

$$\begin{bmatrix} y_1(x) \\ y_2(x) \end{bmatrix}' = \begin{bmatrix} 1 & 0 \\ 0 & 10^q \end{bmatrix} \begin{bmatrix} y_1(x) \\ y_2(x) \end{bmatrix}, \quad \begin{bmatrix} y_1(0) \\ y_2(0) \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad (5.39)$$

or

$$\mathbf{y}' = A\mathbf{y}, \quad \mathbf{y}(0) = \mathbf{y}_0.$$

Since the eigenvalues of A are

$$\lambda_1 = 1, \quad \lambda_2 = -10^q,$$

the stiffness ratio (5.38) of the system is

$$r = 10^q.$$

The solution is

$$\begin{bmatrix} y_1(x) \\ y_2(x) \end{bmatrix} = \begin{bmatrix} e^{-x} \\ e^{-10^q x} \end{bmatrix}.$$

Even though the second part of the solution containing the fast decaying factor $\exp(-10^q t)$ for large q numerically disappears quickly, the large stiffness ratio continues to restrict the step size of any explicit schemes, including predictor-corrector schemes.

EXAMPLE 5.13. Study the effect of the stiffness ratio on the number of steps used by the five MATLAB ode codes in solving problem (5.39) with $q = 1$ and $q = 5$.

SOLUTION. The function M-file `exp5_13.m` is

```
function uprime = exp5_13(x,u); % Example 5.13
global q % global variable
A=[-1 0;0 -10^q]; % matrix A
uprime = A*u;
```

The following commands solve the non-stiff initial value problem with $q = 1$, and hence $r = e^{10}$, with relative and absolute tolerances equal to 10^{-12} and 10^{-14} , respectively. The option `stats on` requires that the code keeps track of the number of function evaluations.

```
clear;
global q; q=1;
tspan = [0 1]; y0 = [1 1]';
options = odeset('RelTol',1e-12,'AbsTol',1e-14,'Stats','on');
[x23,y23] = ode23('exp5_13',tspan,y0,options);
[x45,y45] = ode45('exp5_13',tspan,y0,options);
[x113,y113] = ode113('exp5_13',tspan,y0,options);
[x23s,y23s] = ode23s('exp5_13',tspan,y0,options);
[x15s,y15s] = ode15s('exp5_13',tspan,y0,options);
```

Similarly, when $q = 5$, and hence $r = \exp(10^5)$, the program solves a pseudo-stiff initial value problem (5.39). Table 5.9 lists the number of steps used with $q = 1$ and $q = 5$ by each of the five methods of the ODE suite.

It is seen from the table that nonstiff solvers are hopelessly slow and very expensive in solving pseudo-stiff equations. $\square$

TABLE 5.9. Number of steps used by each method with $q = 1$ and $q = 5$ with default relative and absolute tolerance $RT = 10^{-3}$ and $AT = 10^{-6}$ respectively, and same tolerance set at tolerances 10^{-12} and 10^{-14} , respectively.

(RT, AT)	$(10^{-3}, 10^{-6})$		$(10^{-12}, 10^{-14})$	
q	1	5	1	5
ode23	29	39 823	24 450	65 944
ode45	13	30 143	601	30 856
ode113	28	62 371	132	64 317
ode23s	37	57	30 500	36 925
ode15s	43	89	773	1 128

We consider another example of a second-order equation, with one real parameter q , which we first solve analytically. We shall obtain a coupled system in this case.

EXAMPLE 5.14. Solve the initial value problem

$$y'' + (10^q + 1)y' + 10^q = 0 \quad \text{on } [0, 1],$$

with initial conditions

$$y(0) = 2, \quad y'(0) = -10^q - 1,$$

and real parameter q .

SOLUTION. Substituting

$$y(x) = e^{\lambda x}$$

in the differential equation, we obtain the characteristic polynomial and eigenvalues:

$$\lambda^2 + (10^q + 1)\lambda + 10^q = (\lambda + 10^q)(\lambda + 1) = 0 \implies \lambda_1 = -10^q, \quad \lambda_2 = -1.$$

Two independent solutions are

$$y_1 = e^{-10^q x}, \quad y_2(x) = e^{-x}.$$

The general solution is

$$y(x) = c_1 e^{-10^q x} + c_2 e^{-x}.$$

Using the initial conditions, one finds that $c_1 = 1$ and $c_2 = 1$. Thus the unique solution is

$$y(x) = e^{-10^q x} + e^{-x}. \quad \square$$

In view of solving the problem in Example 5.14 with numeric Matlab, we reformulate it into a system of two first-order equations.

EXAMPLE 5.15. Reformulate the initial value problem

$$y'' + (10^q + 1)y' + 10^q y = 0 \quad \text{on } [0, 1],$$

with initial conditions

$$y(0) = 2, \quad y'(0) = -10^q - 1,$$

and real parameter q , into a system of two first-order equations and find its vector solution.

SOLUTION. Set

$$u_1 = y, \quad u_2 = y'.$$

Hence,

$$u_2 = u_1', \quad u_2' = y'' = -10^q u_1 - (10^q + 1)u_2.$$

Thus we have the system $\mathbf{u}' = A\mathbf{u}$,

$$\begin{bmatrix} u_1(x) \\ u_2(x) \end{bmatrix}' = \begin{bmatrix} 0 & 1 \\ -10^q & -(10^q + 1) \end{bmatrix} \begin{bmatrix} u_1(x) \\ u_2(x) \end{bmatrix}, \quad \text{with} \quad \begin{bmatrix} u_1(0) \\ u_2(0) \end{bmatrix} = \begin{bmatrix} 2 \\ -10^q - 1 \end{bmatrix}.$$

Substituting the vector function

$$\mathbf{u}(x) = \mathbf{c} e^{\lambda x}$$

in the differential system, we obtain the matrix eigenvalue problem

$$(A - \lambda I)\mathbf{c} = \begin{bmatrix} -\lambda & 1 \\ -10^q & -(10^q + 1) - \lambda \end{bmatrix} \mathbf{c} = 0,$$

This problem has a nonzero solution $\mathbf{c}$ if and only if

$$\det(A - \lambda I) = \lambda^2 + (10^q + 1)\lambda + 10^q = (\lambda + 10^q)(\lambda + 1) = 0.$$

Hence the eigenvalues are

$$\lambda_1 = -10^q, \quad \lambda_2 = -1.$$

The eigenvectors are found by solving the linear systems

$$(A - \lambda_i I)\mathbf{v}_i = 0.$$

Thus,

$$\begin{bmatrix} 10^q & 1 \\ -10^q & -1 \end{bmatrix} \mathbf{v}_1 = 0 \implies \mathbf{v}_1 = \begin{bmatrix} 1 \\ -10^q \end{bmatrix}$$

and

$$\begin{bmatrix} 1 & 1 \\ -10^q & -10^q \end{bmatrix} \mathbf{v}_2 = 0 \implies \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

The general solution is

$$\mathbf{u}(x) = c_1 e^{-10^q x} \mathbf{v}_1 + c_2 e^{-x} \mathbf{v}_2.$$

The initial conditions implies that $c_1 = 1$ and $c_2 = 1$. Thus the unique solution is

$$\begin{bmatrix} u_1(x) \\ u_2(x) \end{bmatrix} = \begin{bmatrix} 1 \\ -10^q \end{bmatrix} e^{-10^q x} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} e^{-x}. \quad \square$$

We see that the stiffness ratio of the equation in Example 5.15 is

$$10^q.$$

EXAMPLE 5.16. Use the five Matlab ode solvers to solve the nonstiff differential equations

$$y'' + (10^q + 1)y' + 10^q = 0 \quad \text{on} \quad [0, 1],$$

with initial conditions

$$y(0) = 2, \quad y'(0) = -10^q - 1,$$

for $q = 1$ and compare the number of steps used by the solvers.

SOLUTION. The function M-file `exp5_16.m` is

```
function uprime = exp5_16(x,u)
global q
A=[0 1;-10^q -1-10^q];
uprime = A*u;
```

The following commands solve the initial value problem.

```
>> clear
>> global q; q = 1;
>> xspan = [0 1]; u0 = [2 -(10^q + 1)]';
>> [x23,u23] = ode23('exp5_16',xspan,u0);
>> [x45,u45] = ode45('exp5_16',xspan,u0);
>> [x113,u113] = ode113('exp5_16',xspan,u0);
>> [x23s,u23s] = ode23s('exp5_16',xspan,u0);
>> [x15s,u15s] = ode15s('exp5_16',xspan,u0);
>> whos
```

Name	Size	Bytes	Class
q	1x1	8	double array (global)
u0	2x1	16	double array
u113	26x2	416	double array
u15s	32x2	512	double array
u23	20x2	320	double array
u23s	25x2	400	double array
u45	49x2	784	double array
x113	26x1	208	double array
x15s	32x1	256	double array
x23	20x1	160	double array
x23s	25x1	200	double array
x45	49x1	392	double array
xspan	1x2	16	double array

Grand total is 461 elements using 3688 bytes

From the table produced by the command `whos` one sees that the nonstiff ode solvers `ode23`, `ode45`, `ode113`, and the stiff ode solvers `ode23s`, `ode15s`, use 20, 49, 26, and 25, 32 steps, respectively. $\square$

EXAMPLE 5.17. Use the five Matlab ode solvers to solve the stiff differential equations

$$y'' + (10^q + 1)y' + 10^q = 0 \quad \text{on } [0, 1],$$

with initial conditions

$$y(0) = 2, \quad y'(0) = -10^q - 1,$$

for $q = 5$ and compare the number of steps used by the solvers.

SOLUTION. Setting the value $q = 5$ in the program of Example 5.16 we obtain the following results for the `whos` command.

```
clear
global q; q = 5;
xspan = [0 1]; u0 = [2 -(10^q + 1)]';
```

```

[x23,u23] = ode23('exp5_16',xspan,u0);
[x45,u45] = ode45('exp5_16',xspan,u0);
[x113,u113] = ode113('exp5_16',xspan,u0);
[x23s,u23s] = ode23s('exp5_16',xspan,u0);
[x15s,u15s] = ode15s('exp5_16',xspan,u0);
whos

```

Name	Size	Bytes	Class
q	1x1	8	double array (global)
u0	2x1	16	double array
u113	62258x2	996128	double array
u15s	107x2	1712	double array
u23	39834x2	637344	double array
u23s	75x2	1200	double array
u45	120593x2	1929488	double array
x113	62258x1	498064	double array
x15s	107x1	856	double array
x23	39834x1	318672	double array
x23s	75x1	600	double array
x45	120593x1	964744	double array
xspan	1x2	16	double array

Grand total is 668606 elements using 5348848 bytes

From the table produced by the command `whos` one sees that the nonstiff ode solvers `ode23`, `ode45`, `ode113`, and the stiff ode solvers `ode23s`, `ode15s`, use 39 834, 120 593, 62 258, and 75, 107 steps, respectively. It follows that nonstiff solvers are hopelessly slow and expensive to solve stiff equations. $\square$

Numeric MATLAB has four solvers with “free” interpolants for stiff systems. The first three are low order solvers.

- The code `ode23s` is an implementation of a new modified Rosenbrock (2,3) pair. Local extrapolation is not done. By default, Jacobians are generated numerically.
- The code `ode23t` is an implementation of the trapezoidal rule.
- The code `ode23tb` is an in an implicit two-stage Runge–Kutta formula.
- The variable-step variable-order Matlab solver `ode15s` is a quasi-constant step size implementation in terms of backward differences of the Klopfenstein–Shampine family of Numerical Differentiation Formulae of orders 1 to 5. Local extrapolation is not done. By default, Jacobians are generated numerically.

Details on these methods are to be found in *The MATLAB ODE Suite*, L. F. Shampine and M. W. Reichelt, SIAM Journal on Scientific Computing, **18**(1), 1997.

CHAPTER 6

The Matlab ODE Suite

6.1. Introduction

The MATLAB ODE suite is a collection of seven user-friendly finite-difference codes for solving initial value problems given by first-order systems of ordinary differential equations and plotting their numerical solutions. The three codes `ode23`, `ode45`, and `ode113` are designed to solve non-stiff problems and the four codes `ode23s`, `ode23t`, `ode23tb` and `ode15s` are designed to solve both stiff and non-stiff problems. This chapter is a survey of the seven methods of the ODE suite. A simple example illustrates the performance of the seven methods on a system with a small and a large stiffness ratio. The available options in the MATLAB codes are listed. The 19 problems solved by the MATLAB `odedemo` are briefly described. These standard problems, which are found in the literature, have been designed to test ode solvers.

6.2. The Methods in the Matlab ODE Suite

The MATLAB ODE suite contains three explicit methods for nonstiff problems:

- The explicit Runge–Kutta pair `ode23` of orders 3 and 2,
- The explicit Runge–Kutta pair `ode45` of orders 5 and 4, of Dormand–Prince,
- The Adams–Bashforth–Moulton predictor-corrector pairs `ode113` of orders 1 to 13,

and four implicit methods for stiff systems:

- The implicit Runge–Kutta pair `ode23s` of orders 2 and 3,
- `ode23t` is an implementation of the trapezoidal rule,
- `ode23tb` is a two-stage implicit Runge-Kutta method,
- The implicit numerical differentiation formulae `ode15s` of orders 1 to 5.

All these methods have a built-in local error estimate to control the step size. Moreover `ode113` and `ode15s` are variable-order packages which use higher order methods and smaller step size when the solution varies rapidly.

The command `odeset` lets one create or alter the ode option structure.

The ODE suite is presented in a paper by Shampine and Reichelt [5] and the MATLAB `help` command supplies precise information on all aspects of their use. The codes themselves are found in the `toolbox/matlab/funfun` folder of MATLAB 6. For MATLAB 4.2 or later, it can be downloaded for free by `ftp` on `ftp.mathworks.com` in the `pub/mathworks/toolbox/matlab/funfun` directory.

In MATLAB 6, the command

`odedemo`

lets one solve 4 nonstiff problems and 15 stiff problems by any of the five methods in the suite. The four methods for stiff problems are also designed to solve nonstiff problems. The three nonstiff methods are poor at solving very stiff problems.

For graphing purposes, all seven methods use interpolants to obtain, by default, four or, if specified by the user, more intermediate values of y between y_n and y_{n+1} to produce smooth solution curves.

6.2.1. The ode23 method. The code `ode23` consists in a four-stage pair of embedded explicit Runge–Kutta methods of orders 2 and 3 with error control. It advances from y_n to y_{n+1} with the third-order method (so called local extrapolation) and controls the local error by taking the difference between the third-order and the second-order numerical solutions. The four stages are:

$$\begin{aligned} k_1 &= hf(x_n, y_n), \\ k_2 &= hf(x_n + (1/2)h, y_n + (1/2)k_1), \\ k_3 &= hf(x_n + (3/4)h, y_n + (3/4)k_2), \\ k_4 &= hf(x_n + h, y_n + (2/9)k_1 + (1/3)k_2 + (4/9)k_3), \end{aligned}$$

The first three stages produce the solution at the next time step:

$$y_{n+1} = y_n + (2/9)k_1 + (1/3)k_2 + (4/9)k_3,$$

and all four stages give the local error estimate:

$$E = -\frac{5}{72}k_1 + \frac{1}{12}k_2 + \frac{1}{9}k_3 - \frac{1}{8}k_4.$$

However, this is really a three-stage method since the first step at x_{n+1} is the same as the last step at x_n , that is $k_1^{[n+1]} = k_4^{[n]}$ (that is, a FSAL method).

The natural interpolant used in `ode23` is the two-point Hermite polynomial of degree 3 which interpolates y_n and $f(x_n, y_n)$ at $x = x_n$, and y_{n+1} and $f(x_{n+1}, y_{n+1})$ at $t = x_{n+1}$.

6.2.2. The ode45 method. The code `ode45` is the Dormand–Prince pair DP(5,4)7M with a high-quality “free” interpolant of order 4 that was communicated to Shampine and Reichelt [5] by Dormand and Prince. Since `ode45` can use long step size, the default is to use the interpolant to compute solution values at four points equally spaced within the span of each natural step.

6.2.3. The ode113 method. The code `ode113` is a variable step variable order method which uses Adams–Bashforth–Moulton predictor–correctors of order 1 to 13. This is accomplished by monitoring the integration very closely. In the MATLAB graphics context, the monitoring is expensive. Although more than graphical accuracy is necessary for adequate resolution of moderately unstable problems, the high accuracy formulae available in `ode113` are not nearly as helpful in the present context as they are in general scientific computation.

6.2.4. The ode23s method. The code `ode23s` is a triple of modified implicit Rosenbrock methods of orders 3 and 2 with error control for stiff systems. It advances from y_n to y_{n+1} with the second-order method (that is, without local extrapolation) and controls the local error by taking the difference between the third- and second-order numerical solutions. Here is the algorithm:

$$\begin{aligned}
 f_0 &= hf(x_n, y_n), \\
 k_1 &= W^{-1}(f_0 + hdT), \\
 f_1 &= f(x_n + 0.5h, y_n + 0.5hk_1), \\
 k_2 &= W^{-1}(f_1 - k_1) + k_1, \\
 y_{n+1} &= y_n + hk_2, \\
 f_2 &= f(x_{n+1}, y_{n+1}), \\
 k_3 &= W^{-1}[f_2 - c_{32}(k_2 - f_1) - 2(k_1 - f_0) + hdt], \\
 \text{error} &\approx \frac{h}{6}(k_1 - 2k_2 + k_3),
 \end{aligned}$$

where

$$W = I - hdJ, \quad d = 1/(2 + \sqrt{2}), \quad c_{32} = 6 + \sqrt{2},$$

and

$$J \approx \frac{\partial f}{\partial y}(x_n, y_n), \quad T \approx \frac{\partial f}{\partial t}(x_n, y_n).$$

This method is FSAL (First Step As Last). The interpolant used in `ode23s` is the quadratic polynomial in s :

$$y_{n+s} = y_n + h \left[\frac{s(1-s)}{1-2d} k_1 + \frac{s(s-2d)}{1-2d} k_2 \right].$$

6.2.5. The ode23t method. The code `ode23t` is an implementation of the trapezoidal rule. It is a low order method which integrates moderately stiff systems of differential equations of the forms $y' = f(t, y)$ and $m(t)y' = f(t, y)$, where the mass matrix $m(t)$ is nonsingular and usually sparse. A free interpolant is used.

6.2.6. The ode23tb method. The code `ode23tb` is an implementation of TR-BDF2, an implicit Runge-Kutta formula with a first stage that is a trapezoidal rule (TR) step and a second stage that is a backward differentiation formula (BDF) of order two. By construction, the same iteration matrix is used in evaluating both stages. It is a low order method which integrates moderately stiff systems of differential equations of the forms $y' = f(t, y)$ and $m(t)y' = f(t, y)$, where the mass matrix $m(t)$ is nonsingular and usually sparse. A free interpolant is used.

6.2.7. The ode15s method. The code `ode15s` for stiff systems is a quasi-constant step size implementation of the NDF's of order 1 to 5 in terms of backward differences. Backward differences are very suitable for implementing the NDF's in MATLAB because the basic algorithms can be coded compactly and efficiently and the way of changing step size is well-suited to the language. Options allow integration with the BDF's and integration with a maximum order less than the default 5. Equations of the form $M(t)y' = f(t, y)$ can be solved by the code `ode15s` for stiff problems with the `Mass` option set to `on`.

6.3. The `odeset` Options

Options for the seven ode solvers can be listed by the `odeset` command (the default values are in curly brackets):

```
odeset
    AbsTol: [ positive scalar or vector {1e-6} ]
    BDF: [ on | {off} ]
    Events: [ on | {off} ]
    InitialStep: [ positive scalar ]
    Jacobian: [ on | {off} ]
    JConstant: [ on | {off} ]
    JPattern: [ on | {off} ]
    Mass: [ on | {off} ]
    MassConstant: [ on | off ]
    MaxOrder: [ 1 | 2 | 3 | 4 | {5} ]
    MaxStep: [ positive scalar ]
    NormControl: [ on | {off} ]
    OutputFcn: [ string ]
    OutputSel: [ vector of integers ]
    Refine: [ positive integer ]
    RelTol: [ positive scalar {1e-3} ]
    Stats: [ on | {off} ]
```

The following commands solve a problem with different methods and different options.

```
[t, y]=ode23('exp2', [0 1], 0, odeset('RelTol', 1e-9, 'Refine', 6));
[t, y]=ode45('exp2', [0 1], 0, odeset('AbsTol', 1e-12));
[t, y]=ode113('exp2', [0 1], 0, odeset('RelTol', 1e-9, 'AbsTol', 1e-12));
[t, y]=ode23s('exp2', [0 1], 0, odeset('RelTol', 1e-9, 'AbsTol', 1e-12));
[t, y]=ode15s('exp2', [0 1], 0, odeset('JConstant', 'on'));
```

The ode options are used in the demo problems in Sections 8 and 9 below. Others ways of inserting the options in the ode M-file are explained in [7].

The command `ODESET` creates or alters `ODE OPTIONS` structure as follows

- `OPTIONS = ODESET('NAME1', VALUE1, 'NAME2', VALUE2, ...)` creates an integrator options structure `OPTIONS` in which the named properties have the specified values. Any unspecified properties have default values. It is sufficient to type only the leading characters that uniquely identify the property. Case is ignored for property names.
- `OPTIONS = ODESET(OLDOPTS, 'NAME1', VALUE1, ...)` alters an existing options structure `OLDOPTS`.
- `OPTIONS = ODESET(OLDOPTS, NEWOPTS)` combines an existing options structure `OLDOPTS` with a new options structure `NEWOPTS`. Any new properties overwrite corresponding old properties.
- `ODESET` with no input arguments displays all property names and their possible values.

Here is the list of the `odeset` properties.

- **RelTol** : Relative error tolerance [positive scalar 1e-3] This scalar applies to all components of the solution vector and defaults to 1e-3

(0.1% accuracy) in all solvers. The estimated error in each integration step satisfies $e(i) \leq \max(\text{RelTol} \cdot \text{abs}(y(i)), \text{AbsTol}(i))$.

- **AbsTol** : Absolute error tolerance [positive scalar or vector 1e-6] A scalar tolerance applies to all components of the solution vector. Elements of a vector of tolerances apply to corresponding components of the solution vector. AbsTol defaults to 1e-6 in all solvers.
- **Refine** : Output refinement factor [positive integer] This property increases the number of output points by the specified factor producing smoother output. Refine defaults to 1 in all solvers except ODE45, where it is 4. Refine does not apply if $\text{length}(\text{TSPAN}) > 2$.
- **OutputFcn** : Name of installable output function [string] This output function is called by the solver after each time step. When a solver is called with no output arguments, OutputFcn defaults to 'odeplot'. Otherwise, OutputFcn defaults to ''.
- **OutputSel** : Output selection indices [vector of integers] This vector of indices specifies which components of the solution vector are passed to the OutputFcn. OutputSel defaults to all components.
- **Stats** : Display computational cost statistics [on | {off}]
- **Jacobian** : Jacobian available from ODE file [on | {off}] Set this property 'on' if the ODE file is coded so that $F(t, y, 'jacobian')$ returns dF/dy .
- **JConstant** : Constant Jacobian matrix dF/dy [on | {off}] Set this property 'on' if the Jacobian matrix dF/dy is constant.
- **JPattern** : Jacobian sparsity pattern available from ODE file [on | {off}] Set this property 'on' if the ODE file is coded so $F([], [], 'jpattern')$ returns a sparse matrix with 1's showing nonzeros of dF/dy .
- **Vectorized** : Vectorized ODE file [on | {off}] Set this property 'on' if the ODE file is coded so that $F(t, [y1\ y2\ \dots])$ returns $[F(t, y1)\ F(t, y2)\ \dots]$.
- **Events** : Locate events [on — off] Set this property 'on' if the ODE file is coded so that $F(t, y, 'events')$ returns the values of the event functions. See ODEFILE.
- **Mass** : Mass matrix available from ODE file [on | {off}] Set this property 'on' if the ODE file is coded so that $F(t, [], 'mass')$ returns time dependent mass matrix $M(t)$.
- **MassConstan** : Constant mass matrix available from ODE file [on | {off}] Set this property 'on' if the ODE file is coded so that $F(t, [], 'mass')$ returns a constant mass matrix M .
- **MaxStep** : Upper bound on step size [positive scalar] MaxStep defaults to one-tenth of the tspan interval in all solvers.
- **InitialStep** : Suggested initial step size [positive scalar] The solver will try this first. By default the solvers determine an initial step size automatically.
- **MaxOrder** : Maximum order of ODE15S [1 | 2 | 3 | 4 | {5}]
- **BDF** : Use Backward Differentiation Formulae in ODE15S [on | {off}] This property specifies whether the Backward Differentiation Formulae (Gear's methods) are to be used in ODE15S instead of the default Numerical Differentiation Formulae.

- **NormControl** : Control error relative to norm of solution [on | {off}]
Set this property 'on' to request that the solvers control the error in each integration step with $\text{norm}(e) \leq \max(\text{RelTol} \cdot \text{norm}(y), \text{AbsTol})$. By default the solvers use a more stringent component-wise error control.

6.4. Nonstiff Problems of the Matlab odedemo

6.4.1. The orbitode problem. ORBITODE is a restricted three-body problem. This is a standard test problem for non-stiff solvers stated in Shampine and Gordon, p. 246 ff in [8]. The first two solution components are coordinates of the body of infinitesimal mass, so plotting one against the other gives the orbit of the body around the other two bodies. The initial conditions have been chosen so as to make the orbit periodic. Moderately stringent tolerances are necessary to reproduce the qualitative behavior of the orbit. Suitable values are $1e-5$ for RelTol and $1e-4$ for AbsTol.

Because this function returns event function information, it can be used to test event location capabilities.

6.4.2. The orbt2ode problem. ORBT2ODE is the non-stiff problem D5 of Hull *et al.* [9] This is a two-body problem with an elliptical orbit of eccentricity 0.9. The first two solution components are coordinates of one body relative to the other body, so plotting one against the other gives the orbit. A plot of the first solution component as a function of time shows why this problem needs a small step size near the points of closest approach. Moderately stringent tolerances are necessary to reproduce the qualitative behavior of the orbit. Suitable values are $1e-5$ for RelTol and $1e-5$ for AbsTol. See [10], p. 121.

6.4.3. The rigidode problem. RIGIDODE solves Euler's equations of a rigid body without external forces.

This is a standard test problem for non-stiff solvers proposed by Krogh. The analytical solutions are Jacobi elliptic functions accessible in MATLAB. The interval of integration $[t_0, t_f]$ is about 1.5 periods; it is that for which solutions are plotted on p. 243 of Shampine and Gordon [8].

RIGIDODE([], [], 'init') returns the default TSPAN, Y0, and OPTIONS values for this problem. These values are retrieved by an ODE Suite solver if the solver is invoked with empty TSPAN or Y0 arguments. This example does not set any OPTIONS, so the third output argument is set to empty [] instead of an OPTIONS structure created with ODESET.

6.4.4. The vdpode problem. VDPODE is a parameterizable van der Pol equation (stiff for large μ). VDPODE(T, Y) or VDPODE(T, Y, [], MU) returns the derivatives vector for the van der Pol equation. By default, MU is 1, and the problem is not stiff. Optionally, pass in the MU parameter as an additional parameter to an ODE Suite solver. The problem becomes stiffer as MU is increased.

For the stiff problem, see Sections 5.9 and 6.5.

6.5. Stiff Problems of the Matlab odedemo

6.5.1. The a2ode and a3ode problems. A2ODE and A3ODE are stiff linear problems with real eigenvalues (problem A2 of [11]). These nine- and four-equation systems from circuit theory have a constant tridiagonal Jacobian and also a constant partial derivative with respect to t because they are autonomous.

REMARK 6.1. When the ODE solver JConstant property is set to 'off', these examples test the effectiveness of schemes for recognizing when Jacobians need to be refreshed. Because the Jacobians are constant, the ODE solver property JConstant can be set to 'on' to prevent the solvers from unnecessarily recomputing the Jacobian, making the integration more reliable and faster.

6.5.2. The b5ode problem. B5ODE is a stiff problem, linear with complex eigenvalues (problem B5 of [11]). See Ex. 5, p. 298 of Shampine [10] for a discussion of the stability of the BDFs applied to this problem and the role of the maximum order permitted (the MaxOrder property accepted by ODE15S). ODE15S solves this problem efficiently if the maximum order of the NDFs is restricted to 2. Remark 6.1 applies to this example.

This six-equation system has a constant Jacobian and also a constant partial derivative with respect to t because it is autonomous.

6.5.3. The buiode problem. BUIODE is a stiff problem with analytical solution due to Bui. The parameter values here correspond to the stiffest case of [12]; the solution is

$$y(1) = e^{-4t}, \quad y(2) = e^{-t}.$$

6.5.4. The brussode problem. BRUSSODE is a stiff problem modelling a chemical reaction (the Brusselator) [1]. The command BRUSSODE(T, Y) or BRUSSODE(T, Y, [], N) returns the derivatives vector for the Brusselator problem. The parameter $N \geq 2$ is used to specify the number of grid points; the resulting system consists of $2N$ equations. By default, N is 2. The problem becomes increasingly stiff and increasingly sparse as N is increased. The Jacobian for this problem is a sparse matrix (banded with bandwidth 5).

BRUSSODE([], [], 'jpattern') or BRUSSODE([], [], 'jpattern', N) returns a sparse matrix of 1's and 0's showing the locations of nonzeros in the Jacobian $\partial F/\partial Y$. By default, the stiff solvers of the ODE Suite generate Jacobians numerically as full matrices. However, if the ODE solver property JPattern is set to 'on' with ODESET, a solver calls the ODE file with the flag 'jpattern'. The ODE file returns a sparsity pattern that the solver uses to generate the Jacobian numerically as a sparse matrix. Providing a sparsity pattern can significantly reduce the number of function evaluations required to generate the Jacobian and can accelerate integration. For the BRUSSODE problem, only 4 evaluations of the function are needed to compute the $2N \times 2N$ Jacobian matrix.

6.5.5. The chm6ode problem. CHM6ODE is the stiff problem CHM6 from Enright and Hull [13]. This four-equation system models catalytic fluidized bed dynamics. A small absolute error tolerance is necessary because $y(:,2)$ ranges from $7e-10$ down to $1e-12$. A suitable AbsTol is $1e-13$ for all solution components. With this choice, the solution curves computed with ode15s are plausible. Because the step sizes span 15 orders of magnitude, a loglog plot is appropriate.

6.5.6. The chm7ode problem. CHM7ODE is the stiff problem CHM7 from [13]. This two-equation system models thermal decomposition in ozone.

6.5.7. The `chm9ode` problem. CHM9ODE is the stiff problem CHM9 from [13]. It is a scaled version of the famous Belousov oscillating chemical system. There is a discussion of this problem and plots of the solution starting on p. 49 of Aiken [14]. Aiken provides a plot for the interval $[0, 5]$, an interval of rapid change in the solution. The default time interval specified here includes two full periods and part of the next to show three periods of rapid change.

6.5.8. The `d1ode` problem. D1ODE is a stiff problem, nonlinear with real eigenvalues (problem D1 of [11]). This is a two-equation model from nuclear reactor theory. In [11] the problem is converted to autonomous form, but here it is solved in its original non-autonomous form. On page 151 in [15], van der Houwen provides the reference solution values

$$t = 400, \quad y(1) = 22.24222011, \quad y(2) = 27.11071335$$

6.5.9. The `fem1ode` problem. FEM1ODE is a stiff problem with a time-dependent mass matrix,

$$M(t)y' = f(t, y).$$

REMARK 6.2. FEM1ODE(T, Y) or FEM1ODE(T, Y, [], N) returns the derivatives vector for a finite element discretization of a partial differential equation. The parameter N controls the discretization, and the resulting system consists of N equations. By default, N is 9.

FEM1ODE(T, [], 'mass') or FEM1ODE(T, [], 'mass', N) returns the time-dependent mass matrix M evaluated at time T . By default, ODE15S solves systems of the form

$$y' = f(t, y).$$

However, if the ODE solver property `Mass` is set to 'on' with ODESET, the solver calls the ODE file with the flag 'mass'. The ODE file returns a mass matrix that the solver uses to solve

$$M(t)y' = f(t, y).$$

If the mass matrix is a constant M , then the problem can also be solved with ODE23S.

FEM1ODE also responds to the flag 'init' (see RIGIDODE).

For example, to solve a 20×20 system, use

```
[t, y] = ode15s('fem1ode', [], [], [], [], 20);
```

6.5.10. The `fem2ode` problem. FEM2ODE is a stiff problem with a time-independent mass matrix,

$$My' = f(t, y).$$

Remark 6.2 applies to this example, which can also be solved by `ode23s` with the command

```
[t, y] = ode23s('fem2ode', [], [], [], [], 20).
```

6.5.11. The `gearode` problem. GEARODE is a simple stiff problem due to Gear as quoted by van der Houwen [15] who, on page 148, provides the reference solution values

$$t = 50, \quad y(1) = 0.5976546988, \quad y(2) = 1.40234334075$$

6.5.12. The hb1ode problem. HB1ODE is the stiff problem 1 of Hindmarsh and Byrne [16]. This is the original Robertson chemical reaction problem on a very long interval. Because the components tend to a constant limit, it tests reuse of Jacobians. The equations themselves can be unstable for negative solution components, which is admitted by the error control. Many codes can, therefore, go unstable on a long time interval because a solution component goes to zero and a negative approximation is entirely possible. The default interval is the longest for which the Hindmarsh and Byrne code EPISODE is stable. The system satisfies a conservation law which can be monitored:

$$y(1) + y(2) + y(3) = 1.$$

6.5.13. The hb2ode problem. HB2ODE is the stiff problem 2 of [16]. This is a non-autonomous diurnal kinetics problem that strains the step size selection scheme. It is an example for which quite small values of the absolute error tolerance are appropriate. It is also reasonable to impose a maximum step size so as to recognize the scale of the problem. Suitable values are an AbsTol of 1e-20 and a MaxStep of 3600 (one hour). The time interval is 1/3; this interval is used by Kahaner, Moler, and Nash, p. 312 in [17], who display the solution on p. 313. That graph is a semilog plot using solution values only as small as 1e-3. A small threshold of 1e-20 specified by the absolute error control tests whether the solver will keep the size of the solution this small during the night time. Hindmarsh and Byrne observe that their variable order code resorts to high orders during the day (as high as 5), so it is not surprising that relatively low order codes like ODE23S might be comparatively inefficient.

6.5.14. The hb3ode problem. HB3ODE is the stiff problem 3 of Hindmarsh and Byrne [16]. This is the Hindmarsh and Byrne mockup of the diurnal variation problem. It is not nearly as realistic as HB2ODE and is quite special in that the Jacobian is constant, but it is interesting because the solution exhibits quasi-discontinuities. It is posed here in its original non-autonomous form. As with HB2ODE, it is reasonable to impose a maximum step size so as to recognize the scale of the problem. A suitable value is a MaxStep of 3600 (one hour). Because $y(:,1)$ ranges from about 1e-27 to about 1.1e-26, a suitable AbsTol is 1e-29.

Because of the constant Jacobian, the ODE solver property JConstant prevents the solvers from recomputing the Jacobian, making the integration more reliable and faster.

6.5.15. The vdpode problem. VDPODE is a parameterizable van der Pol equation (stiff for large μ) [18]. VDPODE(T, Y) or VDPODE(T, Y, [], MU) returns the derivatives vector for the van der Pol equation. By default, MU is 1, and the problem is not stiff. Optionally, pass in the MU parameter as an additional parameter to an ODE Suite solver. The problem becomes more stiff as MU is increased.

When MU is 1000 the equation is in relaxation oscillation, and the problem becomes very stiff. The limit cycle has portions where the solution components change slowly and the problem is quite stiff, alternating with regions of very sharp change where it is not stiff (quasi-discontinuities). The initial conditions are close to an area of slow change so as to test schemes for the selection of the initial step size.

VDPODE(T, Y, 'jacobian') or VDPODE(T, Y, 'jacobian', MU) returns the Jacobian matrix $\partial F/\partial Y$ evaluated analytically at (T, Y). By default, the stiff solvers of the ODE Suite approximate Jacobian matrices numerically. However, if the ODE Solver property Jacobian is set to 'on' with ODESET, a solver calls the ODE file with the flag 'jacobian' to obtain $\partial F/\partial Y$. Providing the solvers with an analytic Jacobian is not necessary, but it can improve the reliability and efficiency of integration.

VDPODE([], [], 'init') returns the default TSPAN, Y0, and OPTIONS values for this problem (see RIGIDODE). The ODE solver property Vectorized is set to 'on' with ODESET because VDPODE is coded so that calling VDPODE(T, [Y1 Y2 ...]) returns [VDPODE(T, Y1) VDPODE(T, Y2) ...] for scalar time T and vectors Y1, Y2, ... The stiff solvers of the ODE Suite take advantage of this feature when approximating the columns of the Jacobian numerically.

6.6. Concluding Remarks

Ongoing research in explicit and implicit Runge–Kutta pairs, and hybrid methods, which incorporate function evaluations at off-step points in order to lower the stepnumber of a linear multistep method without reducing its order, may, in the future, improve the MATLAB ODE suite.

Bibliography

- [1] E. Hairer and G. Wanner, *Solving ordinary differential equations II, stiff and differential-algebraic problems*, Springer-Verlag, Berlin, 1991, pp. 5–8.
- [2] J. D. Lambert, *Numerical methods for ordinary differential equations. The initial value problem*, Wiley, Chichester, 1991.
- [3] J. R. Dormand and P. J. Prince, *A family of embedded Runge–Kutta formulae*, J. Computational and Applied Mathematics, **6**(2) (1980), 19–26.
- [4] E. Hairer and G. Wanner, *On the instability of the BDF formulae*, SIAM J. Numer. Anal., **20**(6) (1983), 1206–1209.
- [5] L. F. Shampine and M. W. Reichelt, *The Matlab ODE suite*, SIAM J. Sci. Comput., **18**(1), (1997) 1–22.
- [6] R. Ashino and R. Vaillancourt, *Hayawakari Matlab (Introduction to Matlab)*, Kyoritsu Shuppan, Tokyo, 1997, xvi–211 pp., 6th printing, 1999 (in Japanese). (Korean translation, 1998.)
- [7] *Using MATLAB*, Version, 5.1, The MathWorks, Chapter 8, Natick, MA, 1997.
- [8] L. F. Shampine and M. K. Gordon, *Computer solution of ordinary differential equations*, W.H. Freeman & Co., San Francisco, 1975.
- [9] T. E. Hull, W. H. Enright, B. M. Fellen, and A. E. Sedgwick, *Comparing numerical methods for ordinary differential equations*, SIAM J. Numer. Anal., **9**(4) (1972) 603–637.
- [10] L. F. Shampine, *Numerical solution of ordinary differential equations*, Chapman & Hall, New York, 1994.
- [11] W. H. Enright, T. E. Hull, and B. Lindberg, *Comparing numerical methods for stiff systems of ODEs*, BIT **15**(1) (1975), 10–48.
- [12] L. F. Shampine, *Measuring stiffness*, Appl. Numer. Math., **1**(2) (1985), 107–119.
- [13] W. H. Enright and T. E. Hull, *Comparing numerical methods for the solution of stiff systems of ODEs arising in chemistry*, in Numerical Methods for Differential Systems, L. Lapidus and W. E. Schiesser eds., Academic Press, Orlando, FL, 1976, pp. 45–67.
- [14] R. C. Aiken, ed., *Stiff computation*, Oxford Univ. Press, Oxford, 1985.
- [15] P. J. van der Houwen, *Construction of integration formulas for initial value problems*, North-Holland Publishing Co., Amsterdam, 1977.
- [16] A. C. Hindmarsh and G. D. Byrne, *Applications of EPISODE: An experimental package for the integration of ordinary differential equations*, in Numerical Methods for Differential Systems, L. Lapidus and W. E. Schiesser eds., Academic Press, Orlando, FL, 1976, pp. 147–166.
- [17] D. Kahaner, C. Moler, and S. Nash, *Numerical methods and software*, Prentice-Hall, Englewood Cliffs, NJ, 1989.
- [18] L. F. Shampine, *Evaluation of a test set for stiff ODE solvers*, ACM Trans. Math. Soft., **7**(4) (1981) 409–420.

CHAPTER 7

Orthogonal polynomials

Orthogonal polynomials are solutions of Sturm–Liouville problems given by second-order differential equations with boundary conditions. These polynomials have desirable properties in the applications.

7.1. Fourier–Legendre Series

Properties of the Legendre polynomials are listed in Section 8.1. We present simple examples of expansions in Fourier–Legendre series.

EXAMPLE 7.1. Expand the polynomial

$$p(x) = x^3 - 2x^2 + 4x + 1$$

over $[-1, 1]$ in terms of the Legendre polynomials $P_0(x), P_1(x), \dots$.

SOLUTION. We express the powers of x in terms of the basis of Legendre polynomials:

$$\begin{aligned} P_0(x) = 1 &\implies 1 = P_0(x), \\ P_1(x) = x &\implies x = P_1(x), \\ P_2(x) = \frac{1}{2}(3x^2 - 1) &\implies x^2 = \frac{2}{3}P_2(x) + \frac{1}{3}P_0(x), \\ P_3(x) = \frac{1}{2}(5x^3 - 3x) &\implies x^3 = \frac{2}{5}P_3(x) + \frac{3}{5}P_1(x). \end{aligned}$$

This way, one avoids computing integrals. Thus

$$\begin{aligned} p(x) &= \frac{2}{5}P_3(x) + \frac{3}{5}P_1(x) - \frac{4}{3}P_2(x) - \frac{2}{3}P_0(x) + 4P_1(x) + P_0(x) \\ &= \frac{2}{5}P_3(x) - \frac{4}{3}P_2(x) + \frac{23}{5}P_1(x) + \frac{1}{3}P_0(x). \quad \square \end{aligned}$$

EXAMPLE 7.2. Expand the polynomial

$$p(x) = 2 + 3x + 5x^2$$

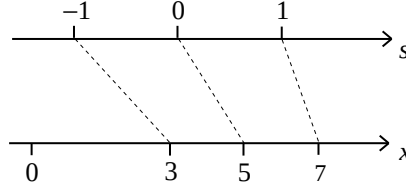
over $[3, 7]$ in terms of the Legendre polynomials $P_0(x), P_1(x), \dots$.

SOLUTION. To map the segment $x \in [3, 7]$ onto the segment $s \in [-1, 1]$ (see Fig. 7.1) we consider the affine transformation

$$s \mapsto x = \alpha s + \beta, \quad \text{such that} \quad -1 \mapsto 3 = -\alpha + \beta, \quad 1 \mapsto 7 = \alpha + \beta.$$

Solving for α and β , we have

$$x = 2s + 5. \tag{7.1}$$

FIGURE 7.1. Affine mapping of $x \in [3, 7]$ onto $s \in [-1, 1]$.

Then

$$\begin{aligned}
 p(x) &= p(2s + 5) \\
 &= 2 + 3(2s + 5) + 5(2s + 5)^2 \\
 &= 142 + 106s + 20s^2 \\
 &= 142P_0(s) + 106P_1(s) + 20 \left[\frac{2}{3}P_2(s) + \frac{1}{3}P_0(s) \right];
 \end{aligned}$$

consequently, we have

$$p(x) = \left(142 + \frac{20}{3}\right) P_0\left(\frac{x-5}{2}\right) + 106P_1\left(\frac{x-5}{2}\right) + \frac{40}{3}P_2\left(\frac{x-5}{2}\right). \quad \square$$

EXAMPLE 7.3. Compute the first three terms of the Fourier–Legendre expansion of the function

$$f(x) = \begin{cases} 0, & -1 < x < 0, \\ x, & 0 < x < 1. \end{cases}$$

SOLUTION. Putting

$$f(x) = \sum_{m=0}^{\infty} a_m P_m(x), \quad -1 < x < 1,$$

we have

$$a_m = \frac{2m+1}{2} \int_{-1}^1 f(x) P_m(x) dx.$$

Hence

$$\begin{aligned}
 a_0 &= \frac{1}{2} \int_{-1}^1 f(x) P_0(x) dx = \frac{1}{2} \int_0^1 x dx = \frac{1}{4}, \\
 a_1 &= \frac{3}{2} \int_{-1}^1 f(x) P_1(x) dx = \frac{3}{2} \int_0^1 x^2 dx = \frac{1}{2}, \\
 a_2 &= \frac{5}{2} \int_{-1}^1 f(x) P_2(x) dx = \frac{5}{2} \int_0^1 x \frac{1}{2} (3x^2 - 1) dx = \frac{5}{16}.
 \end{aligned}$$

Thus we have the approximation

$$f(x) \approx \frac{1}{4} P_0(x) + \frac{1}{2} P_1(x) + \frac{5}{16} P_2(x). \quad \square$$

EXAMPLE 7.4. Compute the first three terms of the Fourier–Legendre expansion of the function

$$f(x) = e^x, \quad 0 \leq x \leq 1.$$

SOLUTION. To use the orthogonality of the Legendre polynomials, we transform the domain of $f(x)$ from $[0, 1]$ to $[-1, 1]$ by the substitution

$$s = 2 \left(x - \frac{1}{2} \right), \quad \text{that is} \quad x = \frac{s}{2} + \frac{1}{2}.$$

Then

$$f(x) = e^x = e^{(1+s)/2} = \sum_{m=0}^{\infty} a_m P_m(s), \quad -1 \leq s \leq 1,$$

where

$$a_m = \frac{2m+1}{2} \int_{-1}^1 e^{(1+s)/2} P_m(s) ds.$$

We first compute the following three integrals by recurrence:

$$\begin{aligned} I_0 &= \int_{-1}^1 e^{s/2} ds = 2 \left(e^{1/2} - e^{-1/2} \right), \\ I_1 &= \int_{-1}^1 s e^{s/2} ds = 2s e^{s/2} \Big|_{-1}^1 - 2 \int_{-1}^1 e^{s/2} ds \\ &= 2 \left(e^{1/2} + e^{-1/2} \right) - 2I_0 \\ &= -2e^{1/2} + 6e^{-1/2}, \\ I_2 &= \int_{-1}^1 s^2 e^{s/2} ds = 2s^2 e^{s/2} \Big|_{-1}^1 - 4 \int_{-1}^1 s e^{s/2} ds \\ &= 2 \left(e^{1/2} - e^{-1/2} \right) - 4I_1 \\ &= 10e^{1/2} - 26e^{-1/2}. \end{aligned}$$

Thus

$$\begin{aligned} a_0 &= \frac{1}{2} e^{1/2} I_0 = e - 1 \approx 1.7183, \\ a_1 &= \frac{3}{2} e^{1/2} I_1 = -3e + 9 \approx 0.8452, \\ a_2 &= \frac{5}{2} e^{1/2} \frac{1}{2} (3I_2 - I_0) = 35e - 95 \approx 0.1399. \end{aligned}$$

We finally have the approximation

$$f(x) \approx 1.7183P_0(2x-1) + 0.8452P_1(2x-1) + 0.1399P_2(2x-1). \quad \square$$

7.2. Derivation of Gaussian Quadratures

We easily obtain the n -point Gaussian quadrature formula by means of the Legendre polynomials. We restrict attention to the cases $n = 2$ and $n = 3$. We immediately remark that the number of points n refers to the n points at which we need to evaluate the integrand over the interval $[-1, 1]$, and not to the numbers of subintervals into which one usually breaks the whole interval of integration $[a, b]$ in order to have a smaller error in the numerical value of the integral.

EXAMPLE 7.5. Determine the four parameters of the two-point Gaussian quadrature formula,

$$\int_{-1}^1 f(x) dx = af(x_1) + bf(x_2).$$

SOLUTION. By symmetry, it is expected that the nodes will be negative to each other, $x_1 = -x_2$, and the weights will be equal, $a = b$. Since there are four free parameters, the formula will be exact for polynomials of degree three or less. By Example 7.1, it suffices to consider the polynomials $P_0(x), \dots, P_3(x)$. Since $P_0(x) = 1$ is orthogonal to $P_n(x)$, $n = 1, 2, \dots$, we have

$$2 = \int_{-1}^1 P_0(x) dx = aP_0(x_1) + bP_0(x_2) = a + b, \quad (7.2)$$

$$0 = \int_{-1}^1 1 \times P_1(x) dx = aP_1(x_1) + bP_1(x_2) = ax_1 + bx_2, \quad (7.3)$$

$$0 = \int_{-1}^1 1 \times P_2(x) dx = aP_2(x_1) + bP_2(x_2), \quad (7.4)$$

$$0 = \int_{-1}^1 1 \times P_3(x) dx = aP_3(x_1) + bP_3(x_2), \quad (7.5)$$

To satisfy (7.4) we choose x_1 and x_2 such that

$$P_2(x_1) = P_2(x_2) = 0,$$

that is,

$$P_2(x) = \frac{1}{2}(3x^2 - 1) = 0 \Rightarrow -x_1 = x_2 = \frac{1}{\sqrt{3}} = 0.577\,350\,27.$$

Hence, by (7.3), we have

$$a = b.$$

Moreover, (7.5) is automatically satisfied since $P_3(x)$ is odd. Finally, by (7.2), we have

$$a = b = 1.$$

Thus the two-point Gaussian quadrature formula is

$$\int_{-1}^1 f(x) dx = f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right). \quad \square \quad (7.6)$$

EXAMPLE 7.6. Determine the six parameters of the three-point Gaussian quadrature formula,

$$\int_{-1}^1 f(x) dx = af(x_1) + bf(x_2) + cf(x_3).$$

SOLUTION. By symmetry, it is expected that the two extremal nodes are negative to each other, $x_1 = -x_3$, and the middle node is at the origin, $x_2 = 0$. Moreover, the extremal weights should be equal, $a = c$, and the central one be larger than the other two, $b > a = c$. Since there are six free parameters, the

formula will be exact for polynomials of degree five or less. By Example 7.1, it suffices to consider the basis $P_0(x), \dots, P_5(x)$. Thus,

$$2 = \int_{-1}^1 P_0(x) dx = aP_0(x_1) + bP_0(x_2) + cP_0(x_3), \quad (7.7)$$

$$0 = \int_{-1}^1 P_1(x) dx = aP_1(x_1) + bP_1(x_2) + cP_1(x_3), \quad (7.8)$$

$$0 = \int_{-1}^1 P_2(x) dx = aP_2(x_1) + bP_2(x_2) + cP_2(x_3), \quad (7.9)$$

$$0 = \int_{-1}^1 P_3(x) dx = aP_3(x_1) + bP_3(x_2) + cP_3(x_3), \quad (7.10)$$

$$0 = \int_{-1}^1 P_4(x) dx = aP_4(x_1) + bP_4(x_2) + cP_4(x_3), \quad (7.11)$$

$$0 = \int_{-1}^1 P_5(x) dx = aP_5(x_1) + bP_5(x_2) + cP_5(x_3). \quad (7.12)$$

To satisfy (7.10), we let x_1, x_2, x_3 be the three zeros of

$$P_3(x) = \frac{1}{2}(5x^3 - 3x) = \frac{1}{2}x(5x^2 - 3)$$

that is,

$$-x_1 = x_3 = \sqrt{\frac{3}{5}} = 0.7745967, \quad x_2 = 0.$$

Hence (7.8) implies

$$-\sqrt{\frac{3}{5}}a + \sqrt{\frac{3}{5}}c = 0 \Rightarrow a = c.$$

We immediately see that (7.12) is satisfied since $P_5(x)$ is odd. Moreover, by substituting $a = c$ in (7.9), we have

$$a \frac{1}{2} \left(3 \times \frac{3}{5} - 1 \right) + b \left(-\frac{1}{2} \right) + a \frac{1}{2} \left(3 \times \frac{3}{5} - 1 \right) = 0,$$

that is,

$$4a - 5b + 4a = 0 \quad \text{or} \quad 8a - 5b = 0. \quad (7.13)$$

Now, it follows from (7.7) that

$$2a + b = 2 \quad \text{or} \quad 10a + 5b = 10. \quad (7.14)$$

Adding the second expressions in (7.13) and (7.14), we have

$$a = \frac{10}{18} = \frac{5}{9} = 0.555.$$

Thus

$$b = 2 - \frac{10}{9} = \frac{8}{9} = 0.888.$$

Finally, we verify that (7.11) is satisfied. Since

$$P_4(x) = \frac{1}{8}(35x^4 - 30x^2 + 3),$$

we have

$$\begin{aligned}
 2 \times \frac{5 \times 1}{9 \times 8} \left(35 \times \frac{9}{25} - 30 \times \frac{3}{5} + 3 \right) + \frac{8}{9} \times \frac{3}{8} &= \frac{2 \times 5}{9 \times 8} \left(\frac{315 - 450 + 75}{25} \right) + \frac{8}{9} \times \frac{3}{8} \\
 &= \frac{2 \times 5}{9 \times 8} \times \frac{(-60)}{25} + \frac{8 \times 3}{9 \times 8} \\
 &= \frac{-24 + 24}{9 \times 8} = 0.
 \end{aligned}$$

Therefore, the three-point Gaussian quadrature formula is

$$\int_{-1}^1 f(x) dx = \frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\right). \quad \square \quad (7.15)$$

REMARK 7.1. The interval of integration in the Gaussian quadrature formulae is normalized to $[-1, 1]$. To integrate over the interval $[a, b]$ we use the change of independent variable (see Example 7.2)

$$s \mapsto x = \alpha s + \beta, \quad \text{such that} \quad -1 \mapsto a = -\alpha + \beta, \quad 1 \mapsto b = \alpha + \beta,$$

leading to

$$x = \frac{(b-a)t + b + a}{2}, \quad dx = \left(\frac{b-a}{2}\right) dt.$$

Then,

$$\int_a^b f(x) dx = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{(b-a)t + b + a}{2}\right) dt.$$

EXAMPLE 7.7. Evaluate

$$I = \int_0^{\pi/2} \sin x dx$$

by applying the two-point Gaussian quadrature formula once over the interval $[0, \pi/2]$ and over the half-intervals $[0, \pi/4]$ and $[\pi/4, \pi/2]$.

SOLUTION. Let

$$x = \frac{(\pi/2)t + \pi/2}{2}, \quad dx = \frac{\pi}{4} dt.$$

At $t = -1$, $x = 0$ and, at $t = 1$, $x = \pi/2$. Hence

$$\begin{aligned}
 I &= \frac{\pi}{4} \int_{-1}^1 \sin\left(\frac{\pi t + \pi}{4}\right) dt \\
 &\approx \frac{\pi}{4} [1.0 \times \sin(0.10566\pi) + 1.0 \times \sin(0.39434\pi)] \\
 &= 0.99847.
 \end{aligned}$$

The error is 1.53×10^{-3} . Over the half-intervals, we have

$$\begin{aligned}
 I &= \frac{\pi}{8} \int_{-1}^1 \sin\left(\frac{\pi t + \pi}{8}\right) dt + \frac{\pi}{8} \int_{-1}^1 \sin\left(\frac{\pi t + 3\pi}{8}\right) dt \\
 &\approx \frac{\pi}{8} \left[\sin \frac{\pi}{8} \left(-\frac{1}{\sqrt{3}} + 1\right) + \sin \frac{\pi}{8} \left(\frac{1}{\sqrt{3}} + 1\right) \right. \\
 &\quad \left. + \sin \frac{\pi}{8} \left(-\frac{1}{\sqrt{3}} + 3\right) + \sin \frac{\pi}{8} \left(\frac{1}{\sqrt{3}} + 3\right) \right] \\
 &= 0.99991016676989.
 \end{aligned}$$

The error is 8.983×10^{-5} . The Matlab solution is as follows. For generality, it is convenient to set up a function M-file `exp7_7.m`,

```
function f=exp7_7(t)
% evaluate the function f(t)
f=sin(t);
```

The two-point Gaussian quadrature is programmed as follows.

```
>> clear
>> a = 0; b = pi/2; c = (b-a)/2; d= (a+b)/2;
>> weight = [1 1]; node = [-1/sqrt(3) 1/sqrt(3)];
>> syms x t
>> x = c*node+d;
>> nv1 = c*weight*exp7_7(x)' % numerical value of integral
    nv1 = 0.9985
>> error1 = 1 - nv1 % error in solution
    error1 = 0.0015
```

The other part is done in a similar way. □

REMARK 7.2. The Gaussian quadrature formulae are the most accurate integration formulae for a given number of nodes. The error in the n -point formula is

$$E_n(f) = \frac{2}{(2n+1)!} \left[\frac{2^n (n!)^2}{(2n)!} \right]^2 f^{(2n)}(\xi), \quad -1 < \xi < 1.$$

This formula is therefore exact for polynomials of degree $2n - 1$ or less.

Matlab's adaptive Simpson's rule `quad` and adaptive Newton-Cotes 8-panel rule `quad8` evaluate the integral of Example 7.7 as follows.

```
>> v1 = quad('sin',0,pi/2)
    v1 = 1.00000829552397
>> v2 = quad8('sin',0,pi/2)
    v2 = 1.00000000000000
```

respectively, within a relative error of 10^{-3} .

Uniformly spaced composite rules that are exact for degree d polynomials are efficient if the $(d+1)$ st derivative $f^{(d+1)}$ is uniformly behaved across the interval of integration $[a, b]$. However, if the magnitude of this derivative varies widely across this interval, the error control process may result in an unnecessary number of function evaluations. This is because the number n of nodes is determined by an interval-wide derivative bound M_{d+1} . In regions where $f^{(d+1)}$ is small compared to this value, the subintervals are (possibly) much shorter than necessary. *Adaptive quadrature* methods addresses this problem by discovering where the integrand is ill behaved and shortening the subintervals accordingly. See Section 3.9 for an example.

7.3. Numerical Solution of Integral Equations of the Second Kind

The theory and application of integral equations is an important subject in applied mathematics, science and engineering. In this section we restrict attention

to Fredholm integral equations of the second kind in one variable. The general form of such equation is

$$f(t) = \lambda \int_a^b K(t, s) f(s) ds + g(t), \quad \lambda \neq 0. \quad (7.16)$$

We shall assume that the kernel $K(t, s)$ is continuous on the square $[a, b] \times [a, b] \in \mathbb{R}^2$.

A significant use of Gaussian quadrature formulae is in the numerical solution of Fredholm integral equations of the second kind by the Nyström method. We explain this method.

Let a numerical integration scheme be given:

$$\int_a^b y(s) ds \approx \sum_{j=1}^N w_j y(s_j), \quad (7.17)$$

where the N numbers $\{w_j\}$ are the weights of the quadrature rule and the N points $\{s_j\}$ are the nodes used by the method. One may use the trapezoidal or Simpson's rules, but for smooth nonsingular problems Gaussian quadrature seems by far superior.

If we apply the numerical integration scheme to the integral equation (7.16), we get

$$f(t) = \lambda \sum_{j=1}^N w_j K(t, s_j) f(s_j) + g(t), \quad (7.18)$$

where, for simplicity, we have written $f(t)$ for $f_N(t)$. We evaluate this equation at the quadrature points:

$$f(t_j) = \lambda \sum_{j=1}^N w_j K(t_j, s_j) f(s_j) + g(t_j). \quad (7.19)$$

Let f_i be the vector $f(t_i)$, g_i the vector $g(t_i)$, K_{ij} the matrix $K(t_i, s_j)$, and define

$$\tilde{K}_{ij} = K_{ij} w_j.$$

Then, in matrix notation, the previous equation becomes

$$(I - \lambda \tilde{K}) \mathbf{f} = \mathbf{g}. \quad (7.20)$$

This is a set of N linear algebraic equations in N unknowns that can be solved by the LU decomposition (see Chapter 4).

Having obtained the solution at the quadrature points $\{t_i\}$, how do we get the solution at some other point t ? We do not simply use polynomial interpolation since this destroys the accuracy we worked hard to achieve. Nyström's key observation is to use (7.18) as an interpolatory formula which maintains the accuracy of the solution.

In Example 7.8, we compare the performance of the three-point Simpson rule and three-point Gaussian quadrature, respectively.

EXAMPLE 7.8. Consider the integral equation

$$f(t) = \lambda \int_0^1 e^{ts} f(s) ds + g(t), \quad 0 \leq x \leq 1, \quad (7.21)$$

with $\lambda = 0.5$ and $f(t) = e^t$. Compare the errors in the numerical solutions at the nodes of Simpson's rule and three-point Gaussian quadrature, respectively.

SOLUTION. Substituting $f(t) = e^t$ in the integral equation, we see that the function $g(t)$ on the right-hand side is

$$g(t) = e^t - \frac{1}{2(t+1)} (1 - e^{t+1}).$$

This is easily obtained by the symbolic Matlab commands

```
>> clear; syms s t; lambda = 1/2;
>> g = exp(t)-lambda*int(exp((t+1)*s),s,0,1)
g = exp(t)-1/2/(t+1)*exp(t+1)+1/2/(t+1)
```

Applying Simpson's rule to equation (7.21), with nodes

$$t_1 = 0, \quad t_2 = 0.5, \quad t_3 = 1,$$

and solving the resulting algebraic system (7.20), say, by the LU decomposition, we have the error in the solution f_3 :

$$\begin{bmatrix} f(0) \\ f(0.5) \\ f(1) \end{bmatrix} - \begin{bmatrix} f_3(0) \\ f_3(0.5) \\ f_3(1) \end{bmatrix} = \begin{bmatrix} -0.0047 \\ -0.0080 \\ -0.0164 \end{bmatrix}$$

Applying the three-point Gaussian quadrature to equation (7.21), with nodes

$$t_1 = \frac{1 - \sqrt{0.6}}{2} \approx 0.11270167, \quad t_2 = 0.5, \quad t_3 = \frac{1 + \sqrt{0.6}}{2} \approx 0.88729833,$$

and solving the resulting algebraic system (7.20), say, by the LU decomposition, we have the error in the solution f_3 :

$$\begin{bmatrix} f(t_1) \\ f(t_2) \\ f(t_3) \end{bmatrix} - \begin{bmatrix} f_3(t_1) \\ f_3(t_2) \\ f_3(t_3) \end{bmatrix} = \begin{bmatrix} 0.2099 \times 10^{-4} \\ 0.3195 \times 10^{-4} \\ 0.6315 \times 10^{-4} \end{bmatrix} \quad (7.22)$$

which is much smaller than with Simpson's rule when using the same number of nodes.

The function M-file `exp7_8.m`:

```
function g=exp7_8(t) % Example 7.8
% evaluate right-hand side
global lambda
syms s
g = exp(t)-lambda*int(exp(t*s)*exp(s),s,0,1);
\end{verbatim}
computes the value of the function $g(t)$, and
the following Matlab commands produce these results.
\begin{verbatim}
clear; global lambda
lambda = 1/2; h = 1/2;
snode = [0 1/2 1]; sweight = [1/3 4/3 1/3]; % Simpson's rule
sK = h*exp(snode'*snode)*diag(sweight);
sA = eye(3)-lambda*sK;
sb = double(exp7_8(snode)');
\end{verbatim}
```

TABLE 7.1. Nyström–trapezoidal method in Example 7.9.

N	E_1	Ratio	E_2	Ratio
2	5.35E-03		5.44E-3	
4	1.35E-03	3.9	1.37E-03	4.0
8	3.39E-04	4.0	3.44E-04	4.0
16	8.47E-05	4.0	8.61E-05	4.0

```

sf3 = sA\sB;
serror = exp(snode)-sf3'
serror =
    -0.0047    -0.0080    -0.0164
gnode = [(1-sqrt(0.6))/2 1/2 (1+sqrt(0.6))/2]; % Gaussian quadrature
gweight = [5/18 8/18 5/18];
gK = exp(gnode'*gnode)*diag(gweight);
gA = eye(3)-lambda*gK;
gb = double(exp7_8(gnode)');
gf3 = gA\gb;
gerror = exp(gnode)-gf3'
gerror = 1.0e-04 *
    0.2099    0.3195    0.6315

```

Note that the use of matrices in computing sK and gK avoids recourse to loops.

Quadratic interpolation can be used to extend the numerical solution to all other $t \in [0, 1]$, but it generally results in a much larger error. For example,

$$f(1.0) - P_2 f_3(1.0) = 0.0158,$$

where $P_2 f_3(t)$ denotes the quadratic polynomial interpolating the Nyström solution at the Gaussian quadrature nodes given above. In contrast, the Nyström formula (7.18) gives errors that are consistent in size with those in (7.22). For example,

$$f(1.0) - f_3(1.0) = 8.08 \times 10^{-5}. \quad \square$$

EXAMPLE 7.9. Consider the integral equation of Example 7.8 with $\lambda = 1/50$ and $f(t) = e^t$. Compare the errors in the Nyström–trapezoidal method and Nyström–Gaussian method, respectively.

SOLUTION. In Table 7.1 we give numerical results when using the trapezoidal rule with n nodes, with $N = 2, 4, 8, 16$. In Table 7.2 we give results when using n -point Gaussian quadratures for $N = 1, 2, 3, 4, 5$. The following norms are used

$$E_1 = \max_{1 \leq i \leq N} |f(t_i) - f_N(t_i)|, \quad E_2 = \max_{0 \leq x \leq 1} |f(x) - f_N(x)|.$$

For E_2 , $f_N(x)$ is obtained using the Nyström interpolation formula (7.18). The results for the trapezoidal rule show clearly the $O(h^2)$ behavior of the error. It is seen that the use of Gaussian quadrature leads to very rapid convergence of f_N to $f(x)$. $\square$

TABLE 7.2. Nyström–Gaussian method in Example 7.9.

N	E_1	Ratio	E_2	Ratio
1	4.19E-03		9.81E-03	
2	1.22E-04	34	2.18E-04	45
3	1.20E-06	100	1.86E-06	117
4	5.09E-09	200	8.47E-09	220
5	1.74E-11	340	2.39E-11	354

CHAPTER 8

Formulae and Tables

8.1. Legendre Polynomials $P_n(x)$ on $[-1, 1]$

- (1) The Legendre differential equation is

$$(1 - x^2)y'' - 2xy' + n(n + 1)y = 0, \quad -1 \leq x \leq 1.$$

- (2) The solution $y(x) = P_n(x)$ is given by the series

$$P_n(x) = \frac{1}{2^n} \sum_{m=0}^{[n/2]} (-1)^m \binom{n}{m} \binom{2n-2m}{n} x^{n-2m},$$

where $[n/2]$ denotes the greatest integer smaller than or equal to $n/2$.

- (3) The three-point recurrence relation is

$$(n + 1)P_{n+1}(x) = (2n + 1)xP_n(x) - nP_{n-1}(x).$$

- (4) The standardization is

$$P_n(1) = 1.$$

- (5) The norm of $P_n(x)$ is

$$\int_{-1}^1 [P_n(x)]^2 dx = \frac{2}{2n + 1}.$$

- (6) Rodrigues's formula is

$$P_n(x) = \frac{(-1)^n}{2^n n!} \frac{d^n}{dx^n} [(1 - x^2)^n]. \quad (8.1)$$

- (7) The generating function is

$$\frac{1}{\sqrt{1 - 2xt + t^2}} = \sum_{n=0}^{\infty} P_n(x)t^n, \quad -1 < x < 1, |t| < 1. \quad (8.2)$$

- (8) The $P_n(x)$ satisfy the inequality

$$|P_n(x)| \leq 1, \quad -1 \leq x \leq 1.$$

- (9) The first six Legendre polynomials are:

$$\begin{aligned} P_0(x) &= 1, & P_1(x) &= x, \\ P_2(x) &= \frac{1}{2}(3x^2 - 1), & P_3(x) &= \frac{1}{2}(5x^3 - 3x), \\ P_4(x) &= \frac{1}{8}(35x^4 - 30x^2 + 3), & P_5(x) &= \frac{1}{8}(63x^5 - 70x^3 + 15x). \end{aligned}$$

The graphs of the first five $P_n(x)$ are shown in Fig. 8.1.

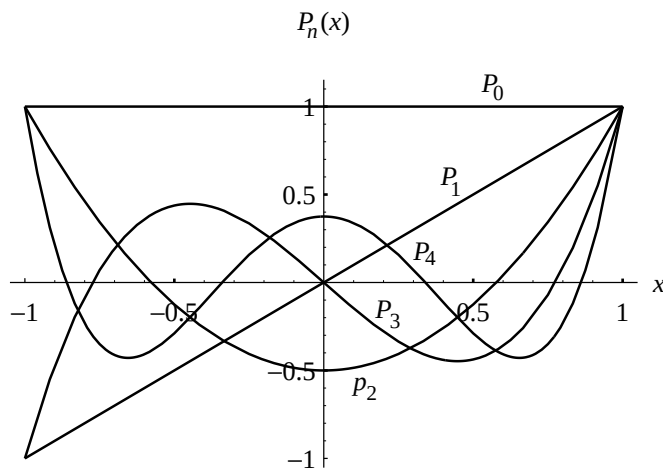


FIGURE 8.1. Plot of the first five Legendre polynomials.

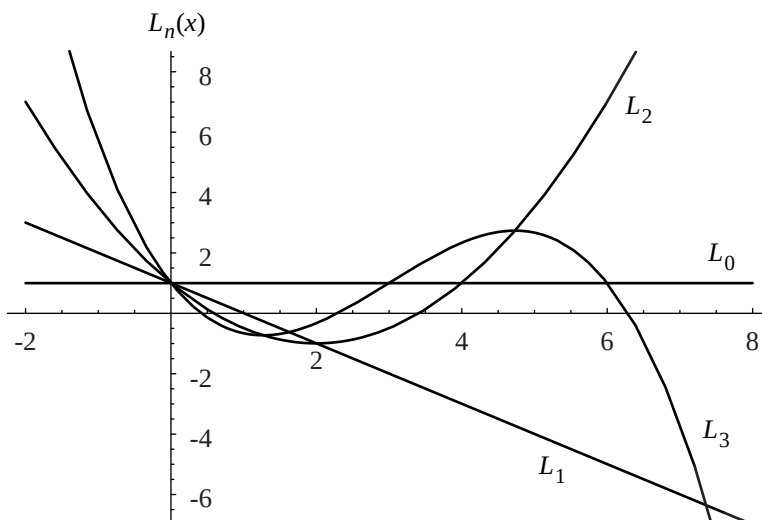


FIGURE 8.2. Plot of the first four Laguerre polynomials.

8.2. Laguerre Polynomials on $0 \leq x < \infty$

Laguerre polynomials on $0 \leq x < \infty$ are defined by the expression

$$L_n(x) = \frac{e^x}{n!} \frac{d^n(x^n e^{-x})}{dx^n}, \quad n = 0, 1, \dots$$

The first four Laguerre polynomials are (see figure 8.2)

$$\begin{aligned} L_0(x) &= 1, & L_1(x) &= 1 - x, \\ L_2(x) &= 1 - 2x + \frac{1}{2}x^2, & L_3(x) &= 1 - 3x + \frac{3}{2}x^2 - \frac{1}{6}x^3. \end{aligned}$$

The $L_n(x)$ can be obtained by the three-point recurrence formula

$$(n+1)L_{n+1}(x) = (2n+1-x)L_n(x) - nL_{n-1}(x).$$

The $L_n(x)$ are solutions of the differential equation

$$xy'' + (1-x)y' + ny = 0$$

and satisfy the orthogonality relations with weight $p(x) = e^{-x}$

$$\int_0^\infty e^{-x} L_m(x) L_n(x) dx = \begin{cases} 0, & m \neq n, \\ 1, & m = n. \end{cases}$$

8.3. Fourier-Legendre Series Expansion

The Fourier-Legendre series expansion of a function $f(x)$ on $[-1, 1]$ is

$$f(x) = \sum_{n=0}^{\infty} a_n P_n(x), \quad -1 \leq x \leq 1,$$

where

$$a_n = \frac{2n+1}{2} \int_{-1}^1 f(x) P_n(x) dx, \quad n = 0, 1, 2, \dots$$

This expansion follows from the orthogonality relations

$$\int_{-1}^1 P_m(x) P_n(x) dx = \begin{cases} 0, & m \neq n, \\ \frac{2}{2n+1}, & m = n. \end{cases} \quad (8.3)$$

Exercises for Numerical Methods

Angles are always in radian measure.

Exercises for Chapter 1

1.1. Use the bisection method to find x_3 for $f(x) = \sqrt{x} - \cos x$ on $[0, 1]$. Angles in radian measure.

1.2. Use the bisection method to find x_3 for

$$f(x) = 3(x+1)\left(x - \frac{1}{2}\right)(x-1)$$

on the following intervals:

$$[-2, 1.5], \quad [-1.25, 2.5].$$

1.3. Use the bisection method to find a solution accurate to 10^{-3} for $f(x) = x - \tan x$ on $[4, 4.5]$. Angles in radian measure.

1.4. Use the bisection method to find an approximation to $\sqrt{3}$ correct to within 10^{-4} . [*Hint:* Consider $f(x) = x^2 - 3$.]

1.5. Show that the fixed point iteration

$$x_{n+1} = \sqrt{2x_n + 3}$$

for the solving the equation $f(x) = x^2 - 2x - 3 = 0$ converges in the interval $[2, 4]$.

1.6. Use a fixed point iteration method, other than Newton's method, to determine a solution accurate to 10^{-2} for $f(x) = x^3 - x - 1 = 0$ on $[1, 2]$. Use $x_0 = 1$.

1.7. Use a fixed point iteration method to find an approximation to $\sqrt{3}$ correct to within 10^{-4} . Compare your result and the number of iterations required with the answer obtained in Exercise 1.4.

1.8. Do five iterations of the fixed point method $g(x) = \cos(x - 1)$. Take $x_0 = 2$. Use at least 6 decimals. Find the order of convergence of the method. Angles in radian measure.

1.9. Do five iterations of the fixed point method $g(x) = 1 + \sin^2 x$. Take $x_0 = 1$. Use at least 6 decimals. Find the order of convergence of the method. Angles in radian measure.

1.10. Sketch the function $f(x) = 2x - \tan x$ and compute a root of the equation $f(x) = 0$ to six decimals by means of Newton's method with $x_0 = 1$. Find the order of convergence of the method.

1.11. Sketch the function $f(x) = e^{-x} - \tan x$ and compute a root of the equation $f(x) = 0$ to six decimals by means of Newton's method with $x_0 = 1$. Find the order of convergence of the method.

1.12 Compute a root of the equation $f(x) = 2x - \tan x$ given in Exercise 1.10 with the secant method with starting values $x_0 = 1$ and $x_1 = 0.5$. Find the order of convergence to the root.

1.13. Repeat Exercise 1.12 with the method of false position. Find the order of convergence of the method.

1.14. Repeat Exercise 1.11 with the secant method with starting values $x_0 = 1$ and $x_1 = 0.5$. Find the order of convergence of the method.

1.15. Repeat Exercise 1.14 with the method of false position. Find the order of convergence of the method.

1.16. Consider the fixed point method of Exercise 1.5:

$$x_{n+1} = \sqrt{2x_n + 3}.$$

Complete the table:

n	x_n	Δx_n	$\Delta^2 x_n$
1	$x_1 = 4.000$		
2	$x_2 =$		
3	$x_3 =$		

Accelerate convergence by Aitken.

$$\hat{x} = x_1 - \frac{(\Delta x_1)^2}{\Delta^2 x_1} = \text{}$$

1.17. Apply Steffensen's method to the result of Exercise 1.9. Find the order of convergence of the method.

1.18. Use Müller's method to find the three zeros of

$$f(x) = x^3 + 3x^2 - 1.$$

1.19. Use Müller's method to find the four zeros of

$$f(x) = x^4 + 2x^2 - x - 3.$$

1.20. Sketch the function $f(x) = x - \tan x$ and compute a root of the equation $f(x) = 0$ to six decimals by means of Newton's method with $x_0 = 1$. Find the multiplicity of the root and the order of convergence of Newton's method to this root.

Exercises for Chapter 2

2.1. Given the function $f(x) = \ln(x+1)$ and the points $x_0 = 0$, $x_1 = 0.6$ and $x_2 = 0.9$. Construct the Lagrange interpolating polynomials of degrees exactly one and two to approximate $f(0.45)$ and find the actual errors.

2.2. Consider the data

$$f(8.1) = 16.94410, \quad f(8.3) = 17.56492, \quad f(8.6) = 18.50515, \quad f(8.7) = 18.82091.$$

Interpolate $f(8.4)$ by Lagrange interpolating polynomials of degree one, two and three.

2.3. Construct the Lagrange interpolating polynomial of degree 2 for the function $f(x) = e^{2x} \cos 3x$, using the values of f at the points $x_0 = 0$, $x_1 = 0.3$ and $x_2 = 0.6$.

2.4. The three points

$$(0.1, 1.0100502), \quad (0.2, 1.04081077), \quad (0.4, 1.1735109)$$

lie on the graph of a certain function $f(x)$. Use these points to estimate $f(0.3)$.

2.5. Complete the following table of divided differences:

i	x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$f[x_i, x_{i+1}, x_{i+2}, x_{i+3}]$
0	3.2	22.0			
1	2.7	17.8	8.400		
2	1.0	14.2		2.856	
3	4.8	38.3			-0.528
4	5.6	5.17			

Write the interpolating polynomial of degree 3 that fits the data at all points from $x_0 = 3.2$ to $x_3 = 4.8$.

2.6. Interpolate the data

$$(-1, 2), \quad (0, 0), \quad (1.5, -1), \quad (2, 4),$$

by means of Newton's divided difference interpolating polynomial of degree three. Plot the data and the interpolating polynomial on the same graph.

2.7. Repeat Exercise 2.1 using Newton's divided difference interpolating polynomials.

2.8. Repeat Exercise 2.2 using Newton's divided difference interpolating polynomials.

2.9. Interpolate the data

$$(-1, 2), \quad (0, 0), \quad (1, -1), \quad (2, 4),$$

by means of Gregory–Newton’s interpolating polynomial of degree three.

2.10. Interpolate the data

$$(-1, 3), \quad (0, 1), \quad (1, 0), \quad (2, 5),$$

by means of Gregory–Newton’s interpolating polynomial of degree three.

2.11. Approximate $f(0.05)$ using the following data and Gregory–Newton’s forward interpolating polynomial of degree four.

x	0.0	0.2	0.4	0.6	0.8
$f(x)$	1.00000	1.22140	1.49182	1.82212	2.22554

2.12. Approximate $f(0.65)$ using the data in Exercise 2.11 and Gregory–Newton’s backward interpolating polynomial of degree four.**2.13.** Construct a Hermite interpolating polynomial of degree three for the data

x	$f(x)$	$f'(x)$
8.3	17.56492	3.116256
8.6	18.50515	3.151762

Exercises for Chapter 3**3.1.** Consider the formulae

$$(4.4) \quad f'(x_0) = \frac{1}{2h}[-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)] + \frac{h^2}{3}f^{(3)}(\xi),$$

$$(4.5) \quad f'(x_0) = \frac{1}{2h}[f(x_0 + h) - f(x_0 - h)] - \frac{h^3}{6}f^{(3)}(\xi),$$

$$(4.6) \quad f'(x_0) = \frac{1}{12h}[f(x_0 - 2h) - 8f(x_0 - h) + 8f(x_0 + h) - f(x_0 + 2h)] + \frac{h^4}{30}f^{(5)}(\xi),$$

$$(4.7) \quad f'(x_0) = \frac{1}{12h}[-25f(x_0) + 48f(x_0 + h) - 36f(x_0 + 2h) + 16f(x_0 + 3h) - 3f(x_0 + 4h) + \frac{h^4}{5}f^{(5)}(\xi),$$

and the table $\{x_n, f(x_n)\}$:

```
x = 1:0.1:1.8; format long; table = [x', (cosh(x)-sinh(x))']
table =
```

1.000000000000000	0.36787944117144
1.100000000000000	0.33287108369808
1.200000000000000	0.30119421191220
1.300000000000000	0.27253179303401
1.400000000000000	0.24659696394161
1.500000000000000	0.22313016014843
1.600000000000000	0.20189651799466
1.700000000000000	0.18268352405273
1.800000000000000	0.1652988822159

For each of the four formulae (4.4)–(4.7) with $h = 0.1$,

- (a) compute the numerical values

$$ndf = f'(1.2)$$

(deleting the error term in the formulae),

- (b) compute the exact value at
- $x = 1.2$
- of the derivative
- $df = f'(x)$
- of the given function

$$f(x) = \cosh x - \sinh x,$$

- (c) compute the error

$$\varepsilon = ndf - df;$$

- (d) verify that
- $|\varepsilon|$
- is bounded by the absolute value of the error term.

3.2. Use Richardson's extrapolation with $h = 0.4$, $h/2$ and $h/4$ to improve the value $f'(1.4)$ obtained by formula (4.5).

3.3. Evaluate $\int_0^1 \frac{dx}{1+x}$ by the trapezoidal rule with $n = 10$.

3.4. Evaluate $\int_0^1 \frac{dx}{1+x}$ by Simpson's rule with $n = 2m = 10$.

3.5. Evaluate $\int_0^1 \frac{dx}{1+2x^2}$ by the trapezoidal rule with $n = 10$.

3.6. Evaluate $\int_0^1 \frac{dx}{1+2x^2}$ by Simpson's rule with $n = 2m = 10$.

3.7. Evaluate $\int_0^1 \frac{dx}{1+x^3}$ by the trapezoidal rule with h for an error of 10^{-4} .

3.8. Evaluate $\int_0^1 \frac{dx}{1+x^3}$ by Simpson's rule with h for an error of 10^{-6} .

3.9. Determine the values of h and n to approximate

$$\int_1^3 \ln x \, dx$$

to 10^{-3} by the following composite rules: trapezoidal, Simpson's, and midpoint.

3.10. Same as Exercise 3.9 with

$$\int_0^2 \frac{1}{x+4} \, dx$$

to 10^{-5} .

3.11. Use Romberg integration to compute $R_{3,3}$ for the integral

$$\int_1^{1.5} x^2 \ln x \, dx.$$

3.12. Use Romberg integration to compute $R_{3,3}$ for the integral

$$\int_1^{1.6} \frac{2x}{x^2-4} \, dx.$$

3.13. Apply Romberg integration to the integral

$$\int_0^1 x^{1/3} dx$$

until $R_{n-1,n-1}$ and $R_{n,n}$ agree to within 10^{-4} .

Exercises for Chapter 4

Solve the following system by the LU decomposition **without pivoting**.

4.1.

$$\begin{array}{rrcr} 2x_1 & + & 2x_2 & + & 2x_3 & = & 4 \\ -x_1 & + & 2x_2 & - & 3x_3 & = & 32 \\ 3x_1 & & & - & 4x_3 & = & 17 \end{array}$$

4.2.

$$\begin{array}{rrcr} x_1 & + & x_2 & + & x_3 & = & 5 \\ x_1 & + & 2x_2 & + & 2x_3 & = & 6 \\ x_1 & + & 2x_2 & + & 3x_3 & = & 8 \end{array}$$

Solve the following system by the LU decomposition **with partial pivoting**.

4.3.

$$\begin{array}{rrcr} 2x_1 & - & x_2 & + & 5x_3 & = & 4 \\ -6x_1 & + & 3x_2 & - & 9x_3 & = & -6 \\ 4x_1 & - & 3x_2 & & & = & -2 \end{array}$$

4.4.

$$\begin{array}{rrcr} 3x_1 & + & 9x_2 & + & 6x_3 & = & 23 \\ 18x_1 & + & 48x_2 & - & 39x_3 & = & 136 \\ 9x_1 & - & 27x_2 & + & 42x_3 & = & 45 \end{array}$$

4.5. Scale each equation in the l_∞ -norm, so that the largest coefficient of each row on the left-hand side be equal to 1 in absolute value, and solve the scaled system by the LU decomposition with partial pivoting.

$$\begin{array}{rrcr} x_1 & - & x_2 & + & 2x_3 & = & 3.8 \\ 4x_1 & + & 3x_2 & - & x_3 & = & -5.7 \\ 5x_1 & + & 10x_2 & + & 3x_3 & = & 2.8 \end{array}$$

4.6. Find the inverse of the Gaussian transformation

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ -a & 1 & 0 & 0 \\ -b & 0 & 1 & 0 \\ -c & 0 & 0 & 1 \end{bmatrix}.$$

4.7. Find the product of the three Gaussian transformations

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ a & 1 & 0 & 0 \\ b & 0 & 1 & 0 \\ c & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & d & 1 & 0 \\ 0 & e & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & f & 1 \end{bmatrix}.$$

4.8. Find the Cholesky decomposition of

$$A = \begin{bmatrix} 1 & -1 & -2 \\ -1 & 5 & -4 \\ -2 & -4 & 22 \end{bmatrix}, \quad B = \begin{bmatrix} 9 & 9 & 9 & 0 \\ 9 & 13 & 13 & -2 \\ 9 & 13 & 14 & -3 \\ 0 & -2 & -3 & 18 \end{bmatrix}.$$

Solve the following systems by the Cholesky decomposition.

4.9.

$$\begin{bmatrix} 16 & -4 & 4 \\ -4 & 10 & -1 \\ 4 & -1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -12 \\ 3 \\ 1 \end{bmatrix}.$$

4.10.

$$\begin{bmatrix} 4 & 10 & 8 \\ 10 & 26 & 26 \\ 8 & 26 & 61 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 44 \\ 128 \\ 214 \end{bmatrix}.$$

Do three iterations of Gauss–Seidel’s scheme on the following properly permuted systems with given initial values $\mathbf{x}^{(0)}$.

4.11.

$$\begin{array}{rclclcl} 6x_1 & + & x_2 & - & x_3 & = & 3 & \text{with } x_1^{(0)} & = & 1 \\ -x_1 & + & x_2 & + & 7x_3 & = & -17 & x_2^{(0)} & = & 1 \\ x_1 & + & 5x_2 & + & x_3 & = & 0 & x_3^{(0)} & = & 1 \end{array}$$

4.12.

$$\begin{array}{rclclcl} -2x_1 & + & x_2 & + & 6x_3 & = & 22 & \text{with } x_1^{(0)} & = & 1 \\ x_1 & + & 4x_2 & + & 2x_3 & = & 13 & x_2^{(0)} & = & 1 \\ 7x_1 & + & 2x_2 & - & x_3 & = & -6 & x_3^{(0)} & = & 1 \end{array}$$

4.13. Using least squares, fit a straight line to (s, F) :

$$(0.9, 10), \quad (0.5, 5), \quad (1.6, 15), \quad (2.1, 20),$$

where s is the elongation of an elastic spring under a force F , and estimate from it the spring modulus $k = F/s$. ($F = ks$ is called Hooke’s law).

4.14. Using least squares, fit a parabola to the data

$$(-1, 2), \quad (0, 0), \quad (1, 1), \quad (2, 2).$$

4.15. Using least squares, fit $f(x) = a_0 + a_1 \cos x$ to the data

$$(0, 3.7), \quad (1, 3.0), \quad (2, 2.4), \quad (3, 1.8).$$

Note: x in radian measures.

4.16. Using least-squares, approximate the data

x_i	-1	-0.5	0	0.25	0.5	0.75	1
y_i	e^{-1}	$e^{-1/2}$	1	$e^{1/4}$	$e^{1/2}$	$e^{3/4}$	e

by means of

$$f(x) = a_0 P_0(x) + a_1 P_1(x) + a_2 P_2(x),$$

where P_0 , P_1 and P_2 are the Legendre polynomials of degree 0, 1 and 2 respectively. Plot $f(x)$ and $g(x) = e^x$ on the same graph.

Using Theorem 4.3, determine and sketch disks that contain the eigenvalues of the following matrices.

4.17.

$$\begin{bmatrix} -i & 0.1 + 0.1i & 0.5i \\ 0.3i & 2 & 0.3 \\ 0.2 & 0.3 + 0.4i & i \end{bmatrix}.$$

4.18.

$$\begin{bmatrix} -2 & 1/2 & i/2 \\ 1/2 & 0 & i/2 \\ -i/2 & -i/2 & 2 \end{bmatrix}.$$

4.19. Find the l_1 -norm of the matrix in exercise 17 and the l_∞ -norm of the matrix in exercise 18.

Do three iterations of the power method to find the largest eigenvalue, in absolute value, and the corresponding eigenvector of the following matrices.

4.20. $\begin{bmatrix} 10 & 4 \\ 4 & 2 \end{bmatrix}$ with $\mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

4.21. $\begin{bmatrix} 3 & 2 & 3 \\ 2 & 6 & 6 \\ 3 & 6 & 3 \end{bmatrix}$ with $\mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$.

Exercises for Chapter 5

Use Euler's method with $h = 0.1$ to obtain a four-decimal approximation for each initial value problem on $0 \leq x \leq 1$ and plot the numerical solution.

5.1. $y' = e^{-y} - y + 1, \quad y(0) = 1.$

5.2. $y' = x + \sin y, \quad y(0) = 0.$

5.3. $y' = x + \cos y, \quad y(0) = 0.$

5.4. $y' = x^2 + y^2, \quad y(0) = 1.$

5.5. $y' = 1 + y^2, \quad y(0) = 0.$

Use the improved Euler method with $h = 0.1$ to obtain a four-decimal approximation for each initial value problem on $0 \leq x \leq 1$ and plot the numerical solution.

5.6. $y' = e^{-y} - y + 1, \quad y(0) = 1.$

5.7. $y' = x + \sin y, \quad y(0) = 0.$

5.8. $y' = x + \cos y, \quad y(0) = 0.$

5.9. $y' = x^2 + y^2, \quad y(0) = 1.$

5.10. $y' = 1 + y^2, \quad y(0) = 0.$

Use the Runge-Kutta method of order 4 with $h = 0.1$ to obtain a six-decimal approximation for each initial value problem on $0 \leq x \leq 1$ and plot the numerical solution.

5.11. $y' = x^2 + y^2, \quad y(0) = 1.$

5.12. $y' = x + \sin y, \quad y(0) = 0.$

5.13. $y' = x + \cos y, \quad y(0) = 0.$

5.14. $y' = e^{-y}, \quad y(0) = 0.$

5.15. $y' = y^2 + 2y - x, \quad y(0) = 0.$

Use the Matlab `ode23` embedded pair of order 3 with $h = 0.1$ to obtain a six-decimal approximation for each initial value problem on $0 \leq x \leq 1$ and estimate the local truncation error by means of the given formula.

5.16. $y' = x^2 + 2y^2, \quad y(0) = 1.$

5.17. $y' = x + 2 \sin y, \quad y(0) = 0.$

5.18. $y' = x + 2 \cos y, \quad y(0) = 0.$

5.19. $y' = e^{-y}, \quad y(0) = 0.$

5.20. $y' = y^2 + 2y - x, \quad y(0) = 0.$

Use the Adams–Bashforth–Moulton three-step predictor-corrector method with $h = 0.1$ to obtain a six-decimal approximation for each initial value problem on $0 \leq x \leq 1$, estimate the local error at $x = 0.5$, and plot the numerical solution.

5.21. $y' = x + \sin y, \quad y(0) = 0.$

5.22. $y' = x + \cos y, \quad y(0) = 0.$

5.23. $y' = y^2 - y + 1, \quad y(0) = 0.$

Use the Adams–Bashforth–Moulton four-step predictor-corrector method with $h = 0.1$ to obtain a six-decimal approximation for each initial value problem on $0 \leq x \leq 1$, estimate the local error at $x = 0.5$, and plot the numerical solution.

5.24. $y' = x + \sin y, \quad y(0) = 0.$

5.25. $y' = x + \cos y, \quad y(0) = 0.$

5.26. $y' = y^2 - y + 1, \quad y(0) = 0.$

Solutions to Exercises for Numerical Methods

Solutions to Exercises for Chapter 1

Ex. 1.11. Sketch the function

$$f(x) = e^{-x} - \tan x$$

and compute a root of the equation $f(x) = 0$ to six decimals by means of Newton's method with $x_0 = 1$.

SOLUTION. We use the `newton1_11` M-file

```
function f = newton1_11(x); % Exercise 1.11.  
f = x - (exp(-x) - tan(x))/(-exp(-x) - sec(x)^2);
```

We iterate Newton's method and monitor convergence to six decimal places.

```
>> xc = input('Enter starting value:'); format long;  
Enter starting value:1  
>> xc = newton1_11(xc)  
xc = 0.68642146135728  
>> xc = newton1_11(xc)  
xc = 0.54113009740473  
>> xc = newton1_11(xc)  
xc = 0.53141608691193  
>> xc = newton1_11(xc)  
xc = 0.53139085681581  
>> xc = newton1_11(xc)  
xc = 0.53139085665216
```

All the digits in the last value of xc are exact. Note the convergence of order 2. Hence the root is $xc = 0.531391$ to six decimals.

We plot the two functions and their difference. The x -coordinate of the point of intersection of the two functions is the root of their difference.

```
x=0:0.01:1.3;  
subplot(2,2,1); plot(x,exp(-x),x,tan(x));  
title('Plot of exp(-x) and tan(x)'); xlabel('x'); ylabel('y(x)');  
subplot(2,2,2); plot(x,exp(-x)-tan(x),x,0);  
title('Plot of exp(-x)-tan(x)'); xlabel('x'); ylabel('y(x)');  
print -deps Fig9_2
```

□

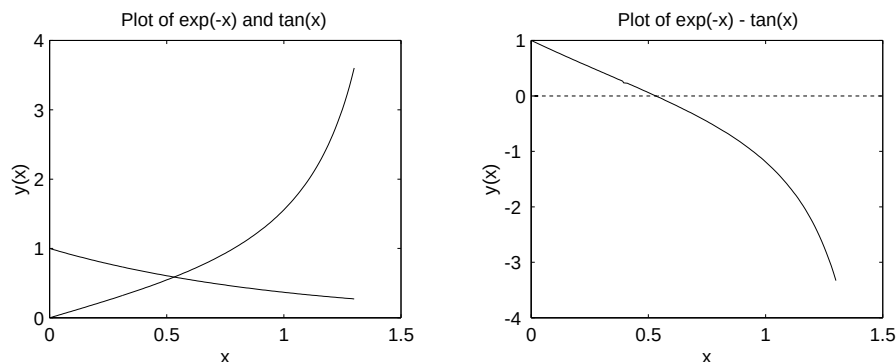


FIGURE 8.3. Graph of two functions and their difference for Exercise 1.11.

Ex. 1.12 Compute a root of the equation $f(x) = x - \tan x$ given in Exercise 1.10 with the secant method with starting values $x_0 = 1$ and $x_1 = 0.5$. Find the order of convergence to the root.

```

SOLUTION. x0 = 1; x1 = 0.5; % starting values
x = zeros(20,1);
x(1) = x0; x(2) = x1;
for n = 3:20
    x(n) = x(n-1) - (x(n-1)-x(n-2)) ...
        / (x(n-1)-tan(x(n-1))-x(n-2)+tan(x(n-2)))*(x(n-1)-tan(x(n-1)));
end
dx = abs(diff(x));
p = 1; % checking convergence of order 1
dxr = dx(2:19)./(dx(1:18).^p);
table = [[0:19]' x [0; dx] [0; 0; dxr]]
table =
    n          x_n      x_n - x_{n-1}      |x_n - x_{n-1}|
                                   /|x_{n-1} - x_{n-2}|

0    1.000000000000000
1    0.500000000000000    0.500000000000000
2    0.45470356524435    0.04529643475565    0.09059286951131
3    0.32718945543123    0.12751410981312    2.81510256824784
4    0.25638399918811    0.07080545624312    0.55527546204022
5    0.19284144711319    0.06354255207491    0.89742451283310
6    0.14671560243705    0.04612584467614    0.72590481763723
7    0.11082587909404    0.03588972334302    0.77808273420254
8    0.08381567002072    0.02701020907332    0.75258894628889
9    0.06330169146740    0.02051397855331    0.75948980985777
10   0.04780894321090    0.01549274825651    0.75522884145761
11   0.03609714636358    0.01171179684732    0.75595347277403
12   0.02725293456160    0.00884421180198    0.75515413367179
13   0.02057409713542    0.00667883742618    0.75516479882196
14   0.01553163187404    0.00504246526138    0.75499146627099

```

15	0.01172476374403	0.00380686813002	0.75496169684658
16	0.00885088980844	0.00287387393559	0.75491817353192
17	0.00668139206035	0.00216949774809	0.75490358892216
18	0.00504365698583	0.00163773507452	0.75489134568691
19	0.00380735389990	0.00123630308593	0.75488588182657

An approximate solution to the triple root $x = 0$ is $x_{19} = 0.0038$. Since the ratio

$$\frac{|x_n - x_{n-1}|}{|x_{n-1} - x_{n-2}|} \rightarrow 0.75 \approx \text{constant}$$

as n grows, we conclude that the method converges to order 1.

Convergence is slow to a triple root. In fact

$$f(0) = f'(0) = f''(0) = 0, \quad f'''(0) \neq 0.$$

In general, the secant method may not converge at all to a multiple root. $\square$

Solutions to Exercises for Chapter 2

Ex. 2.4. The three points

$$(0.1, 1.0100502), \quad (0.2, 1.04081077), \quad (0.4, 1.1735109)$$

lie on the graph of a certain function $f(x)$. Use these points to estimate $f(0.3)$.

SOLUTION. We have

$$f[0.1, 0.2] = \frac{1.04081077 - 1.0100502}{0.1} = 0.307606,$$

$$f[0.2, 0.4] = \frac{1.1735109 - 1.04081077}{0.2} = 0.663501$$

and

$$f[0.1, 0.2, 0.4] = \frac{0.663501 - 0.307606}{0.3} = 1.18632.$$

Therefore,

$$p_2(x) = 1.0100502 + (x - 0.1) \times 0.307606 + (x - 0.1)(x - 0.2) \times 1.18632$$

and

$$p_2(0.3) = 1.0953. \quad \square$$

Ex. 2.12. Approximate $f(0.65)$ using the data in Exercise 2.10

x	0.0	0.2	0.4	0.6	0.8
$f(x)$	1.00000	1.22140	1.49182	1.82212	2.22554

and Gregory–Newton’s backward interpolating polynomial of degree four.

SOLUTION. We construct the difference table.

```
f = [1 1.2214 1.49182 1.82212 2.22554];
ddt = [f' [0 diff(f)]' [0 0 diff(f,2)]' [0 0 0 diff(f,3)]' ...
      [0 0 0 0 diff(f,4)]']
```

The backward difference table is

n	x_n	f_n	∇f_n	$\nabla^2 f_n$	$\nabla^3 f_n$	$\nabla^4 f_n$
0	0.0	1.0000				
			0.22140			
1	0.2	1.2214		0.04902		
			0.27042		0.01086	
2	0.4	1.4918		0.05998		0.00238
			0.33030		0.01324	
3	0.6	1.8221		0.07312		
			0.40342			
4	0.8	2.2255				

```
s = (0.65-0.80)/0.2 % the variable s
s = -0.7500
format long
p4 = ddt(5,1) + s*ddt(5,2) + s*(s+1)*ddt(5,3)/2 ....
      + s*(s+1)*(s+2)*ddt(5,4)/6 + s*(s+1)*(s+2)*(s+3)*ddt(5,5)/24
p4 = 1.91555051757812
```

□

Solutions to Exercises for Chapter 4

Ex. 4.2. Solve the linear system

$$\begin{aligned} x_1 + x_2 + x_3 &= 5 \\ x_1 + 2x_2 + 2x_3 &= 6 \\ x_1 + 2x_2 + 3x_3 &= 8 \end{aligned}$$

by the LU decomposition **without pivoting**.

SOLUTION. The Matlab numeric solution.— In this case, Matlab need not pivot since L will be unit lower triangular. Hence we can use the LU decomposition obtained by Matlab.

```
clear
>> A = [1 1 1; 1 2 2; 1 2 3]; b = [5 6 8]';
>> [L,U] = lu(A) % LU decomposition of A
L =
     1     0     0
     1     1     0
     1     1     1
U =
     1     1     1
     0     1     1
     0     0     1
>> y = L\b % solution by forward substitution
y =
     5
     1
     2
>> x = U\y % solution by backward substitution
```



```
x =
    4
   -1
    2
```

□

Ex. 4.4. Solve the linear system

$$\begin{array}{rrcr} 3x_1 & + & 9x_2 & + & 6x_3 & = & 23 \\ 18x_1 & + & 48x_2 & - & 39x_3 & = & 136 \\ 9x_1 & - & 27x_2 & + & 42x_3 & = & 45 \end{array}$$

by the *LU* decomposition **with pivoting**.

SOLUTION. The Matlab numeric solution.— In this case, Matlab will pivot since L will be a row permutation of a unit lower triangular matrix. Hence we can use the *LU* decomposition obtained by Matlab.

```
clear
>> A = [3 9 6; 18 48 -39; 9 -27 42]; b = [23 136 45]';
>> [L,U] = lu(A) % LU decomposition of A
L =
    0.1667    -0.0196    1.0000
    1.0000         0         0
    0.5000     1.0000         0
U =
   18.0000   48.0000  -39.0000
         0  -51.0000   61.5000
         0         0   13.7059
>> y = L\b % solution by forward substitution
y =
   136.0000
   -23.0000
   -0.1176
>> x = U\y % solution by backward substitution
x =
    6.3619
    0.4406
   -0.0086
```

□

Ex. 4.6. Find the inverse of the Gaussian transformation

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -a & 1 & 0 & 0 \\ -b & 0 & 1 & 0 \\ -c & 0 & 0 & 1 \end{bmatrix}.$$

SOLUTION. The inverse, M^{-1} , of a Gaussian transformation is obtained by changing the signs of the multipliers, that is, of $-a, -b, -c$. Thus

$$M^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ a & 1 & 0 & 0 \\ b & 0 & 1 & 0 \\ c & 0 & 0 & 1 \end{bmatrix}. \quad \square$$

Ex. 4.7. Find the product of the three Gaussian transformations

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ a & 1 & 0 & 0 \\ b & 0 & 1 & 0 \\ c & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & d & 1 & 0 \\ 0 & e & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & f & 1 \end{bmatrix}.$$

SOLUTION. The product of three Gaussian transformation, $M_1 M_2 M_3$, in the given order is the unit lower triangular matrix whose j th column is the j th column of M_j .

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ a & 1 & 0 & 0 \\ b & d & 1 & 0 \\ c & e & f & 1 \end{bmatrix}. \quad \square$$

Ex. 4.10. Solve the linear system

$$\begin{bmatrix} 4 & 10 & 8 \\ 10 & 26 & 26 \\ 8 & 26 & 61 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 44 \\ 128 \\ 214 \end{bmatrix}.$$

by the Cholesky decomposition.

SOLUTION. The Matlab command `chol` decomposes a positive definite matrix A in the form

$$A = R^T R, \quad \text{where } R \text{ is upper triangular.}$$

```
>> A = [4 10 8; 10 26 26; 8 26 61]; b = [44 128 214]';
>> R = chol(A) % Cholesky decomposition
R =
     2     5     4
     0     1     6
     0     0     3
>> y = R\b % forward substitution
y =
    22
    18
     6
>> x = R\y % backward substitution
x =
    -8
     6
     2
```

□

Ex. 4.11. Do three iterations of Gauss–Seidel’s scheme on the properly permuted system with given initial vector $\mathbf{x}^{(0)}$,

$$\begin{array}{rcccccl} 6x_1 & + & x_2 & - & x_3 & = & 3 & & x_1^{(0)} & = & 1, \\ -x_1 & + & x_2 & + & 7x_3 & = & -17 & \text{with} & x_2^{(0)} & = & 1, \\ x_1 & + & 5x_2 & + & x_3 & = & 0 & & x_3^{(0)} & = & 1. \end{array}$$

SOLUTION. Interchanging rows 2 and 3 and solving for x_1 , x_2 and x_3 , we have

$$\begin{array}{lcl} x_1^{(n+1)} & = & \frac{1}{6} [3 - x_2^{(n)} + x_3^{(n)}] \quad \text{with} \quad x_1^{(0)} = 1, \\ x_2^{(n+1)} & = & \frac{1}{5} [0 - x_1^{(n+1)} - x_3^{(n)}] \quad \text{with} \quad x_2^{(0)} = 1, \\ x_3^{(n+1)} & = & \frac{1}{7} [-17 + x_1^{(n+1)} - x_2^{(n+1)}] \quad \text{with} \quad x_3^{(0)} = 1. \end{array}$$

Hence,

$$\mathbf{x}^{(1)} = \begin{bmatrix} 0.5 \\ -0.3 \\ -2.31429 \end{bmatrix}, \quad \mathbf{x}^{(2)} = \begin{bmatrix} 0.16428 \\ 0.43000 \\ -2.46653 \end{bmatrix}, \quad \mathbf{x}^{(3)} = \begin{bmatrix} 0.01724 \\ 0.48986 \\ -2.49609 \end{bmatrix}.$$

One suspects that

$$\mathbf{x}^{(n)} \rightarrow \begin{bmatrix} 0.0 \\ 0.5 \\ -2.5 \end{bmatrix}$$

as $n \rightarrow \infty$. □

Ex. 4.14. Using least squares, fit a parabola to the data

$$(-1, 2), \quad (0, 0), \quad (1, 1), \quad (2, 2).$$

SOLUTION. We look for a solution of the form

$$f(x) = a_0 + a_1x + a_2x^2.$$

```
>> x = [-1 0 1 2]';
>> A = [x.^0 x x.^2];
>> y = [2 0 1 2]';
>> a = (A'*A \ (A'*y))'
a =
    0.4500    -0.6500     0.7500
```

The parabola is

$$f(x) = 0.45 - 0.65x + 0.75x^2.$$

The Matlab command `A \ y` produces the same answer. It uses the normal equations with the Cholesky or *LU* decomposition, or, perhaps, the *QR* decomposition, □

Ex. 4.18. Determine and sketch the Gershgorin disks that contain the eigenvalues of the matrix

$$A = \begin{bmatrix} -2 & 1/2 & i/2 \\ 1/2 & 0 & i/2 \\ -i/2 & -i/2 & 2 \end{bmatrix}.$$

SOLUTION. The centres, c_i , and radii, r_i , of the disks are

$$\begin{aligned} c_1 &= -2, & r_1 &= |1/2| + |i/2| = 1, \\ c_2 &= 0, & r_2 &= |1/2| + |i/2| = 1, \\ c_3 &= 2, & r_3 &= |-i/2| + |i/2| = 1. \end{aligned}$$

Note that the eigenvalues are real since the matrix A is symmetric, $A^T = A$. $\square$

Solutions to Exercises for Chapter 5

The M-file `exr5_25` for Exercises 5.3, 5.8, 5.13 and 5.12 is

```
function yprime = exr5_25(x,y); % Exercises 12.3, 12.8, 12.13 and 12.25.
yprime = x+cos(y);
```

Ex. 5.3. Use Euler's method with $h = 0.1$ to obtain a four-decimal approximation for the initial value problem

$$y' = x + \cos y, \quad y(0) = 0$$

on $0 \leq x \leq 1$ and plot the numerical solution.

SOLUTION. **The Matlab numeric solution.**— Euler's method applied to the given differential equation:

```
clear
h = 0.1; x0= 0; xf= 1; y0 = 0;
n = ceil((xf-x0)/h); % number of steps
%
count = 2; print_time = 1; % when to write to output
x = x0; y = y0; % initialize x and y
output1 = [0 x0 y0];
for i=1:n
    z = y + h*exr5_25(x,y);
    x = x + h;
    if count > print_time
        output1 = [output1; i x z];
        count = count - print_time;
    end
    y = z;
    count = count + 1;
end
output1
save output1 %for printing the graph
```

The command `output1` prints the values of n , x , and y .

n	x	y
0	0	0
1.000000000000000	0.100000000000000	0.100000000000000
2.000000000000000	0.200000000000000	0.20950041652780
3.000000000000000	0.300000000000000	0.32731391010682
4.000000000000000	0.400000000000000	0.45200484393704
5.000000000000000	0.500000000000000	0.58196216946658
6.000000000000000	0.600000000000000	0.71550074191996

7.000000000000000	0.700000000000000	0.85097722706339
8.000000000000000	0.800000000000000	0.98690209299587
9.000000000000000	0.900000000000000	1.12202980842386
10.000000000000000	1.000000000000000	1.25541526027779

□

Ex. 5.8. Use the improved Euler method with $h = 0.1$ to obtain a four-decimal approximation for the initial value problem

$$y' = x + \cos y, \quad y(0) = 0$$

on $0 \leq x \leq 1$ and plot the numerical solution.

SOLUTION. The Matlab numeric solution.—The improved Euler method applied to the given differential equation:

```
clear
h = 0.1; x0= 0; xf= 1; y0 = 0;
n = ceil((xf-x0)/h); % number of steps
%
count = 2; print_time = 1; % when to write to output
x = x0; y = y0; % initialize x and y
output2 = [0 x0 y0];
for i=1:n
    zp = y + h*exr5_25(x,y); % Euler's method
    z = y + (1/2)*h*(exr5_25(x,y)+exr5_25(x+h,zp));
    x = x + h;
    if count > print_time
        output2 = [output2; i x z];
        count = count - print_time;
    end
    y = z;
count = count + 1;
end
output2
save output2 %for printing the graph
```

The command `output2` prints the values of n , x , and y .

n	x	y
0	0	0
1.000000000000000	0.100000000000000	0.10475020826390
2.000000000000000	0.200000000000000	0.21833345972227
3.000000000000000	0.300000000000000	0.33935117091202
4.000000000000000	0.400000000000000	0.46622105817179
5.000000000000000	0.500000000000000	0.59727677538612
6.000000000000000	0.600000000000000	0.73088021271199
7.000000000000000	0.700000000000000	0.86552867523997
8.000000000000000	0.800000000000000	0.99994084307400
9.000000000000000	0.900000000000000	1.13311147003613
10.000000000000000	1.000000000000000	1.26433264384505

□

Ex. 5.13. Use the Runge-Kutta method of order 4 with $h = 0.1$ to obtain a six-decimal approximation for the initial value problem

$$y' = x + \cos y, \quad y(0) = 0$$

on $0 \leq x \leq 1$ and plot the numerical solution.

SOLUTION. The Matlab numeric solution.— The Runge-Kutta method of order 4 applied to the given differential equation:

```
clear
h = 0.1; x0= 0; xf= 1; y0 = 0;
n = ceil((xf-x0)/h); % number of steps
%
count = 2; print_time = 1; % when to write to output
x = x0; y = y0; % initialize x and y
output3 = [0 x0 y0];
for i=1:n
    k1 = h*exr5_25(x,y);
    k2 = h*exr5_25(x+h/2,y+k1/2);
    k3 = h*exr5_25(x+h/2,y+k2/2);
    k4 = h*exr5_25(x+h,y+k3);
    z = y + (1/6)*(k1+2*k2+2*k3+k4);
    x = x + h;
    if count > print_time
        output3 = [output3; i x z];
        count = count - print_time;
    end
    y = z;
count = count + 1;
end
output3
save output3 % for printing the graph
```

The command `output3` prints the values of n , x , and y .

n	x	y
0	0	0
1.000000000000000	0.100000000000000	0.10482097362427
2.000000000000000	0.200000000000000	0.21847505355285
3.000000000000000	0.300000000000000	0.33956414151249
4.000000000000000	0.400000000000000	0.46650622608728
5.000000000000000	0.500000000000000	0.59763447559658
6.000000000000000	0.600000000000000	0.73130914485224
7.000000000000000	0.700000000000000	0.86602471267959
8.000000000000000	0.800000000000000	1.00049620051241
9.000000000000000	0.900000000000000	1.13371450064800
10.000000000000000	1.000000000000000	1.26496830711844

□

Ex. 5.25. Use the Adams–Bashforth–Moulton four-step predictor-corrector method with $h = 0.1$ to obtain a six-decimal approximation for the initial value problem

$$y' = x + \cos y, \quad y(0) = 0$$

on $0 \leq x \leq 1$, estimate the local error at $x = 0.5$, and plot the numerical solution.

SOLUTION. The Matlab numeric solution.— The initial conditions and the Runge–Kutta method of order 4 are used to obtain the four starting values for the ABM four-step method.

```
clear
h = 0.1; x0= 0; xf= 1; y0 = 0;
n = ceil((xf-x0)/h); % number of steps
%
count = 2; print_time = 1; % when to write to output
x = x0; y = y0; % initialize x and y
output4 = [0 x0 y0 0];
%RK4
for i=1:3
    k1 = h*exr5_25(x,y);
    k2 = h*exr5_25(x+h/2,y+k1/2);
    k3 = h*exr5_25(x+h/2,y+k2/2);
    k4 = h*exr5_25(x+h,y+k3);
    z = y + (1/6)*(k1+2*k2+2*k3+k4);
    x = x + h;
    if count > print_time
        output4 = [output4; i x z 0];
        count = count - print_time;
    end
    y = z;
count = count + 1;
end
% ABM4
for i=4:n
    zp = y + (h/24)*(55*exr5_25(output4(i,2),output4(i,3))-...
        59*exr5_25(output4(i-1,2),output4(i-1,3))+...
        37*exr5_25(output4(i-2,2),output4(i-2,3))-...
        9*exr5_25(output4(i-3,2),output4(i-3,3)) );
    z = y + (h/24)*( 9*exr5_25(x+h,zp)+...
        19*exr5_25(output4(i,2),output4(i,3))-...
        5*exr5_25(output4(i-1,2),output4(i-1,3))+...
        exr5_25(output4(i-2,2),output4(i-2,3)) );
    x = x + h;
    if count > print_time
        errest = -(19/270)*(z-zp);
        output4 = [output4; i x z errest];
        count = count - print_time;
    end
    y = z;
```

```

count = count + 1;
end
output4
save output4 %for printing the grap

```

The command `output4` prints the values of n , x , and y .

n	x	y	Error estimate
0	0	0	0
1.000000000000000	0.100000000000000	0.10482097362427	0
2.000000000000000	0.200000000000000	0.21847505355285	0
3.000000000000000	0.300000000000000	0.33956414151249	0
4.000000000000000	0.400000000000000	0.46650952510670	-0.00000234408483
5.000000000000000	0.500000000000000	0.59764142006542	-0.00000292485029
6.000000000000000	0.600000000000000	0.73131943222018	-0.00000304450366
7.000000000000000	0.700000000000000	0.86603741396612	-0.00000269077058
8.000000000000000	0.800000000000000	1.00050998975914	-0.00000195879670
9.000000000000000	0.900000000000000	1.13372798977088	-0.00000104794662
10.000000000000000	1.000000000000000	1.26498035231682	-0.00000017019624

□

The numerical solutions for Exercises 12.3, 12.8, 12.13 and 12.25 are plotted by the commands:

```

load output1; load output2; load output3; load output4;
subplot(2,2,1); plot(output1(:,2),output1(:,3));
title('Plot of solution y_n for Exercise 5.3');
xlabel('x_n'); ylabel('y_n');
subplot(2,2,2); plot(output2(:,2),output2(:,3));
title('Plot of solution y_n for Exercise 5.8');
xlabel('x_n'); ylabel('y_n');
subplot(2,2,3); plot(output3(:,2),output3(:,3));
title('Plot of solution y_n for Exercise 5.13');
xlabel('x_n'); ylabel('y_n');
subplot(2,2,4); plot(output4(:,2),output4(:,3));
title('Plot of solution y_n for Exercise 5.25');
xlabel('x_n'); ylabel('y_n');
print -deps Fig9_3

```

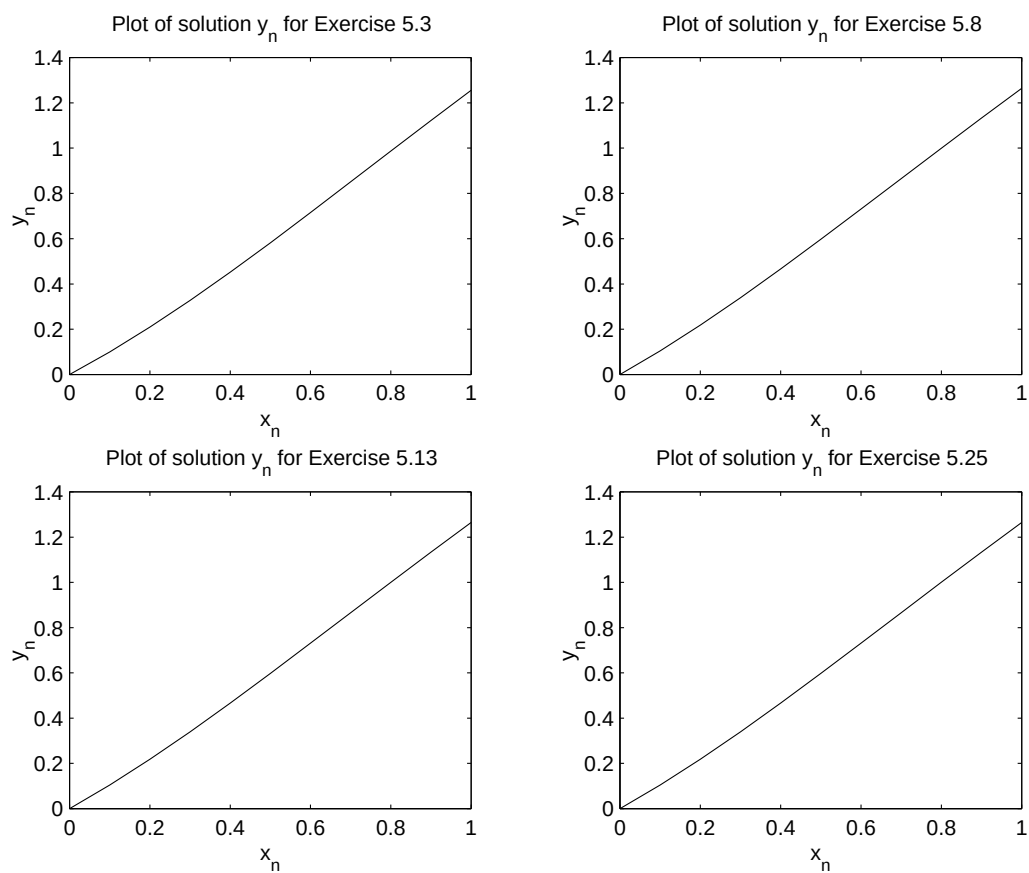



FIGURE 8.4. Graph of numerical solutions of Exercises 12.3 (Euler), 12.8 (improved Euler), 12.13 (RK4) and 12.25 (ABM4).

Index

- absolute error, 1
- absolutely stable method for ODE, 101
- absolutely stable multistep method, 110
- Adams–Bashforth multistep method, 110
- Adams–Bashforth–Moulton method
 - four-step, 113
 - three-step, 112
- Adams–Moulton multistep method, 110
- Aitken’s process, 20
- backward differentiation formula, 123
- BDF (backward differentiation formula), 123
- bisection method, 5
- Butcher tableau, 93
- centered formula for $f''(x)$, 42
- centred formula for $f'(x)$, 42
- Cholesky decomposition, 67
- clamped boundary, 38
- clamped spline, 38
- classic Runge–Kutta method, 94
- composite integration rule
 - midpoint, 50
 - Simpson’s, 54
 - trapezoidal, 52
- condition number of a matrix, 72
- consistent method for ODE, 100
- convergent method for ODE, 99
- corrector, 111
- cubic spline, 38
- diagonally dominant matrix, 68
- divided difference
 - k th, 31
 - first, 29
- divided difference table, 31
- Dormand–Prince pair
 - seven-stage, 106
- DP(5,4)7M, 106
- eigenvalue of a matrix, 78
- eigenvector, 78
- error, 1
- Euclidean matrix norm, 72
- Euler’s method, 88
- exact solution of ODE, 87
- explicit multistep method, 110, 120
- extreme value theorem, 4
- first forward difference, 32
- first-order initial value problem, 87
- fixed point, 8
 - attractive, 8
 - indifferent, 8
 - repulsive, 8
- floating point number, 1
- forward difference
 - k th, 33
 - second, 32
- Fourier–Legendre series, 143
- free boundary, 38
- Frobenius norm of a matrix, 72
- FSAL method for ODE, 107
- function of order p , 87
- Gauss–Seidel iteration, 74
- Gaussian quadrature, 145
 - three-point, 146
 - two-point, 146
- Gaussian transformation, 60
 - inverse, 60
 - product, 61
- Gershgorin
 - disk, 79
 - Theorem, 79
- global Newton-bisection method, 18
- Hermite interpolating polynomial, 36
- Heun’s method
 - of order 2, 93
- Horner’s method, 23
- Householder reflection, 82
- implicit multistep method, 120
- improved Euler’s method, 91
- intermediate value theorem, 4
- interpolating polynomial
 - Gregory–Newton
 - backward-difference, 35
 - forward-difference, 33

- Müller's method, 25
- Newton divided difference, 29
- parabola method, 25
- interval of absolute stability, 101
- inverse power method, 81
- iterative method, 73
- Jacobi iteration, 75
- Jacobi method for eigenvalues, 83
- l_1 -norm
 - of a matrix, 72
 - of a vector, 72
- l_2 -norm
 - of a matrix, 72
 - of a vector, 72
- l_∞ -norm
 - of a matrix, 72
 - of a vector, 72
- Lagrange basis, 27
- Lagrange interpolating polynomial, 27
- Legendre
 - differential equation, 155
 - polynomial $P_n(x)$, 143
- linear regression, 76
- Lipschitz condition, 87
- local approximation, 111
- local error of method for ODE, 100
- local extrapolation, 106
- local truncation error, 88, 89, 100
- MATLAB
 - fzero** function, 20
 - ode113, 121
 - ode15s, 129
 - ode23, 104
 - ode23s, 129
 - ode23t, 129
 - ode23tb, 129
- mean value theorem, 4
 - for integral, 4
 - for sum, 4
- method of false position, 17
- method of order p , 100
- midpoint rule, 47, 48
- multistep method, 110
- natural boundary, 38
- natural spline, 38
- NDF (numerical differentiation formula, 123
- Newton's method, 13
 - modified, 15
- Newton–Raphson method, 13
- normal equations, 76
- normal matrix, 84
- numerical differentiation formula, 123
- numerical solution of ODE, 87
- operation
 - gaxpy, 71
 - saxpy, 70
- order of an iterative method, 13
- overdetermined system, 75, 83
- partial pivoting, 59
- PECE mode, 112
- PECLE mode, 112
- phenomenon of stiffness, 122
- pivot, 60
- positive definite matrix, 67
- power method, 80
- predictor, 111
- principal minor, 67
- QR
 - algorithm, 83
 - decomposition, 82
- quadratic regression, 77
- rate of convergence, 13
- region of absolute stability, 100
- regula falsi, 17
- relative error, 1
- residual, 82
- Richardson's extrapolation, 45
- RKF(4,5), 107
- RKV(5,6), 109
- roundoff error, 1, 43, 90
- Runge–Kutta method
 - four-stage, 94
 - fourth-order, 94
 - second-order, 93
 - third-order, 93
- Runge–Kutta–Fehlberg pair
 - six-stage, 107
- Runge–Kutta–Verner pair
 - eight-stage, 109
- scaling rows, 65
- Schur decomposition, 84
- secant method, 16
- signum function sign, 3
- singular value decomposition, 84
- stability function, 101
- stiff system, 122
 - in an interval, 123
- stiffness ratio, 122
- stopping criterion, 12
- Sturm–Liouville problem, 143
- subordinate matrix norm, 72
- substitution
 - backward, 61
 - forward, 61
- supremum norm
 - of a vector, 72
- three-point formula for $f'(x)$, 42
- trapezoidal rule, 48
- truncation error, 1
- truncation error of a method, 90

- two-point formula for $f'(x)$, 41
- well-posed problem, 87
- zero-stable method for ODE, 100